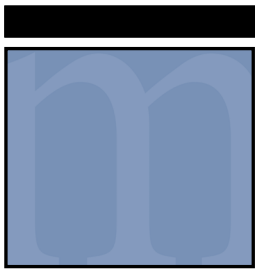
MICHAEL BOWLES

MACHINE LEARNING IN PYTHON®

ESSENTIAL TECHNIQUES FOR PREDICTIVE ANALYSIS

WILEY

Machine Learning in Python®



Machine Learning in Python®

Essential Techniques for Predictive Analysis

Michael Bowles

WILEY

Machine Learning in Python®: Essential Techniques for Predictive Analysis

Published by
John Wiley & Sons, Inc.
10475 Crosspoint Boulevard
Indianapolis, IN 46256
www.wiley.com

Copyright © 2015 by John Wiley & Sons, Inc., Indianapolis, Indiana
Published simultaneously in Canada

ISBN: 978-1-118-96174-2
ISBN: 978-1-118-96176-6 (ebk)
ISBN: 978-1-118-96175-9 (ebk)

Manufactured in the United States of America

10 9 8 7 6 5 4 3 2 1

No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning or otherwise, except as permitted under Sections 107 or 108 of the 1976 United States Copyright Act, without either the prior written permission of the Publisher, or authorization through payment of the appropriate per-copy fee to the Copyright Clearance Center, 222 Rosewood Drive, Danvers, MA 01923, (978) 750-8400, fax (978) 646-8600. Requests to the Publisher for permission should be addressed to the Permissions Department, John Wiley & Sons, Inc., 111 River Street, Hoboken, NJ 07030, (201) 748-6011, fax (201) 748-6008, or online at http://www.wiley.com/go/permissions.

Limit of Liability/Disclaimer of Warranty: The publisher and the author make no representations or warranties with respect to the accuracy or completeness of the contents of this work and specifically disclaim all warranties, including without limitation warranties of fitness for a particular purpose. No warranty may be created or extended by sales or promotional materials. The advice and strategies contained herein may not be suitable for every situation. This work is sold with the understanding that the publisher is not engaged in rendering legal, accounting, or other professional services. If professional assistance is required, the services of a competent professional person should be sought. Neither the publisher nor the author shall be liable for damages arising herefrom. The fact that an organization or Web site is referred to in this work as a citation and/or a potential source of further information does not mean that the author or the publisher endorses the information the organization or website may provide or recommendations it may make. Further, readers should be aware that Internet websites listed in this work may have changed or disappeared between when this work was written and when it is read.

For general information on our other products and services please contact our Customer Care Department within the United States at (877) 762-2974, outside the United States at (317) 572-3993 or fax (317) 572-4002.

Wiley publishes in a variety of print and electronic formats and by print-on-demand. Some material included with standard print versions of this book may not be included in e-books or in print-on-demand. If this book refers to media such as a CD or DVD that is not included in the version you purchased, you may download this material at http://booksupport.wiley.com. For more information about Wiley products, visit www.wiley.com.

Library of Congress Control Number: 2015930541

Trademarks: Wiley and the Wiley logo are trademarks or registered trademarks of John Wiley & Sons, Inc. and/or its affiliates, in the United States and other countries, and may not be used without written permission. Python is a registered trademark of Python Software Foundation. All other trademarks are the property of their respective owners. John Wiley & Sons, Inc. is not associated with any product or vendor mentioned in this book.

To my children, Scott, Seth, and Cayley. Their blossoming lives and selves bring me more joy than anything else in this world.

To my close friends David and Ron for their selfless generosity and steadfast friendship.

To my friends and colleagues at Hacker Dojo in Mountain View, California, for their technical challenges and repartee.

To my climbing partners. One of them, Katherine, says climbing partners make the best friends because "they see you paralyzed with fear, offer encouragement to overcome it, and celebrate when you do."

About the Author

Dr. Michael Bowles (Mike) holds Bachelor's and Master's degrees in Mechanical Engineering, an Sc.D. in Instrumentation, and an MBA. He has worked in academia, technology, and business. Mike currently works with startup companies where machine learning is integral to success. He serves variously as part of the management team, a consultant, or advisor. He also teaches machine learning courses at Hacker Dojo, a co-working space and startup incubator in Mountain View, California.

Mike was born in Oklahoma and earned his Bachelor's and Master's degrees there. Then after a stint in Southeast Asia, Mike went to Cambridge for his Sc.D. and then held the C. Stark Draper Chair at MIT after graduation. Mike left Boston to work on communications satellites at Hughes Aircraft company in Southern California, and then after completing an MBA at UCLA moved to the San Francisco Bay Area to take roles as founder and CEO of two successful venture-backed startups.

Mike remains actively involved in technical and startup-related work. Recent projects include the use of machine learning in automated trading, predicting biological outcomes on the basis of genetic information, natural language processing for website optimization, predicting patient outcomes from demographic and lab data, and due diligence work on companies in the machine learning and big data arenas. Mike can be reached through `www.mbowles.com`.

About the Technical Editor

Daniel Posner holds Bachelor's and Master's degrees in Economics and is completing a Ph.D. in Biostatistics at Boston University. He has provided statistical consultation for pharmaceutical and biotech firms as well as for researchers at the Palo Alto VA hospital.

Daniel has collaborated with the author extensively on topics covered in this book. In the past, they have written grant proposals to develop web-scale gradient boosting algorithms. Most recently, they worked together on a consulting contract involving random forests and spline basis expansions to identify key variables in drug trial outcomes and to sharpen predictions in order to reduce the required trial populations.

Credits

Executive Editor
Robert Elliott

Project Editor
Jennifer Lynn

Technical Editor
Daniel Posner

Production Editor
Dassi Zeidel

Copy Editor
Keith Cline

Manager of Content Development & Assembly
Mary Beth Wakefield

Marketing Director
David Mayhew

Marketing Manager
Carrie Sherrill

Professional Technology & Strategy Director
Barry Pruett

Business Manager
Amy Knies

Associate Publisher
Jim Minatel

Project Coordinator, Cover
Brent Savage

Proofreader
Word One New York

Indexer
Johnna VanHoose Dinse

Cover Designer
Wiley

Acknowledgments

I'd like to acknowledge the splendid support that people at Wiley have offered during the course of writing this book. It began with Robert Elliot, the acquisitions editor, who first contacted me about writing a book; he was very easy to work with. It continued with Jennifer Lynn, who has done the editing on the book. She's been very responsive to questions and very patiently kept me on schedule during the writing. I thank you both.

I also want to acknowledge the enormous comfort that comes from having such a sharp, thorough statistician and programmer as Daniel Posner doing the technical editing on the book. Thank you for that and thanks also for the fun and interesting discussions on machine learning, statistics, and algorithms. I don't know anyone else who'll get as deep as fast.

Contents at a Glance

Introduction		xxiii
Chapter 1	The Two Essential Algorithms for Making Predictions	1
Chapter 2	Understand the Problem by Understanding the Data	23
Chapter 3	Predictive Model Building: Balancing Performance, Complexity, and Big Data	75
Chapter 4	Penalized Linear Regression	121
Chapter 5	Building Predictive Models Using Penalized Linear Methods	165
Chapter 6	Ensemble Methods	211
Chapter 7	Building Ensemble Models with Python	255
Index		319

Contents

Introduction **xxiii**

Chapter 1 The Two Essential Algorithms for Making Predictions **1**
Why Are These Two Algorithms So Useful? 2
What Are Penalized Regression Methods? 7
What Are Ensemble Methods? 9
How to Decide Which Algorithm to Use 11
The Process Steps for Building a Predictive Model 13
Framing a Machine Learning Problem 15
Feature Extraction and Feature Engineering 17
Determining Performance of a Trained Model 18
Chapter Contents and Dependencies 18
Summary 20

Chapter 2 Understand the Problem by Understanding the Data **23**
The Anatomy of a New Problem 24
Different Types of Attributes and Labels Drive Modeling Choices 26
Things to Notice about Your New Data Set 27
Classification Problems: Detecting Unexploded Mines Using Sonar 28
Physical Characteristics of the Rocks Versus Mines Data Set 29
Statistical Summaries of the Rocks versus Mines Data Set 32
Visualization of Outliers Using Quantile-Quantile Plot 35
Statistical Characterization of Categorical Attributes 37
How to Use Python Pandas to Summarize the Rocks Versus Mines Data Set 37

Visualizing Properties of the Rocks versus Mines Data Set 40
Visualizing with Parallel Coordinates Plots 40
Visualizing Interrelationships between Attributes and Labels 42
Visualizing Attribute and Label Correlations Using a Heat Map 49
Summarizing the Process for Understanding Rocks versus Mines Data Set 50
Real-Valued Predictions with Factor Variables: How Old Is Your Abalone? 50
Parallel Coordinates for Regression Problems—Visualize Variable Relationships for Abalone Problem 56
How to Use Correlation Heat Map for Regression—Visualize Pair-Wise Correlations for the Abalone Problem 60
Real-Valued Predictions Using Real-Valued Attributes: Calculate How Your Wine Tastes 62
Multiclass Classification Problem: What Type of Glass Is That? 68
Summary 73

Chapter 3 Predictive Model Building: Balancing Performance, Complexity, and Big Data 75
The Basic Problem: Understanding Function Approximation 76
Working with Training Data 76
Assessing Performance of Predictive Models 78
Factors Driving Algorithm Choices and Performance—Complexity and Data 79
Contrast Between a Simple Problem and a Complex Problem 80
Contrast Between a Simple Model and a Complex Model 82
Factors Driving Predictive Algorithm Performance 86
Choosing an Algorithm: Linear or Nonlinear? 87
Measuring the Performance of Predictive Models 88
Performance Measures for Different Types of Problems 88
Simulating Performance of Deployed Models 99
Achieving Harmony Between Model and Data 101
Choosing a Model to Balance Problem Complexity, Model Complexity, and Data Set Size 102
Using Forward Stepwise Regression to Control Overfitting 103
Evaluating and Understanding Your Predictive Model 108
Control Overfitting by Penalizing Regression Coefficients—Ridge Regression 110
Summary 119

Chapter 4 Penalized Linear Regression 121
Why Penalized Linear Regression Methods Are So Useful 122
Extremely Fast Coefficient Estimation 122
Variable Importance Information 122
Extremely Fast Evaluation When Deployed 123

Reliable Performance 123
Sparse Solutions 123
Problem May Require Linear Model 124
When to Use Ensemble Methods 124
Penalized Linear Regression: Regulating Linear Regression for Optimum Performance 124
Training Linear Models: Minimizing Errors and More 126
Adding a Coefficient Penalty to the OLS Formulation 127
Other Useful Coefficient Penalties—Manhattan and ElasticNet 128
Why Lasso Penalty Leads to Sparse Coefficient Vectors 129
ElasticNet Penalty Includes Both Lasso and Ridge 131
Solving the Penalized Linear Regression Problem 132
Understanding Least Angle Regression and Its Relationship to Forward Stepwise Regression 132
How LARS Generates Hundreds of Models of Varying Complexity 136
Choosing the Best Model from The Hundreds LARS Generates 139
Using Glmnet: Very Fast and Very General 144
Comparison of the Mechanics of Glmnet and LARS Algorithms 145
Initializing and Iterating the Glmnet Algorithm 146
Extensions to Linear Regression with Numeric Input 151
Solving Classification Problems with Penalized Regression 151
Working with Classification Problems Having More Than Two Outcomes 155
Understanding Basis Expansion: Using Linear Methods on Nonlinear Problems 156
Incorporating Non-Numeric Attributes into Linear Methods 158
Summary 163

Chapter 5 Building Predictive Models Using Penalized Linear Methods 165
Python Packages for Penalized Linear Regression 166
Multivariable Regression: Predicting Wine Taste 167
Building and Testing a Model to Predict Wine Taste 168
Training on the Whole Data Set before Deployment 172
Basis Expansion: Improving Performance by Creating New Variables from Old Ones 178
Binary Classification: Using Penalized Linear Regression to Detect Unexploded Mines 181
Build a Rocks versus Mines Classifier for Deployment 191
Multiclass Classification: Classifying Crime Scene Glass Samples 204
Summary 209

Chapter 6 Ensemble Methods 211
Binary Decision Trees 212
How a Binary Decision Tree Generates Predictions 213
How to Train a Binary Decision Tree 214
Tree Training Equals Split Point Selection 218
How Split Point Selection Affects Predictions 218
Algorithm for Selecting Split Points 219
Multivariable Tree Training—Which Attribute to Split? 219
Recursive Splitting for More Tree Depth 220
Overfitting Binary Trees 221
Measuring Overfit with Binary Trees 221
Balancing Binary Tree Complexity for Best Performance 222
Modifications for Classification and Categorical Features 225
Bootstrap Aggregation: "Bagging" 226
How Does the Bagging Algorithm Work? 226
Bagging Performance—Bias versus Variance 229
How Bagging Behaves on Multivariable Problem 231
Bagging Needs Tree Depth for Performance 235
Summary of Bagging 236
Gradient Boosting 236
Basic Principle of Gradient Boosting Algorithm 237
Parameter Settings for Gradient Boosting 239
How Gradient Boosting Iterates Toward a Predictive Model 240
Getting the Best Performance from Gradient Boosting 240
Gradient Boosting on a Multivariable Problem 244
Summary for Gradient Boosting 247
Random Forest 247
Random Forests: Bagging Plus Random Attribute Subsets 250
Random Forests Performance Drivers 251
Random Forests Summary 252
Summary 252

Chapter 7 Building Ensemble Models with Python 255
Solving Regression Problems with Python Ensemble Packages 255
Building a Random Forest Model to Predict Wine Taste 256
Constructing a RandomForestRegressor Object 256
Modeling Wine Taste with RandomForestRegressor 259
Visualizing the Performance of a Random Forests Regression Model 262
Using Gradient Boosting to Predict Wine Taste 263
Using the Class Constructor for GradientBoostingRegressor 263
Using GradientBoostingRegressor to Implement a Regression Model 267
Assessing the Performance of a Gradient Boosting Model 269

Coding Bagging to Predict Wine Taste 270
Incorporating Non-Numeric Attributes in Python Ensemble Models 275
Coding the Sex of Abalone for Input to Random Forest Regression in Python 275
Assessing Performance and the Importance of Coded Variables 278
Coding the Sex of Abalone for Gradient Boosting Regression in Python 278
Assessing Performance and the Importance of Coded Variables with Gradient Boosting 282
Solving Binary Classification Problems with Python Ensemble Methods 284
Detecting Unexploded Mines with Python Random Forest 285
Constructing a Random Forests Model to Detect Unexploded Mines 287
Determining the Performance of a Random Forests Classifier 291
Detecting Unexploded Mines with Python Gradient Boosting 291
Determining the Performance of a Gradient Boosting Classifier 298
Solving Multiclass Classification Problems with Python Ensemble Methods 302
Classifying Glass with Random Forests 302
Dealing with Class Imbalances 305
Classifying Glass Using Gradient Boosting 307
Assessing the Advantage of Using Random Forest Base Learners with Gradient Boosting 311
Comparing Algorithms 314
Summary 315

Index **319**

Introduction

Extracting actionable information from data is changing the fabric of modern business in ways that directly affect programmers. One way is the demand for new programming skills. Market analysts predict demand for people with advanced statistics and machine learning skills will exceed supply by 140,000 to 190,000 by 2018. That means good salaries and a wide choice of interesting projects for those who have the requisite skills. Another development that affects programmers is progress in developing core tools for statistics and machine learning. This relieves programmers of the need to program intricate algorithms for themselves each time they want to try a new one. Among general-purpose programming languages, Python developers have been in the forefront, building state-of-the-art machine learning tools, but there is a gap between having the tools and being able to use them efficiently.

Programmers can gain general knowledge about machine learning in a number of ways: online courses, a number of well-written books, and so on. Many of these give excellent surveys of machine learning algorithms and examples of their use, but because of the availability of so many different algorithms, it's difficult to cover the details of their usage in a survey.

This leaves a gap for the practitioner. The number of algorithms available requires making choices that a programmer new to machine learning might not be equipped to make until trying several, and it leaves the programmer to fill in the details of the usage of these algorithms in the context of overall problem formulation and solution.

This book attempts to close that gap. The approach taken is to restrict the algorithms covered to two families of algorithms that have proven to give optimum performance for a wide variety of problems. This assertion is supported by their dominant usage in machine learning competitions, their early inclusion in newly developed packages of machine learning tools, and their performance in

comparative studies (as discussed in Chapter 1, "The Two Essential Algorithms for Making Predictions"). Restricting attention to two algorithm families makes it possible to provide good coverage of the principles of operation and to run through the details of a number of examples showing how these algorithms apply to problems with different structures.

The book largely relies on code examples to illustrate the principles of operation for the algorithms discussed. I've discovered in the classes I teach at Hacker Dojo in Mountain View, California, that programmers generally grasp principles more readily by seeing simple code illustrations than by looking at math.

This book focuses on Python because it offers a good blend of functionality and specialized packages containing machine learning algorithms. Python is an often-used language that is well known for producing compact, readable code. That fact has led a number of leading companies to adopt Python for prototyping and deployment. Python developers are supported by a large community of fellow developers, development tools, extensions, and so forth. Python is widely used in industrial applications and in scientific programming, as well. It has a number of packages that support computationally-intensive applications like machine learning, and it is a good collection of the leading machine learning algorithms (so you don't have to code them yourself). Python is a better general-purpose programming language than specialized statistical languages such as R or SAS (Statistical Analysis System). Its collection of machine learning algorithms incorporates a number of top-flight algorithms and continues to expand.

Who This Book Is For

This book is intended for Python programmers who want to add machine learning to their repertoire, either for a specific project or as part of keeping their toolkit relevant. Perhaps a new problem has come up at work that requires machine learning. With machine learning being covered so much in the news these days, it's a useful skill to claim on a resume.

This book provides the following for Python programmers:

- A description of the basic problems that machine learning attacks
- Several state-of-the-art algorithms
- The principles of operation for these algorithms
- Process steps for specifying, designing, and qualifying a machine learning system
- Examples of the processes and algorithms
- Hackable code

To get through this book easily, your primary background requirements include an understanding of programming or computer science and the ability to read and write code. The code examples, libraries, and packages are all Python, so the book will prove most useful to Python programmers. In some cases, the book runs through code for the core of an algorithm to demonstrate the operating principles, but then uses a Python package incorporating the algorithm to apply the algorithm to problems. Seeing code often gives programmers an intuitive grasp of an algorithm in the way that seeing the math does for others. Once the understanding is in place, examples will use developed Python packages with the bells and whistles that are important for efficient use (error checking, handling input and output, developed data structures for the models, defined predictor methods incorporating the trained model, and so on).

In addition to having a programming background, some knowledge of math and statistics will help get you through the material easily. Math requirements include some undergraduate-level differential calculus (knowing how to take a derivative and a little bit of linear algebra), matrix notation, matrix multiplication, and matrix inverse. The main use of these will be to follow the derivations of some of the algorithms covered. Many times, that will be as simple as taking a derivative of a simple function or doing some basic matrix manipulations. Being able to follow the calculations at a conceptual level may aid your understanding of the algorithm. Understanding the steps in the derivation can help you to understand the strengths and weaknesses of an algorithm and can help you to decide which algorithm is likely to be the best choice for a particular problem.

This book also uses some general probability and statistics. The requirements for these include some familiarity with undergraduate-level probability and concepts such as the mean value of a list of real numbers, variance, and correlation. You can always look through the code if some of the concepts are rusty for you.

This book covers two broad classes of machine learning algorithms: penalized linear regression (for example, Ridge and Lasso) and ensemble methods (for example, Random Forests and Gradient Boosting). Each of these families contains variants that will solve regression and classification problems. (You learn the distinction between classification and regression early in the book.)

Readers who are already familiar with machine learning and are only interested in picking up one or the other of these can skip to the two chapters covering that family. Each method gets two chapters—one covering principles of operation and the other running through usage on different types of problems. Penalized linear regression is covered in Chapter 4, “Penalized Linear Regression,” and Chapter 5, “Building Predictive Models Using Penalized Linear Methods.” Ensemble methods are covered in Chapter 6, “Ensemble Methods,” and Chapter 7, “Building Predictive Models with Python.” To

familiarize yourself with the problems addressed in the chapters on usage of the algorithms, you might find it helpful to skim Chapter 2, "Understand the Problem by Understanding the Data," which deals with data exploration. Readers who are just starting out with machine learning and want to go through from start to finish might want to save Chapter 2 until they start looking at the solutions to problems in later chapters.

What This Book Covers

As mentioned earlier, this book covers two algorithm families that are relatively recent developments and that are still being actively researched. They both depend on, and have somewhat eclipsed, earlier technologies.

Penalized linear regression represents a relatively recent development in ongoing research to improve on ordinary least squares regression. Penalized linear regression has several features that make it a top choice for predictive analytics. Penalized linear regression introduces a tunable parameter that makes it possible to balance the resulting model between overfitting and underfitting. It also yields information on the relative importance of the various inputs to the predictions it makes. Both of these features are vitally important to the process of developing predictive models. In addition, penalized linear regression yields best prediction performance in some classes of problems, particularly underdetermined problems and problems with very many input parameters such as genetics and text mining. Furthermore, there's been a great deal of recent development of coordinate descent methods, making training penalized linear regression models extremely fast.

To help you understand penalized linear regression, this book recapitulates ordinary linear regression and other extensions to it, such as stepwise regression. The hope is that these will help cultivate intuition.

Ensemble methods are one of the most powerful predictive analytics tools available. They can model extremely complicated behavior, especially for problems that are vastly overdetermined, as is often the case for many web-based prediction problems (such as returning search results or predicting ad click-through rates). Many seasoned data scientists use ensemble methods as their first try because of their performance. They are also relatively simple to use, and they also rank variables in terms of predictive performance.

Ensemble methods have followed a development path parallel to penalized linear regression. Whereas penalized linear regression evolved from overcoming the limitations of ordinary regression, ensemble methods evolved to overcome the limitations of binary decision trees. Correspondingly, this book's coverage of ensemble methods covers some background on binary decision trees because ensemble methods inherit some of their properties from binary

decision trees. Understanding them helps cultivate intuition about ensemble methods.

How This Book Is Structured

This book follows the basic order in which you would approach a new prediction problem. The beginning involves developing an understanding of the data and determining how to formulate the problem, and then proceeds to try an algorithm and measure the performance. In the midst of this sequence, the book outlines the methods and reasons for the steps as they come up. Chapter 1 gives a more thorough description of the types of problems that this book covers and the methods that are used. The book uses several data sets from the UC Irvine data repository as examples, and Chapter 2 exhibits some of the methods and tools that you can use for developing insight into a new data set. Chapter 3, "Predictive Model Building: Balancing Performance, Complexity, and Big Data," talks about the difficulties of predictive analytics and techniques for addressing them. It outlines the relationships between problem complexity, model complexity, data set size, and predictive performance. It discusses overfitting and how to reliably sense overfitting. It talks about performance metrics for different types of problems. Chapters 4 and 5, respectively, cover the background on penalized linear regression and its application to problems explored in Chapter 2. Chapters 6 and 7 cover background and application for ensemble methods.

What You Need to Use This Book

To run the code examples in the book, you need to have Python 2.x, SciPy, NumPy, Pandas, and scikit-learn. These can be difficult to install due to cross-dependencies and version issues. To make the installation easy, I've used a free distribution of these packages that's available from Continuum Analytics (`http://continuum.io/`). Their Anaconda product is a free download and includes Python 2.x and all the packages you need to run the code in this book (and more). I've run the examples on Ubuntu 14.04 Linux but haven't tried them on other operating systems.

Conventions

To help you get the most from the text and keep track of what's happening, we've used a number of conventions throughout the book.

WARNING **Boxes like this one hold important, not-to-be forgotten information that is directly relevant to the surrounding text.**

NOTE **Notes, tips, hints, tricks, and asides to the current discussion are offset and appear like this.**

As for styles in the text:

- We *highlight* new terms and important words when we introduce them.
- We show keyboard strokes like this: Ctrl+A.
- We show filenames, URLs, and code within the text like so: `persistence.properties`.
- We present code in two different ways:

```
We use a monofont type with no highlighting for most code examples.
We use bold to emphasize code that's particularly important in
the present context.
```

Source Code

As you work through the examples in this book, you may choose either to type in all the code manually or to use the source code files that accompany the book. All the source code used in this book is available for download from `http://www.wiley.com/go/pythonmachinelearning`. You will find the code snippets from the source code are accompanied by a download icon and note indicating the name of the program so that you know it's available for download and can easily locate it in the download file. Once at the site, simply locate the book's title (either by using the Search box or by using one of the title lists) and click the Download Code link on the book's detail page to obtain all the source code for the book.

NOTE **Because many books have similar titles, you may find it easiest to search by ISBN; this book's ISBN is 978-1-118-96174-2.**

After you download the code, just decompress it with your favorite compression tool.

Errata

We make every effort to ensure that no errors appear in the text or in the code. However, no one is perfect, and mistakes do occur. If you find an error in one

of our books, like a spelling mistake or faulty piece of code, we would be very grateful for your feedback. By sending in errata, you might save another reader hours of frustration, and at the same time you will be helping us provide even higher-quality information.

To find the errata page for this book, go to `http://www.wiley.com` and locate the title using the Search box or one of the title lists. Then, on the book details page, click the Book Errata link. On this page, you can view all errata that has been submitted for this book and posted by Wiley editors.

Machine Learning in Python®

CHAPTER

1

The Two Essential Algorithms for Making Predictions

This book focuses on the machine learning process and so covers just a few of the most effective and widely used algorithms. It does not provide a survey of machine learning techniques. Too many of the algorithms that might be included in a survey are not actively used by practitioners.

This book deals with one class of machine learning problems, generally referred to as *function approximation*. Function approximation is a subset of problems that are called *supervised learning* problems. Linear regression and its classifier cousin, logistic regression, provide familiar examples of algorithms for function approximation problems. Function approximation problems include an enormous breadth of practical classification and regression problems in all sorts of arenas, including text classification, search responses, ad placements, spam filtering, predicting customer behavior, diagnostics, and so forth. The list is almost endless.

Broadly speaking, this book covers two classes of algorithms for solving function approximation problems: penalized linear regression methods and ensemble methods. This chapter introduces you to both of these algorithms, outlines some of their characteristics, and reviews the results of comparative studies of algorithm performance in order to demonstrate their consistent high performance.

This chapter then discusses the process of building predictive models. It describes the kinds of problems that you'll be able to address with the tools covered here and the flexibilities that you have in how you set up your problem and define the features that you'll use for making predictions. It describes process steps involved in building a predictive model and qualifying it for deployment.

Why Are These Two Algorithms So Useful?

Several factors make the penalized linear regression and ensemble methods a useful collection. Stated simply, they will provide optimum or near-optimum performance on the vast majority of predictive analytics (function approximation) problems encountered in practice, including big data sets, little data sets, wide data sets, tall skinny data sets, complicated problems, and simple problems. Evidence for this assertion can be found in two papers by Rich Caruana and his colleagues:

- "An Empirical Comparison of Supervised Learning Algorithms," by Rich Caruana and Alexandru Niculescu-Mizil[1]
- "An Empirical Evaluation of Supervised Learning in High Dimensions," by Rich Caruana, Nikos Karampatziakis, and Ainur Yessenalina[2]

In those two papers, the authors chose a variety of classification problems and applied a variety of different algorithms to build predictive models. The models were run on test data that were not included in training the models, and then the algorithms included in the studies were ranked on the basis of their performance on the problems. The first study compared 9 different basic algorithms on 11 different machine learning (binary classification) problems. The problems used in the study came from a wide variety of areas, including demographic data, text processing, pattern recognition, physics, and biology. Table 1-1 lists the data sets used in the study using the same names given by the study authors. The table shows how many attributes were available for predicting outcomes for each of the data sets, and it shows what percentage of the examples were positive.

The term *positive example* in a classification problem means an experiment (a line of data from the input data set) in which the outcome is positive. For example, if the classifier is being designed to determine whether a radar return signal indicates the presence of an airplane, then the positive example would be those returns where there was actually an airplane in the radar's field of view. The term *positive* comes from this sort of example where the two outcomes represent presence or absence. Other examples include presence or absence of disease in a medical test or presence or absence of cheating on a tax return.

Not all classification problems deal with presence or absence. For example, determining the gender of an author by machine-reading their text or

machine-analyzing a handwriting sample has two classes—male and female—but there's no sense in which one is the absence of the other. In these cases, there's some arbitrariness in the assignment of the designations "positive" and "negative." The assignments of positive and negative can be arbitrary, but once chosen must be used consistently.

Some of the problems in the first study had many more examples of one class than the other. These are called *unbalanced*. For example, the two data sets Letter.p1 and Letter.p2 pose closely related problems in correctly classifying typed uppercase letters in a wide variety of fonts. The task with Letter.p1 is to correctly classify the letter *O* in a standard mix of letters. The task with Letter.p2 is to correctly classify *A–M* versus *N–Z*. The percentage of positives shown in Table 1-1 reflects this difference.

Table 1-1 also shows the number of "attributes" in each of the data sets. Attributes are the variables you have available to base a prediction on. For example, to predict whether an airplane will arrive at its destination on time or not, you might incorporate attributes such as the name of the airline company, the make and year of the airplane, the level of precipitation at the destination airport, the wind speed and direction along the flight path, and so on. Having a lot of attributes upon which to base a prediction can be a blessing and a curse. Attributes that relate directly to the outcomes being predicted are a blessing. Attributes that are unrelated to the outcomes are a curse. Telling the difference between blessed and cursed attributes requires data. Chapter 3, "Predictive Model Building: Balancing Performance, Complexity, and Big Data," goes into that in more detail.

Table 1-1: Sketch of Problems in Machine Learning Comparison Study[3]

DATA SET NAME	NUMBER OF ATTRIBUTES	% OF EXAMPLES THAT ARE POSITIVE
Adult	14	25
Bact	11	69
Cod	15	50
Calhous	9	52
Cov_Type	54	36
HS	200	24
Letter.p1	16	3
Letter.p2	16	53
Medis	63	11
Mg	124	17
Slac	59	50

Table 1-2 shows how the algorithms covered in this book fared relative to the other algorithms used in the study. Table 1-2 shows which algorithms showed the top five performance scores for each of the problems listed in Table 1-1. Algorithms covered in this book are spelled out (boosted decision trees, Random Forests, bagged decision trees, and logistic regression). The first three of these are ensemble methods. Penalized regression was not fully developed when the study was done and wasn't evaluated. Logistic regression is a close relative and is used to gauge the success of regression methods. Each of the 9 algorithms used in the study had 3 different data reduction techniques applied, for a total of 27 combinations. The top five positions represent roughly the top 20 percent of performance scores. The row next to the heading Covt indicates that the boosted decision trees algorithm was the first and second best relative to performance, Random Forests algorithm was the fourth and fifth best, and bagged decision trees algorithm was the third best. In the cases where algorithms not covered here were in the top five, an entry appears in the Other column. The algorithms that show up there are *k nearest neighbors* (KNNs), *artificial neural nets* (ANNs), and *support vector machines* (SVMs).

Table 1-2: How the Algorithms Covered in This Book Compare on Different Problems

ALGORITHM	BOOSTED DECISION TREES	RANDOM FORESTS	BAGGED DECISION TREES	LOGISTIC REGRESSION	OTHER
Covt	1, 2	4, 5	3		
Adult	1, 4	2	3, 5		
LTR.P1	1				SVM, KNN
LTR.P2	1, 2	4, 5			SVM
MEDIS		1, 3		5	ANN
SLAC		1, 2, 3	4, 5		
HS	1, 3				ANN
MG		2, 4, 5	1, 3		
CALHOUS	1, 2	5	3, 4		
COD	1, 2		3, 4, 5		
BACT	2, 5		1, 3, 4		

Logistic regression captures top-five honors in only one case in Table 1-2. The reason for that is that these data sets have few attributes (at most 200) relative to examples (5,000 in each data set). There's plenty of data to resolve a model with so few attributes, and yet the training sets are small enough that the training time is not excessive.

NOTE **As you'll see in Chapter 3 and in the examples covered in Chapter 5, "Building Predictive Models Using Penalized Linear Methods," and Chapter 7, "Building Ensemble Models with Python," the penalized regression methods perform best relative to other algorithms when there are numerous attributes and not enough examples or time to train a more complicated ensemble model.**

Caruana et al. have run a newer study (2008) to address how these algorithms compare when the number of attributes increases. That is, how do these algorithms compare on big data? A number of fields have significantly more attributes than the data sets in the first study. For example, genomic problems have several tens of thousands of attributes (one attribute per gene), and text mining problems can have millions of attributes (one attribute per distinct word or per distinct pair of words). Table 1-3 shows how linear regression and ensemble methods fare as the number of attributes grows. The results in Table 1-3 show the ranking of the algorithms used in the second study. The table shows the performance on each of the problems individually and in the far right column shows the ranking of each algorithm's average score across all the problems. The algorithms used in the study are broken into two groups. The top group of algorithms are ones that will be covered in this book. The bottom group will not be covered.

The problems shown in Table 1-3 are arranged in order of their number of attributes, ranging from 761 to 685,569. Linear (logistic) regression is in the top three for 5 of the 11 test cases used in the study. Those superior scores were concentrated among the larger data sets. Notice that boosted decision tree (denoted by BSTDT in Table 1-3) and Random Forests (denoted by RF in Table 1-3) algorithms still perform near the top. They come in first and second for overall score on these problems.

The algorithms covered in this book have other advantages besides raw predictive performance. An important benefit of the penalized linear regression models that the book covers is the speed at which they train. On big problems, training speed can become an issue. In some problems, model training can take days or weeks. This time frame can be an intolerable delay, particularly early in development when iterations are required to home in on the best approach. Besides training very quickly, after being deployed a trained linear model can produce predictions very quickly—quickly enough for high-speed trading or Internet ad insertions. The study demonstrates that penalized linear regression can provide the best answers available in many cases and be near the top even in cases where they are not the best.

In addition, these algorithms are reasonably easy to use. They do not have very many tunable parameters. They have well-defined and well-structured input types. They solve several types of problems in regression and classification. It is not unusual to be able to arrange the input data and generate a first trained model and performance predictions within an hour or two of starting a new problem.

Table 1-3: How the Algorithms Covered in This Book Compare on Big Data Problems

	761	761	780	927	1344	3448	20958	105354	195203	405333	685569	
DIM	STURN	CALAM	DIGITS	TIS	CRYST	KDD98	R-S	CITE	DSE	SPAM	IMDB	MEAN
BSTDT	8	1	2	6	1	3	8	1	7	6	3	1
RF	9	4	3	3	2	1	6	5	3	1	3	2
BAGDT	5	2	6	4	3	1	9	1	6	7	3	4
BSTST	2	3	7	7	7	1	7	4	8	8	5	7
LR	4	8	9	1	4	1	2	2	2	4	4	6
SVM	3	5	5	2	5	2	1	1	5	5	3	3
ANN	6	7	4	5	8	1	4	2	1	3	3	5
KNN	1	6	1	9	6	2	10	1	7	9	6	8
PRC	7	9	8	8	7	1	3	3	4	2	2	9
NB	10	10	10	10	9	1	5	1	9	10	7	10

One of their most important features is that they indicate which of their input variables is most important for producing predictions. This turns out to be an invaluable feature in a machine learning algorithm. One of the most time-consuming steps in the development of a predictive model is what is sometimes called *feature selection* or *feature engineering*. This is the process whereby the data scientist chooses the variables that will be used to predict outcomes. By ranking features according to importance, the algorithms covered in this book aid in the feature-engineering process by taking some of the guesswork out of the development process and making the process more sure.

What Are Penalized Regression Methods?

Penalized linear regression is a derivative of *ordinary least squares* (OLS) regression—a method developed by Gauss and Legendre roughly 200 years ago. Penalized linear regression methods were designed to overcome some basic limitations of OLS regression. The basic problem with OLS is that sometimes it overfits the problem. Think of OLS as fitting a line through a group of points, as in Figure 1-1. This is a simple prediction problem: predicting *y*, the target value given a single attribute *x*. For example, the problem might be to predict men's salaries using only their heights. Height is slightly predictive of salaries for men (but not for women).

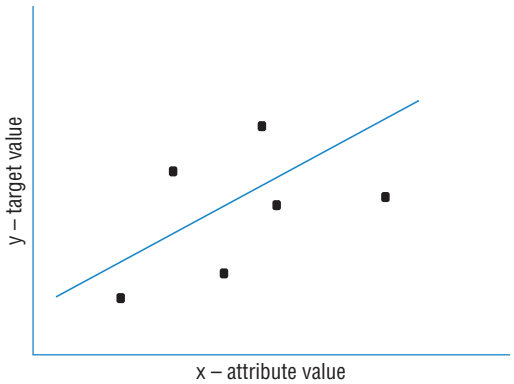


Figure 1-1: Ordinary least squares fit

The points represent men's salaries versus their heights. The line in Figure 1-1 represents the OLS solution to this prediction problem. In some sense, the line is the best predictive model for men's salaries given their heights. The data set has six points in it. Suppose that the data set had only two points in it. Imagine that there's a population of points, like the ones in Figure 1-1, but that you do not get to see all the points. Maybe they are too expensive to generate, like the

genetic data mentioned earlier. There are enough humans available to isolate the gene that is the culprit; the problem is that you do not have gene sequences for many of them because of cost.

To simulate this in the simple example, imagine that instead of six points you're given only two of the six points. How would that change the nature of the line fit to those points? It would depend on which two points you happened to get. To see how much effect that would have, pick any two points from Figure 1-1 and imagine a line through them. Figure 1-2 shows some of the possible lines through pairs of points from Figure 1-1. Notice how much the lines vary depending on the choice of points.

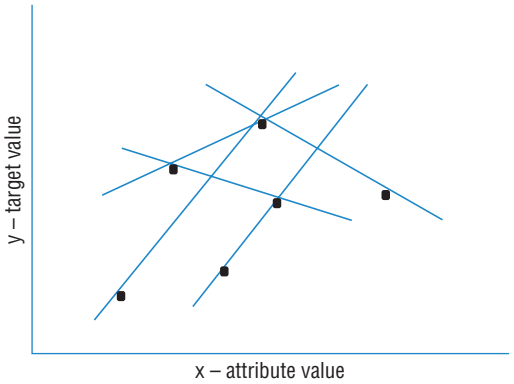


Figure 1-2: Fitting lines with only two points

The problem with having only two points to fit a line is that there is not enough data for the number of degrees of freedom. A line has two degrees of freedom. Having two degrees of freedom means that there are two independent parameters that uniquely determine a line. You can imagine grabbing hold of a line in the plane and sliding it up and down in the plane or twisting it to change its slope. So, vertical position and slope are independent. They can be changed separately, and together they completely specify a line. The degrees of freedom of a line can be expressed in several equivalent ways (where it intercepts the y-axis and its slope, two points that are on the line, and so on). All of these representations of a line require two parameters to specify.

When the number of degrees of freedom is equal to the number of points, the predictions are not very good. The lines hit the points used to draw them, but there is a lot of variation among lines drawn with different pairs of points. You cannot place much faith in a prediction that has as many degrees of freedom as the number of points in your data set. The plot in Figure 1-1 had six points and fit a line (two degrees of freedom) through them. That is six points and two degrees of freedom. The thought problem of determining the genes causing a heritable condition illustrated that having more genes to choose from makes it

necessary to have more data in order to isolate a cause from among the 20,000 or so possible human genes. The 20,000 different genes represent 20,000 degrees of freedom. Data from even 20,000 different persons will not suffice to get a reliable answer, and in many cases, all that can be afforded within the scope of a reasonable study is a sample from 500 or so persons. That is where penalized linear regression may be the best algorithm choice.

Penalized linear regression provides a way to systematically reduce degrees of freedom to match the amount of data available and the complexity of the underlying phenomena. These methods have become very popular for problems with very many degrees of freedom. They are a favorite for genetic problems where the number of degrees of freedom (that is, the number of genes) can be several tens of thousands and for problems like text classification where the number of degrees of freedom can be more than a million. Chapter 4, "Penalized Linear Regression," gives more detail on how these methods work, sample code that illustrates the mechanics of these algorithms, and examples of the process for implementing machine learning systems using available Python packages.

What Are Ensemble Methods?

The other family of algorithms covered in this book is ensemble methods. The basic idea with ensemble methods is to build a horde of different predictive models and then combine their outputs—by averaging the outputs or taking the majority answer (voting). The individual models are called *base learners*. Some results from computational learning theory show that if the base learners are just slightly better than random guessing, the performance of the ensemble can be very good if there is a sufficient number of independent models.

One of the problems spurring the development of ensemble methods has been the observation that some particular machine learning algorithms exhibit instability. For example, the addition of fresh data to the data set might result in a radical change in the resulting model or its performance. Binary decision trees and traditional neural nets exhibit this sort of instability. This instability causes high variance in the performance of models, and averaging many models can be viewed as a way to reduce the variance. The trick is how to generate large numbers of independent models, particularly if they are all using the same base learner. Chapter 6, "Ensemble Methods," will get into the details of how this is done. The techniques are ingenious, and it is relatively easy to understand their basic principles of operation. Here is a preview of what's in store.

The ensemble methods that enjoy the widest availability and usage incorporate binary decision trees as their base learners. Binary decision trees are often portrayed as shown in Figure 1-3. The tree in Figure 1-3 takes a real number, called x, as input at the top, and then uses a series of binary (two-valued) decisions to decide what value should be output in response to x. The first decision

is whether x is less than 5. If the answer to that question is "no," the binary decision tree outputs the value 4 indicated in the circle below the No leg of the upper decision box. Every possible value for x leads to some output y from the tree. Figure 1-4 plots the output (y) as a function of the input to the tree (x).

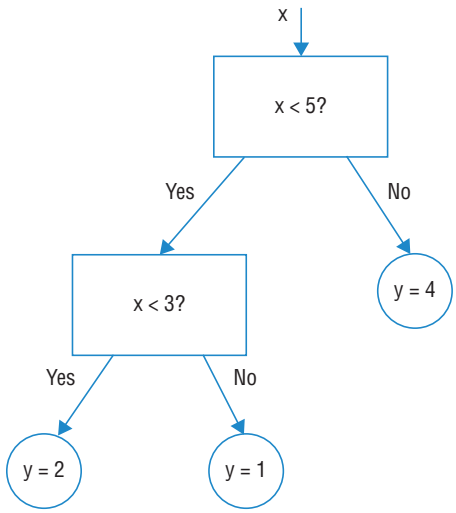


Figure 1-3: Binary decision tree example

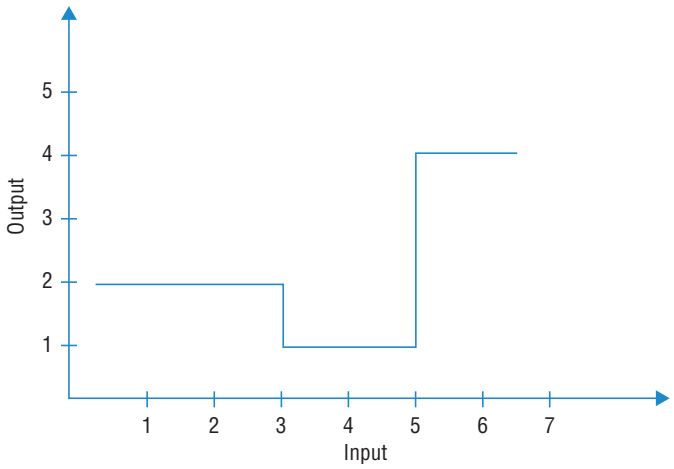


Figure 1-4: Input-output graph for the binary decision tree example

This description raises the question of where the comparisons (for example, $x < 5$?) come from and where the output values (in the circles at the bottom of the tree) come from. These values come from training the binary tree on the input data. The algorithm for doing that training is not difficult to understand and is

covered in Chapter 6. The important thing to note at this point is that the values in the trained binary decision tree are fixed, given the data. The process for generating the tree is deterministic. One way to get differing models is to take random samples of the training data and train on these random subsets. That technique is called *Bagging* (short for *bootstrap aggregating*). It gives a way to generate a large number of slightly different binary decision trees. Those are then averaged (or voted for a classifier) to yield a final result. Chapter 6 describes in more detail this technique and other more powerful ones.

How to Decide Which Algorithm to Use

Table 1-4 gives a sketch comparison of these two families of algorithms. Penalized linear regression methods have the advantage that they train very quickly. Training times on large data sets can extend to hours, days, or even weeks. Training usually needs to be done several times before a deployable solution is arrived at. Long training times can stall development and deployment on large problems. The rapid training time for penalized linear methods makes them useful for the obvious reason that shorter is better. Depending on the problem, these methods may suffer some performance disadvantages relative to ensemble methods. Chapter 3 gives more insight into the types of problems where penalized regression might be a better choice and those where ensemble methods might be a better choice. Penalized linear methods can sometimes be a useful first step in your development process even in the circumstance where they yield inferior performance to ensemble methods.

Early in development, a number of training iterations will be necessary for purposes of feature selection and feature engineering and for solidifying the mathematical problem statement. Deciding what you are going to use as input to your predictive model can take some time and thought. Sometimes that is obvious, but usually it requires some iteration. Throwing in everything you can find is not usually a good solution.

Trial and error is typically required to determine the best inputs for a model. For example, if you're trying to predict whether a visitor to your website will click a link for an ad, you might try using demographic data for the visitor. Maybe that does not give you the accuracy that you need, so you try incorporating data regarding the visitor's past behavior on the site—what ad the visitor clicked during past site visits or what products the visitor has bought. Maybe adding data about the site the visitor was on before coming to your site would help. These questions lead to a series of experiments where you incorporate the new data and see whether it hurts or helps. This iteration is generally time-consuming both for the data manipulations and for training your predictive model. Penalized linear regression will generally be faster than an ensemble method, and the time difference can be a material factor in the development process.

For example, if the training set is on the order of a gigabyte, training times may be on the order of 30 minutes for penalized linear regression and 5 or 6 hours for an ensemble method. If the feature engineering process requires 10 iterations to select the best feature set, the computation time alone comes to the difference between taking a day or taking a week to accomplish feature engineering. A useful process, therefore, is to train a penalized linear model in the early stages of development, feature engineering, and so on. That gives the data scientist a feel for which variables are going to be useful and important as well as a baseline performance for comparison with other algorithms later in development.

Besides enjoying a training time advantage, penalized linear methods generate predictions much faster than ensemble methods. Generating a prediction involves using the trained model. The trained model for penalized linear regression is simply a list of real numbers—one for each feature being used to make the predictions. The number of floating-point operations involved is the number of variables being used to make predictions. For highly time-sensitive predictions such as high-speed trading or Internet ad insertions, computation time makes the difference between making money and losing money.

Table 1-4: High-Level Tradeoff between Penalized Linear Regression and Ensemble Algorithms

	TRAINING SPEED	PREDICTION SPEED	PROBLEM COMPLEXITY	DEALS WITH WIDE ATTRIBUTE
Penalized Linear Regression	+	+	–	+
Ensemble Methods	–	–	+	–

For some problems, linear methods may give equivalent or even better performance than ensemble methods. Some problems do not require complicated models. Chapter 3 goes into some detail about the nature of problem complexity and how the data scientist's task is to balance problem complexity, predictive model complexity, and data set size to achieve the best deployable model. The basic idea is that on problems that are not complex and problems for which sufficient data are not available, linear methods may achieve better overall performance than more complicated ensemble methods. Genetic data provide a good illustration of this type of problem.

The general perception is that there's an enormous amount of genetic data around. Genetic data sets are indeed large when measured in bytes, but in terms of generating accurate predictions, they aren't very large. To understand this distinction, consider the following thought experiment. Suppose that you have two people, one with a heritable condition and the other without. If you

had genetic sequences for the two people, could you determine which gene was responsible for the condition? Obviously, that's not possible because many genes will differ between the two persons. So how many people would it take? At a minimum, it would take gene sequences for as many people as there are genes, and given any noise in the measurements, it would take even more. Humans have roughly 20,000 genes, depending on your count. And each datum costs roughly $1,000. So having just enough data to resolve the disease with perfect measurements would cost $20 million.

This situation is very similar to fitting a line to two points, as discussed earlier in this chapter. Models need to have fewer degrees of freedom than the number of data points. The data set typically needs to be a multiple of the degrees of freedom in the model. Because the data set size is fixed, the degrees of freedom in the model need to be adjustable. The chapters dealing with penalized linear regression will show you how the adjustability is built into penalized linear regression and how to use it to achieve optimum performance.

NOTE **The two broad categories of algorithms addressed in this book match those that Jeremy Howard and I presented at Strata Conference in 2012. Jeremy took ensemble methods, and I took penalized linear regression. We had fun arguing about the relative merits of the two groups. In reality, however, those two cover something like 80 percent of the model building that I do, and there are good reasons for that.**

Chapter 3 goes into more detail about why one algorithm or another is a better choice for a given problem. It has to do with the complexity of the problem and the number of degrees of freedom inherent in the algorithms. The linear models tend to train rapidly and often give equivalent performance to nonlinear ensemble methods, especially if the data available are somewhat constrained. Because they're so rapid to train, it is often convenient to train linear models for early feature selection and to ballpark achievable performance for a specific problem. The linear models considered in this book can give information about variable importance to aid in the feature selection process. The ensemble methods often give better performance if there are adequate data and also give somewhat indirect measures of relative variable importance.

The Process Steps for Building a Predictive Model

Using machine learning requires several different skills. One is the required programming skill, which this book does not address. The other skills have to do with getting an appropriate model trained and deployed. These other skills are what the book does address. What do these other skills include?

Initially, problems are stated in somewhat vague language-based terms like "Show site visitors links that they're likely to click on." To turn this into a working system requires restating the problem in concrete mathematical terms, finding data to base the prediction on, and then training a predictive model that will predict the likelihood of site visitors clicking the links that are available for presentation. Stating the problem in mathematical terms makes assumptions about what features will be extracted from the available data sources and how they will be structured.

How do you get started with a new problem? First, you look through the available data to determine which of the data might be of use in prediction. "Looking through the data" means running various statistical tests on the data to get a feel for what they reveal and how they relate to what you're trying to predict. Intuition can guide you to some extent. You can also quantify the outcomes and test the degree to which potential prediction features correlate with these outcomes. Chapter 2, "Understand the Problem by Understanding the Data," goes through this process for the data sets that are used to characterize and compare the algorithms outlined in the rest of the book.

By some means, you develop a set of features and start training the machine learning algorithm that you have selected. That produces a trained model and estimates its performance. Next, you want to consider making changes to the features set, including adding new ones or removing some that proved unhelpful, or perhaps changing to a different type of training objective (also called a *target*) to see whether it improves performance. You'll iterate various design decisions to determine whether there's a possibility of improving performance. You may pull out the examples that show the worst performance and then attempt to determine if there's something that unites these examples. That may lead to another feature to add to the prediction process, or it might cause you to bifurcate the data and train different models on different populations.

The goal of this book is to make these processes familiar enough to you that you can march through these development steps confidently. That requires your familiarity with the input data structures required by different algorithms as you frame the problem and begin extracting the data to be used in training and testing algorithms. The process usually includes several of the following steps:

1. Extract and assemble features to be used for prediction.
2. Develop targets for the training.
3. Train a model.
4. Assess performance on test data.

NOTE **The first pass can usually be improved on by trying different sets of features, different types of targets, and so on.**

Machine learning requires more than familiarization with a few packages. It requires understanding and having practiced the process involved in developing a deployable model. This book aims to give you that understanding. It assumes basic undergraduate math and some basic ideas from probability and statistics, but the book doesn't presuppose a background in machine learning. At the same time, it intends to arm readers with the very best algorithms for a wide class of problems, not necessarily to survey all machine learning algorithms or approaches. There are a number of algorithms that are interesting but that don't get used often, for a variety of reasons. For example, perhaps they don't scale well, maybe they don't give insight about what is going on inside, maybe they're difficult to use, and so on. It is well known, for example, that Random Forests (one of the algorithms covered here) is the leading winner of online machine competitions by a wide margin. There are good reasons why some algorithms are more often used by practitioners, and this book will succeed to the extent that you understand these when you've finished reading.

Framing a Machine Learning Problem

Beginning work on a machine learning competition presents a simulation of a real machine learning problem. The competition presents a brief description (for example, announcing that an insurance company would like to better predict loss rates on their automobile policies). As a competitor, your first step is to open the data set, take a look at the data available, and identify what form a prediction needs to take to be useful. The inspection of the data will give an intuitive feel for what the data represent and how they relate to the prediction job at hand. The data can give insight regarding approaches. Figure 1-5 depicts the process of starting from a general language statement of objective and moving toward an arrangement of data that will serve as input for a machine learning algorithm.

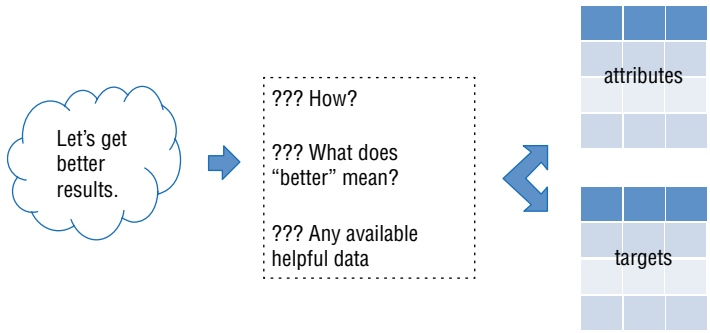


Figure 1-5: Framing a machine learning problem

The generalized statement caricatured as "Let's get better results" has first to be converted into specific goals that can be measured and optimized. For a website owner, specific performance might be improved click-through rates or more sales (or more contribution margin). The next step is to assemble data that might make it possible to predict how likely a given customer is to click various links or to purchase various products offered online. Figure 1-5 depicts these data as a matrix of attributes. For the website example, they might include other pages the visitor has viewed or items the visitor has purchased in the past. In addition to attributes that will be used to make predictions, the machine learning algorithms for this type of problem need to have correct answers to use for training. These are denoted as targets in Figure 1-5. The algorithms covered in this book learn by detecting patterns in past behaviors, but it is important that they not merely memorize past behavior; after all, a customer might not repeat a purchase of something he bought yesterday. Chapter 3 discusses in detail how this process of training without memorizing works.

Usually, several aspects of the problem formulation can be done in more than one way. This leads to some iteration between framing the problem, selecting and training a model, and producing performance estimates. Figure 1-6 depicts this process.

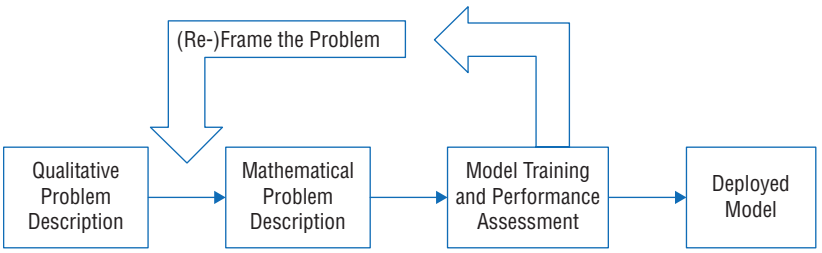


Figure 1-6: Iteration from formulation to performance

The problem may come with specific quantitative training objectives, or part of the job might be extracting these data (called *targets* or *labels*). Consider, for instance, the problem of building a system to automatically trade securities. To trade automatically, a first step might be to predict changes in the price of a security. The prices are easily available, so it is conceptually simple to use historical data to build training examples for which the future price changes are known. But even that involves choices and experimentation. Future price change could be computed in several different ways. The change could be the difference between the current price and the price 10 minutes in the future. It could also be the change between the current price and the price 10 days in the future. It could also be the difference between the current price and the maximum/minimum price over the next 10 minutes. The change in price could be characterized by a two-state variable taking values "higher" of "lower"

depending on whether the price is higher or lower 10 minutes in the future. Each of these choices will lead to a predictive model, and the predictions will be used for deciding whether to buy or sell the security. Some experimentation will be required to determine the best choice.

Feature Extraction and Feature Engineering

Deciding which variables to use for making predictions can also involve experimentation. This process is known as *feature extraction* and *feature engineering*. Feature extraction is the process of taking data from a free-form arrangement, such as words in a document or on a web page, and arranging them into rows and columns of numbers. For example, a spam-filtering problem begins with text from emails and might extract things such as the number of capital letters in the document and the number of words in all caps, the number of times the word "buy" appears in the document and other numeric features selected to highlight the differences between spam and non-spam emails.

Feature engineering is the process of manipulating and combining features to arrive at more informative ones. Building a system for trading securities involves feature extraction and feature engineering. Feature extraction would be deciding what things will be used to predict prices. Past prices, prices of related securities, interest rates, and features extracted from news releases have all been incorporated into various trading systems that have been discussed publicly. In addition, securities prices have a number of engineered features with names like stochastic, MACD (*moving average convergence divergence*), and RSI (*relative strength index*) that are basically functions of past prices that their inventors believed to be useful in securities trading.

After a reasonable set of features is developed, you can train a predictive model like the ones described in this book, assess its performance, and make a decision about deploying the model. Generally, you'll want to make changes to the features used, if for no other reason than to confirm that your model's performance is adequate. One way to determine which features to use is to try all combinations, but that can take a lot of time. Inevitably, you'll face competing pressures to improve performance but also to get a trained model into use quickly. The algorithms discussed in this book have the beneficial property of providing metrics on the utility of each attribute in producing predictions. One training pass will generate rankings on the features to indicate their relative importance. This information helps speed the feature engineering process.

NOTE **Data preparation and feature engineering is estimated to take 80 to 90 percent of the time required to develop a machine learning model.**

The model training process, which begins each time a baseline set of features is attempted, also involves a process. A modern machine learning algorithm,

such as the ones described in this book, trains something like 100 to 5,000 different models that have to be winnowed down to a single model for deployment. The reason for generating so many models is to provide models of all different shades of complexity. This makes it possible to choose the model that is best suited to the problem and data set. You don't want a model that's too simple or you give up performance, but you don't want a model that's too complicated or you'll overfit the problem. Having models in all shades of complexity lets you pick one that is just right.

Determining Performance of a Trained Model

The fit of a model is determined by how well it performs on data that were not used to train the model. This is an important step and conceptually simple. Just set aside some data. Don't use it in training. After the training is finished, use the data you set aside to determine the performance of your algorithm. This book discusses several systematic ways to hold out data. Different methods have different advantages, depending mostly on the size of the training data. As easy as it sounds, people continually figure out complicated ways to let the test data "leak" into the training process. At the end of the process, you'll have an algorithm that will sift through incoming data and make accurate predictions for you. It might need monitoring as changing conditions alter the underlying statistics.

Chapter Contents and Dependencies

Different readers may want to take different paths through this book, depending on their backgrounds and whether they have time to understand the basic principles. Figure 1-7 shows how chapters in the book depend on one another.

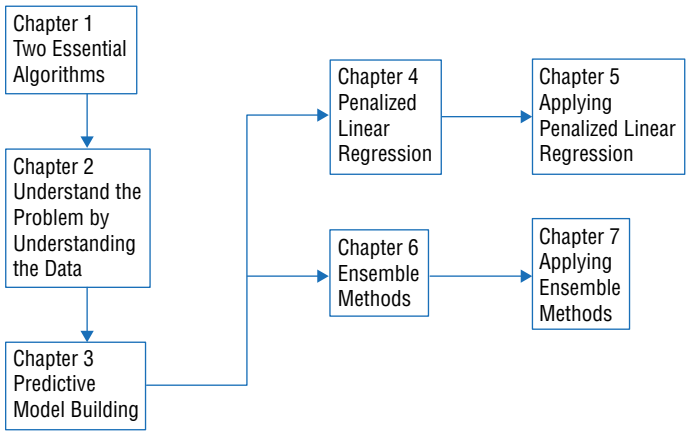


Figure 1-7: Dependence of chapters on one another

Chapter 2 goes through the various data sets that will be used for problem examples to illustrate the use of the algorithms that will be developed and to compare algorithms to each other based on performance and other features. The starting point with a new machine learning problem is digging into the data set to understand it better and to learn its problems and idiosyncrasies. Part of the point of Chapter 2 is to demonstrate some of the tools available in Python for data exploration. You might want to go through some but not all of the examples shown in Chapter 2 to become familiar with the process and then come back to Chapter 2 when diving into the solution examples later.

Chapter 3 explains the basic tradeoffs in a machine learning problem and introduces several key concepts that are used throughout the book. One key concept is the mathematical description of predictive problems. The basic distinctions between classification and regression problems are shown. Chapter 3 also introduces the concept of using out-of-sample data for determining the performance of a predictive model. Out-of-sample data are data that have not been included in the training of the model. Good machine learning practice demands that a developer produce solid estimates of how a predictive model will perform when it is deployed. This requires excluding some data from the training set and using it to simulate fresh data. The reasons for this requirement, the methods for accomplishing it, and the tradeoffs between different methods are described. Another key concept is that there are numerous measures of system performance. Chapter 3 outlines these methods and discusses tradeoffs between them. Readers who are already familiar with machine learning can browse this chapter and scan the code examples instead of reading it carefully and running the code.

Chapter 4 shows the core ideas of the algorithms for training penalized regression models. The chapter introduces the basic concepts and shows how the algorithms are derived. Some of the examples introduced in Chapter 3 are used to motivate the penalized linear regression methods and algorithms for their solution. The chapter runs through code for the core algorithms for solving penalized linear regression training. Chapter 4 also explains several extensions to linear regression methods. One of these extensions shows how to code factor variables as real numbers so that linear regression methods can be applied. Linear regression can be used only on problems where the predictors are real numbers; that is, the quantities being used to make predictions have to be numeric. Many practical and important problems have variables like "single, married, or divorced" that can be helpful in making predictions. To incorporate variables of this type (called *categorical variables*) in a linear regression model, means have been devised to convert categorical variables to real number variables. Chapter 4 covers those methods. In addition, Chapter 4 also shows methods (called *basis expansion*) for getting nonlinear functions out of nonlinear regression. Sometimes basis expansion can be used to squeeze a little more performance out of linear regression.

Chapter 5 applies the penalized regression algorithms developed in Chapter 4 to a number of the problems outlined in Chapter 2. The chapter outlines the Python packages that implement penalized regression methods and uses them to solve problems. The objective is to cover a wide enough variety of problems that practitioners can find a problem close to the one that they have in front of them to solve. Besides quantifying and comparing predictive performance, Chapter 5 looks at other properties of the trained algorithms. Variable selection and variable ranking are important to understand. This understanding will help speed development on new problems.

Chapter 6 develops ensemble methods. Because ensemble methods are most frequently based on binary decision trees, the first step is to understand the principles of training and using binary decision trees. Many of the properties of ensemble methods are ones that they inherit directly from binary decision trees. With that understanding in place, the chapter explains the three principal ensemble methods covered in the book. The common names for these are Bagging, boosting, and Random Forest. For each of these, the principles of operation are outlined and the code for the core algorithm is developed so that you can understand the principles of operation.

Chapter 7 uses ensemble methods to solve problems from Chapter 2 and then compares the various algorithms that have been developed. The comparison involves a number of elements. Predictive performance is one element of comparison. The time required for training and performance is another element. All the algorithms covered give variable importance ranking, and this information is compared on a given problem across several different algorithms.

In my experience, teaching machine learning to programmers and computer scientists, I've learned that code examples work better than mathematics for some people. The approach taken here is to provide some mathematics, algorithm sketches, and code examples to illustrate the important points. Nearly all the methods that are discussed will be found in the code included in the book and on the website. The intent is to provide hackable code to help you get up and running on your own problems as quickly as possible.

Summary

This chapter has given a specification for the kinds of problems that you'll be able to solve and a description of the process steps for building predictive models. The book concentrates on two algorithm families. Limiting the number of algorithms covered allows for a more thorough explanation of the background for these algorithms and of the mechanics of using them. This chapter showed some comparative performance results to motivate the choice of these two particular families. The chapter discussed the different strengths and characteristics of

these two families and gave some description of the types of problems that would favor one or the other of the two.

The chapter also laid out the steps in the process of developing a predictive model and elaborated on the tradeoffs and outcomes for each step. The use of data not included in model training was suggested for generating performance estimates for predictive models.

This book's goal is to bring programmers with little or no machine learning experience to the point where they feel competent and comfortable incorporating machine learning into projects. The book does not survey a wide number of algorithms. Instead, it covers several best-in-class algorithms that can offer you performance, flexibility, and clarity. Once you understand a little about how these work and have some experience using them, you'll find them easy and quick to use. They will enable you to solve a wide variety of problems without having to do a lot of fussing to get them trained, and they'll give you insight into the sources of their performance.

References

1. Caruana, Rich, and Alexandru Niculescu-Mizil. "An Empirical Comparison of Supervised Learning Algorithms." *Proceedings of the 23rd International Conference on Machine Learning.* ACM, 2006.
2. Caruana, Rich, Nikos Karampatziakis, and Ainur Yessenalina. "An Empirical Evaluation of Supervised Learning in High Dimensions." *Proceedings of the 25th International Conference on Machine Learning.* ACM, 2008.

CHAPTER

2

Understand the Problem by Understanding the Data

A new data set (problem) is a wrapped gift. It's full of promise and anticipation at the miracles you can wreak once you've solved it. But it remains a mystery until you've opened it. This chapter is about opening up your new data set so you can see what's inside, get an appreciation for what you'll be able to do with the data, and start thinking about how you'll approach model building with it.

This chapter has two purposes. One is to familiarize you with data sets that will be used later as examples of different types of problems to be solved using the algorithms you'll learn in Chapter 4, "Penalized Linear Regression," and Chapter 6, "Ensemble Methods." The other purpose is to demonstrate some of the tools available in Python for data exploration.

The chapter uses a simple example to review some basic problem structure, nomenclature, and characteristics of a machine learning data set. The language introduced in this section will be used throughout the rest of the book. After establishing some common language, the chapter goes one by one through several different types of function approximation problems. These problems illustrate common variations of machine learning problems so that you'll know how to recognize the variants when you see them and will know how to handle them (and will have code examples for them).

The Anatomy of a New Problem

The algorithms covered in this book start with a matrix (or table) full of numbers and perhaps some character variables. The example in Table 2-1 establishes some nomenclature and represents a small machine learning data set in a two-dimensional table. The table will give you a mental image of a data set so that references to "columns corresponding to attributes" or rows corresponding to individual examples will be familiar. In this example, the predictive analytics problem is to predict how much money individuals will spend buying books online over the next year.

Table 2-1: Data for a Machine Learning Problem

USERID	ATTRIBUTE 1	ATTRIBUTE 2	ATTRIBUTE 3	LABELS
001	6.5	Male	12	$120
004	4.2	Female	17	$270
007	5.7	Male	3	$75
008	5.8	Female	8	$600

The data are arranged into rows and columns. Each row represents an individual case (also called an *instance*, *example*, or *observation*). The columns in Table 2-1 are given designations that indicate the roles they will play in the machine learning problem. The columns designated as attributes will be used to make predictions of the dollars spent on books. In the column designated as labels, you'll see how much each customer spent last year on books.

NOTE **Machine learning data sets are most commonly arranged with columns corresponding to a single attribute and rows corresponding to a single observation, but not always. For example, some text mining literature arranges the matrix the other way around—with columns corresponding to an observation and rows corresponding to an attribute.**

In Table 2-1, a row represents an individual customer, and the data in the row all pertain to that individual. The first column is called UserID and contains an identifier that is unique for each row (case). A unique identifier may or may not be present in your problem. For instance, websites typically tag site visitors with a user ID that is associated with them for the duration of their visit. If a user does not register with the site, the same user gets a different ID with each visit. The ID is usually assigned to each observation, which will be the subject of the prediction you're going to build. Columns 2, 3, and 4 are called Attributes instead of being given more specific names like Height or Gender. The point is

to highlight their role in the prediction process. Attributes are data available about the case that will be used to make predictions.

Labels are the things you want to predict. In this example, UserID is a simple number, Attribute 1 is height, Attribute 2 is gender, and Attribute 3 is how many books the person read last year. The column under Labels contains how much money the individual spent on books online last year. What are the roles that these different categories of data will play? What use does a machine learning algorithm make of user ID, attributes, and labels? The short answer is this: You ignore the user ID. You use the attributes to predict the labels.

The unique ID is for bookkeeping purposes and allows you to refer back to the other data available for the specific case. Generally, the unique ID does not get used directly in a machine learning algorithm. Attributes are the things that you've chosen to use for making predictions. Labels are observed outcomes that the machine learning algorithm will use to build a predictive model.

User ID doesn't usually get used for making predictions because it is too specific. It pertains to only a single example. The trick with machine learning is to build a model that generalizes to new cases (not merely memorizing past cases). To achieve that, the algorithm must be derived so that it is forced to pay attention to more than one row of data. One possible exception to excluding user ID is when the user ID is numeric and assigned in the order that users are signed up. Basically, it's indicating signup date in that case and can be useful because users with close IDs signed up at similar times and can be considered as a group on that basis.

The process of building a predictive model is called *training*. The way the process proceeds depends on the algorithm, and later chapters cover the details, but it is often iterative. The algorithm postulates a predictive relationship between the attributes and the labels, observes the mistakes that it makes, and makes some correction, and then iterates on that process until a sound model is achieved. A number of technicalities are addressed later, but that's the basic idea.

WHAT'S IN A NAME?

Attributes and labels go by a variety of names, and new machine learners can get tripped up by the name switching from one author to another or even one paragraph to another from a single author.

Attributes (the variables being used to make predictions) are also known as the following:

- **Predictors**
- **Features**

- Independent variables
- Inputs

Labels are also known as the following:

- Outcomes
- Targets
- Dependent variables
- Responses

Different Types of Attributes and Labels Drive Modeling Choices

The attributes shown in Table 2-1 come in two different types: numeric variables and categorical (or factor) variables. Attribute 1 (height) is a numeric variable and is the most usual type of attribute. Attribute 2 is gender and is indicated by the entry Male or Female. This type of attribute is called a *categorical* or *factor* variable. Categorical variables have the property that there's no order relation between the various values. There's no sense to Male < Female (despite centuries of squabbling). Categorical variables can be two-valued, like Male Female, or multivalued, like states (AL, AK, AR . . . WY). Other distinctions can be drawn regarding attributes (integer versus float, for example), but they do not have the same impact on machine learning algorithms. The reason for this is that many machine learning algorithms take numeric attributes only; they cannot handle categorical or factor variables. Penalized regression algorithms deal only with numeric attributes. The same is true for support vector machines, kernel methods, and K-nearest neighbors. Chapter 4 will cover methods for converting categorical variables to numeric variables. The nature of the variables will shape your algorithm choices and the direction you take in developing a predictive model, so it's one of the things you need to pay attention to when you face a new problem.

A similar dichotomy arises for the labels. The labels shown in Table 2-1 are numeric: the amount of money that the individual spent on books online last year. In other problems, though, the labels may also be categorical. For example, if the job with Table 2-1 were to predict which individuals would spend more than $200 next year the problem would change, and the problem approach would change. The new problem of predicting which customers would spend more than $200 would have new labels. The new labels would take one of two values. Table 2-2 shows the relationship between the labels given in Table 2-1 and new labels based on the logical proposition Spending > $200. The new labels shown in Table 2-2 take one of two values—True or False.

Table 2-2: Numeric Targets versus Categorical Targets

TABLE 1 LABELS	>$200 ?
$120	False
$270	True
$75	False
$600	True

When the labels are numeric, the problem is called a *regression problem*. When the labels are categorical, the problem is called a *classification problem*. If the categorical target takes only two values, the problem is called a *binary classification problem*. If it takes more than two values, the problem is called a *multiclass classification problem*.

In many cases, the choice of problem type is up to the designer. You've just seen that this example problem can be converted from a regression problem to a binary classification problem by the simple transformation of the labels. These are tradeoffs that you may might to make as part of your attack on a problem. For example, classification targets might better support a decision between two courses of action.

The classification problem might also be simpler than the regression problem. Consider, for instance, the difference in complexity between a topographic map with a single contour line (say the 100-foot contour line) and a topographic map with contour lines every 10 feet. The single contour divides the map into the areas that are higher than 100 feet and those that are lower and contains considerably less information than the more detailed contour map. A classifier is trying to compute a single dividing contour without regard for behavior distant from the decision boundary, whereas regression is trying to draw the whole map.

Things to Notice about Your New Data Set

You'll want to ascertain a number of other features of the data set as part of your initial inspection of the data. The following is a checklist and a sequence of things to learn about your data set to familiarize yourself with the data and to formulate the predictive model development steps that you want to follow. These are simple things to check and directly impact your next steps. In addition, the process gets you moving around the data and learning its properties.

Items to Check

Number of rows and columns

Number of categorical variables and number of unique values for each

Missing values

Summary statistics for attributes and labels

One of the first things to check is the size and shape of the data. Read the data into a list of lists; then the dimension of the outer list is the number of rows, and the dimension of one of the inner lists is the number of columns. The next section shows the concrete application of this to one of the data sets that you'll see used later to illustrate the properties of an algorithm that will be developed.

The next step in the process is to determine how many missing values there are in each row. The reason for doing it on a row-by-row basis is that the simplest way to deal with missing values is to throw away instances that aren't complete (examples with at least one missing value). In many situations, this can bias the results, but just a few incomplete examples will not make a material difference. By counting the rows with missing data (in addition to the total number of missing entries), you'll know how much of the data set you have to discard if you use the easy method.

If you have a large number of rows, as you might if you're collecting web data, the number you'll lose may be small compared to the number of rows of data you have available. If you're working on biological problems where the data are expensive and you have many attributes, you might not be able to afford to throw data out. In that case, you'll have to figure out some ways to fill in the missing values or use an algorithm that can deal with them. Filling them in is called *imputation*. The easiest way to impute the missing data is to fill in the missing entries using average values of the entries in each row. A more sophisticated method is to use one of the predictive methods covered in Chapters 4 and 6. To use a predictive method, you treat a column of attributes with missing values as though it were labels. Be sure to remove the original problem labels before undertaking this process.

The next several sections are going to go through the process outlined here and will introduce some methods for characterizing your data set to help you decide how to attack the modeling process.

Classification Problems: Detecting Unexploded Mines Using Sonar

This section steps through several checks that you might make on a classification problem as you begin digging into it. It starts with simple measurements of size and shape, reporting data types, counting missing values, and so forth. Then it moves on to statistical properties of the data and interrelationships between attributes and between attributes and the labels. The data set comes from the UC Irvine Data Repository [Ref 1.]. The data result from some experiments to determine if sonar can be used to detect unexploded mines left in harbors subsequent to military actions. The sonar signal is what's called a *chirped signal*. That means that the signal rises (or falls) in frequency over the duration of the

sound pulse. The measurements in the data set represent the power measurements collected in the sonar receiver at different points in the returned signal. For roughly half of the examples, the sonar is illuminating a rock, and for the other half a metal cylinder having the shape of a mine. The data set goes by the name of "Rocks versus Mines."

Physical Characteristics of the Rocks Versus Mines Data Set

The first thing to do with a new data set is to determine its size and shape. Listing 2-1 shows code for determining the size and shape of the "Rocks versus Mines" data set from the UC Irvine Data Repository: the rocks versus mines data. Later in this chapter, you'll learn more about this data set, and the book will use it for example purposes as the algorithms are introduced. The process for determining the number of rows and columns is pretty simple in this case. The file is comma delimited, with the data for one experiment occupying one line of text. This makes it a simple matter to read a line, split it on the comma delimiters, and stack the resulting lists into an outer list containing the whole data set.

Listing 2-1: Sizing Up a New Data Set—rockVmineSummaries.py (Output: outputRocksVMinesSummaries.txt)

```
__author__ = 'mike_bowles'
import urllib2
import sys

#read data from uci data repository
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")

data = urllib2.urlopen(target_url)

#arrange data into list for labels and list of lists for attributes
xList = []
labels = []
for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)

sys.stdout.write("Number of Rows of Data = " + str(len(xList)) + '\n')
sys.stdout.write("Number of Columns of Data = " + str(len(xList[1])))


Output:
Number of Rows of Data = 208
Number of Columns of Data = 61
```

As you can see in the sample output, this data set has 208 rows (lines) and 61 columns (fields per line). What difference does this make? The number of rows and columns has several impacts on how you proceed. First, the overall size gives you a rough idea of how long your training times are going to be. For a small data set like the rocks versus mines data, training time will be less than a minute, which will facilitate iterating through the process of training and tweaking. If the data set grows to 1,000 x 1,000, the training times will grow to a fraction of a minute for penalized linear regression and a few minutes for an ensemble method. As the data set gets to several tens of thousands of rows and columns, the training times will expand to 3 or 4 hours for penalized linear regression and 12 to 24 hours for an ensemble method. The larger training times will have an impact on your development time because you'll iterate a number of times.

The second important observation regarding row and column counts is that if the data set has many more columns than rows, you may be more likely to get the best prediction with penalized linear regression and vice versa. Chapter 3, "Predictive Model Building: Balancing Performance, Complexity, and Big Data," and the examples you'll run later will give you a better understanding of why that's true.

The next step in the checklist is to determine how many of the columns of data are numeric versus categorical. Listing 2-2 shows code to accomplish this for the rocks versus mine data set. The code runs down each column and adds up the number of entries that are numeric (int or float), the number of entries that are nonempty strings, and the number that are empty. The result is that the first 60 columns contain all numeric values and the last column contains all strings. The string values are the labels. Generally, categorical variables are presented as strings, as in this example. In some cases, binary-valued categorical variables are presented as a 0,1 numeric variable.

Listing 2-2: Determining the Nature of Attributes—rockVmineContents.py (Output: outputRocksVMinesContents.txt)

```
__author__ = 'mike_bowles'
import urllib2
import sys

#read data from uci data repository
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")

data = urllib2.urlopen(target_url)


#arrange data into list for labels and list of lists for attributes
xList = []
labels = []
```

```
for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)
nrow = len(xList)
ncol = len(xList[1])

type = [0]*3
colCounts = []

for col in range(ncol):
    for row in xList:
        try:
            a = float(row[col])
            if isinstance(a, float):
                type[0] += 1
        except ValueError:
            if len(row[col]) > 0:
                type[1] += 1
            else:
                type[2] += 1

    colCounts.append(type)
    type = [0]*3

sys.stdout.write("Col#" + '\t' + "Number" + '\t' +
                 "Strings" + '\t ' + "Other\n")
iCol = 0
for types in colCounts:
    sys.stdout.write(str(iCol) + '\t\t' + str(types[0]) + '\t\t' +
                     str(types[1]) + '\t\t' + str(types[2]) + "\n")
    iCol += 1


Output:
Col#   Number  Strings Other
 0      208      0      0
 1      208      0      0
 2      208      0      0
 3      208      0      0
 4      208      0      0
 5      208      0      0
 6      208      0      0
 7      208      0      0
 8      208      0      0
 9      208      0      0
10      208      0      0
11      208      0      0
 .        .      .      .
 .        .      .      .
 .        .      .      .
```

continues

continued

```
54      208      0      0
55      208      0      0
56      208      0      0
57      208      0      0
58      208      0      0
59      208      0      0
60      0      208      0
```

Statistical Summaries of the Rocks versus Mines Data Set

After determining which attributes are categorical and which are numeric, you'll want some descriptive statistics for the numeric variables and a count of the unique categories in each categorical attribute. Listing 2-3 gives some examples of these two procedures.

Listing 2-3: Summary Statistics for Numeric and Categorical Attributes—rVMSummaryStats.py (Output: outputSummaryStats.txt)

```
__author__ = 'mike_bowles'
import urllib2
import sys
import numpy as np

#read data from uci data repository
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")
data = urllib2.urlopen(target_url)


#arrange data into list for labels and list of lists for attributes
xList = []
labels = []

for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)
nrow = len(xList)
ncol = len(xList[1])

type = [0]*3
colCounts = []

#generate summary statistics for column 3 (e.g.)
col = 3
colData = []
for row in xList:
    colData.append(float(row[col]))

colArray = np.array(colData)
```

```
colMean = np.mean(colArray)
colsd = np.std(colArray)
sys.stdout.write("Mean = " + '\t' + str(colMean) + '\t\t' +
            "Standard Deviation = " + '\t ' + str(colsd) + "\n")


#calculate quantile boundaries
ntiles = 4

percentBdry = []

for i in range(ntiles+1):
    percentBdry.append(np.percentile(colArray, i*(100)/ntiles))

sys.stdout.write("\nBoundaries for 4 Equal Percentiles \n")
print(percentBdry)
sys.stdout.write(" \n")


#run again with 10 equal intervals
ntiles = 10

percentBdry = []

for i in range(ntiles+1):
    percentBdry.append(np.percentile(colArray, i*(100)/ntiles))

sys.stdout.write("Boundaries for 10 Equal Percentiles \n")
print(percentBdry)
sys.stdout.write(" \n")


#The last column contains categorical variables

col = 60
colData = []
for row in xList:
    colData.append(row[col])

unique = set(colData)
sys.stdout.write("Unique Label Values \n")
print(unique)

#count up the number of elements having each value

catDict = dict(zip(list(unique),range(len(unique))))

catCount = [0]*2

for elt in colData:
    catCount[catDict[elt]] += 1
```

continues

continued

```
sys.stdout.write("\nCounts for Each Value of Categorical Label \n")
print(list(unique))
print(catCount)

Output:
Mean =    0.053892307      Standard Deviation =     0.046415983

Boundaries for 4 Equal Percentiles
 [0.0057999999999999996, 0.024375000000000001, 0.044049999999999999,
 0.064500000000000002, 0.4264]

Boundaries for 10 Equal Percentiles
[0.00579999999999, 0.0141, 0.022740000000, 0.0278699999999,
0.0362200000000, 0.0440499999999, 0.050719999999, 0.0599599999999,
0.0779400000000, 0.10836, 0.4264]
Unique Label Values
set(['R', 'M'])

Counts for Each Value of Categorical Label
['R', 'M']
[97, 111]
```

The first section of the code picks up one column of numeric data, and then generates some statistics for it. The first step is to calculate the mean and standard deviation for the chosen attribute. Knowing these will undergird your intuition as you're developing models.

The next section of code looks for outliers. Here's how that works. Suppose that you're trying to determine whether you've got an outlier in the following list of numbers = [0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 4]. This example is constructed to have an outlier. The last number (4) is clearly out of scale with the rest of the numbers.

One way to reveal this sort of mismatch is to divide a set of numbers into percentiles. For example, the 25th percentile contains the smallest 25 percent of the data. The 50th percentile contains the smallest 50 percent of the data. The easiest way to visualize forming these groupings is to imagine that the data are sorted into numeric order. The numbers in the preceding list are arranged in numeric order. That makes it easy to see where the percentile boundaries go. Some often used percentiles are given special names. The percentiles defined by dividing the set into equal quarters, fifths, and tenths are called respectively *quartiles*, *quintiles*, and *deciles*.

With the preceding list, it's easy to define the quartiles because the list is ordered and there are eight elements in the list. The first quartile contains 0.1 and 0.15 and so on. Notice how wide these quartiles are. The first quartile has a range of 0.5 (0.15–0.1). The second quartile is roughly the same. However, the last quartile has a range of 4.6, which is 100 times larger than the range of the other quartiles.

You can see similar behavior in the quartile boundaries that are calculated in Listing 2-3. First the program calculates the quartiles. That shows that the upper quartile is much wider than the others. To be more certain, the decile boundaries are also calculated and similarly demonstrate that the upper decile is unusually wide. Some widening is normal because distributions often thin out in the tails.

Visualization of Outliers Using Quantile-Quantile Plot

One way to study outliers in more detail is to plot the distribution of the data in question relative to some reasonable distributions to see whether the relative numbers match up. Listing 2-4 shows how to use the Python function `probplot` to help determine whether the data has outliers or not. The resulting plot shows how the boundaries associated with empirical percentiles in the data compare to the boundaries for the same percentiles of a Gaussian distribution. If the data being analyzed comes from a Gaussian distribution, the point being plotted will lie on a straight line. Figure 2-1 shows that a couple of points from column 4 of the rocks versus mines data are very far from the line. That means that the tails of the rocks versus mines data contain more examples than the tails of a Gaussian density.

Listing 2-4: Quantile-Quantile Plot for 4th Rocks versus Mines Attribute—qqplotAttribute.py

```
__author__ = 'mike bowles'
import numpy as np
import pylab
import scipy.stats as stats
import urllib2
import sys

target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")

data = urllib2.urlopen(target_url)


#arrange data into list for labels and list of lists for attributes
xList = []
labels = []

for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)
nrow = len(xList)
ncol = len(xList[1])
```

continues

continued

```
type = [0]*3
colCounts = []

#generate summary statistics for column 3 (e.g.)
col = 3
colData = []
for row in xList:
    colData.append(float(row[col]))


stats.probplot(colData, dist="norm", plot=pylab)
pylab.show()
```

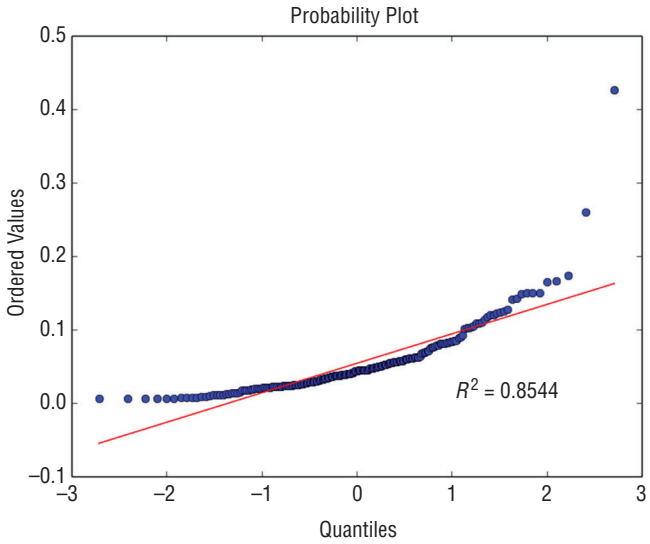


Figure 2-1: Quantile-quantile plot of attribute 4 from rocks versus mines data

What do you do with this information? Outliers may cause trouble either for model building or prediction. After you've trained a model on this data set, you can look at the errors your model makes and see whether the errors are correlated with these outliers. If they are, you can then take steps to correct them. For example, you can replicate the poor-performing examples to force them to be more heavily represented. You can segregate them out and train on them as a separate class. You can also edit them out of the data if they represent an abnormality that won't be present in the data your model will see when deployed. A reasonable process for this might be to generate quartile boundaries during the exploration phase and note potential outliers to get a feel for how much of a problem you might (or might not) have with it. Then when you're evaluating

performance data, use quantile-quantile (Q-Q) plots to determine which points to call outliers for use in your error analysis.

Statistical Characterization of Categorical Attributes

The process just described applies to numeric attributes. But what about categorical attributes? You want to check to see how many categories they have and how many examples there are from each category. You want to learn these things for a couple of reasons. The gender attribute has two possible values (Male and Female), but if the attribute had been the state of the United States, there would have been 50 possible categories. As the number of attributes grows, the complexity of dealing with them mounts. Most binary tree algorithms, which are the basis for ensemble methods, have a cutoff on how many categories they can handle. The popular Random Forests package written by Breiman and Cutler (the inventors of the algorithm) has a cutoff of 32 categories. If an attribute has more than 32 categories, you'll need to aggregate them.

You'll see later that training involves taking a random subset of the data and training a series of models on it. Suppose, for instance, that the category is the state of the United States and that Idaho has only two examples. A random draw of training examples might not get any from Idaho. You need to see those kinds of problems before they occur so that you can address them. In the case of the two Idaho examples, you might merge them with Montana or Wyoming, you might duplicate them, or you might manage the random draw so that you ensure getting Idaho examples (a procedure called *stratified sampling*).

How to Use Python Pandas to Summarize the Rocks Versus Mines Data Set

The Python package Pandas can help automate the process of data inspection and handling. It proves particularly useful for the early stages of data inspection and preprocessing. The Pandas package makes it possible to read data into a specialized data structure called a *data frame*. The data frame is modeled after the CRAN-R data structure of the same name.

NOTE **The Pandas package can be difficult to install because it has a number of dependencies that need to be correctly versioned and each of those has to be correctly matched to one another (and so on). An easy way around this hurdle is to use the Anaconda Python distribution available for free download from Continuum Analytics (`http://continuum.io`). The installation procedures are easy to follow and result in compatible installations of a wide variety of packages for data analysis and machine learning.**

You can think of a data frame as a table or matrix-like structure as in Table 2-1. The data frame is oriented with a row representing a single case (experiment, example, measurement) and columns representing particular attributes. The structure is matrix-like, but not a matrix because the elements in various columns may be of different types. Formally, a matrix is defined over a field (like the real numbers, binary numbers, complex numbers), and all the entries in a matrix are elements from that field. For statistical problems, the matrix is too confining because statistical samples typically have a mix of different types.

The simple example in Table 2-1 has real values in the Attribute 1 column, categorical variables in the Attribute 2 column, and integer variables in the Attribute 3 column. Within a column, the entries are all the same type, but they differ from one column to the next. The data frame structure enables access to individual elements through an index roughly similar to addressing an entry in a Python Numpy array or a list of lists. Similarly, index slicing can be used to address an entry in the array. In addition, the Pandas data frame enables addressing rows and columns by means of their names. This turns out to be very handy, particularly for a small to medium number of columns. (A search on "Pandas introduction" will give you a number of links that can guide you through the basics of using Pandas.)

Listing 2-5 show how simple it is to read in the rocks versus mines CSV file from the UC Irvine Data Repository website. The output shown as part of the listing is truncated from the actual output. You can get the full version by running the code for yourself.

Listing 2-5: Using Python Pandas to Read and Summarize Data—pandasReadSummarize.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
import matplotlib.pyplot as plot
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")

#read rocks versus mines data into pandas data frame
rocksVMines = pd.read_csv(target_url,header=None, prefix="V")

#print head and tail of data frame
print(rocksVMines.head())
print(rocksVMines.tail())

#print summary of data frame
summary = rocksVMines.describe()
print(summary)

Output (truncated):

       V0      V1      V2      ...      V57      V58      V59  V60
```

```
0  0.0200  0.0371  0.0428     ...  0.0084  0.0090  0.0032    R
1  0.0453  0.0523  0.0843     ...  0.0049  0.0052  0.0044    R
2  0.0262  0.0582  0.1099     ...  0.0164  0.0095  0.0078    R
3  0.0100  0.0171  0.0623     ...  0.0044  0.0040  0.0117    R
4  0.0762  0.0666  0.0481     ...  0.0048  0.0107  0.0094    R

[5 rows x 61 columns]

         V0      V1      V2     ...      V57     V58     V59  V60
203  0.0187  0.0346  0.0168     ...   0.0115  0.0193  0.0157    M
204  0.0323  0.0101  0.0298     ...   0.0032  0.0062  0.0067    M
205  0.0522  0.0437  0.0180     ...   0.0138  0.0077  0.0031    M
206  0.0303  0.0353  0.0490     ...   0.0079  0.0036  0.0048    M
207  0.0260  0.0363  0.0136     ...   0.0036  0.0061  0.0115    M

[5 rows x 61 columns]



                V0          V1         ...          V58         V59
count  208.000000  208.000000         ...   208.000000  208.000000
mean     0.029164    0.038437         ...     0.007941    0.006507
std      0.022991    0.032960         ...     0.006181    0.005031
min      0.001500    0.000600         ...     0.000100    0.000600
25%      0.013350    0.016450         ...     0.003675    0.003100
50%      0.022800    0.030800         ...     0.006400    0.005300
75%      0.035550    0.047950         ...     0.010325    0.008525
max      0.137100    0.233900         ...     0.036400    0.043900
```

After reading in the file, the first section of the program prints out head and tail. Notice that all the heads have R labels, and the tails have M labels. With this data set, the Rs all come first and the Ms second. Note things like that during your inspection of the data. You'll see in later sections that determining the quality of your models requires sampling the data. Structure in the way the data are stored might need to be factored into your approach for doing subsequent sampling. The last bit of the code snippet prints out summaries of the real-valued columns in the data set.

Pandas makes it possible to automate the steps of calculating mean, variance, and quantiles. Notice that the summary produced by the `describe` function is itself a data frame so that you can automate the process of screening for attributes that have outliers. To do that, you can compare the differences between the various quantiles and raise a flag if any of the differences for an attribute are out of scale with the other differences for the same attributes. The attributes that are shown in the output indicate that several of them have outliers. It would be worth looking to determine how many rows are involved in the outliers. They might all come from a handful of examples. This can point out data that needs to be inspected more closely.

Visualizing Properties of the Rocks versus Mines Data Set

Visualizations can sometimes give you insights into your data that would be difficult to see in tables of numbers. This section introduces several that you may find useful. Some of the visualizations take slightly different forms for classification problems than for regression problems. You'll see the regression variants of the methods in the sections covering the abalone data set and the wine quality data set.

Visualizing with Parallel Coordinates Plots

One visualization that is useful for problems with more than a few attributes is called a *parallel coordinates plot*. Figure 2-2 depicts the construction of a parallel coordinates plot. The vector of numbers on the right-hand side of the figure represents a row of attribute data from a machine learning data set. The parallel coordinates plot of that vector of numbers is shown in the line plot in Figure 2-2. The line plots the value of each attribute versus its index. The parallel coordinates plot for the whole data set has a line for each row of attributes in the data set. Color-coding based on the labels can help you see some types of systematic relationships between the attribute values and the labels. Plot the real-valued attributes from a row versus the index of the attribute. (Search "parallel coordinates" and check out the Wikipedia page for some more examples.)

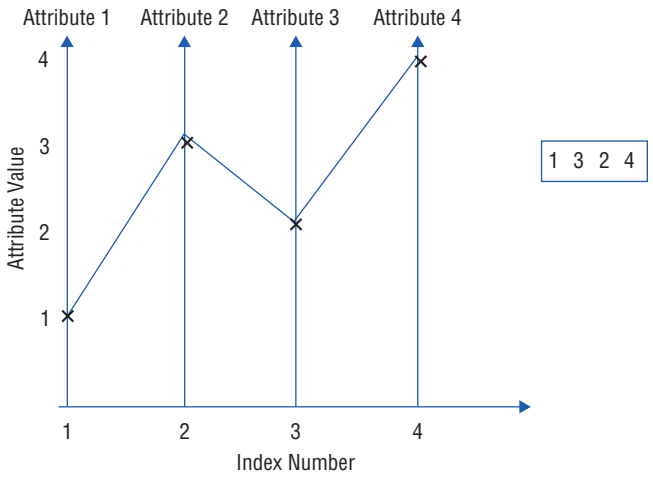


Figure 2-2: Constructing a parallel coordinates plot

Listing 2-6 shows how this process works for the rocks versus mines data set. Figure 2-3 shows the resulting plotted line graphs. The lines are color coded according to their labels: blue for R (rock), and red for M (mine). Sometimes a plot of this type will show clear areas of separation between the classes. The famous "Iris data" show very clear separation that machine learning algorithms will exploit for classification purposes. For the rocks versus mines data set, no extremely clear separation is evident in the line plot, but there are some areas where the blues and reds are separated. Along the bottom of the plot, the blues stand out a bit, and in the range of attribute indices from 30 to 40, the blues are somewhat higher than the reds. These kinds of insights can help in interpreting and confirming predictions made by your trained model.

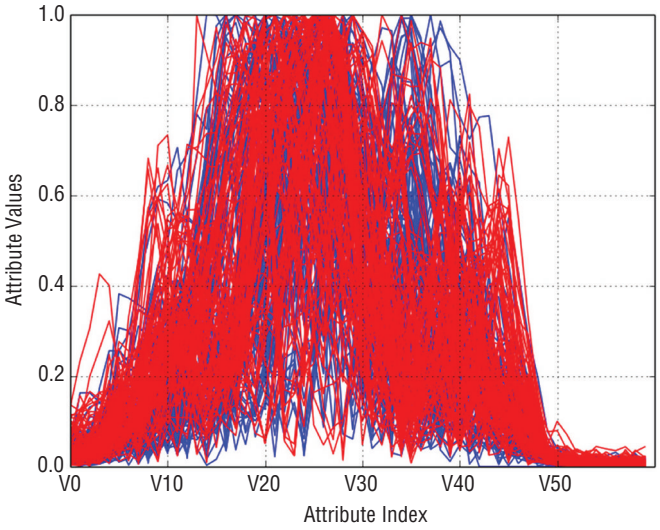


Figure 2-3: Parallel coordinates graph of rocks versus mines attributes

Listing 2-6: Parallel Coordinates Graph for Real Attribute Visualization—linePlots.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
import matplotlib.pyplot as plot
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")

#read rocks versus mines data into pandas data frame
rocksVMines = pd.read_csv(target_url,header=None, prefix="V")

for i in range(208):
    #assign color based on "M" or "R" labels
    if rocksVMines.iat[i,60] == "M":
        pcolor = "red"
```

continues

continued

```
    else:
        pcolor = "blue"

    #plot rows of data as if they were series data
    dataRow = rocksVMines.iloc[i,0:60]
    dataRow.plot(color=pcolor)

plot.xlabel("Attribute Index")
plot.ylabel(("Attribute Values"))
plot.show()
```

Visualizing Interrelationships between Attributes and Labels

Another question you might ask of the data is how the various attributes relate to one another. One quick way to get an idea of pair-wise relationships is to cross-plot the attributes with the labels. Listing 2-7 shows what's required to generate cross-plots for a couple of representative pairs of attributes. These cross-plots (also called *scatter plots*) show you how closely related the pairs of variables are.

Listing 2-7: Cross Plotting Pairs of Attributes—corrPlot.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
import matplotlib.pyplot as plot
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")

#read rocks versus mines data into pandas data frame
rocksVMines = pd.read_csv(target_url,header=None, prefix="V")

#calculate correlations between real-valued attributes
dataRow2 = rocksVMines.iloc[1,0:60]
dataRow3 = rocksVMines.iloc[2,0:60]

plot.scatter(dataRow2, dataRow3)


plot.xlabel("2nd Attribute")
plot.ylabel(("3rd Attribute"))
plot.show()

dataRow21 = rocksVMines.iloc[20,0:60]

plot.scatter(dataRow2, dataRow21)


plot.xlabel("2nd Attribute")
plot.ylabel(("21st Attribute"))
plot.show()
```

Figures 2-4 and 2-5 show the scatter plots for two pairs of attributes from the rocks versus mines data set. The rocks versus mines attributes are samples from sonar returns. The sonar signal is called a *chirped* waveform because it's a pulse that starts at low frequency and rises higher over the duration of the pulse. The attributes in the rocks versus mines data set are time samples of the sound waves that bounce off the rock or mine. These returned acoustic signals bear the same relationship between time and frequency as the outgoing transmission. The 60 attributes in the rocks versus mines data are samples of the return taken at 60 different times (and therefore 60 different frequencies). You'd expect that adjacent attributes would be more correlated than attributes separated in time from one another because there's not much difference in frequency between adjacent time samples.

This intuition is borne out in Figures 2-4 and 2-5. The points in the scatter plot in Figure 2-4 are more closely grouped around a straight line than those in Figure 2-5. If you want to develop your intuition about the relation between numeric correlation and the shape of the scatter plot, just search "correlation" and have a look at the Wikipedia page that comes up. That shows some scatter plots and the associated numeric correlation. Basically, if the points in the scatter plot lie along a thin straight line, the two variables are highly correlated; if they form a ball of points, they're uncorrelated.

You can apply the same principle to plotting the correlation between each of the attributes and the target. For a problem where the targets are real numbers (a regression problem), the plots look much the same as Figures 2-4 and 2-5. The rocks versus mines data set is a classification problem. The targets are two-valued. You can follow the same general procedure.

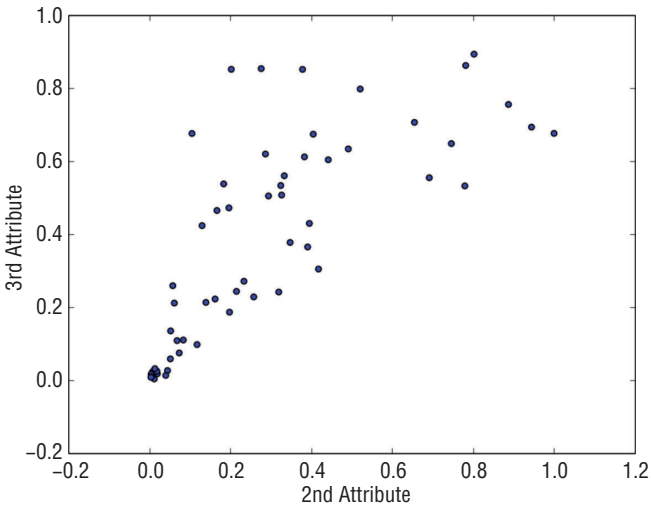


Figure 2-4: Cross-plot of rocks versus mines attributes 2 and 3

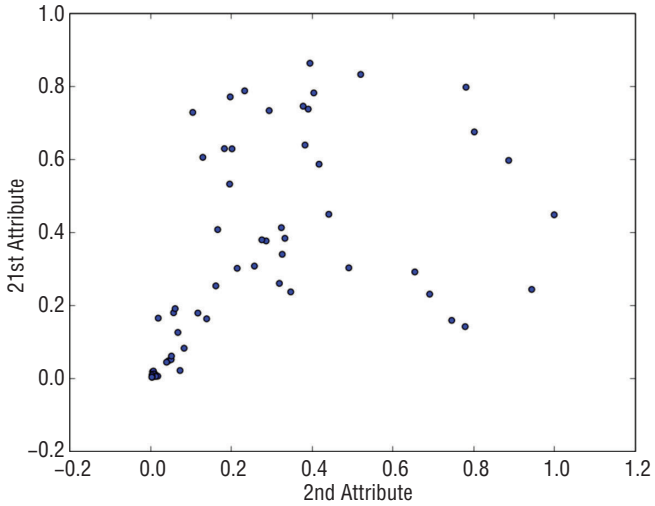


Figure 2-5: Cross- plot of rocks versus mines attributes 2 and 21

Listing 2-8 shows the code for plotting a scatter plot between the targets and attribute 35. The idea of using attribute 35 for the example showing correlation with the target came from the parallel coordinates graph in Figure 2-3. That graph shows some separation between the rocks and mines (red lines and blue lines) around index value 35. The correlation between the target and one of the attributes around that index value should also show some separation. Figures 2-6 and 2-7 plot the results.

Listing 2-8: Correlation between Classification Target and Real Attributes—targetCorr.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
import matplotlib.pyplot as plot
from random import uniform
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")

#read rocks versus mines data into pandas data frame
rocksVMines = pd.read_csv(target_url,header=None, prefix="V")

#change the targets to numeric values
target = []
for i in range(208):
    #assign 0 or 1 target value based on "M" or "R" labels
    if rocksVMines.iat[i,60] == "M":
        target.append(1.0)
```

```
    else:
        target.append(0.0)

#plot 35th attribute
dataRow = rocksVMines.iloc[0:208,35]
plot.scatter(dataRow, target)

plot.xlabel("Attribute Value")
plot.ylabel("Target Value")
plot.show()

#
#To improve the visualization, this version dithers the points a little
# and makes them somewhat transparent
target = []
for i in range(208):

#assign 0 or 1 target value based on "M" or "R" labels
    # and add some dither

    if rocksVMines.iat[i,60] == "M":
        target.append(1.0 + uniform(-0.1, 0.1))
    else:
        target.append(0.0 + uniform(-0.1, 0.1))

    #plot 35th attribute with semi-opaque points
dataRow = rocksVMines.iloc[0:208,35]
plot.scatter(dataRow, target, alpha=0.5, s=120)

plot.xlabel("Attribute Value")
plot.ylabel("Target Value")
plot.show()
```

The plots show what happens if you make a list corresponding to the list of R or M targets but with the substitution of 1 for M and 0 for R. Then you can plot a scatter plot as shown in Figure 2-6. Figure 2-6 highlights a common problem with cross-plots. When one of the variables being plotted takes on a small number of values, the points get plotted on top of one another. If there are a lot of them, you just get a thick dark line, and you don't get a feel for how the points are distributed along the line.

The code in Listing 2-8 generates a second plot with two small changes to overcome this problem. A small random number is added to each of the points and takes a small number of discrete values (the targets in this case). The target values are either 0 or 1 by construction. In the code, you'll see that the added random number is uniformly distributed between –0.1 and 0.1. That spreads the points apart, but not so far as to confuse the two lines. Second, the points are plotted with alpha=0.5 in order that the points are only partially opaque.

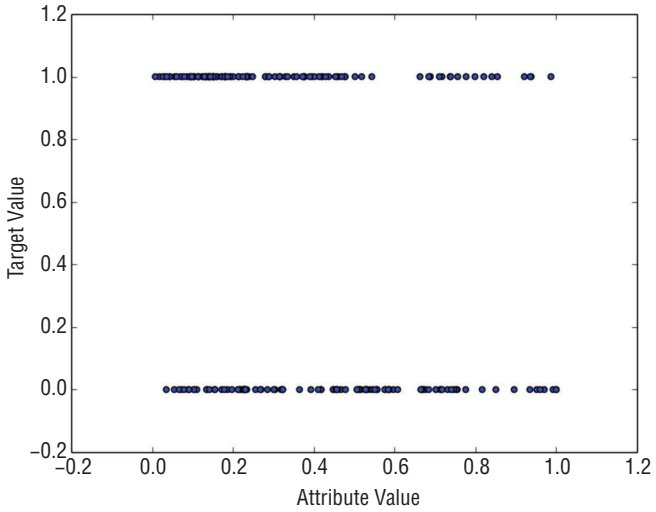


Figure 2-6: Target-attribute cross-plot

Then any overplotting shows up as a darkened region in the scatter plot. You may have to adjust these numbers a little to make the plot show you what you need to know.

Figure 2-7 shows the effect of these two alterations. Notice the somewhat higher concentration of attribute 35 on the left end of the upper band of points, whereas the points are more uniformly spread from right to left in the lower band. The upper band of points corresponds to mines. The lower band corresponds to rocks. You could build a classifier for this problem by testing whether attribute 35 is greater than or less than 0.5. If it is greater than 0.5 call it a rock,

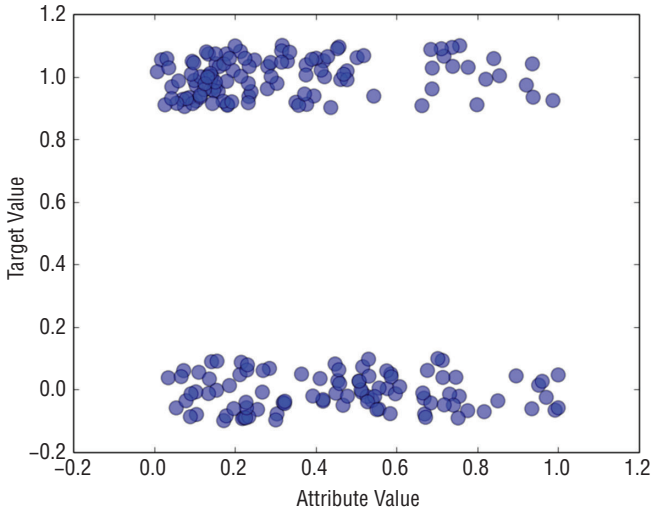


Figure 2-7: Target-attribute cross-plot with point dither and partial opacity

and if it is less than 0.5, call it a mine. The examples where attribute 35 is less than 0.5 contain a higher concentration of mines than rock, and the examples where attribute 35 is less than 0.5 contain a lower density, so you'd get better performance than you would with random guessing.

NOTE **You'll see much more systematic approaches to building classifiers in Chapters 5, "Building Predictive Models Using Penalized Linear Methods," and Chapter 7, "Building Ensemble Models with Python." They'll use all the attributes instead of just one or two. However, when you look at what they're using to make their decisions, you can refer back to these types of studies to help you gain confidence that what they're doing is sensible.**

The degree of correlation between two attributes (or an attribute and a target) can be quantified using Pearson's correlation coefficient. Pearson's correlation coefficient is defined for two equal length vectors u and v, as follows (see Equations 2-1 and 2-2). First subtract the mean value of u from all the elements of u (see Equation 2-3) and do the same for v.

$$u = \begin{matrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{matrix}$$

Equation 2-1: Elements of a vector u

$$\overline{u} = avg(u)$$

Equation 2-2: Average values of the entries in u

$$\Delta u = \begin{matrix} u_1 - \overline{u} \\ u_2 - \overline{u} \\ \vdots \\ u_n - \overline{u} \end{matrix}$$

Equation 2-3: Subtract the average from each element in u.

For the second vector v, define a vector Δv in the same way as Δu was defined corresponding to the vector u.

Then Pearson's correlation between u and v is shown in Equation 2-4.

$$corr(u,v) = \frac{\Delta u^T * \Delta v}{\sqrt{(\Delta u^T * \Delta u) * (\Delta v^T * \Delta v)}}$$

Equation 2-4: Definition of Pearson's correlation coefficient

Listing 2-9 shows a Python implementation of this function to calculate correlation for the pairs of attributes plotted in Figures 2-3 and 2-5. The correlation numbers agree with plotted data. The attributes that have close index numbers have relatively higher correlations than those that are separated further.

Listing 2-9: Pearson's Correlation Calculation for Attributes 2 versus 3 and 2 versus 21—corrCalc.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
from math import sqrt
import sys
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")

#read rocks versus mines data into pandas data frame
rocksVMines = pd.read_csv(target_url,header=None, prefix="V")

#calculate correlations between real-valued attributes
dataRow2 = rocksVMines.iloc[1,0:60]
dataRow3 = rocksVMines.iloc[2,0:60]
dataRow21 = rocksVMines.iloc[20,0:60]

mean2 = 0.0; mean3 = 0.0; mean21 = 0.0
numElt = len(dataRow2)
for i in range(numElt):
    mean2 += dataRow2[i]/numElt
    mean3 += dataRow3[i]/numElt
    mean21 += dataRow21[i]/numElt

var2 = 0.0; var3 = 0.0; var21 = 0.0
for i in range(numElt):
    var2 += (dataRow2[i] - mean2) * (dataRow2[i] - mean2)/numElt
    var3 += (dataRow3[i] - mean3) * (dataRow3[i] - mean3)/numElt
    var21 += (dataRow21[i] - mean21) * (dataRow21[i] - mean21)/numElt

corr23 = 0.0; corr221 = 0.0
for i in range(numElt):

corr23 += (dataRow2[i] - mean2) * \
              (dataRow3[i] - mean3) / (sqrt(var2*var3) * numElt)
    corr221 += (dataRow2[i] - mean2) * \
                (dataRow21[i] - mean21) / (sqrt(var2*var21) * numElt)

sys.stdout.write("Correlation between attribute 2 and 3 \n")
print(corr23)
sys.stdout.write(" \n")
```

```
sys.stdout.write("Correlation between attribute 2 and 21 \n")
print(corr221)
sys.stdout.write(" \n")


Output:
Correlation between attribute 2 and 3
0.770938121191

Correlation between attribute 2 and 21
0.466548080789
```

Visualizing Attribute and Label Correlations Using a Heat Map

Calculating the correlations and printing them or drawing cross-plots works fine for a few correlations, but it is difficult to get a grasp of a large table of numbers, and it is difficult to squeeze all the cross-plots onto a page if the problem has 100 attributes.

One way to check correlations with a large number of attributes is to calculate the Pearson's correlation coefficient for pairs of attributes, arrange those correlations into a matrix where the ij-th entry is the correlation between the ith attribute and the jth attribute, and then plot them in a heat map. Listing 2-10 gives the code to make this plot. Figure 2-8 shows the plot. The light areas along the diagonal confirm that attributes close to one another in index have relatively high correlations. As mentioned earlier, this is due to the way in which the data are generated. Close indices are sampled at short time intervals from one another and consequently have similar frequencies. Similar frequencies reflect off the targets similarly (and so on).

Listing 2-10: Presenting Attribute Correlations Visually—sampleCorrHeatMap.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
import matplotlib.pyplot as plot
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")

#read rocks versus mines data into pandas data frame
rocksVMines = pd.read_csv(target_url,header=None, prefix="V")

#calculate correlations between real-valued attributes
corMat = DataFrame(rocksVMines.corr())

#visualize correlations using heatmap
plot.pcolor(corMat)
plot.show()
```

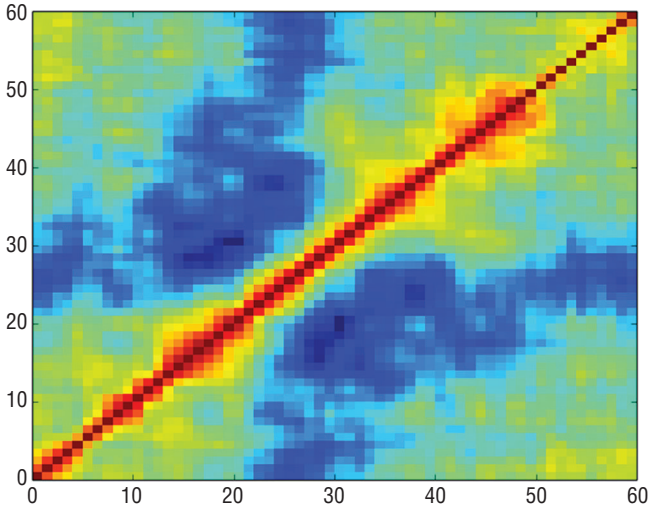


Figure 2-8: Heat map showing attribute cross-correlations

Perfect correlation (correlation = 1) between attributes means that you may have made a mistake and included the same thing twice. Very high correlation between a set of attributes (pairwise correlations > 0.7) is known as *multicollinearity* and can lead to unstable estimates. Correlation with the targets is a different matter. Having an attribute that's correlated with the target generally indicates a predictive relation.

Summarizing the Process for Understanding Rocks versus Mines Data Set

In the process of understanding the rocks versus mines data set, this section has introduced a number of tools for you to use to gain understanding and intuition about your data sets. The section has gone into some detail to make their derivation and use clear. The next sections will use several of these same tools to inspect the other data sets that the book will use to develop machine learning algorithms. Since you're now familiar with the tools for doing data inspection, the next sections will comment on the tools only to the extent that they need to be modified because of the different nature of a problem.

Real-Valued Predictions with Factor Variables: How Old Is Your Abalone?

Most of the tools you've seen used for understanding the problem of detecting unexploded mines can be applied to regression problems. Predicting the age of an abalone, given physical measurements, provides an example of such a

problem. The abalone attributes also include an attribute that is a factor variable, which will illustrate the differences involved with factor variables.

The abalone data set poses the problem of predicting the age of an abalone by taking several measurements. It is possible to get a precise reading on the age of an abalone by slicing the shell and counting growth rings, much like gauging the age of a tree by counting rings. The problem for scientists studying abalone populations is that it is expensive and time-consuming to slice the shells and count the rings under a microscope. It would be more convenient and economical to be able to make simple physical measurements like length, width, weight, and so forth and then to use a predictive model to process the measurements and make an accurate determination of the age of the abalone. There are a myriad of scientific applications for predictive analytics, and one of the benefits of studying machine learning is being able to contribute to an interesting array of different problems.

The data for this problem are available through the UC Irvine Data Repository. The URL for this data set is `http://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data`. This data set is in the form of a comma-delimited file with no column headers. The names of the columns are in a separate file. Listing 2-11 reads the abalone data set into a Pandas data frame and runs through some of the same analyses that you saw in "Classification Problems: Detecting Unexploded Mines Using Sonar." For the rocks versus mines data set, the column names were somewhat generic because of the nature of the data. For the abalone data set, the different columns of data have meanings that can be critical to cultivating an intuitive understanding of your progress toward an acceptable model. For this reason, you'll see in the code that the column names have been copy-pasted into the code and attached to the data set to help you make sense of what subsequent machine learning algorithms are doing to make predictions. The columns of data available for building a predictive model are Sex, Length, Diameter, Height, Whole Weight, Shucked Weight, Viscera Weight, Shell Weight, and Rings. The last column, Rings, is measured by the laborious process of sawing the shell and counting under a microscope. This is the usual arrangement for a supervised learning problem. You've got a special data set for which the answer is known so as to build a model that will generate predictions when the answer is not known.

In addition to showing the code for producing the summaries, Listing 2-11 shows the printed output from the summarization. The first section prints the head and tail of the data set. Only the head is shown in the output to save space. When you run the code for yourself, you'll see both. Most of the data frame is filled with floating-point numbers. The first column, which contains the gender of the animal, contains the letters M (male), F (female), and I (indeterminate). The gender of an abalone is not determined at birth, but after it has matured a little. Therefore, the gender is indeterminate for younger abalones. The gender of the abalone is a three-valued categorical variable. Categorical attributes require special attention. Some algorithms only deal with real-valued attributes (for

example, support vector machines, K-nearest neighbors, and penalized linear regression, which is introduced in Chapter 4). Chapter 4 discusses techniques for translating categorical variables into real-valued variables so that you can employ these algorithms. Listing 2-11 also shows the column-by-column statistical summaries for the real-valued attributes.

Listing 2-11: Read and Summarize the Abalone Data Set—abaloneSummary.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
from pylab import *
import matplotlib.pyplot as plot

target_url = ("http://archive.ics.uci.edu/ml/machine-"
              "learning-databases/abalone/abalone.data")
#read abalone data
abalone = pd.read_csv(target_url,header=None, prefix="V")
abalone.columns = ['Sex', 'Length', 'Diameter', 'Height',
                   'Whole weight','Shucked weight', 'Viscera weight',
                   'Shell weight', 'Rings']


print(abalone.head())
print(abalone.tail())

#print summary of data frame
summary = abalone.describe()
print(summary)

#box plot the real-valued attributes
#convert to array for plot routine
array = abalone.iloc[:,1:9].values
boxplot(array)
plot.xlabel("Attribute Index")
plot.ylabel(("Quartile Ranges"))
show()

#the last column (rings) is out of scale with the rest
# - remove and replot
array2 = abalone.iloc[:,1:8].values
boxplot(array2)
plot.xlabel("Attribute Index")
plot.ylabel(("Quartile Ranges"))
show()

#removing is okay but renormalizing the variables generalizes better.
#renormalize columns to zero mean and unit standard deviation
#this is a common normalization and desirable for other operations
# (like k-means clustering or k-nearest neighbors
```

```
abaloneNormalized = abalone.iloc[:,1:9]


for i in range(8):
    mean = summary.iloc[1, i]
    sd = summary.iloc[2, i]

abaloneNormalized.iloc[:,i:(i + 1)] = (
                    abaloneNormalized.iloc[:,i:(i + 1)] - mean) / sd


array3 = abaloneNormalized.values
boxplot(array3)
plot.xlabel("Attribute Index")
plot.ylabel(("Quartile Ranges - Normalized "))
show()

Printed Output: (partial)
  Sex  Length  Diameter  Height  Whole wt  Shucked wt  Viscera wt
0   M   0.455     0.365   0.095    0.5140      0.2245      0.1010
1   M   0.350     0.265   0.090    0.2255      0.0995      0.0485
2   F   0.530     0.420   0.135    0.6770      0.2565      0.1415
3   M   0.440     0.365   0.125    0.5160      0.2155      0.1140
4   I   0.330     0.255   0.080    0.2050      0.0895      0.0395

   Shell weight  Rings
0         0.150     15
1         0.070      7
2         0.210      9
3         0.155     10
4         0.055      7
     Sex  Length  Diameter  Height  Whole weight  Shucked weight
4172   F   0.565     0.450   0.165        0.8870          0.3700
4173   M   0.590     0.440   0.135        0.9660          0.4390
4174   M   0.600     0.475   0.205        1.1760          0.5255
4175   F   0.625     0.485   0.150        1.0945          0.5310
4176   M   0.710     0.555   0.195        1.9485          0.9455

      Viscera weight  Shell weight  Rings
4172          0.2390        0.2490     11
4173          0.2145        0.2605     10
4174          0.2875        0.3080      9
4175          0.2610        0.2960     10
4176          0.3765        0.4950     12
            Length     Diameter       Height    Whole wt  Shucked wt
count  4177.000000  4177.000000  4177.000000 4177.000000 4177.000000
mean      0.523992     0.407881     0.139516    0.828742    0.359367
std       0.120093     0.099240     0.041827    0.490389    0.221963
min       0.075000     0.055000     0.000000    0.002000    0.001000
25%       0.450000     0.350000     0.115000    0.441500    0.186000
50%       0.545000     0.425000     0.140000    0.799500    0.336000
```

continues

continued

```
75%         0.615000      0.480000      0.165000     1.153000     0.502000
max         0.815000      0.650000      1.130000     2.825500     1.488000

       Viscera weight  Shell weight        Rings
count     4177.000000   4177.000000  4177.000000
mean         0.180594      0.238831     9.933684
std          0.109614      0.139203     3.224169
min          0.000500      0.001500     1.000000
25%          0.093500      0.130000     8.000000
50%          0.171000      0.234000     9.000000
75%          0.253000      0.329000    11.000000
max          0.760000      1.005000    29.000000
```

As an alternative to the listing of the statistical summaries, Listing 2-11 generates box plots for each of the real-valued columns of data. The first of these is shown in Figure 2-9. In Figure 2-9, the statistical summaries are represented by box plots, which are also called *box and whisker* plots. These plots show a small rectangle with a red line through it. The red line marks the median value (or 50th percentile) for the column of data. The top and bottom of the rectangle mark the 25th percentile and the 75th percentile, respectively. You can compare the numbers in the printed summary to the levels in the box plot to confirm this. Above and below the box, you'll see small horizontal ticks, the so-called whiskers. These are drawn in at levels that are 1.4 times the interquartile spacing above and below the box. Interquartile spacing is the difference between the 75th percentile and the 25th percentile. In other words, the space between the top of the box and the upper whisker is 1.4 times the height of the box. The 1.4x spacing for the whisker is adjustable; see the box plot documentation. You'll

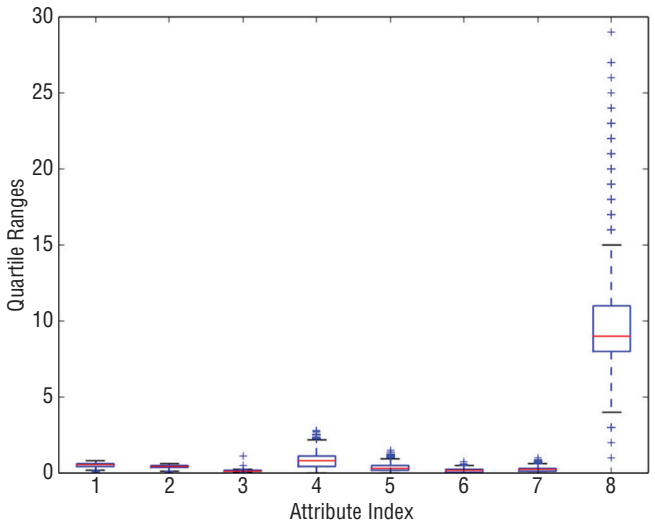


Figure 2-9: Box plot of real-valued attributes from abalone data set

notice that in some cases the whiskers are closer than the 1.4x spacing. For these cases the data values do not extend all the way to the calculated whisker locations. In these cases, the whisker is placed at the most extreme data point. In other cases, the data extend for a considerable distance beyond the calculated whisker locations. These points can be considered outliers.

The box plot in Figure 2-9 is a faster, more visual way to identify outliers than the printed data, but the scale on the rings attributes (the rightmost box plot) causes the other attributes to be compressed (making them hard to see). One way to deal with this is to simply eliminate the larger-scale attributes. The result of that is shown in Figure 2-10. But that approach is unsatisfying because it doesn't automate or scale very well.

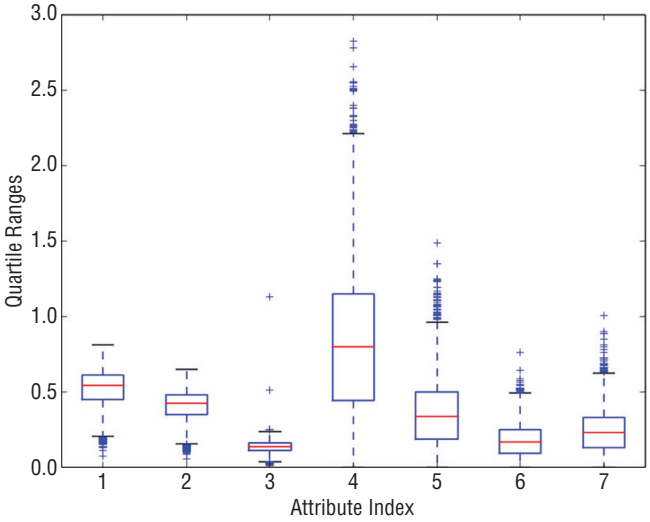


Figure 2-10: Box plot of real-valued attributes from the abalone data set

The last section of the code in Listing 2-11 normalizes all the data columns before box plotting. *Normalization* in this case means centering and scaling each column so that a unit of attribute number 1 means the same thing as a unit of attribute number 2. A number of algorithms and operations in data science require this type of normalization. For example, K-means clustering builds clusters based on vector distance between rows of data. Distance is measured by subtracting one point from another and squaring. If the units are different, the numeric distances are different. The distance to the grocery store can be 1 if measured in miles or 5,280 if measured in feet. The normalization indicated in Listing 2-11 adjusts the variables so that they all have 0 mean and a standard deviation of 1. This is a very common normalization. The calculations for the normalization make use of the numbers generated by the `summary()` function. The results are plotted in Figure 2-11.

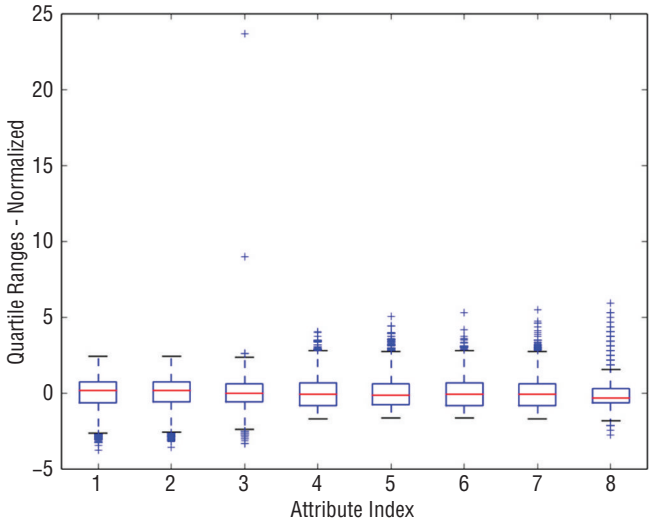


Figure 2-11: Box plot of normalized abalone attributes

Notice that normalizing to standard deviation of 1.0 does not mean that the data all fit between –1.0 and +1.0. It more or less places the lower and upper edges of the boxes at –1.0 and +1.0, but much of the data are outside these boundaries.

Parallel Coordinates for Regression Problems—Visualize Variable Relationships for Abalone Problem

The next step is to get some ideas about the relationship among the attributes and between attributes and labels. For the rocks versus mines data, the color-coded parallel coordinates plot portrayed these relationships graphically. That approach needs some modification to work for the abalone problem. Rocks versus mines was a classifier problem. The parallel coordinates plot for that problem color-coded the lines representing rows of data according to their true classification. That helps to visualize the relationship between prediction and predictors. The abalone problem is a regression problem, so the color-coding in this example needs to be shades of color corresponding to higher or lower target values. To assign shades of color to real values, the real values need to be compressed into the interval [0.0, 1.0]. Listing 2-12 uses the min and max values generated by the `summary()` function from Pandas to accomplish this. Figure 2-12 shows the results.

Listing 2-11: Parallel Coordinate Plot for Abalone Data—abaloneParallelPlot.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
```

```
import matplotlib.pyplot as plot
from math import exp

target_url = ("http://archive.ics.uci.edu/ml/machine-"
              "learning-databases/abalone/abalone.data")
#read abalone data
abalone = pd.read_csv(target_url,header=None, prefix="V")
abalone.columns = ['Sex', 'Length', 'Diameter', 'Height',
                   'Whole Wt', 'Shucked Wt',
                   'Viscera Wt', 'Shell Wt', 'Rings']
#get summary to use for scaling
summary = abalone.describe()
minRings = summary.iloc[3,7]
maxRings = summary.iloc[7,7]
nrows = len(abalone.index)

for i in range(nrows):
    #plot rows of data as if they were series data
    dataRow = abalone.iloc[i,1:8]
    labelColor = (abalone.iloc[i,8] - minRings) / (maxRings - minRings)
    dataRow.plot(color=plot.cm.RdYlBu(labelColor), alpha=0.5)

plot.xlabel("Attribute Index")
plot.ylabel(("Attribute Values"))
plot.show()

#renormalize using mean and standard variation, then compress
# with logit function
meanRings = summary.iloc[1,7]
sdRings = summary.iloc[2,7]

for i in range(nrows):
    #plot rows of data as if they were series data
    dataRow = abalone.iloc[i,1:8]
    normTarget = (abalone.iloc[i,8] - meanRings)/sdRings
    labelColor = 1.0/(1.0 + exp(-normTarget))
    dataRow.plot(color=plot.cm.RdYlBu(labelColor), alpha=0.5)

plot.xlabel("Attribute Index")
plot.ylabel(("Attribute Values"))
plot.show()
```

The parallel coordinates plot in Figure 2-12 illustrates a direct relationship between abalone age (number of shell rings) and the attributes available for predicting age. The color scale used to produce this plot ranges from very dark reddish brown through lighter shades, yellow, light blue, and very dark blue. The box plot in Figure 2-11 shows that the maximum and minimum values are widely separated from the bulk of the data. This has the effect of compressing the scale so that most of the data are mid-range on the color scale. Nonetheless,

Figure 2-12 indicates significant correlation between each of the attributes and the number of rings measured for each of the examples. Similar shades of color are grouped together at similar values of several of the attributes. This correlation suggests that you'll be able to build an accurate predictive model. Contrary to the generally favorable correlation between attributes and target, some faint blue lines are mixed among the darker orange areas of the graph, indicating that there are some examples that will be difficult to correctly predict.

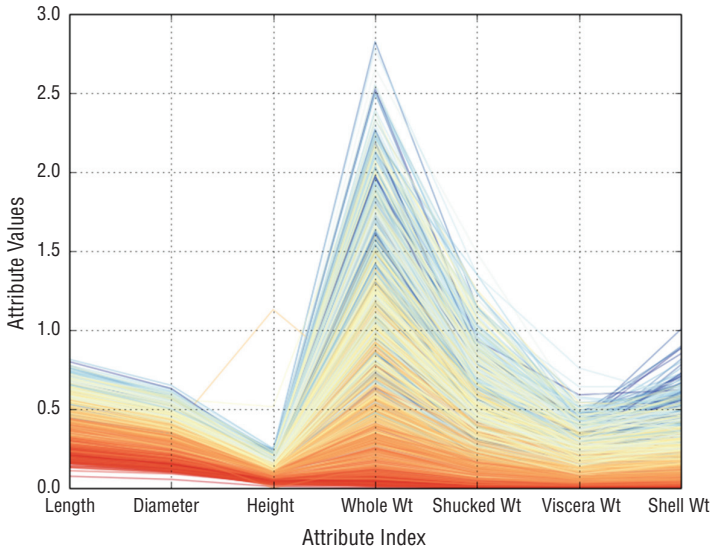


Figure 2-12: Color-coded parallel coordinate plot for abalone

Changing the color mapping can help you visualize relationships at different levels of target values. The last section of the code in Listing 2-11 uses the normalization that you saw used in the box plot graphs. That normalization doesn't make all the values fit between 0 and 1. For one thing, the resulting values take as many negative values as positive ones. The program in Listing 2-11 employs the logit transform to get values in (0, 1). The logit transform is given by the expression shown in Equation 2-5. The plot for this function is given in Figure 2-13.

$$logit\ transform(x) = 1 / (1 + e^{-x})$$

Equation 2-5: Using logit transform for soft range compression

The plot for this function is given in Figure 2-13. As you can see, the logit transform maps large negative values to 0 (almost) and large positive numbers to 1 (almost); it maps 0 to 0.5. You'll see the logit function again in Chapter 4, where it plays a critical role relating a linear function to a probability.

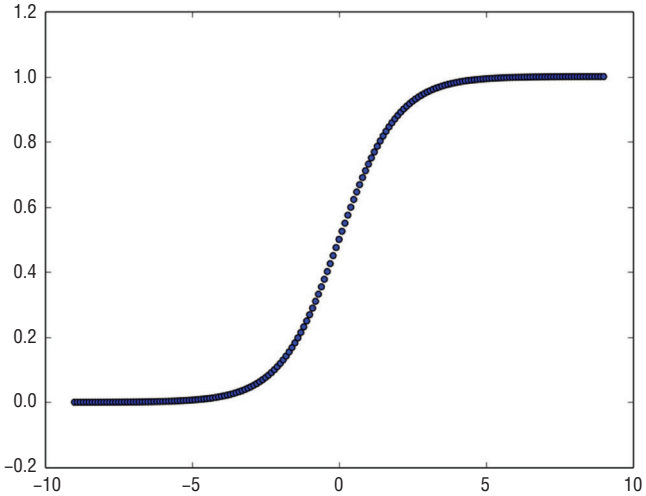


Figure 2-13: Graph of the logit function

Figure 2-14 shows the results of these steps. These transformations have resulted in better usage of the full range of colors available. Notice that there are several darker blue lines (corresponding to specimens with large numbers of rings) mixed in among lighter blue examples, and even yellow and light red specimens for the graphs in the area of Whole Weight and Shucked Weight.

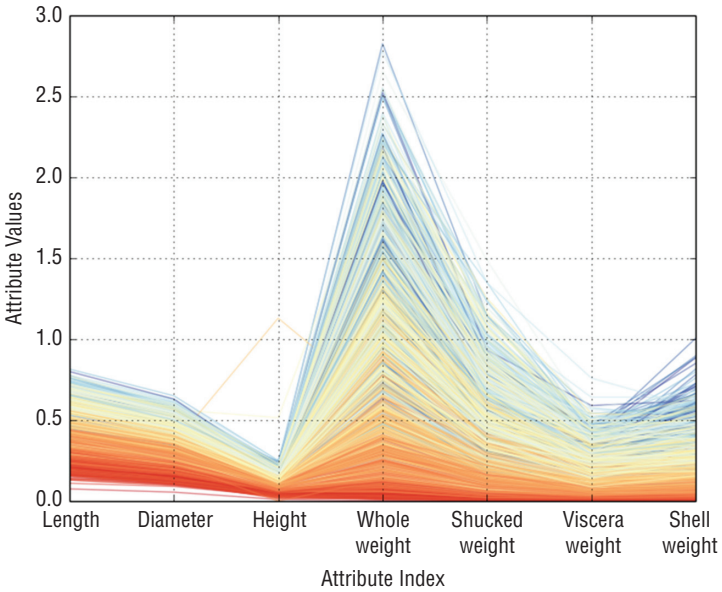


Figure 2-14: Parallel coordinate plot for the abalone data

That suggests that those attributes might not be enough to correctly predict the ages (number of rings) in the older specimens. Fortunately, some of the other attributes (Diameter and Shell Weight) do a better job of correctly ordering the dark blue lines. Those observations will prove helpful when you're analyzing the prediction errors later.

How to Use Correlation Heat Map for Regression—Visualize Pair-Wise Correlations for the Abalone Problem

The last step is to have a look at the correlations between the various attributes and between the attributes and the targets. Listing 2-12 shows the code for generating a correlation heat map and a correlation matrix for the abalone data. These calculations follow the same method outlined for the rocks versus mines data, but with one important difference: Because the abalone problem calls for making real number predictions, the correlation calculations can include the targets in the correlation matrix.

Listing 2-12: Correlation Calculations for Abalone Data—abaloneCorrHeat.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
import matplotlib.pyplot as plot

target_url = ("http://archive.ics.uci.edu/ml/machine-"
              "learning-databases/abalone/abalone.data")
#read abalone data
abalone = pd.read_csv(target_url,header=None, prefix="V")
abalone.columns = ['Sex', 'Length', 'Diameter', 'Height',
                   'Whole weight', 'Shucked weight',
                   'Viscera weight', 'Shell weight', 'Rings']

#calculate correlation matrix
corMat = DataFrame(abalone.iloc[:,1:9].corr())
#print correlation matrix
print(corMat)

#visualize correlations using heatmap
plot.pcolor(corMat)
plot.show()
```

```
                  Length  Diameter    Height  Whole Wt  Shucked Wt
Length          1.000000  0.986812  0.827554  0.925261    0.897914
Diameter        0.986812  1.000000  0.833684  0.925452    0.893162
Height          0.827554  0.833684  1.000000  0.819221    0.774972
Whole weight    0.925261  0.925452  0.819221  1.000000    0.969405
Shucked weight  0.897914  0.893162  0.774972  0.969405    1.000000
Viscera weight  0.903018  0.899724  0.798319  0.966375    0.931961
```

```
Shell weight     0.897706   0.905330   0.817338    0.955355    0.882617
Rings            0.556720   0.574660   0.557467    0.540390    0.420884

                 Viscera weight  Shell weight      Rings
Length                 0.903018      0.897706   0.556720
Diameter               0.899724      0.905330   0.574660
Height                 0.798319      0.817338   0.557467
Whole weight           0.966375      0.955355   0.540390
Shucked weight         0.931961      0.882617   0.420884
Viscera weight         1.000000      0.907656   0.503819
Shell weight           0.907656      1.000000   0.627574
Rings                  0.503819      0.627574   1.000000
```

Figure 2-15 shows the correlation heat map. In this map, red indicates high correlation, and blue represents weak correlation. The targets (the number of rings in the shell) are the last item, which is the top row of the heat map and the rightmost column. The blue values in those positions mean that the attributes are weakly correlated with the targets. The light blue corresponds to the correlation between the target and the shell weight. That confirms what you saw in the parallel coordinates plot. The reddish values in the other off-diagonal cell in Figure 2-15 indicate that the attributes are highly correlated with one another. This somewhat contradicts the picture given by the parallel coordinates map where visually the correspondence between the target and the attributes seemed fairly tight. Listing 2-12 shows the numeric values for correlation.

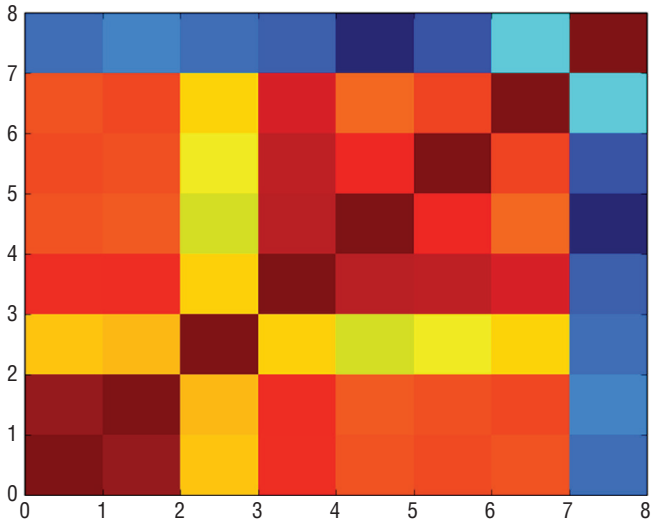


Figure 2-15: Correlation heat map for the abalone data

In this section you've seen how to modify the tools described for a classification problem (rocks versus mines) to a regression problem (abalone). The

modifications all stemmed from the basic difference between the two problem types—labels that are real numbers for a regression problem versus labels that are two-valued for a binary classification problem. The next section will conduct the same set of studies on a regression problem having all numeric attributes. Because it's a regression problem, the same tools used in this section for the abalone problem can be used. Because it has all numeric attributes, all of the attributes can be included in the studies, like correlation and plotting along the real number line.

Real-Valued Predictions Using Real-Valued Attributes: Calculate How Your Wine Tastes

The wine taste data set contains data for approximately 1,500 red wines. For each wine there are a number of measurements of chemical composition, including things like alcohol content, volatile acidity, and sulphites. Each wine also has a taste score determined by averaging the scores given by three professional wine tasters. The problem is to build a model that will incorporate the chemical measurements and predict taste scores to match those given by the human tasters.

Listing 2-13 shows the code for producing summaries of the wine data set. The code prints out a numeric summary of the data, which is included at the bottom of the listing. The code also generates a box plot of the normalized variables so that you can visualize the outliers in the data. Figure 2-16 shows the box plots. The numeric summaries and the box plots indicate numerous outlying values. This is something to keep in mind during training on this data set. When analyzing the performance of the trained models, these outlying examples will be one place to look to understand the source of errors in your models.

Listing 2-13: Wine Data Summary—wineSummary.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
from pylab import *
import matplotlib.pyplot as plot

target_url = ("http://archive.ics.uci.edu/ml/machine-"
 "learning-databases/wine-quality/winequality-red.csv")
wine = pd.read_csv(target_url,header=0, sep=";")

print(wine.head())

#generate statistical summaries
```

```
summary = wine.describe()
print(summary)

wineNormalized = wine
ncols = len(wineNormalized.columns)

for i in range(ncols):
    mean = summary.iloc[1, i]
    sd = summary.iloc[2, i]

wineNormalized.iloc[:,i:(i + 1)] = \
        (wineNormalized.iloc[:,i:(i + 1)] - mean) / sd
array = wineNormalized.values
boxplot(array)
plot.xlabel("Attribute Index")
plot.ylabel(("Quartile Ranges - Normalized "))
show()

Output - [filename - wineSummary.txt]
   fixed acidity  volatil acid  citric acid  resid sugar  chlorides
0            7.4          0.70         0.00          1.9      0.076
1            7.8          0.88         0.00          2.6      0.098
2            7.8          0.76         0.04          2.3      0.092
3           11.2          0.28         0.56          1.9      0.075
4            7.4          0.70         0.00          1.9      0.076

   free sulfur dioxide  tot sulfur dioxide  density    pH  sulphates
0                   11                  34   0.9978  3.51       0.56
1                   25                  67   0.9968  3.20       0.68
2                   15                  54   0.9970  3.26       0.65
3                   17                  60   0.9980  3.16       0.58
4                   11                  34   0.9978  3.51       0.56

   alcohol  quality
0      9.4        5
1      9.8        5
2      9.8        5
3      9.8        6
4      9.4        5
       fixed acidity  volatile acidity  citric acid  residual sugar
count    1599.000000       1599.000000  1599.000000     1599.000000
mean        8.319637          0.527821     0.270976        2.538806
std         1.741096          0.179060     0.194801        1.409928
min         4.600000          0.120000     0.000000        0.900000
25%         7.100000          0.390000     0.090000        1.900000
50%         7.900000          0.520000     0.260000        2.200000
75%         9.200000          0.640000     0.420000        2.600000
max        15.900000          1.580000     1.000000       15.500000

        chlorides  free sulfur dioxide tot sulfur dioxide   density
```

continues

continued

```
count  1599.000000          1599.000000      1599.000000 1599.000000
mean      0.087467            15.874922        46.467792    0.996747
std       0.047065            10.460157        32.895324    0.001887
min       0.012000             1.000000         6.000000    0.990070
25%       0.070000             7.000000        22.000000    0.995600
50%       0.079000            14.000000        38.000000    0.996750
75%       0.090000            21.000000        62.000000    0.997835
max       0.611000            72.000000       289.000000    1.003690

                pH    sulphates      alcohol      quality
count  1599.000000  1599.000000  1599.000000  1599.000000
mean      3.311113     0.658149    10.422983     5.636023
std       0.154386     0.169507     1.065668     0.807569
min       2.740000     0.330000     8.400000     3.000000
25%       3.210000     0.550000     9.500000     5.000000
50%       3.310000     0.620000    10.200000     6.000000
75%       3.400000     0.730000    11.100000     6.000000
max       4.010000     2.000000    14.900000     8.000000
```

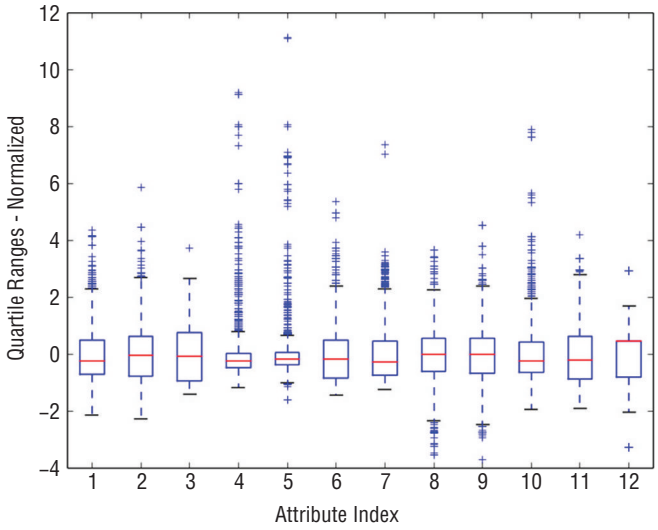


Figure 2-16: Attribute and target box plots of normalized wine data

A color-coded parallel coordinates plot for the wine data will give some idea of how well correlated the attributes are with the targets. Listing 2-14 shows the code for producing that plot. Figure 2-17 shows the resulting parallel coordinates plot. The plot in Figure 2-17 suffers from compressing the graph along the variable directions that have smaller scale values.

To overcome this limitation, Listing 2-14 normalizes the wine data and re-plots it. Figure 2-18 shows the resulting parallel coordinates plot.

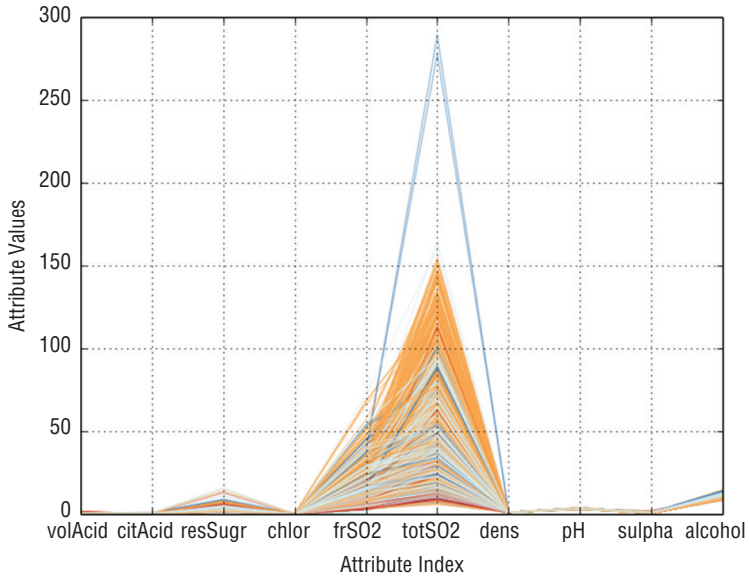


Figure 2-17: Parallel coordinate plot for wine data

Listing 2-14: Producing a Parallel Coordinate Plot for Wine Data—wineParallelPlot.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
from pylab import *
import matplotlib.pyplot as plot
from math import exp

target_url = "http://archive.ics.uci.edu/ml/machine-learning-databases/
wine-quality/winequality-red.csv"
wine = pd.read_csv(target_url,header=0, sep=";")

#generate statistical summaries
summary = wine.describe()
nrows = len(wine.index)
tasteCol = len(summary.columns)
meanTaste = summary.iloc[1,tasteCol - 1]
sdTaste = summary.iloc[2,tasteCol - 1]
nDataCol = len(wine.columns) -1

for i in range(nrows):
    #plot rows of data as if they were series data
    dataRow = wine.iloc[i,1:nDataCol]
    normTarget = (wine.iloc[i,nDataCol] - meanTaste)/sdTaste
    labelColor = 1.0/(1.0 + exp(-normTarget))
```

continues

continued

```
    dataRow.plot(color=plot.cm.RdYlBu(labelColor), alpha=0.5)

plot.xlabel("Attribute Index")
plot.ylabel(("Attribute Values"))
plot.show()

wineNormalized = wine
ncols = len(wineNormalized.columns)

for i in range(ncols):
    mean = summary.iloc[1, i]
    sd = summary.iloc[2, i]
    wineNormalized.iloc[:,i:(i + 1)] =
    (wineNormalized.iloc[:,i:(i + 1)] - mean) / sd

#Try again with normalized values
for i in range(nrows):
    #plot rows of data as if they were series data
    dataRow = wineNormalized.iloc[i,1:nDataCol]
    normTarget = wineNormalized.iloc[i,nDataCol]
    labelColor = 1.0/(1.0 + exp(-normTarget))
    dataRow.plot(color=plot.cm.RdYlBu(labelColor), alpha=0.5)

plot.xlabel("Attribute Index")
plot.ylabel(("Attribute Values"))
plot.show()
```

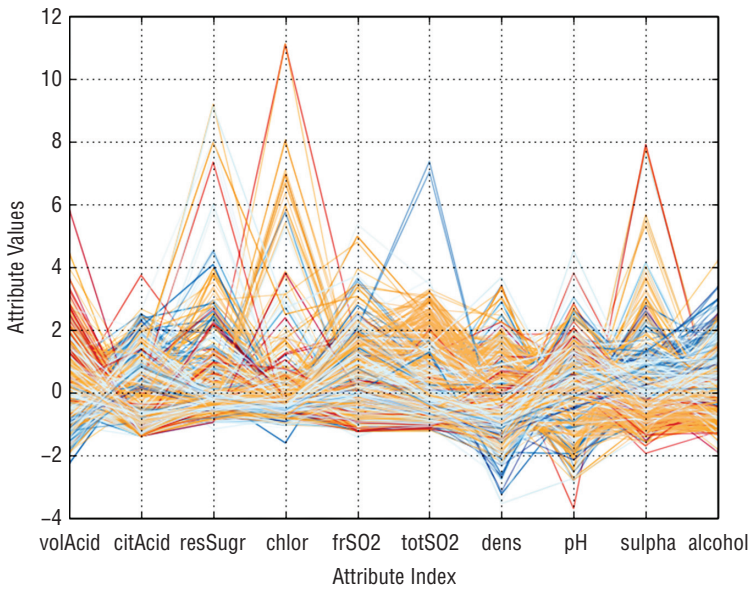


Figure 2-18: Parallel coordinates plot for normalized wine data

The plot of the normalized wine data gives a better simultaneous view of the correlation with the targets along all the coordinate directions. Figure 2-18 shows a clear correlation between several of the attributes. On the far right of the plot, dark blue lines (high taste scores) aggregate at high values of alcohol. On the far left, the dark red lines (low taste scores) aggregate at high values of volatile acidity. Those are the most obviously correlated attributes. The predictive models that you'll see in Chapters 5 and 7 will rank attributes on the basis of their importance in generating predictions. You'll see how these visual observations are supported by the predictive models.

Figure 2-19 shows the heat map of the correlations between attributes and other attributes and between the attributes and the target. In the heat map, hot colors correspond to high levels (the opposite of the color scale used in the parallel coordinates plots). The heat map for the wine data shows relatively high correlation between taste (the last column) and alcohol (the next-to-last column), and very low levels (high correlation but with negative sign) for several of the other attributes, including the first one (volatile acidity).

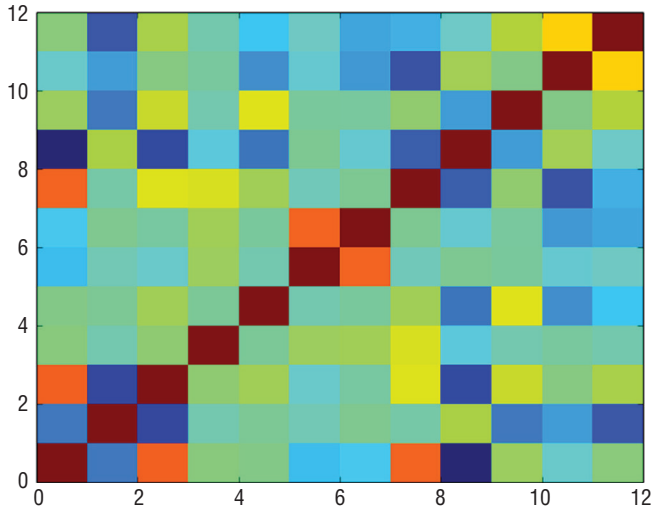


Figure 2-19: Correlation heat map for the wine data

Exploration of the wine data set was accomplished with tools that have already been explained and used. The wine data set shows off what these tools can reveal. Both the parallel coordinates plot and the correlation heat map show that high levels of alcohol go with high taste scores, while high levels of volatile acidity go with low taste scores. You'll see in Chapters 5 and 7 that the variable importance studies that come as part of predictive modeling will echo these findings. The wine data gives a good example of how far data exploration can

take you toward building and qualifying a predictive modeling. The next section will explore data for a multiclass classification problem.

Multiclass Classification Problem: What Type of Glass Is That?

Multiclass classifications are similar to binary classifications, with the difference that there are several possible discrete outcomes instead of just two. Recall that the problem of detecting unexploded mines involved two possible outcomes: that the object being illuminated by the sonar was a rock or that it was a mine. The problem of determining wine taste from measurements of chemical composition had several possible outcomes (taste scores from 3 to 8). But with the wine problem, an order relationship existed among the scores. A wine that had a score of 5 was better than one with a score of 3, but worse than one with a score of 8. With a multiclass problem, no sense of order exists among the outcomes. The glass problem described in this section provides an example of a multiclass problem.

In this section, the glass problem presents chemical compositions of various types of glass. The objective of the problem is to determine the use for the glass. The possible types of glass include glass from building windows, glass from vehicle windows, glass containers, and so on. The motivation for determining the type of glass is forensics. At the scene of an accident or a crime, there are fragments of glass, and determining their origin can help determine who is at fault or who committed the crime. Listing 2-15 shows the code for generating summaries of the glass data set. Figure 2-20 shows the box plot on the normalized data. The box plot shows a fair number of extreme values.

Listing 2-15: Summary of Glass Data Set—glassSummary.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
from pylab import *
import matplotlib.pyplot as plot

target_url = ("https://archive.ics.uci.edu/ml/machine-"
              "learning-databases/glass/glass.data")

glass = pd.read_csv(target_url,header=None, prefix="V")
glass.columns = ['Id', 'RI', 'Na', 'Mg', 'Al', 'Si',
                 'K', 'Ca', 'Ba', 'Fe', 'Type']

print(glass.head())

#generate statistical summaries
summary = glass.describe()
```

```
print(summary)
ncol1 = len(glass.columns)

glassNormalized = glass.iloc[:, 1:ncol1]
ncol2 = len(glassNormalized.columns)
summary2 = glassNormalized.describe()

for i in range(ncol2):
    mean = summary2.iloc[1, i]
    sd = summary2.iloc[2, i]

glassNormalized.iloc[:,i:(i + 1)] = \
        (glassNormalized.iloc[:,i:(i + 1)] - mean) / sd

array = glassNormalized.values
boxplot(array)
plot.xlabel("Attribute Index")
plot.ylabel(("Quartile Ranges - Normalized "))
show()

Output:  [filename - ]
print(glass.head())
```

	Id	RI	Na	Mg	Al	Si	K	Ca	Ba	Fe	Type
0	1	1.52101	13.64	4.49	1.10	71.78	0.06	8.75	0	0	1
1	2	1.51761	13.89	3.60	1.36	72.73	0.48	7.83	0	0	1
2	3	1.51618	13.53	3.55	1.54	72.99	0.39	7.78	0	0	1
3	4	1.51766	13.21	3.69	1.29	72.61	0.57	8.22	0	0	1
4	5	1.51742	13.27	3.62	1.24	73.08	0.55	8.07	0	0	1

```
print(summary) - Abridged
```

	Id	RI	Na	Mg	Al
count	214.000000	214.000000	214.000000	214.000000	214.000000
mean	107.500000	1.518365	13.407850	2.684533	1.444907
std	61.920648	0.003037	0.816604	1.442408	0.499270
min	1.000000	1.511150	10.730000	0.000000	0.290000
25%	54.250000	1.516523	12.907500	2.115000	1.190000
50%	107.500000	1.517680	13.300000	3.480000	1.360000
75%	160.750000	1.519157	13.825000	3.600000	1.630000
max	214.000000	1.533930	17.380000	4.490000	3.500000

	K	Ca	Ba	Fe	Type
count	214.000000	214.000000	214.000000	214.000000	214.000000
mean	0.497056	8.956963	0.175047	0.057009	2.780374
std	0.652192	1.423153	0.497219	0.097439	2.103739
min	0.000000	5.430000	0.000000	0.000000	1.000000
25%	0.122500	8.240000	0.000000	0.000000	1.000000
50%	0.555000	8.600000	0.000000	0.000000	2.000000
75%	0.610000	9.172500	0.000000	0.100000	3.000000
max	6.210000	16.190000	3.150000	0.510000	7.000000

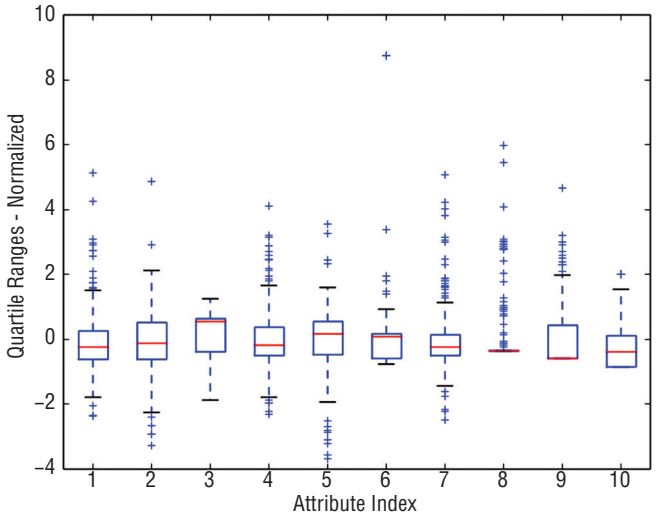


Figure 2-20: Box plot of the glass data

The box plot of the glass data attributes shows a remarkable number of outliers—remarkable at least by comparison to some of the other example problems. The glass data have a couple of elements that may drive the outlier behavior. One is that the problem is a classification problem. There's not necessarily any continuity in relationship between attribute values and class membership—no reason to expect proximity of attribute values across classes. Another unique feature of the glass data is that it is somewhat unbalanced. The number of examples of each class runs from 76 for the most populous class to 9 for the least populous. The average statistics can be dominated by the values for the most populous classes and there's no reason to expect members of other classes to have similar attribute values. The radical behavior can be a good thing for distinguishing classes from one another, but it also means that a method for making predictions has to be able to trace a fairly complicated boundary between the different classes. You'll learn in Chapter 3 that ensemble methods are producing more complicated decision boundaries than penalized linear regression if they are given enough data, and you'll see in Chapters 5 and 7 which family performs better on this data set.

The parallel coordinates plot might shed some more light on the behavior of these data. Figure 2-21 shows the parallel coordinates plot. The data is plotted using discrete colors for each possible output classification. Some of the variables in the plot show fairly distinct paths of color. For example, the dark blue lines group together fairly well and are well separated from the other classes along a number of the attributes. The dark blue lines are at the edges of the data for several attributes—in other words, outliers along those attributes. The light blue lines are less numerous than the dark blue ones and are at the edges for

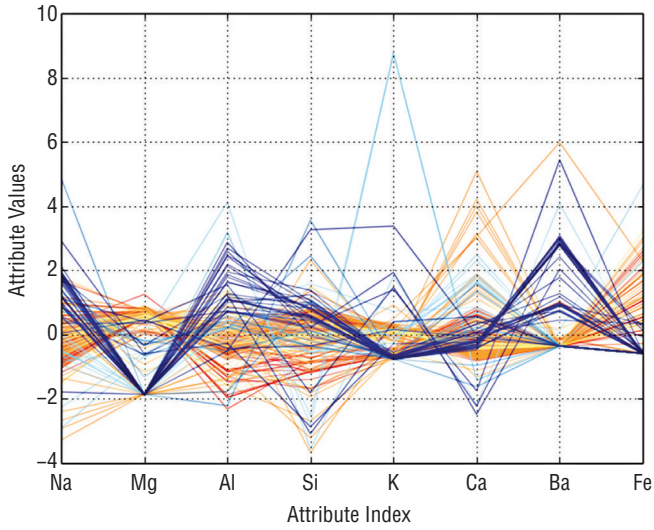


Figure 2-21: Parallel coordinate plot for the glass data

some of the same attributes as dark blue, but not for all of the same attributes. The middle brown lines also group together but toward the mid-range in value.

Listing 2-16: Parallel Coordinate Plot for the Glass Data—glassParallelPlot.py

```
__author__ = 'mike_bowles'
import pandas as pd
from pandas import DataFrame
from pylab import *
import matplotlib.pyplot as plot

target_url = ("https://archive.ics.uci.edu/ml/machine-"
              "learning-databases/glass/glass.data")

glass = pd.read_csv(target_url,header=None, prefix="V")
glass.columns = ['Id', 'RI', 'Na', 'Mg', 'Al', 'Si',
                 'K', 'Ca', 'Ba', 'Fe', 'Type']

glassNormalized = glass
ncols = len(glassNormalized.columns)
nrows = len(glassNormalized.index)
summary = glassNormalized.describe()
nDataCol = ncols - 1

#normalize except for labels
for i in range(ncols - 1):
    mean = summary.iloc[1, i]
    sd = summary.iloc[2, i]

glassNormalized.iloc[:,i:(i + 1)] = \
```

continues

continued

```
        (glassNormalized.iloc[:,i:(i + 1)] - mean) / sd

#Plot Parallel Coordinate Graph with normalized values
for i in range(nrows):
    #plot rows of data as if they were series data
    dataRow = glassNormalized.iloc[i,1:nDataCol]
    labelColor = glassNormalized.iloc[i,nDataCol]/7.0
    dataRow.plot(color=plot.cm.RdYlBu(labelColor), alpha=0.5)

plot.xlabel("Attribute Index")
plot.ylabel(("Attribute Values"))
plot.show()
```

Listing 2-16 shows the code to produce a parallel coordinates plot of the glass data. With the rocks versus mines problem, the lines in the parallel coordinates plot were two-colored to account for the two different label values. In the regression problems (wine taste and abalone age), the labels could take any real value, and the lines in the plots were drawn in a spectrum of different colors. In this multiclass problem, each class gets its own color. There are six discrete colors. The labels run from 1 to 7; there are no 4s. The calculation of the color is similar to the calculation done in the regression problem—divide the numeric label by its maximum value. The resulting lines in the plots take six discrete colors. Figure 2-22 shows the correlation heat map for the glass data. The plot shows mostly low correlation between attributes. That means the attributes are mostly independent of one another, which is a good thing. The targets are not included in the correlation map because the problem has targets that take on one of several discrete values. This robs the correlation heat map of some explanatory power.

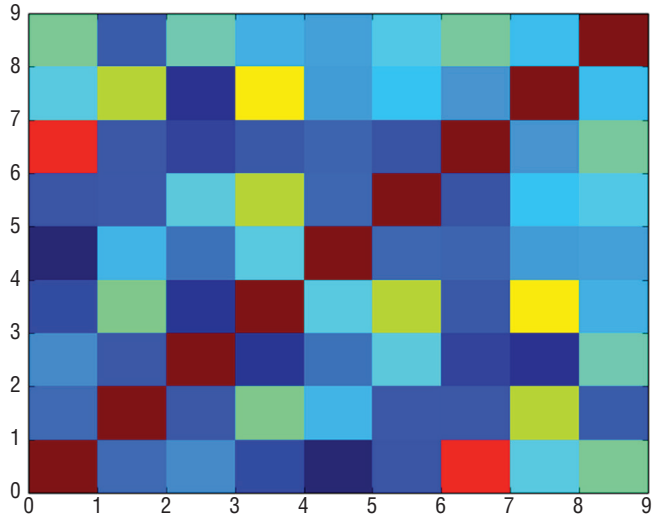


Figure 2-22: Correlation heat map for the glass problem

Exploratory studies for the glass data have revealed a very interesting problem. In particular, the box plots, coupled with the parallel coordinates plot, suggest that a good choice of algorithm might be an ensemble method if there's enough data to fit it. The sets of attributes corresponding to one class or another apparently have a complicated boundary between them. What algorithm will give the best predictive performance remains to be seen. The exploratory methods you have learned have done their job. They have given a good understanding of the tradeoffs for this problem, leading to some guesses about what algorithm will give the best performance.

Summary

This chapter introduced you to several tools for delving into new data sets and coming away with an understanding of how to proceed to building predictive models. The tools began with simply learning the size and shape of the data set and determining the types of attributes and labels. These facts about your data set will help you set your course through preprocesing the data and training predictive models. The chapter also covered several different statistical studies that can help you understand your data set. These included simple descriptive statistics (mean, variance, and quantiles) and second order statistics like correlations between attributes and correlations between attributes and labels. The correlation of attributes and binary labels required some techniques different from real number (regression labels). The chapter also introduced several visualization techniques. One was a Q-Q plot for visualizing outlier behavior in your data. Another was the parallel coordinates plot for visualizing the relationship between attributes and labels. All of these were applied to the problems that will be used in the rest of the book for demonstrating the algorithms covered and for comparing them.

Reference

1. Gorman, R. P., and Sejnowski, T. J. (1988). UCI Machine Learning Repository. https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+%28Sonar, +Mines+vs.+Rocks%29. Irvine, CA: University of California, School of Information and Computer Science.

CHAPTER

3

Predictive Model Building: Balancing Performance, Complexity, and Big Data

This chapter discusses the factors affecting the performance of machine learning models. The chapter provides technical definitions of *performance* for different types of machine learning problems. In an e-commerce application, for example, good performance might mean returning correct search results or presenting ads that site visitors frequently click. In a genetic problem, it might mean isolating a few genes responsible for a heritable condition. The chapter describes relevant performance measures for these different problems.

The goal of selecting and fitting a predictive algorithm is to achieve the best possible performance. Achieving performance goals involves three factors: complexity of the problem, complexity of the algorithmic model employed, and the amount and richness of the data available. The chapter includes some visual examples that demonstrate the relationship between problem and model complexity and then provides technical guidelines for use in design and development.

The Basic Problem: Understanding Function Approximation

The algorithms covered in this book address a specific class of predictive problem. The problem statement for these problems has two types of variables:

- The variable that you are attempting to predict (for example, whether a visitor to a website will click an ad)
- Other variables (for example, the visitor's demographics or past behavior on the site) that you can use to make the prediction

Problems of this type are referred to as *function approximation problems* because the goal is to construct a model generating predictions of the first of these as a function of the second.

In a function approximation problem, the designer starts with a collection of historical examples for which the correct answer is known. For example, historical web log files will indicate whether a visitor clicked an ad when shown the ad. The data scientist next has to find other data that can be used to build a predictive model. For example, to predict whether a site visitor will click an ad, the data scientist might try using other pages that the visitor viewed before seeing the ad. If the user is registered with the site, data on past purchases or pages viewed might be available for making a prediction.

The variable being predicted is referred to by a number of different names, such as *target*, *label*, and *outcome*. The variables being used to make the predictions are variously called *predictors*, *regressors*, *features*, and *attributes*. These terms are used interchangeably in this text, as they are in general practice. Determining what attributes to use for making predictions is called *feature engineering*. Data cleaning and feature engineering take 80 percent to 90 percent of a data scientist's time.

Feature engineering usually requires a manual, iterative process for selecting features, determining optimal potential, and experimenting with different combinations of features. The algorithms covered in this book assign numeric importance values to each attribute. These values indicate the relative importance of attributes in making predictions. That information helps speed up the feature engineering process.

Working with Training Data

The data scientist starts algorithm development with a training set. The training set consists of outcome examples and the assemblage of features chosen by the data scientist. The training set comprises two types of data:

- The outcomes you want to predict
- The features available for making the prediction

Table 3-1 provides an example of a training set. The leftmost column contains outcomes (whether a site visitor clicked a link) and features to be used to make predictions about whether visitors will click the link in the future.

Table 3-1: *Example Training Set*

OUTCOMES: CLICKED ON LINK	FEATURE1: GENDER	FEATURE2: MONEY SPENT ON SITE	FEATURE3: GENDER
Yes	M	0	25
No	F	250	32
Yes	F	12	17

The predictor values (a.k.a., features, attributes, and so on) can be arranged in the form of a matrix (see Equation 3-1). The notational convention used in this book is as follows. The table of predictors will be called *X*, and it has the following form:

$$X = \begin{matrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{matrix}$$

Equation 3-1: Notation for set of predictors

Referring to the data set in Table 3-1, x_{11} would be M (gender), x_{12} would be 0.00 (money spent on site), x_{21} would be F (gender), and so on.

Sometimes it will be convenient to refer to all the attribute values for a particular example. For that purpose, x_i (with a single index) will refer to the *i*th row of X. For the data set in Table 3-1, x_2 would be a row vector containing the values F, 250, 32.

Strictly speaking, the X is not a matrix because the predictors may not all be the same type of variable. (A proper matrix contains variables that are all the same type. Predictors, however come in different types.) Using the example of predicting ad clicks, the predictors might include demographic data about the site visitor. Those data could include marital status and yearly income, among other things. Yearly income is a real number, and marital status is a categorical variable. That means that marital status does not admit arithmetic operations such as adding or multiplication and that no order relation exists between *single*, *married*, and *divorced*. The entries in a column from X all have the same type, but the type may vary from one column to the next.

Attributes such as marital status, gender, or the state of residence go by several different designations. They may be called *factor* or *categorical*. Attributes like age or income that are represented by numbers are called *numeric* or *real-valued*.

The distinction between these two types of attributes is important because some algorithms may not handle one type or the other. For example, linear methods, including the ones covered in this book, require numeric attributes. (Chapter 4, "Penalized Linear Regression," which covers linear methods, shows methods for converting [or coding] categorical variables to numeric in order to apply linear methods to problems with categorical variables.)

The targets corresponding to each row in X are arranged in a column vector Y (see Equation 3-2), as follows:

$$Y = \begin{matrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{matrix}$$

Equation 3-2: Notation for vector of targets

The target y_i corresponds to x_i—the predictors in the *i*th row of X. Referring to the data in Table 3-1, y_1 is Yes, and y_2 is No.

Targets may be of several different forms. For example, they may be real numbers, like if the objective were to predict how much a customer will spend. When the targets are real numbers, the problem is called a *regression problem*. Linear regression implies using a linear method to solve a regression problem. (This book covers both linear and nonlinear regression methods.)

If the targets are two-valued, as in Table 3-1, the problem is called a *binary classification problem*. Predicting whether a customer will click an advertisement is a binary classification problem. If the targets contain several discrete values, the problem is a *multiclass classification problem*. Predicting which of several ads a customer will click would be a multiclass classification problem.

The basic problem is to find a prediction function, `pred()`, that uses the attributes to predict outcomes (see Equation 3-3):

$$y_t \sim pred(x_t)$$

Equation 3-3: Basic equation for making predictions

The function `pred()` uses the attribute x_i to predict y_i. This book describes some of the very best current methods for producing the function `pred()`.

Assessing Performance of Predictive Models

Good performance means using the attributes x_i to generate a prediction that is close to y_i, but *close* has different meanings for different problems. For a regression problem where y_i is a real number, performance is measured in terms like the mean squared error (MSE) or the mean absolute error (MAE) (see Equation 3-4).

$$Mean\,squared\,error = \left(\frac{1}{m}\right)\sum_{i=1}^{m}\left(y_i - pred\left(x_i\right)\right)^2$$

Equation 3-4: Performance measure for a regression problem

In a regression problem, the target (y_i) and the prediction, pred(x_i), are both real numbers, so it makes sense to describe the error as being the numeric difference between them. Equation 3-4 for MSE squares the errors and averages over the data set to produce a measure of the overall level of errors. MAE averages the absolute values of the errors (see Equation 3-5) instead of averaging the squares of the errors.

$$Mean\,absolute\,error = \left(\frac{1}{m}\right)\sum_{i=1}^{m}\left|y_i - pred\left(x_i\right)\right|$$

Equation 3-5: Another performance measure for regression

If the problem is a classification problem, you must use some other measure of performance. One of the most used is the misclassification error—that is, the fraction of examples that the function `pred()` predicts incorrectly. The section "Performance Measures for Different Types of Problems" shows how to calculate misclassification.

For our function `pred()` to be useful for making predictions, there must be some way to predict what level of errors it will generate on new examples as they arrive. What is the performance on new data—data that were not involved in developing the function `pred()`? This chapter covers the best methods for estimating performance on new data.

This section introduced the basic type of prediction problem that will be addressed in this book and described how constructing these prediction models amounts to constructing a function that maps attributes (or features) into predicted outcomes. It also gave an overview of how the errors in these predictions can be assessed. Performing these steps leads to several complications. The remaining sections of this chapter describe these complications, how to deal with them, and how to arrive at the best possible model given the constraints of the problem and the data available.

Factors Driving Algorithm Choices and Performance—Complexity and Data

Several factors affect the overall performance of a predictive algorithm. Among these factors are the complexity of the problem, the complexity of the model used, and the amount of training data available. The following sections describe how these factors interrelate to determine performance.

Contrast Between a Simple Problem and a Complex Problem

The preceding section of this chapter described several ways to quantify performance and highlighted the importance of performance on new data. The goal of designing a predictive model is to make accurate predictions on new examples (such as new visitors to your site). As a practicing data scientist, you will want an estimate of an algorithm's performance so that you can set expectations with your customer and compare algorithms with one another. Best practice in predictive modeling requires that you hold out some data from the training set. These held-out examples have labels associated with them and can be compared to predictions produced by models training on the remaining data. Statisticians refer to this technique as *out-of-sample error* because it is an error on data not used in training. (The section "Measuring Performance of Predictive Models" later in this chapter goes into more detail about the mechanics of this process.) The important thing is that the only performance that counts is the performance of the model when it is run against new examples.

One of the factors affecting performance is the complexity of the problem being solved. Figure 3-1 shows a relatively simple classification problem in two dimensions. There are two groups of points: dark and light points. The dark points are randomly drawn from a 2D Gaussian distribution centered at (1,0) with unit variance in both dimensions. The light points are also drawn from a Gaussian distribution having the same variance but centered at (0,1). The

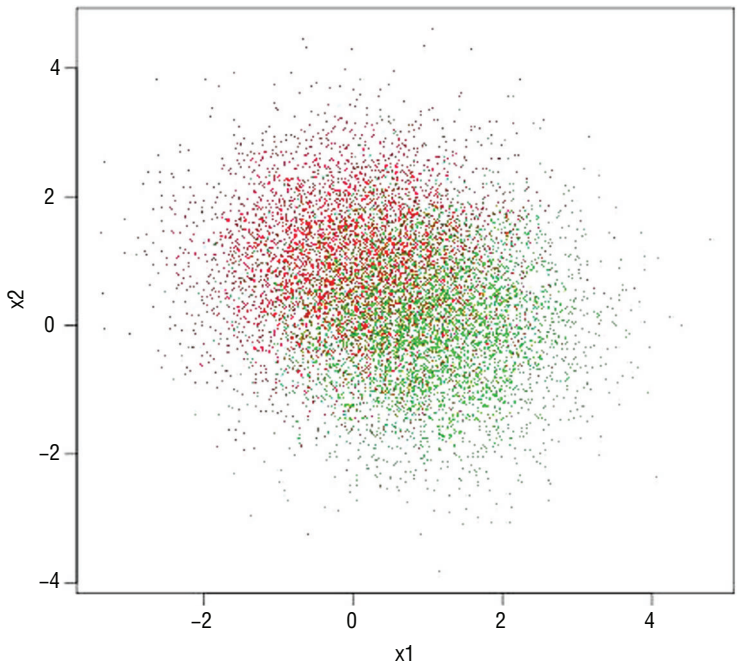


Figure 3-1: A simple classification problem

attributes for the problem are the two axes in the plot: x_1 and x_2. The classification task is to draw some boundaries in the x_1, x_2 plane to separate the light points from the dark points. About the best that can be done in this circumstance is to draw a 45-degree line in the plot—that is the line where x_1 equals x_2. In a precise probabilistic sense, that is the best possible classifier for this problem. Because a straight line separates the lights and darks as well as possible, a linear classifier will do as well as nonlinear classifier. The linear methods covered in this book will do a splendid job on this problem.

Figure 3-2 depicts a more complicated problem. The points shown in Figure 3-2 are generated by drawing points at random. The main difference from the random draw that generated Figure 3-1 is that the points in Figure 3-2 are drawn from several distributions for the light points and several different ones for dark. This is called a *mixture model*. The general goal is basically the same: draw boundaries in the x_1, x_2 plane to separate the light points from the dark points. In Figure 3-2, however, it is clear that a linear boundary will not separate the points as well as a curve. The ensemble methods covered in Chapter 6, "Ensemble Methods," will work well on this problem.

However, complexity of the decision boundaries is not the only factor influencing whether linear or nonlinear methods will deliver better performance. Another important factor is the size of the data set. Figure 3-3 illustrates this element of performance. The points plotted in Figure 3-3 are a 1 percent subsample of data plotted in Figure 3-2.

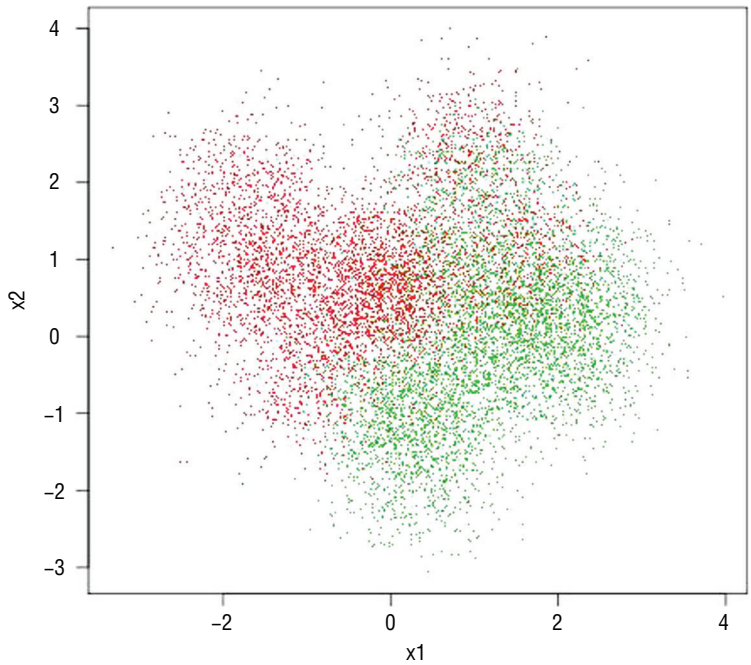


Figure 3-2: A complicated classification problem

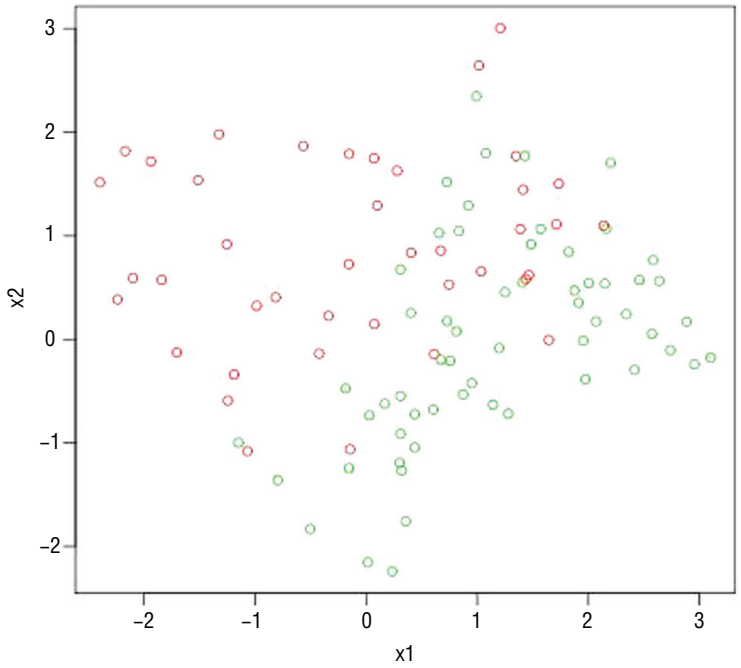


Figure 3-3: A complicated classification problem without much data

In Figure 3-2, there was enough data to visualize the curved boundaries delineating the sets of light and dark points. Without as much data, the sets are not so easily discerned visually, and in this circumstance, a linear model may give equal or better performance than a nonlinear model. With less data, the boundaries are harder to visualize, and they are more difficult to compute. This gives a graphic demonstration of the value of having a large volume of data. If the underlying problem is complicated (for example, personalizing responses for individual shoppers), a complicated model with a lot of data can produce accurate results. However, if the model is not complicated, as in Figure 3-1, or there is not sufficient data, as in Figure 3-3, a linear model may produce the best answer.

Contrast Between a Simple Model and a Complex Model

The previous section showed visual comparisons between simple and complex problems. This section describes how the various models available to solve these problems differ from one another. Intuitively, it seems that a complex model should be fit to a complex problem, but the visual example from the last section demonstrates that data set size may dictate that a simple model fits a complex problem better than a complex model.

Another important concept is that modern machine learning algorithms generate families of models, not just single models. The algorithms covered

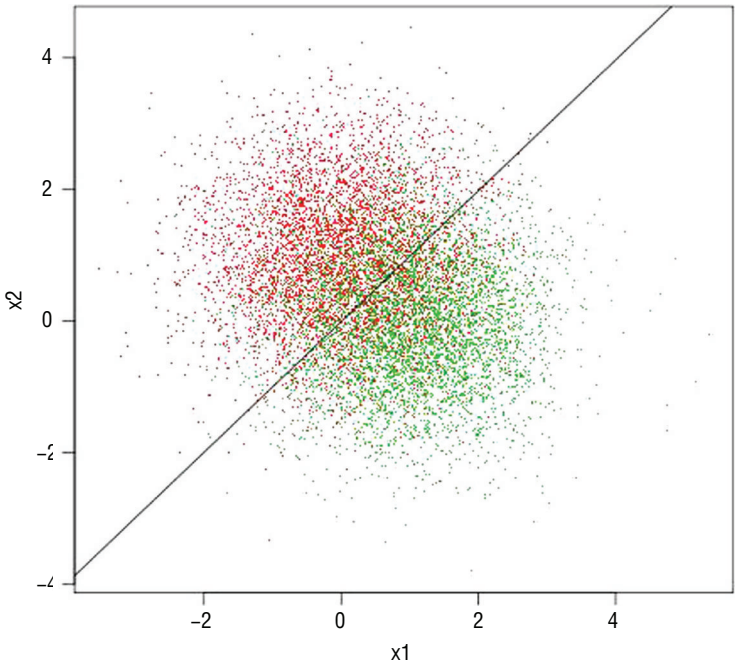


Figure 3-4: Linear model fit to simple data

in this book each generate hundreds or even thousands of different models. Generally, the ensemble methods covered in Chapter 6 yield more complex models than linear methods covered in Chapter 4, but both of these methods generate multiple models of varying complexity. (This will become clearer in Chapters 4 and 6, which cover linear and ensemble techniques in detail.)

Figure 3-4 shows a linear model fit to the simple problem introduced in the previous section. The linear model shown in Figure 3-4 was generated using the glmnet algorithm (covered in Chapter 4). The linear model fit to these data divides the data roughly in half. The line in the figure is given by Equation 3-6.

$$x_2 = -0.01 + 0.99x_1$$

Equation 3-6: Linear model fit to simple problem

This is very close to the line where x_2 equals x_1, which is the best possible boundary in a probabilistic sense. The boundary appears sensible from a visual intuitive standpoint. Fitting a more complicated model to this simple problem is not going to improve performance.

A more complicated problem with more complicated decision boundaries gives a complicated model an opportunity to outperform a simple linear model.

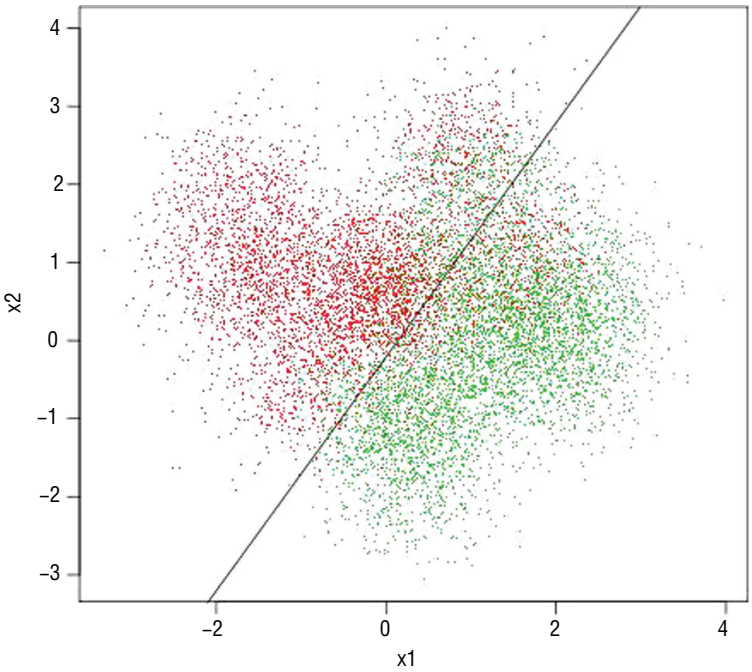


Figure 3-5: Linear model fit to complex data

Figure 3-5 shows a linear model fit to data indicating a nonlinear decision boundary. In this circumstance, the linear model misclassifies regions as dark when they should be light and vice versa.

Figure 3-6 shows how much better a complicated model can do with complicated data. The model used to generate this decision boundary is an ensemble (collection) of 1,000 binary decision trees constructed using the gradient boosting algorithm. (Gradient boosted decision trees are covered in detail in Chapter 6 on ensemble methods.) The nonlinear decision boundary curves are used to better delineate regions where the dark points are denser and regions where the light points are denser.

It is tempting to draw the conclusion that the best approach is to use complicated models for complicated problems and simple models for simple problems. But, you must consider one more dimension to the problem. As mentioned in the previous section, you must consider data set size. Figures 3-7 and 3-8 show 1 percent of the data from a complicated problem. Figure 3-7 shows a linear model fit to the data, and Figure 3-8 shows an ensemble model fit to the data. Count the number of points that are misclassified. There are 100 points in the data set. The linear model in Figure 3-7 misclassifies 11 points, for a misclassification error rate of 11 percent. The complex model misclassifies 8, for an 8 percent error rate. Their performance is roughly equal.

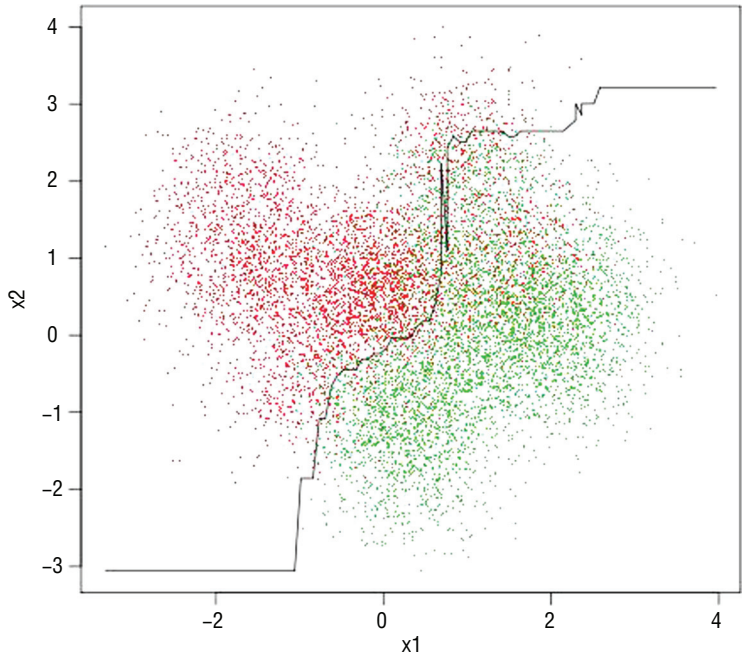


Figure 3-6: Ensemble model fit to complex data

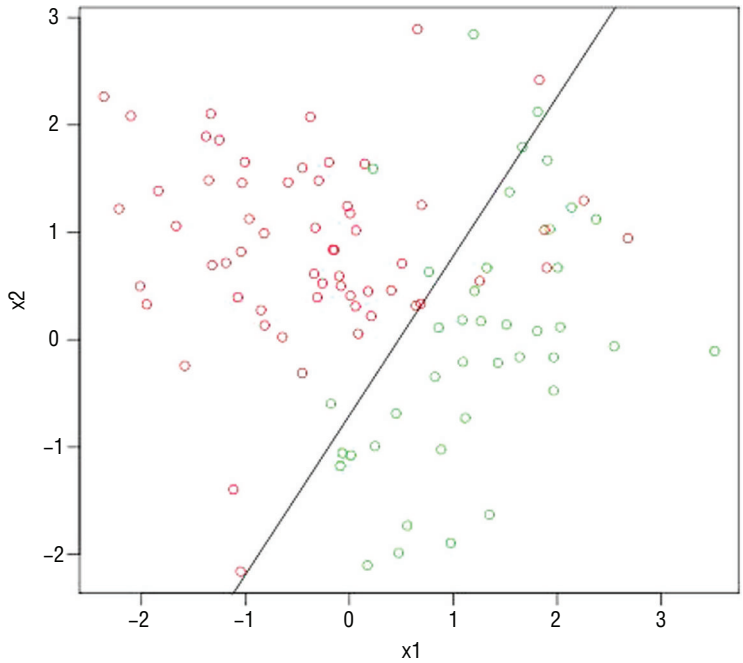


Figure 3-7: Linear model fit to small sample of complex data

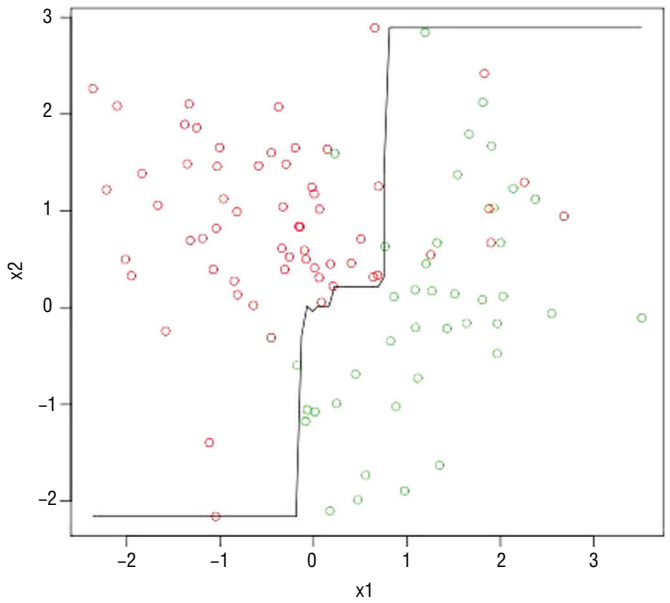


Figure 3-8: Ensemble model fit to small sample of complex data

Factors Driving Predictive Algorithm Performance

These results explain the excitement over large volumes of data. Accurate predictions for complicated problems require large volumes of data. But the size isn't quite a precise enough measure. The shape of the data also matters.

Equation 3-1 portrayed predictor data as a matrix having a number of rows (height) and a number of columns (width). The number of entries in the matrix is the product of the number of rows and the number of columns. An important difference exists between the number of rows and the number of columns when the data are being used for predictive modeling. Adding a column means adding a new attribute. Adding a new row means getting an additional historical example of the existing attributes. To understand how the effects of a new row differ from the effects of a new column, consider a linear model relating the attributes from Equation 3-1 to the labels of Equation 3-2.

Assume a model of the following form (see Equation 3-7):

$$y_i \sim x_i * \beta$$
$$= x_{i1} * \beta 1 + x_{i2} * \beta 2 + \ldots + x_i m * \beta m$$

Equation 3-7: Linear relation between attributes and outcomes

Here, x_i is a row of attributes, and β is a column vector of coefficients to be determined. Adding a column to the matrix of attributes adds another β coefficient that needs to be determined. This added coefficient is also called *degree*

of freedom. Adding another degree of freedom is making the model more complicated. The preceding examples demonstrated that making the model more complicated required more data. For this reason, it is common to think in terms of the ratio of rows to columns—the aspect ratio.

Biological data sets and natural language processing data sets are examples that are quite large because they have a lot of columns, but they are sometimes not large enough to get good performance out of a complex modeling approach. In biology, genomic data sets can easily contain 10,000 to 50,000 attributes. Even with tens of thousands of individual experiments (rows of data), a genomic data set may not be enough to train a complex ensemble model. A linear model may give equivalent or better performance.

Genomic data are expensive. One of the experiments (rows) can cost upward of $5,000, making the full data set cost upward of $50 million. Text can be relatively inexpensive to collect and store, but can also be even wider than genomic data. In some natural language processing problems, the attributes are words, and rows are documents. Entries in the matrix of attributes are the number of times a word appears in a document. The number of columns is the vocabulary size for a document collection. Depending on preprocessing (for example, removing common words like *a*, *and*, and *of*), the vocabulary can be from a few thousand to a few tens of thousands. The attribute matrix for text becomes very wide when n-grams are counted alongside words. N-grams are groups of two, three, or four words that appear next to one another (or close enough to be a phrase). When groups of two, three, or four words are also counted, the attribute space for natural language processing can grow to more than a million attributes. Once again, a linear model may give equivalent or better performance than a more complicated ensemble model.

Choosing an Algorithm: Linear or Nonlinear?

The visual examples you have just seen give some idea of the performance tradeoffs between linear and nonlinear predictive models. Linear models are preferable when the data set has more columns than rows or when the underlying problem is simple. Nonlinear models are preferable for complex problems with many more rows than columns of data. An additional factor is training time. Fast linear techniques train much faster than nonlinear techniques. (You will have more of a basis for making this decision after you've covered the techniques described in Chapter 4 and Chapter 6 and have worked through some examples.)

Choosing a nonlinear model (say an ensemble method) entails training a number of different models of differing complexity. For example, the ensemble model that generated the decision boundary in Figure 3-6 was one of roughly a thousand different models generated during the training process. These models had a variety of different complexities. Some of them would have given a much cruder approximation to the boundaries that are visually apparent in Figure 3-6. The model that generated the decision boundary in Figure 3-6 was chosen because it performed the best on out-of-sample data. This process holds for many modern machine learning

algorithms. Examples will be covered in covered in the section "Choosing a Model to Balance Problem Complexity, Model Complexity, and Data Set Size."

This section has used data sets and classifier solutions that can be visualized in order to give you an intuitive grasp of the factors affecting the performance of the predictive models you build. Generally, you'll use numeric measures of performance instead of relying on pictures. The next section describes the methods and considerations for producing numeric performance measures for predictive models and how to use these to estimate the performance your models will achieve when deployed.

Measuring the Performance of Predictive Models

This section covers two broad areas relating to performance measures for predictive models. The first one is the different metrics that you can use for different types of problems (for example, using MSE for a regression problem and misclassification error for a classification problem). In the literature (and in machine learning competitions), you will also see measures like receiver operating curves (ROC curves) and area under the curve (AUC). Besides that, these ideas are useful for optimizing performance.

The second broad area consists of techniques for gathering out-of-sample error estimates. Recall that out-of-sample errors are meant to simulate errors on new data. It's an important part of design practice to use these techniques to compare different algorithms and to select the best model complexity for a given problem complexity and data set size. That process is discussed in detail later in this chapter and is then used in examples throughout the rest of the book.

Performance Measures for Different Types of Problems

Performance measures for regression problems are relatively straightforward. In a regression problem, both the target and the prediction are real numbers. Error is naturally defined as the difference between the target and the prediction. It is useful to generate statistical summaries of the errors for comparisons and for diagnostics. The most frequently used summaries are the mean squared error (MSE) and the mean absolute error (MAE). Listing 3-1 compares the calculation of the MSE, MAE, and root MSE (RMSE, which is the square root of MSE).

Listing 3-1: Comparison of MSE, MAE and RMSE—regressionErrorMeasures.py

```
__author__ = 'mike-bowles'


#here are some made-up numbers to start with
target = [1.5, 2.1, 3.3, -4.7, -2.3, 0.75]
prediction = [0.5, 1.5, 2.1, -2.2, 0.1, -0.5]
```

```
error = []
for i in range(len(target)):
    error.append(target[i] - prediction[i])

#print the errors
print("Errors ",)
print(error)
#ans:  [1.0, 0.60000000000000009, 1.1999999999999997, -2.5,
#-2.3999999999999999, 1.25]



#calculate the squared errors and absolute value of errors
squaredError = []
absError = []
for val in error:
    squaredError.append(val*val)
    absError.append(abs(val))


#print squared errors and absolute value of errors
print("Squared Error")
print(squaredError)
#ans: [1.0, 0.3600000000000001, 1.4399999999999993, 6.25,
#5.7599999999999998, 1.5625]
print("Absolute Value of Error")
print(absError)
#ans: [1.0, 0.60000000000000009, 1.1999999999999997, 2.5,
#2.3999999999999999, 1.25]


#calculate and print mean squared error MSE
print("MSE = ", sum(squaredError)/len(squaredError))
#ans: 2.72875


from math import sqrt
#calculate and print square root of MSE (RMSE)
print("RMSE = ", sqrt(sum(squaredError)/len(squaredError)))
#ans: 1.65189285367


#calculate and print mean absolute error MAE
print("MAE = ", sum(absError)/len(absError))
#ans: 1.49166666667


#compare MSE to target variance
targetDeviation = []
targetMean = sum(target)/len(target)
for val in target:
    targetDeviation.append((val - targetMean)*(val - targetMean))
```

continues

continued

```
#print the target variance
print("Target Variance = ", sum(targetDeviation)/len(targetDeviation))
#ans: 7.5703472222222219

#print the the target standard deviation (square root of variance)
print("Target Standard Deviation = ", sqrt(sum(targetDeviation)
   /len(targetDeviation)))
#ans: 2.7514263977475797
```

The example starts with some made-up numbers for the targets and the predictions. First, it calculates the errors by simple subtraction; then it shows the calculation of MSE, MAE, and RMSE. Notice that MSE comes out markedly different in magnitude than MAE and RMSE. That's because MSE is in squared units. For that reason, the RMSE is usually a more usable number to calculate. At the bottom of the listing is a calculation of the variance (mean squared deviation from the mean) and the standard deviation (square root of variance) of the targets. These quantities are useful to compare (respectively) to the MSE and RMSE of the prediction errors. For example, if the MSE of the prediction error is roughly the same as the target variance (or the RMSE is roughly the same as target standard deviation), the prediction algorithm is not performing well. You could replace the prediction algorithm with a simple calculation of the mean of the targets and perform as well. The errors in Listing 3-1 have RMSE that's about half the standard deviation of the targets. That is fairly good performance.

Besides calculating summary statistics for the error, it may sometimes be useful for analyzing sources and magnitudes of error to look at things like histogram of the error or tail behavior (quantile or decile boundaries), degree of normality, and so forth. Sometimes those investigations will yield insights into error sources and potential performance improvements.

Classification problems require different treatment. The approaches to classification problems generally revolve around misclassification error rates—the fraction of examples that are incorrectly classified. Suppose, for instance, that the classification problem is to predict click or not-click on a link being considered for presentation to a site visitor. Generally, algorithms for doing classification can present predictions in the form of a probability instead of a hard click versus not-click decision. The algorithms considered in this book all output probabilities.

Here's why that's useful. If the prediction of click or not-click is given as a probability—say 80 percent chance of click (and correspondingly 20 percent chance of not-click)—the data scientist has the option to use 50 percent as a threshold for presenting the link or not presenting the link. In some cases, however, a higher or lower threshold value will give a better end result.

Suppose, for example, that the problem is fraud detection (for credit cards, automatic clearinghouses [checking], insurance claims, and so on). The actions that proceed from making a fraud-or-not decision are to have a call center representative intervene in the transaction or to let it go. There are costs involved

with either decision. If the call is made, there's the call center cost and the cost of the customer's reaction. If the call isn't made, there's the cost of the potential fraud. If the costs of taking the action are very low relative to the costs of not taking the action, the minimum total comes at a relatively low threshold. More transactions get flagged for intervention.

But where do you draw the line for interrupting your customer's checkout and requiring the customer to call card services to proceed? Do you interrupt the transactions where your predictive algorithm indicates a 20 percent, 50 percent, or 80 percent probability that the transaction is fraudulent? If you place the threshold for interruption at 20 percent, you'll be intervening more frequently—preventing more fraudulent transactions—but also irritating more customers and keeping many call center reps busy. Maybe it is better to place the threshold higher (say 80 percent) and to accept more fraud.

A useful way to think about this is to arrange the possible outcomes into what is called a *confusion matrix* or *contingency table* (`http://en.wikipedia.org/wiki/Confusion_matrix`). Figure 3-9 shows a toy example of a contingency matrix. The numbers in the contingency table represent the performance based on a choice for the threshold value discussed in the last paragraph. The contingency matrix in Figure 3-9 summarizes the results of making predictions for 135 test examples for a particular choice of the threshold probability. The matrix has two columns representing the possible predictions. It also has two rows representing the truth (label) for each example. So, each example in the test set can be assigned to one of the four cells in the table. The two classifications portrayed in Figure 3-9 are "click" and "not click," appropriate for selecting an ad. These could also correspond to "fraud" and "not fraud"—or other pairs—depending on the specific problem being addressed.

The upper-left cell contains examples that are predicted as click and where that matches the label (truth). These are called *true positive* and are generally abbreviated as TP. The entries in the lower-left box correspond to examples where the prediction was positive (click) but the truth was negative (not-click).

	Predicted Class		
Actual Class		Positive (Click)	Negative (Not Click)
	Positive (Click)	True Positive 10	False Negative 7
	Negative (Not-click)	False Positive 22	True Negative 96

Figure 3-9: Confusion matrix example

These are called *false positive* and abbreviated as FP. The right column of the matrix contains the examples that were predicted not-click. The examples in the upper right were click in truth and are called *false negatives* or FN. The lower-right examples were predicted not-click and agree with the real outcome. They are called *false negative* or FN.

What happens when the probability threshold is changed? Consider the extreme values. If the probability threshold is set to 0.0, no matter what probability your model predicts, it will get designated as a click. All the examples wind up in the left column. There are only 0s in the right column. The number of TPs would go up to 17. The number of FPs would go up to 118. If there were no cost for an FP and no reward for a true negative (TN), that might be a good choice, but no predictive algorithm is required to assume click for every input example. Similarly, if there is no cost for an FN and no benefit for a TP, the threshold can be set at 1.0 so that all examples are classified as not-click. These extremes aid understanding, but they're not useful in a deployed system. The following example shows how the process would work to build a classifier for the rocks-versus-mines data set.

The rocks-versus-mines data set presents the problem of building a classifier that uses sonar data to determine whether seabed objects are rocks or mines. (For a more thorough discussion and exploration of the data set, see Chapter 2, "Understand the Problem by Understanding the Data.") Listing 3-2 shows the Python code for training a simple classifier on the rocks-versus-mines data set and then predicts performance for the classifier.

Listing 3-2: Measuring Performance for Classifier Trained on Rocks-Versus-Mines—classifierPerformance_RocksVMines.py

```
__author__ = 'mike-bowles'
#use scikit learn package to build classified on rocks-versus-mines data
#assess classifier performance

import urllib2
import numpy
import random
from sklearn import datasets, linear_model
from sklearn.metrics import roc_curve, auc
import pylab as pl


def confusionMatrix(predicted, actual, threshold):
    if len(predicted) != len(actual): return -1
    tp = 0.0
    fp = 0.0
    tn = 0.0
    fn = 0.0
    for i in range(len(actual)):
        if actual[i] > 0.5: #labels that are 1.0  (positive examples)
```

```
            if predicted[i] > threshold:
                tp += 1.0 #correctly predicted positive
            else:
                fn += 1.0 #incorrectly predicted negative
        else:              #labels that are 0.0 (negative examples)
            if predicted[i] < threshold:
                tn += 1.0 #correctly predicted negative
            else:
                fp += 1.0 #incorrectly predicted positive
    rtn = [tp, fn, fp, tn]
    return rtn


#read in the rocks versus mines data set from uci.edu data repository
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")
data = urllib2.urlopen(target_url)

#arrange data into list for labels and list of lists for attributes
xList = []
labels = []
for line in data:
    #split on comma
    row = line.strip().split(",")
    #assign label 1.0 for "M" and 0.0 for "R"
    if(row[-1] == 'M'):
        labels.append(1.0)
    else:
        labels.append(0.0)
    #remove lable from row
    row.pop()
    #convert row to floats
    floatRow = [float(num) for num in row]
    xList.append(floatRow)

#divide attribute matrix and label vector into training(2/3 of data)
#and test sets (1/3 of data)
indices = range(len(xList))
xListTest = [xList[i] for i in indices if i%3 == 0 ]
xListTrain = [xList[i] for i in indices if i%3 != 0 ]
labelsTest = [labels[i] for i in indices if i%3 == 0]
labelsTrain = [labels[i] for i in indices if i%3 != 0]

#form list of list input into numpy arrays to match input class
#for scikit-learn linear model
xTrain = numpy.array(xListTrain); yTrain = numpy.array(labelsTrain)
xTest = numpy.array(xListTest); yTest = numpy.array(labelsTest)

#check shapes to see what they look like
print("Shape of xTrain array", xTrain.shape)
print("Shape of yTrain array", yTrain.shape)
```

continues

continued

```
print("Shape of xTest array", xTest.shape)
print("Shape of yTest array", yTest.shape)

#train linear regression model
rocksVMinesModel = linear_model.LinearRegression()
rocksVMinesModel.fit(xTrain,yTrain)

#generate predictions on in-sample error
trainingPredictions = rocksVMinesModel.predict(xTrain)
print("Some values predicted by model", trainingPredictions[0:5],
 trainingPredictions[-6:-1])

#generate confusion matrix for predictions on training set (in-sample
confusionMatTrain = confusionMatrix(trainingPredictions, yTrain, 0.5)
#pick threshold value and generate confusion matrix entries
tp = confusionMatTrain[0]; fn = confusionMatTrain[1]
fp = confusionMatTrain[2]; tn = confusionMatTrain[3]

print("tp = " + str(tp) + "\tfn = " + str(fn) + "\n" + "fp = " +
str(fp) + "\ttn = " + str(tn) + '\n')

#generate predictions on out-of-sample data
testPredictions = rocksVMinesModel.predict(xTest)

#generate confusion matrix from predictions on out-of-sample data
conMatTest = confusionMatrix(testPredictions, yTest, 0.5)
#pick threshold value and generate confusion matrix entries
tp = conMatTest[0]; fn = conMatTest[1]
fp = conMatTest[2]; tn = conMatTest[3]
print("tp = " + str(tp) + "\tfn = " + str(fn) + "\n" + "fp = " +
str(fp) + "\ttn = " + str(tn) + '\n')

#generate ROC curve for in-sample

fpr, tpr, thresholds = roc_curve(yTrain,trainingPredictions)
roc_auc = auc(fpr, tpr)
print( 'AUC for in-sample ROC curve: %f' % roc_auc)

# Plot ROC curve
pl.clf()
pl.plot(fpr, tpr, label='ROC curve (area = %0.2f)' % roc_auc)
pl.plot([0, 1], [0, 1], 'k-')
pl.xlim([0.0, 1.0])
pl.ylim([0.0, 1.0])
pl.xlabel('False Positive Rate')
pl.ylabel('True Positive Rate')
pl.title('In sample ROC rocks versus mines')
pl.legend(loc="lower right")
pl.show()

#generate ROC curve for out-of-sample
fpr, tpr, thresholds = roc_curve(yTest,testPredictions)
```

```
roc_auc = auc(fpr, tpr)
print( 'AUC for out-of-sample ROC curve: %f' % roc_auc)

# Plot ROC curve
pl.clf()
pl.plot(fpr, tpr, label='ROC curve (area = %0.2f)' % roc_auc)
pl.plot([0, 1], [0, 1], 'k-')
pl.xlim([0.0, 1.0])
pl.ylim([0.0, 1.0])
pl.xlabel('False Positive Rate')
pl.ylabel('True Positive Rate')
pl.title('Out-of-sample ROC rocks versus mines')
pl.legend(loc="lower right")
pl.show()
```

The first section of the code reads the input data from the University of California Irvine data repository and then formats it as a list for the labels and a list of lists for the attributes. The next step is to break the data (labels and attributes) into two subsets: a test set that contains one third of the data, and a training set that contains the other two thirds. The data labeled *test* will not be used in training the classifier, but will be reserved for assessing performance after the classifier is trained. This step simulates the behavior of the classifier on new data examples after it has been deployed. Later, this chapter discusses a variety of different methods for holding out data and making estimates of performance on new data.

The classifier is trained by converting the labels M (for mine) and R (for rock) in the original data set into numeric values—1.0 corresponding to mine, and 0.0 corresponding to rock—and then using the ordinary least squares regression to fit a linear model. This is a fairly simple method to understand and to implement and will often generate very similar performance to the more sophisticated algorithms discussed later. The program in Listing 3-2 employs the linear regression class from scikit-learn to train the ordinary least squares model. Then the trained model is used to generate predictions on the training set and on the test set.

The code prints out some representative values for the predictions. The linear regression model generates numbers that are mostly in the interval between 0.0 and 1.0, but not entirely. The predictions aren't quite probabilities. They can still be used to generate predicted classifications by comparing to a threshold value. The function `confusionMatrix()` produces the values for a confusion matrix, similar to Figure 3-9. It takes the predictions, the corresponding actual values (labels), and a threshold value as input. It compares the predictions to the threshold to determine whether to assign each example to the "predicted positive" or "predicted negative" column in the confusion matrix. It uses the actual value to make the assignment to the appropriate row of the confusion matrix.

The error rates for each threshold value can be read out of the confusion matrix. The total number of errors is the sum of FPs and FNs. The example code produces confusion matrices for the in-sample data and the out-of-sample data and prints them both out. The misclassification error rate on the in-sample data is about 8 percent, and about 26 percent on the out-of-sample data. Generally, the out-of-sample performance will be worse than performance on in-sample data. It will also be more representative of the expected error on new examples.

The misclassification error changes when the thresholds are changed. Table 3-2 shows how misclassification error rate changes as the threshold value changes. The numbers in the table are based on out-of-sample results. That will be generally true of numbers characterizing performance throughout the book. Any in-sample errors will have warning labels attached: "Warning: These are in-sample errors." If the goal is to minimize the misclassification error, the best threshold value is 0.25.

Table 3-2: *Dependence of Misclassification Error on Decision Threshold*

DECISION THRESHOLD	MISCLASSIFICATION ERROR RATE
0.0	28.6 percent
0.25	24.3 percent
0.5	25.7 percent
0.75	30.0 percent
1.0	38.6 percent

The best value for the threshold may be the one that minimizes the misclassification error. Sometimes, however, there's more cost associated with one type of error than with another. Suppose, for instance, that for the rocks-versus-mines problem it costs $100 to send a diver to do a visual inspection and that unexploded mines cost $1,000 in expected injuries and property damage if not removed. An FP costs $100, and an FN costs $1,000. Given these assumptions, Table 3-3 summarizes the dollar cost of mistakes for different threshold values. The higher cost of mistaking a mine for a rock (and leaving it in place to threaten health and safety) has pushed the decision threshold down to zero. That means more FNs, but they aren't as expensive. A more thorough analysis could include the costs associated with TP and TN. For example, the TP might have costs associated with removing the mine and a benefit of +$1,000 associated with its removal. If these figures are available (or can be reasonably approximated) in your problem, it behooves you to use them to derive better threshold values.

Table 3-3: *Cost of Mistakes for Different Decision Thresholds*

DECISION THRESHOLD	FALSE NEGATIVE COST	FALSE POSITIVE COST	TOTAL COST
0.0	1,000	1,900	2,900
0.25	3,000	1,400	4,400
0.5	9,000	900	9,900
0.75	18,000	300	18,300
1.00	26,000	100	26,100

Note that the relative cost of total FPs versus FNs depends on the proportion of positive and negative examples in the data set. The rocks-versus-mines data set has an equal number of positives and negatives (mines and rocks). That was presumably determined by an experimental protocol. The proportion of positives and negatives encountered in actual practice may differ. If the numbers are likely to be different when the system is deployed, you need to make some adjustments to account for the proportions in actual use.

The data scientist may not have the costs available but may still want a method to characterize the overall performance of the classifier instead of using the misclassification error rate for a particular decision threshold. A common technique for doing that is called the *receiver operating characteristic* or ROC curve (`http://en.wikipedia.org/wiki/Receiver_operating_characteristic`).

ROC inherits its name from its original application—processing returns from a radar receiver to determine the presence or absence of hostile aircraft. The ROC curve yields a single plot that summarizes all of these different contingency tables. The ROC curve plots the true positive rate (abbreviated TPR) versus the false positive rate (FPR). TPR is the proportion of positive examples that are correctly classified as positive (see Equation 3-8). FPR is the number of FPs relative to the total number of actual negatives (see Equation 3-9). In terms of the elements of the contingency table, these are given by the following formulas:

$$TPR = \frac{TP}{TP + FN}$$

Equation 3-8: True positive rate

$$FPR = \frac{FP}{TN + FP}$$

Equation 3-9: False positive rate

As a simple thought experiment, consider using an extremely low value for the decision threshold. For a low value, every example is predicted as positive.

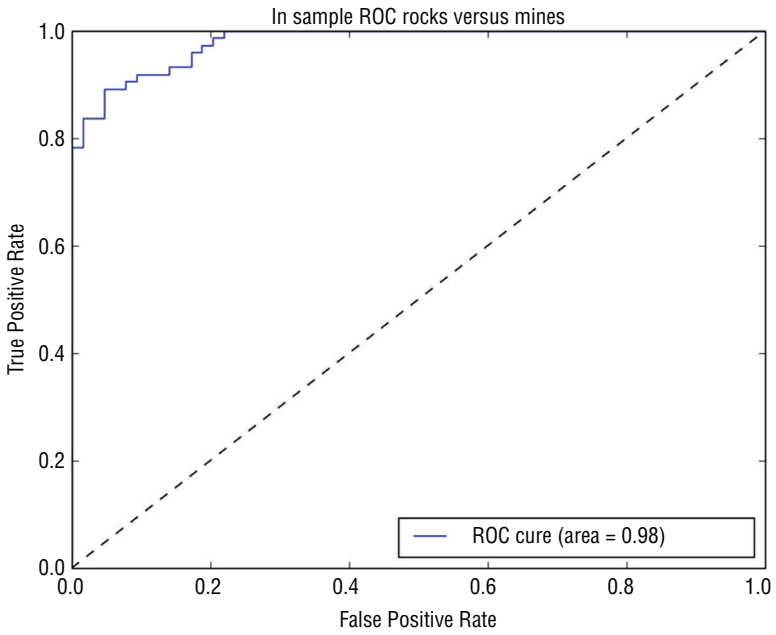


Figure 3-10: In-sample ROC for rocks-versus-mines classifier

That gives 1.0 for TPR. Because everything is classified as positive, there are no FNs (FN is 0.0). It also gives 1.0 for FPR because nothing gets classified as negative (TN is 0.0). However, when the decision threshold is set very high, TP is equal to zero, and so TPR is also zero and FP is also zero because nothing gets classified as positive. Therefore, FPR is also zero. The following two figures were drawn using the `pylab roc_curve()` and `auc()` functions. Figure 3-10 shows the ROC curve-based performance on in-sample data. Figure 3-11 shows the ROC curve based on out-of-sample data.

The ROC curve for the classifier that operates by randomly deciding rock or mine forms a diagonal line from the lower-left corner to the upper-right corner of the plot. That line is often drawn onto ROC curves as a reference point. For a perfect classifier, the ROC curve steps straight up from (0, 0) to (0, 1) and then goes straight across to (1, 1). Not surprisingly, Figure 3-10 (on in-sample data) comes closer to perfection than Figure 3-11 (on out-of-sample data). The closer that a classifier can come to hitting the upper-left corner, the better it is. If the ROC curve drops significantly below the diagonal line, it usually means that the data scientist has gotten a sign switched somewhere and should examine his code carefully.

Figures 3-10 and 3-11 also show the area under the curve (AUC) numbers. AUC, as the name suggests, is the area under the ROC curve. A perfect classifier has an AUC of 1.0, and random guessing has an AUC of 0.5. AUCs for Figures 3-10 and 3-11 provide another demonstration that performance estimates based

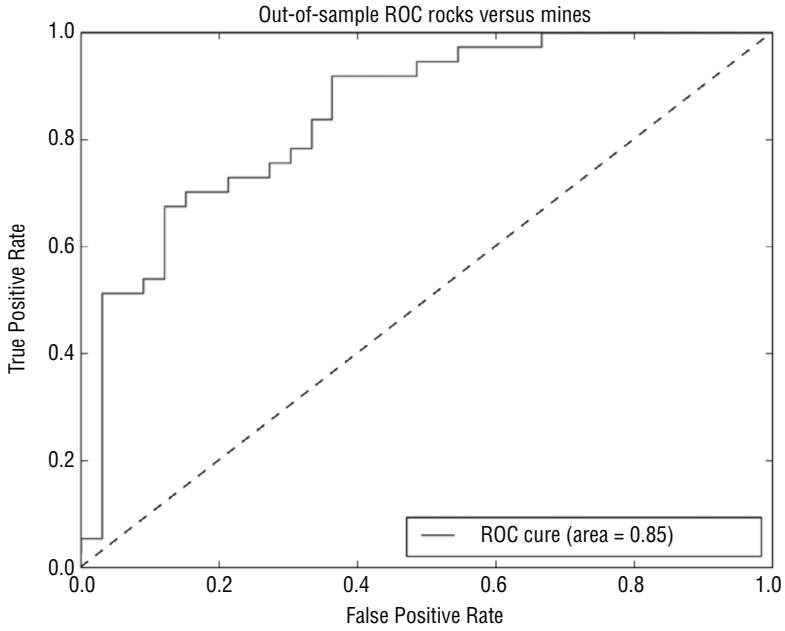


Figure 3-11: Out-of-sample ROC for rocks-versus-mines classifier

on the error on the training set (in-sample data) overestimate performance. The AUC on in-sample data is 0.98. The AUC on out-of-sample data is 0.85.

Some of the methods used for measuring binary classifier performance will also work for multiclass classifiers. Misclassification error still makes sense, and the confusion matrix also works. There are also multiclass generalizations of the ROC curve and AUC.[1]

Simulating Performance of Deployed Models

The examples from the preceding section demonstrated the need for testing performance on data not included in the training set to get a useful estimate of expected performance once a predictive model is deployed. The example broke the available labeled data into two subsets. One subset, called the *training set*, contained approximately two-thirds of the available data and was used to fitting an ordinary least squares model. The second subset, which contained the remaining third of the available data, was called the *test set* and was used only for determining performance (not used during training of the model). This is a standard procedure in machine learning.

Test set sizes range from 25 percent to 35 percent of the data, although there aren't any hard-and-fast rules about the sizes. One thing to keep in mind is that the performance of the trained model deteriorates as the size of the training data set shrinks. Taking out too much data from the training set can prove detrimental to end performance.

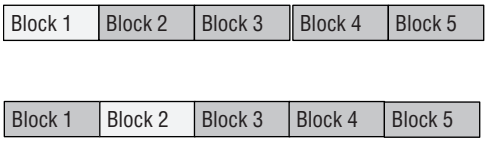


Figure 3-12: N-fold cross-validation

Another approach to holding out data is called *n-fold cross-validation*. Figure 3-12 shows schematically how a data set is divided up for training and testing with n-fold cross-validation. The set is divided into *n* disjointed sets of roughly equal sizes. In the figure, *n* is 5. Several training and testing passes are made through the data. In the first pass, the first block of data is held out for testing, and the remaining *n*-1 are used for training. In the second pass, the second block is held out for testing, and the other *n*-1 are used for training. This process is continued until all the data have been held out (five times for the five-fold example depicted in Figure 3-12).

The n-fold cross-validation process yields an estimate of the prediction error and has several samples of the error so that it can estimate error bounds on the error. It can keep more of the data in the training set, which generally gives lower generalization errors and better final performance. For example, if the 10-fold cross-validation is chosen, then only 10 percent of the data is held out for each training pass. These features of n-fold cross-validation come at the expense of taking more training time. The approach of taking a fixed holdout set has the advantage of faster training, because it employs only one pass through the training data. Taking a fixed holdout set is probably a better choice when the training times are unbearable with n-fold cross-validation and when there's so much training data available that some extra in the holdout set won't adversely affect performance.

Another thing to keep in mind is that the sample should be representative of the whole data set. The sampling plan used in the example in the last section was not a random sample. It was a sample of every third data point. Spreading the samples uniformly through the data usually works fine. However, you do need to avoid sampling in a way that introduces a bias in training and test sets. For example, if you were given data that was sampled once per day and arranged in order of sampling date, then coding seven-fold cross-validation and sampling every seventh point should be avoided.

Sampling may need to be carefully controlled if the phenomenon being studied has unusual statistics. Care may have to be taken to preserve the statistical peculiarities in the test sample. Examples of this include predicting rare events like fraud or ad clicks. The events being modeled are so infrequent that random sampling may over- or under-represent them in the test set and lead to erroneous estimates of performance. Stratified sampling (`http://en.wikipedia.org/wiki/Stratified_sampling`) divides the data into separate subsets that are

separately sampled and then recombined. When the labels are rare events, you might need to separately sample the fraudulent examples and the legitimate examples and then combine them for the test set to match the training set and, more importantly, the new data upon which the model will be used.

After a model has been trained and tested, it is good practice to recombine the training and test data into a single set and retrain the model on the larger data set. The out-of-sample testing procedure will have already given good estimates of the expected prediction errors. That was the purpose of holding out some of the data. The model will perform better and generalize better if trained on more data. The deployed model should be trained on all the data.

This section supplied you with tools to quantify the performance of your predictive model. The next section shows you how to replace the intuitive graphical comparisons of model and problem complexity that you saw in the section "Factors Driving Algorithm Choices and Performance" with numerical comparison. This replacement makes it possible to mechanize some of the selection process.

Achieving Harmony Between Model and Data

This section uses ordinary least squares (OLS) regression to illustrate several things. First, it illustrates how OLS can sometimes *overfit* a problem. Overfitting means that there's a significant discrepancy between errors on the training data and errors on the test data, such as you saw in the previous section where OLS was used to solve the rocks-versus-mines classification problem. Second, it introduces two methods for overcoming the overfit problem with OLS. These methods will cultivate your intuition and set the stage for the penalized linear regression methods that are covered in more depth in Chapter 4. In addition, the methods for overcoming overfitting have a property that is common to most modern machine learning algorithms. Modern algorithms generate a number of models of varying complexity and then use out-of-sample performance to balance model complexity, problem complexity, and data set richness and thus determine which model to deploy. This process will be used repeatedly throughout the rest of the book.

Ordinary least squares regression serves as a good prototype for machine learning algorithms in general. It's a supervised algorithm that has a training procedure and a deployment procedure. It can be overfit in some circumstances. It shares these features with other more modern function approximation algorithms. OLS is missing an important feature of modern algorithms, however. In its original formulation (the most familiar formulation), there's no means to throttle it back when it overfits. It's like having a car that only runs at full throttle (great when there's plenty of road, but tough to use in tight circumstances). Fortunately, there's been a lot of work on ordinary least squares regression

since its invention more than 200 years ago by Gauss and Legendre. This section introduces two of the methods for adjusting the throttle on ordinary least squares regression. One is called *forward stepwise regression*; the other is called *ridge regression*.

Choosing a Model to Balance Problem Complexity, Model Complexity, and Data Set Size

A couple of examples will illustrate how modern machine learning techniques can be tuned to best fit a given problem and data set. The first example is a modification to ordinary least squares regression called forward stepwise regression. Here's how it works. Recall Equations 3-1 and 3-2, which define the problem being solved (see Equations 3-10 and 3-11 here, which repeat those equations). The vector Y contains the labels. And the matrix X contains the attributes available to predict the labels.

$$Y = \begin{matrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{matrix}$$

Equation 3-10: Vector of numeric labels

$$X = \begin{matrix} x_{11} & x_{12} & & x_{1n} \\ x_{21} & x_{22} & & x_{2n} \\ \vdots & \vdots & \ddots & \\ x_{m1} & x_{m2} & & x_{mn} \end{matrix}$$

Equation 3-11: Matrix of numeric attributes

If this is a regression problem, then Y is a column vector of real numbers, and the linear problem is to find a column vector of weights β and a scalar $\beta 0$ (see Equation 3-12).

$$\beta = \begin{matrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{matrix}$$

Equation 3-12: Vector of coefficients for linear model

The values for β are selected so that Y is well approximated (see Equation 3-13).

$$Y \sim X\beta + \begin{matrix} \beta_0 \\ \beta_0 \\ \vdots \\ \beta_0 \end{matrix}$$

Equation 3-13: Approximating labels as linear function of attributes

If the number of columns of X is the same as the number of rows of X and the columns of X are independent (not linear multiples of one another), then X can be inverted and the ~ can be replaced with =. A coefficient vector β will make the linear fit the labels exactly. That's too good to be true. The problem is one of overfitting (that is, getting terrific performance on the training data that cannot be replicated on new data). In real problems, this is not a good outcome. The source of overfitting is having too many columns of data in X. The answer might be to get rid of some of the columns of X. However, getting rid of some involves deciding how many to eliminate and which ones should be eliminated. The brute-force method is called *best subset selection*.

Using Forward Stepwise Regression to Control Overfitting

The following code provides an outline of the algorithm for best subset selection. The basic idea is to impose a constraint (say nCol) on the number of columns and then take all subsets of the columns of X that have that number of columns, perform ordinary least squares regression, identify the nCol subset that has the least out-of-sample error, increment nCol, and repeat. The process results in a list of the best choice of one-column subsets: two-column subsets up to the full matrix X (the all-column subset). It also yields the performance of each of these. Then the next step is to determine whether to deploy the one-column version, the two-column version, and so on. But that's relatively easy; just pick the one with the least errors.

```
Initialize: Out_of_sample_error = NULL
   Break X and Y into test and training sets
for i in range(number of columns in X):
   for each subset of X having i+1 columns:
      fit ordinary least squares model
   Out_of_sample_error.append(least error amoung subsets containing
   i+1 columns)
Pick the subset corresponding to least overall error
```

The problem with best subset selection is that it requires too much calculation for even modest numbers of attributes (columns of X). For example, 10 attributes leads to $2^{10} = 1,000$ subsets. There are several techniques that avoid this. The following code shows the procedure for forward stepwise regression. The idea with forward stepwise regression is to start with one-column subsets and then,

given the best single column, to find the best second column to append instead of evaluating all possible two-column subsets. Pseudo-code for forward stepwise regression is given here.

```
Initialize: ColumnList = NULL
   Out-of-sample-error = NULL
   Break X and Y into test and training sets
For number of column in X:
   For each trialColumn (column not in ColumnList):
      Build submatrix of X using ColumnList + trialColumn
      Train OLS on submatrix and store RSS Error on test data
   ColumnList.append(trialColumn that minimizes RSS Error)
   Out-of-sample-error.append(minimum RSS Error)
```

Best subset selection and forward stepwise regression have similar processes. They train a series of models (several for one column, several for two columns, and so on). They result in a parameterized family of models (all linear regression parameterized on number of columns). The models vary in complexity, and the final model is selected from the family on the basis of performance on out-of-sample error.

Listing 3-3 shows Python code implementing forward stepwise regression on the wine data set.

Listing 3-3: Forward Stepwise Regression: Wine Quality Data—fwdStepwiseWine.py

```
import numpy
from sklearn import datasets, linear_model
from math import sqrt
import matplotlib.pyplot as plt

def xattrSelect(x, idxSet):
    #takes X matrix as list of list and returns subset containing
    #columns in idxSet
    xOut = []
    for row in x:
        xOut.append([row[i] for i in idxSet])
    return(xOut)

#read data into iterable
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-
databases/"
"wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)
xList = []
labels = []
names = []
firstLine = True
for line in data:
```

```
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

#divide attributes and labels into training and test sets
indices = range(len(xList))
xListTest = [xList[i] for i in indices if i%3 == 0 ]
xListTrain = [xList[i] for i in indices if i%3 != 0 ]
labelsTest = [labels[i] for i in indices if i%3 == 0]
labelsTrain = [labels[i] for i in indices if i%3 != 0]

#build list of attributes one-at-a-time - starting with empty
attributeList = []
index = range(len(xList[1]))
indexSet = set(index)
indexSeq = []
oosError = []

for i in index:
    attSet = set(attributeList)
    #attributes not in list already
    attTrySet = indexSet - attSet
    #form into list
    attTry = [ii for ii in attTrySet]
    errorList = []
    attTemp = []
    #try each attribute not in set to see which
    #one gives least oos error
    for iTry in attTry:
        attTemp = [] + attributeList
        attTemp.append(iTry)
        #use attTemp to form training and testing sub matrices
        #as list of lists
        xTrainTemp = xattrSelect(xListTrain, attTemp)
        xTestTemp = xattrSelect(xListTest, attTemp)
        #form into numpy arrays
        xTrain = numpy.array(xTrainTemp)
        yTrain = numpy.array(labelsTrain)
        xTest = numpy.array(xTestTemp)
        yTest = numpy.array(labelsTest)
```

continues

continued

```
        #use sci-kit learn linear regression
        wineQModel = linear_model.LinearRegression()
        wineQModel.fit(xTrain,yTrain)
        #use trained model to generate prediction and calculate rmsError
        rmsError = numpy.linalg.norm((yTest-wineQModel.predict(xTest)),
            2)/sqrt(len(yTest))
        errorList.append(rmsError)
        attTemp = []

    iBest = numpy.argmin(errorList)
    attributeList.append(attTry[iBest])
    oosError.append(errorList[iBest])

print("Out of sample error versus attribute set size" )
print(oosError)
print("\n" + "Best attribute indices")
print(attributeList)
namesList = [names[i] for i in attributeList]
print("\n" + "Best attribute names")
print(namesList)

#Plot error versus number of attributes
x = range(len(oosError))
plt.plot(x, oosError, 'k')
plt.xlabel('Number of Attributes')
plt.ylabel('Error (RMS)')
plt.show()

#Plot histogram of out of sample errors for best number of attributes
#Identify index corresponding to min value,
#retrain with the corresponding attributes
#Use resulting model to predict against out of sample data.
#Plot errors (aka residuals)
indexBest = oosError.index(min(oosError))
attributesBest = attributeList[1:(indexBest+1)]

#Define column-wise subsets of xListTrain and xListTest
#and convert to numpy
xTrainTemp = xattrSelect(xListTrain, attributesBest)
xTestTemp = xattrSelect(xListTest, attributesBest)
xTrain = numpy.array(xTrainTemp); xTest = numpy.array(xTestTemp)

#train and plot error histogram
wineQModel = linear_model.LinearRegression()
wineQModel.fit(xTrain,yTrain)
errorVector = yTest-wineQModel.predict(xTest)
plt.hist(errorVector)
plt.xlabel("Bin Boundaries")
plt.ylabel("Counts")
plt.show()
```

```
#scatter plot of actual versus predicted
plt.scatter(wineQModel.predict(xTest), yTest, s=100, alpha=0.10)
plt.xlabel('Predicted Taste Score')
plt.ylabel('Actual Taste Score')
plt.show()
```

The preceding listing includes a small function to extract selected columns from the X matrix (in the form of a list of lists). Then it breaks the X matrix and the vector of labels into training and test sets. After that, the code follows the preceding algorithm description. A pass through the algorithm begins with a subset of attributes that are included in the solution. For the first pass, this subset is empty. For subsequent passes, the subset includes the attributes selected one at a time during earlier passes. Each pass selects a single new attribute to add to the subset of attributes. The attribute to be added is chosen by testing each non-included attribute to see which one results in the best performance when added to the subset. In turn, each attribute is added to the attribute subset and ordinary least squares is used to fit a linear model with the resulting attribute subset. For each attribute tested, the out-of-sample performance is measured. The tested attribute which yields the best root sum of squares (RSS) error is added to the attribute set, and the associated RSS error is captured.

Figure 3-13 plots the RMSEs as a function of the number of attributes included in the regression. The error decreases until nine attributes are included and then increases somewhat.

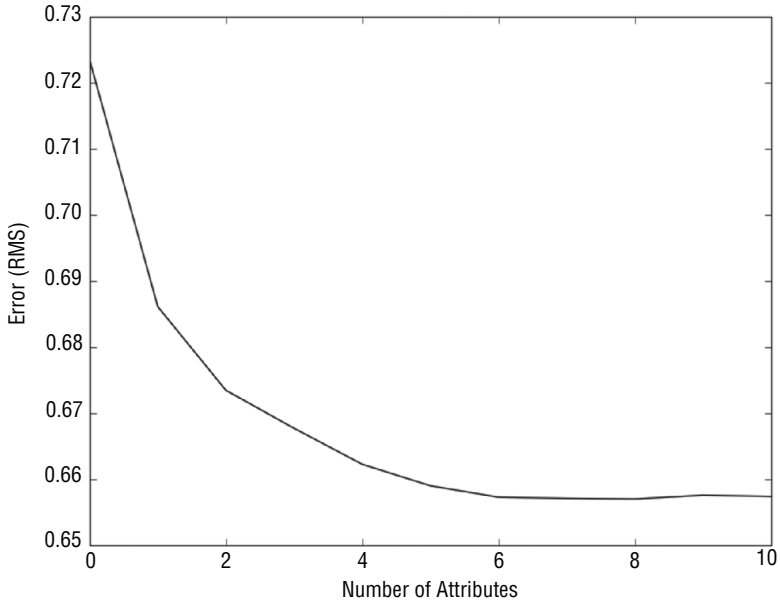


Figure 3-13: Wine quality prediction error using forward stepwise regression

Listing 3-4 shows numeric output for forward stepwise regression applied to wine quality data.

Listing 3-4: Forward Stepwise Regression Output—fwdStepwiseWineOutput.txt

```
Out of sample error versus attribute set size
[0.7234259255116281, 0.68609931528371915, 0.67343650334202809,
0.66770332138977984, 0.66225585685222743, 0.65900047541546247,
0.65727172061430772, 0.65709058062076986, 0.65699930964461406,
0.65758189400434675, 0.65739098690113373]

Best attribute indices
[10, 1, 9, 4, 6, 8, 5, 3, 2, 7, 0]

Best attribute names
['"alcohol"', '"volatile acidity"', '"sulphates"', '"chlorides"',
 '"total sulfur dioxide"', '"pH"', '"free sulfur dioxide"',
 '"residual sugar"', '"citric acid"', '"density"', '"fixed acidity"']
```

The first list shows the RSS error. The error decreases until the 10th element in the list, and then gets larger again. The associated column indices are shown in the next list. The last list gives the names (column headers) of the associated attributes.

Evaluating and Understanding Your Predictive Model

Several other plots are helpful in understanding the performance of a trained algorithm and can point the way to making improvements in its performance. Figure 3-14 shows a scatter plot of the true labels plotted versus the predicted labels for points in the test set. Ideally, all of the points in Figure 3-1 would lie on a 45-degree line—the line where the true labels and the predicted labels are equal. Because the real scores are integers, the scatter plot shows horizontal rows of points. When the true values take on a small number of values, it is useful to make the data points partially transparent so that the darkness can indicate the accumulation of many points in one area of the graph. Actual taste scores of 5 and 6 are reproduced fairly well. The more extreme values are not as well predicted by the system. Generally speaking, machine learning algorithms do worse at the edges of a data set.

Figure 3-15 shows a histogram of the prediction error for forward stepwise prediction predicting wine taste scores. Sometimes the error histogram will have two or more discrete peaks. Perhaps it will have a small peak on the far right or far left of the graph. In that case, it may be possible to find an explanation for the different peaks in the error and to reduce the prediction error by adding a new attribute that explains the membership in one or the other of the groups of points.

You want to note several things about this output. First, let's reiterate the process. The process is to train a family of models (in this case, ordinary linear

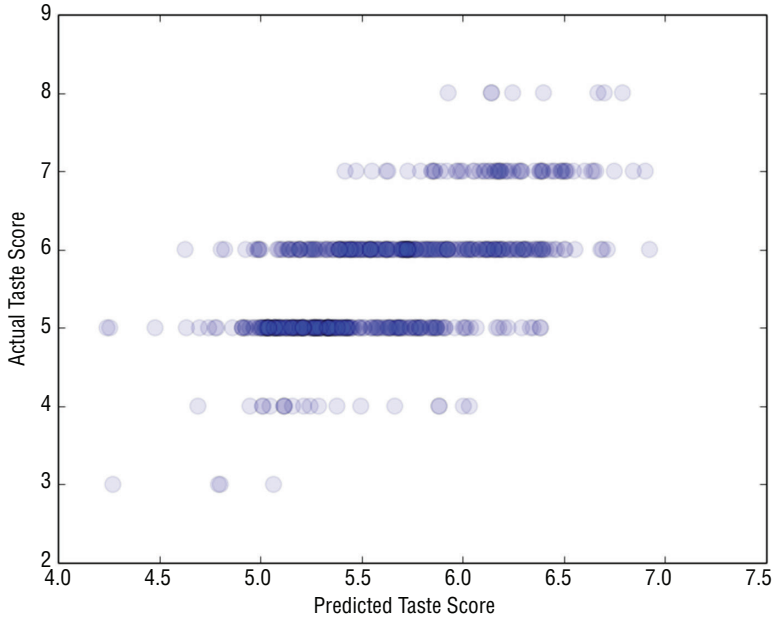


Figure 3-14: Actual taste scores versus predictions generated with forward stepwise regression

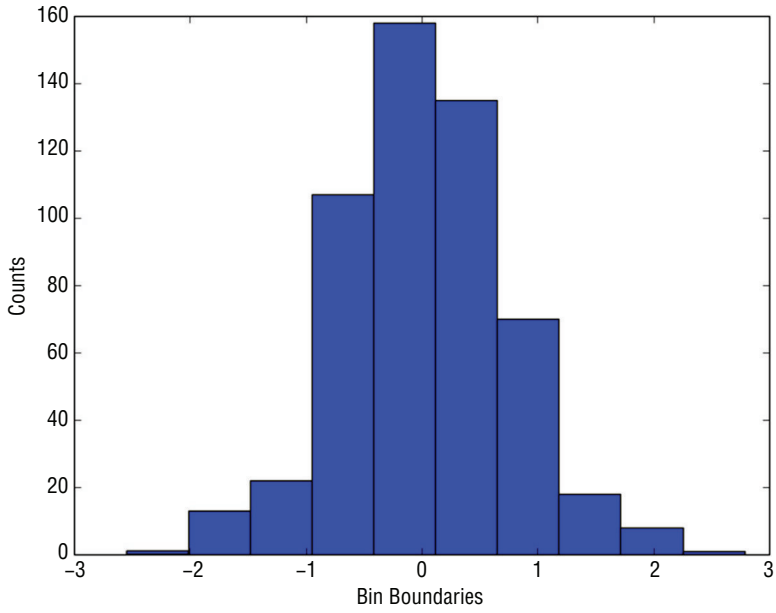


Figure 3-15: Histogram of wine taste prediction error with forward stepwise regression

regression trained on column-wise subsets of X). The series of models is parameterized (in this case, by the number of attributes that are used in the linear model). The model to deploy is chosen to minimize the out-of-sample error. The number of attributes to be incorporated in the solution can be called a *complexity parameter*. Models with larger complexity parameters have more free parameters and are more likely to overfit the data than less-complex models.

Also note that the attributes have become ordered by their importance in predicting quality. In the list of column numbers and the associated list of attribute names, the first in the list is the first attribute chosen, the second was next, and so on. The attributes used come out in a nice ordered list. This is an important and desirable feature of a machine learning technique. Early stages of a machine learning task mostly involve hunting for (or constructing) the best set of attributes for making predictions. Having techniques to rank attributes in order of importance helps in that process. The other algorithms developed in this book will also have this property.

The last observation regards picking a model from the family that machine learning techniques generate. The more complicated the model, the less well it will generalize. It is better to err on the side of a less-complicated model. The earlier example indicates that there's very little degradation in performance between the 9th (best) model and the 10th model (a change in the 4th significant digit). Best practice would be to remove those attributes even if they were better in the 4th significant digit in order to be conservative.

Control Overfitting by Penalizing Regression Coefficients—Ridge Regression

This section describes another method for modifying ordinary least squares regression to control model complexity and to avoid overfitting. This method serves as a first introduction to penalized linear regression. You'll see more coverage of this in Chapter 4.

Ordinary least squares regression seeks to find scalar β_0 and vector β that satisfy (see Equation 3-14).

$$\beta_0^*, \beta^* = argmin_{\beta_0, \beta}\left(\frac{1}{m}\sum_{i=1}^{m}\left(y_i - (\beta_0 + x_i\beta)\right)^2\right)$$

Equation 3-14: OLS minimization problem

The expression *argmin* means the "values of β_0 and β that minimize the expression." The resulting coefficients β_0^*, β^* are the ordinary least squares solution. Best subset regression and forward stepwise regression throttle back ordinary regression by limiting the number of attributes used. That's equivalent to

imposing a constraint that some of the entries in the vector β be equal to zero. Another approach is called *coefficient penalized regression*. Coefficient penalized regression accomplishes the same thing by making all the coefficients smaller instead of making some of them zero. One version of coefficient penalized linear regression is called *ridge regression*. Equation 3-15 shows the problem formulation for ridge regression.

$$\beta_0^*, \beta^* = argmin_{\beta_0,\beta}\left(\frac{1}{m}\sum_{i=1}^{m}\left(y_i - (\beta_0 + x_i\beta)\right)^2 + \alpha\beta^T\beta\right)$$

Equation 3-15: Ridge regression minimization problem

The difference between Equation 3-15 and ordinary least squares (Equation 3-14) is the addition of the $\alpha\beta^T\beta$ term. The $\beta^T\beta$ term is the square of the Euclidean norm of β (the vector of coefficients). The variable β is a complexity parameter for this formulation of the problem. If $\alpha = 0$, the problem becomes ordinary least squares regression. When α becomes large, β (the vector of coefficients) approaches zero, and only the constant term β_0 is available to predict the labels y_i. Ridge regression is available in scikit-learn. Listing 3-5 shows the code for solving the wine taste regression problem using ridge regression.

Listing 3-5: Predicting Wine Taste with Ridge Regression—ridgeWine.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
from sklearn import datasets, linear_model
from math import sqrt
import matplotlib.pyplot as plt

#read data into iterable
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-
databases/"
"wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
```

continues

continued

```
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

#divide attributes and labels into training and test sets
indices = range(len(xList))
xListTest = [xList[i] for i in indices if i%3 == 0 ]
xListTrain = [xList[i] for i in indices if i%3 != 0 ]
labelsTest = [labels[i] for i in indices if i%3 == 0]
labelsTrain = [labels[i] for i in indices if i%3 != 0]

xTrain = numpy.array(xListTrain); yTrain = numpy.array(labelsTrain)
xTest = numpy.array(xListTest); yTest = numpy.array(labelsTest)

alphaList = [0.1**i for i in [0,1, 2, 3, 4, 5, 6]]

rmsError = []
for alph in alphaList:
    wineRidgeModel = linear_model.Ridge(alpha=alph)
    wineRidgeModel.fit(xTrain, yTrain)
    rmsError.append(numpy.linalg.norm((yTest-wineRidgeModel.predict(
       xTest)), 2)/sqrt(len(yTest)))

print("RMS Error             alpha")
for i in range(len(rmsError)):
    print(rmsError[i], alphaList[i])

#plot curve of out-of-sample error versus alpha
x = range(len(rmsError))
plt.plot(x, rmsError, 'k')
plt.xlabel('-log(alpha)')
plt.ylabel('Error (RMS)')
plt.show()

#Plot histogram of out of sample errors for best alpha value and
#scatter plot of actual versus predicted

#Identify index corresponding to min value, retrain with
#the corresponding value of alpha

#Use resulting model to predict against out of sample data.
#Plot errors (aka residuals)
indexBest = rmsError.index(min(rmsError))
alph = alphaList[indexBest]
wineRidgeModel = linear_model.Ridge(alpha=alph)
wineRidgeModel.fit(xTrain, yTrain)
errorVector = yTest-wineRidgeModel.predict(xTest)
```

```
plt.hist(errorVector)
plt.xlabel("Bin Boundaries")
plt.ylabel("Counts")
plt.show()

plt.scatter(wineRidgeModel.predict(xTest), yTest, s=100, alpha=0.10)
plt.xlabel('Predicted Taste Score')
plt.ylabel('Actual Taste Score')
plt.show()
```

Recall that the forward stepwise regression the algorithm produced a sequence of different models—the first with one attribute, the next with two attributes, and so on until the final model included all the attributes. The code for ridge regression also has a sequence of models. Instead of different numbers of attributes, the sequence of ridge regression models have different values of α—the parameter that determines the severity of the penalty on the β's. The construction of sequence of α's decreases them by powers of 10. Generally speaking, you'll want to make them decrease exponentially, not by a fixed increment. The range needs to be fairly wide and may take some experimentation to establish.

Figure 3-16 plots the RMSE as a function of the ridge complexity parameter α. The parameter is arranged from largest value on the left to smallest value on the right. It is conventional to show the least complex model on the left side of the plot and the most complex on the right side. The plot shows much the same character as with forward stepwise regression. The errors are roughly the same, but favor forward stepwise regression slightly.

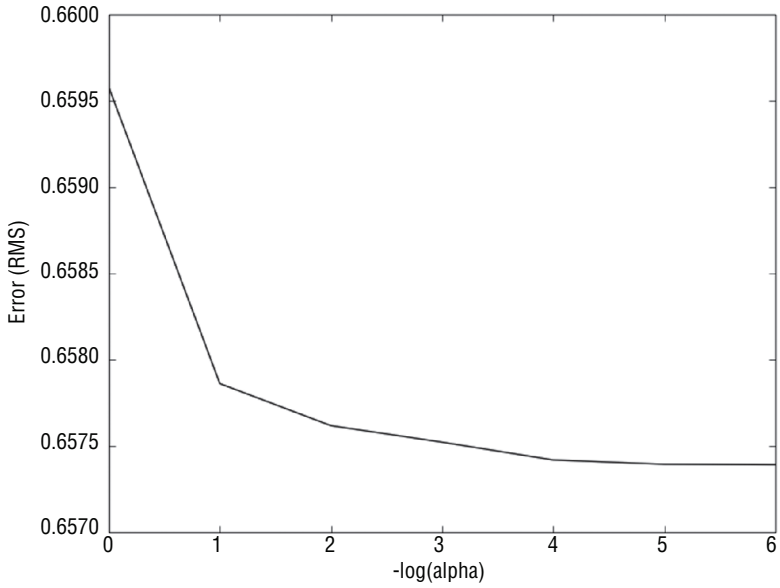


Figure 3-16: Wine quality prediction error using ridge regression

Listing 3-6 shows the output from the ridge regression. The numbers show that ridge regression has roughly the same character as forward stepwise regression. The numbers slightly favor forward stepwise regression.

Listing 3-6: Ridge Regression Output—ridgeWineOutput.txt

```
RMS Error             alpha
(0.65957881763424564, 1.0)
(0.65786109188085928, 0.1)
(0.65761721446402455, 0.010000000000000002)
(0.65752164826417536, 0.0010000000000000002)
(0.65741906801092931, 0.00010000000000000002)
(0.65739416288512531, 1.0000000000000003e-05)
(0.65739130871558593, 1.0000000000000004e-06)
```

Figure 3-17 shows the scatter plot of actual taste score versus predicted taste score for the ridge regression predictor trained on wine taste data. Figure 3-18 shows the histogram of prediction error.

You can apply the same general method to classification problems. The section "Measuring the Performance of Predictive Models" discussed several methods for quantifying classifier performance. The methods outlined included using misclassification error, associating economic costs to the various prediction outcomes, and using the area under the ROC curve (AUC) to quantify performance.

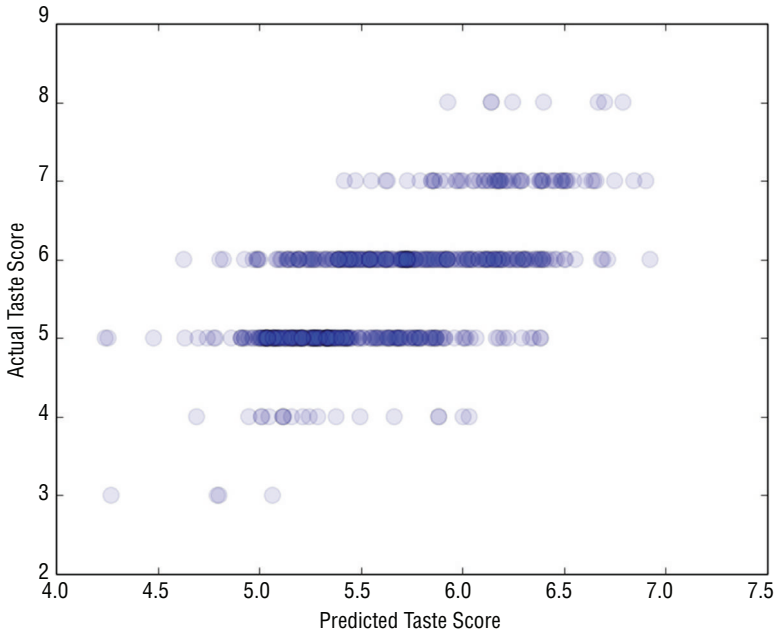


Figure 3-17: Actual taste scores versus predictions generated with ridge regression

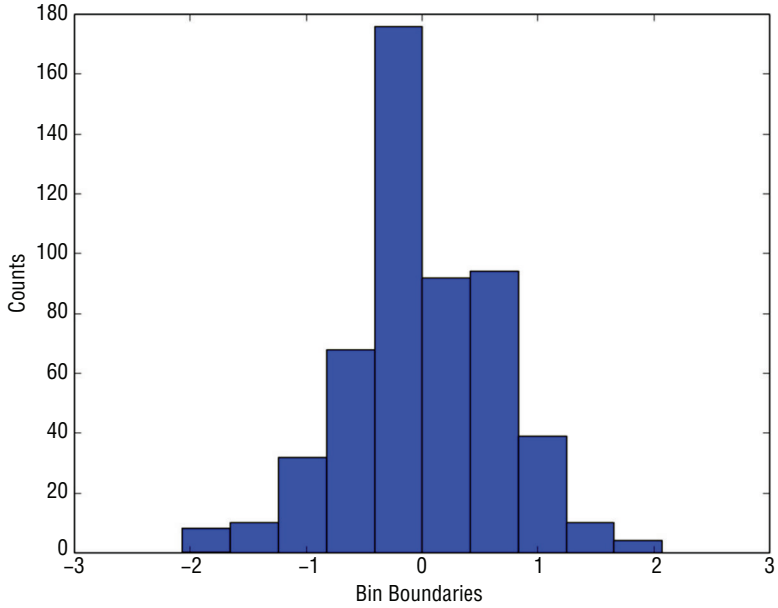


Figure 3-18: Histogram of wine taste prediction error with ridge regression

That section built a classifier using ordinary least squares regression. Listing 3-7 shows Python code that follows that same general plan. Instead of OLS, it uses ridge regression as a regression method (with a complexity tuning parameter) for building the rocks-versus-mines classifier and uses AUC as the performance measure for the classifier. The program in Listing 3-7 is similar to the wine taste prediction with ridge regression. The big difference is that the program uses the predictions on the test data and the test labels as input to the roc_curve program from the scikit-learn package. That makes it easy to calculate the AUC for each pass through the training. These are accumulated, and the printed values are shown in Listing 3-8.

Listing 3-7: Rocks Versus Mines Using Ridge Regression—classifierRidgeRocksVMines.py

```
__author__ = 'mike-bowles'
import urllib2
import numpy
from sklearn import datasets, linear_model
from sklearn.metrics import roc_curve, auc
import pylab as plt

#read data from uci data repository
target_url = ("https://archive.ics.uci.edu/ml/machine-earning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")
data = urllib2.urlopen(target_url)
```

continues

continued

```
#arrange data into list for labels and list of lists for attributes
xList = []
labels = []
for line in data:
    #split on comma
    row = line.strip().split(",")
    #assign label 1.0 for "M" and 0.0 for "R"
    if(row[-1] == 'M'):
        labels.append(1.0)
    else:
        labels.append(0.0)
        #remove lable from row
    row.pop()
    #convert row to floats
    floatRow = [float(num) for num in row]
    xList.append(floatRow)

#divide attribute matrix and label vector into training(2/3 of data)
#and test sets (1/3 of data)
indices = range(len(xList))
xListTest = [xList[i] for i in indices if i%3 == 0 ]
xListTrain = [xList[i] for i in indices if i%3 != 0 ]
labelsTest = [labels[i] for i in indices if i%3 == 0]
labelsTrain = [labels[i] for i in indices if i%3 != 0]

#form list of list input into numpy arrays to match input class for
#scikit-learn linear model
xTrain = numpy.array(xListTrain); yTrain = numpy.array(labelsTrain)
xTest = numpy.array(xListTest); yTest = numpy.array(labelsTest)

alphaList = [0.1**i for i in [-3, -2, -1, 0,1, 2, 3, 4, 5]]

aucList = []
for alph in alphaList:
    rocksVMinesRidgeModel = linear_model.Ridge(alpha=alph)
    rocksVMinesRidgeModel.fit(xTrain, yTrain)
    fpr, tpr, thresholds = roc_curve(yTest,rocksVMinesRidgeModel.
      predict(xTest))
    roc_auc = auc(fpr, tpr)
    aucList.append(roc_auc)


print("AUC             alpha")
for i in range(len(aucList)):
    print(aucList[i], alphaList[i])

#plot auc values versus alpha values
x = [-3, -2, -1, 0,1, 2, 3, 4, 5]
plt.plot(x, aucList)
plt.xlabel('-log(alpha)')
```

```
plt.ylabel('AUC')
plt.show()

#visualize the performance of the best classifier
indexBest = aucList.index(max(aucList))
alph = alphaList[indexBest]
rocksVMinesRidgeModel = linear_model.Ridge(alpha=alph)
rocksVMinesRidgeModel.fit(xTrain, yTrain)

#scatter plot of actual vs predicted
plt.scatter(rocksVMinesRidgeModel.predict(xTest),
   yTest, s=100, alpha=0.25)
plt.xlabel("Predicted Value")
plt.ylabel("Actual Value")
plt.show()
```

Listing 3-8 shows the AUC and associated alpha (multiplier on the coefficient penalty).

Listing 3-8: Output from Classification Model for Rocks Versus Mines Using Ridge Regression—classifierRidgeRocksVMinesOutput.txt

```
AUC             alpha
(0.84111384111384113, 999.9999999999999)
(0.86404586404586403, 99.99999999999999)
(0.9074529074529073, 10.0)
(0.91809991809991809, 1.0)
(0.88288288288288286, 0.1)
(0.8615888615888615, 0.010000000000000002)
(0.85176085176085159, 0.0010000000000000002)
(0.85094185094185093, 0.00010000000000000002)
(0.84930384930384917, 1.0000000000000003e-05)
```

A value of AUC close to 1 means great performance. A value near 0.5 is not good. So the goal with AUC is to maximize it instead of minimizing it, as was done with MSE in the earlier examples. AUC shows a fairly sharp peak at $\alpha = 1.0$. The numbers and the plot show a fairly significant drop off in performance relative to $\alpha = 1.0$. Recall that as alpha gets smaller, the solution approaches the solution to the unconstrained linear regression problem. The drop-off in performance for values of alpha smaller than 1.0 indicates that the unconstrained solution won't perform as well as ridge regression does. In the earlier section "Measuring Performance of Predictive Models," you saw the results for unconstrained ordinary least squares. The AUC on in-sample data was 0.98, and on out-of-sample data it was 0.85—very close to the AUC using ridge regression with a relatively small alpha (1E-5). Ridge regression results in a significant improvement in performance.

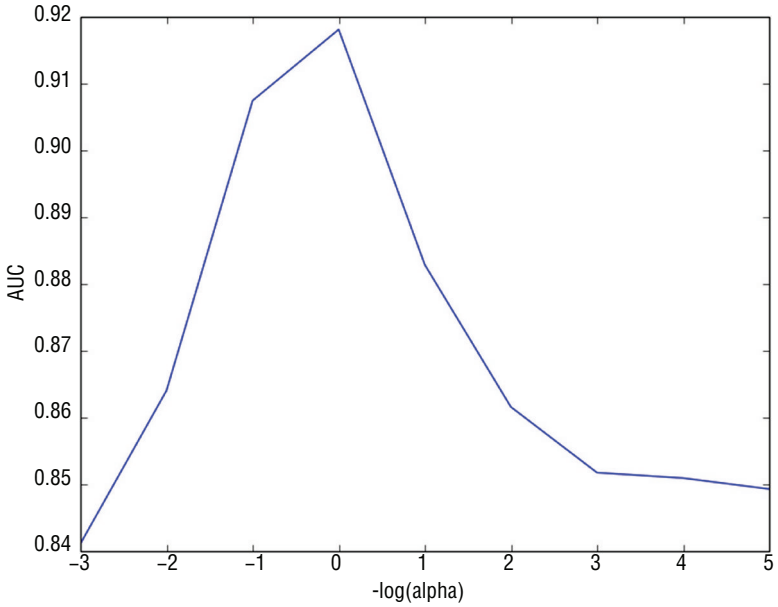


Figure 3-19: AUC for the rocks-versus-mines classifier using ridge regression

The issue here is that the attribute space for the rocks-versus-mines problem is 60 attributes wide while the full data set contains 208 rows of data. After the removal of 70 examples to be used as holdout data, 138 rows of data are available for training. That's more than twice the number of attributes, but the unconstrained (ordinary least squares) solution still overfits the data. This situation might be a good candidate for trying 10-fold cross-validation. That would result in only 20 examples (10 percent of the data set) being held out on each of the folds and might show some consequent improvement in performance. That approach comes up in Chapter 5, "Building Predictive Models Using Penalized Linear Methods."

Figure 3-19 plots the AUC as a function of the alpha parameter. That gives a visual demonstration of the value of reducing the complexity of the ordinary least squares solution by imposing a constraint on the Euclidean length of the coefficient vector.

Figure 3-20 shows the scatter plot of actual classification versus prediction for this classifier. This plot has a similar character to the scatter plot for wine prediction. Because there are a discrete number of actual outcomes, the scatter plot is composed of two horizontal rows of points.

This section introduced and explored two extensions to ordinary least squares regression. These served as illustrations of the process of training and balancing a modern predictive model. In addition, these extensions help introduce the more general penalized regression methods that will be explained in Chapter 4 and used to solve a variety of problem in Chapter 5.

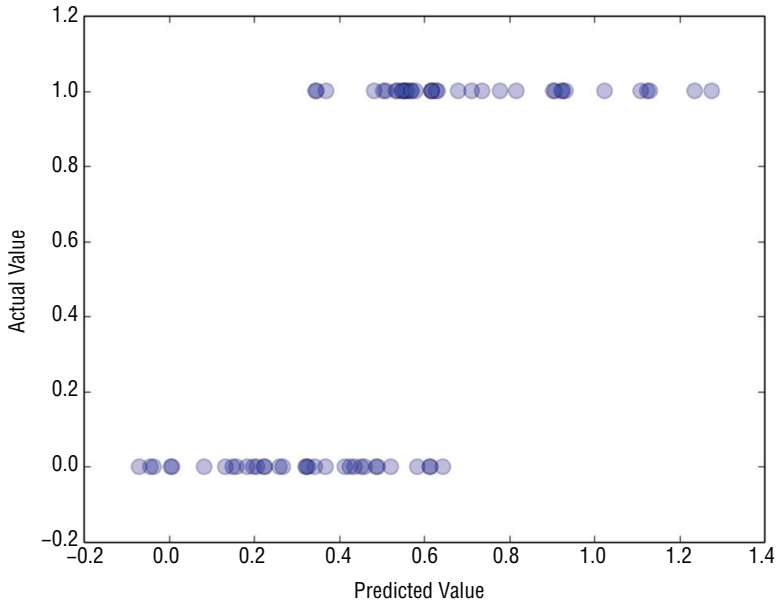


Figure 3-20: Plot of actual versus prediction for the rocks-versus-mines classifier using ridge regression

Summary

This chapter covered several topics that serve as a foundation for what comes later. First, the chapter provided visual demonstrations of problem complexity and model complexity and discussed how those factors and data set sizes conspire to determine classifier performance on a given problem. The discussion then turned to a number of different metrics for prediction performance associated with the different problem types (regression, classification, and multiclass classification) that arise as part of the function approximation problem. The chapter described two methods (holdout and n-fold cross-validation) for estimating performance on new data. The chapter introduced the conceptual framework that a machine learning technique produces a parameterized family of models and that one of these is selected for deployment on the basis of out-of-sample performance. Several examples based on modifications of ordinary least squares regression (forward stepwise regression and ridge regression) then instantiated that conceptual framework.

References

1. David J. Hand and Robert J. Till (2001). A Simple Generalization of the Area Under the ROC Curve for Multiple Class Classification Problems. *Machine Learning*, 45(2), 171–186.

CHAPTER

4

Penalized Linear Regression

As you saw in Chapter 3, "Predictive Model Building: Balancing Performance, Complexity, and Big Data," getting linear regression to work in practice requires some manipulation of the ordinary least squares algorithm. Ordinary least squares regression cannot temper its use of all the data available in an attempt to minimize the error on the training data. Chapter 3 illustrated that this situation can lead to models that perform much worse on new data than on the training data. Chapter 3 showed two extensions of ordinary least squares regression. Both of these involved judiciously reducing the amount of data available to ordinary least squares and using out-of-sample error measurement to determine how much data resulted in the best performance.

Stepwise regression began by letting ordinary least squares regression use exactly one of the attribute columns for making predictions and by picking the best one. It proceeded by adding new attributes to the existing model.

Ridge regression introduced a different type of constraint. Ridge regression imposed a penalty on the magnitude of the coefficients to constrict the solution. Both ridge regression and forward stepwise regression gave better than ordinary least squares (OLS) on example problems.

This chapter develops an extended family of methods for taming the overfitting inherent in OLS. The methods discussed in this chapter are called *penalized linear regression*. Penalized linear regression covers several algorithms that operate similarly to the methods introduced in Chapter 3. Ridge regression is a specific example of a penalized linear regression algorithm. Ridge regression regulates

overfitting by penalizing the sum of the regression coefficients squared. Other penalized regression algorithms use different forms of penalty. This chapter explains how the penalty method determines the nature of the solution and the type of information that is available about the solution.

Why Penalized Linear Regression Methods Are So Useful

Several properties make penalized linear regression methods outstandingly useful, including the following:

- Extremely fast model training
- Variable importance information
- Extremely fast evaluation when deployed
- Reliable performance on a wide variety of problems—particularly on attribute matrices that are not very tall compared to their width or that are sparse. Sparse solutions (that is, a more parsimonious model)
- May require linear model

Here's what these properties mean to you as a designer of machine learning models.

Extremely Fast Coefficient Estimation

Training time matters for several reasons. One reason is that the process of building a model is iterative. You'll find that you use training as part of your feature selection and feature engineering process. You'll pick some features that seem reasonable, train a model, evaluate it on out-of-sample data, want more performance, make some changes, and try again. If the basic training gets done quickly, you don't waste so much time getting coffee while waiting for answers (and reap the health benefit of lowering your caffeine intake). This makes the development process faster. Another reason why training times matter is that you might need to retrain your models to keep them working as conditions change. If you're classifying tweets, you might need to stay on top of changes in vocabulary. If you're training to trade in financial markets, the conditions are always changing. The time taken for training, even without feature reengineering, will dictate how rapidly you can respond to changing conditions.

Variable Importance Information

Both classes of algorithms covered in this book develop variable importance information. Variable importance information consists of a ranking for each of the attributes you've chosen to base your model on. The ranking tells you how

much the model values each attribute compared to others. A highly ranked attribute contributes more to the model's prediction than lesser-ranked attributes. This is crucial information for a variety of reasons. First, it helps you weed out variables during the feature engineering process. The good features will rise to the top of the list, and the not-so-good ones will sink to the bottom. Besides helping you with feature engineering, knowing what variables are driving the predictions helps you understand and explain your models to others (your boss, your customer, subject matter experts in the company, and so on). To the extent that the important attributes are what people expected it gives them confidence that the models make sense. If some of the rankings are surprises, you may gain new insights into your problem. Discussion about the relative importance can give your development group new ideas about where to look for performance improvements.

The two properties of rapid training and variable importance make penalized regression a good algorithm to try first on a new problem. These algorithms help you quickly get your arms around the problems and decide which features are going to be useful.

Extremely Fast Evaluation When Deployed

In some problem settings, fast evaluations are a critical performance parameter. In some electronic markets (for example, Internet ads and automated trading), whoever gets the answer first gets the business. In many other applications (for instance, spam filtering), time might be critical, although not a yes/no criterion. It is hard to beat a linear model for evaluation speed. The number of operations required for the prediction calculation is one multiply and one add for each attribute.

Reliable Performance

Reliable performance means that penalized linear methods will generate reasonable answers to problems of all different shapes and sizes. On some problems, they will equal the best performance available. In some cases, they will outperform all contenders with a little coaxing. This chapter will talk about the sorts of coaxing available. Chapter 6, "Ensemble Methods," revisits this topic and explains some ways to use penalized linear regression in conjunction with ensemble methods to improve performance.

Sparse Solutions

A *sparse solution* means that many of the coefficients in the model are zero. That means that not as many multiplications and sums are required. More important, a sparse model (one with few nonzero coefficients) is easier to interpret. It's easier to see what attributes are driving the predictions that the model is generating.

Problem May Require Linear Model

The last reason for using penalized linear regression is that a linear model might be a requirement of the solution. Calculations of insurance payouts represent one example where linear models are required, where a payout formula is often part of a contract that specifies variables and their coefficients. An ensemble model that involves a thousand trees, each with a thousand parameters, would be nearly impossible to write out in English. Drug testing is another arena where regulatory apparatus requires a linear form for statistical inference.

When to Use Ensemble Methods

The prime reason for not using penalized linear regression is that you might get better performance with another technique, such as an ensemble method. As outlined in Chapter 3, ensembles perform best in complicated problems (for example, highly irregular decision surfaces) with plenty of data to resolve the problem's complexities. In addition, ensemble methods for measuring variable importance can yield more information about the relationship between attributes and predictions. For example, ensembles will give second-order (and higher) information about what pairs of variables are more important together than the sum of their individual importance. That information can actually help squeeze more performance out of penalized regression. You'll read more about that in Chapter 6.

Penalized Linear Regression: Regulating Linear Regression for Optimum Performance

As discussed in Chapter 3, this book addresses a class of problems called *function approximation*. The starting point for training a model for a function approximation problem is a data set containing a number of examples or instances. Each instance has an outcome (also called a *target*, *label*, *endpoint*, and so forth) and a number of attributes that are used to predict the outcome. Chapter 3 gave a simple illustrative example. It is repeated here in slightly modified form as Table 4-1.

Table 4-1: Example Training Set

OUTCOMES	FEATURE 1	FEATURE 2	FEATURE 3
$ Spent 2013	Gender	$ Spent 2012	Age
100	M	0.0	25
225	F	250	32
75	F	12	17

In this table, the outcomes are real-valued—making this a regression problem. The gender attribute (Feature 1) is two-valued, making it a categorical (or factor) attribute. The other two attributes are numeric. The goal with a function approximation problem is to (1) build a function relating the attributes to the outcome and (2) to minimize the error in some sense. Chapter 3 discussed some of the alternative error characterizations that might be employed to quantify overall error.

Data sets of the type shown in Table 4-1 are often represented by a column vector containing the outcomes (the leftmost column) and a matrix containing the attributes (the three columns of features). Asserting that the feature columns fit into a matrix abuses mathematical language a little. Strictly speaking, a matrix contains elements that are all defined over the same field. The contents of a matrix can all be real numbers, integers, complex numbers, binary numbers, and so on. They cannot, however, be a mixture of real numbers and categorical variables.

Here's an important point. Linear methods work with numeric data only. The data in Table 4-1 has non-numeric data, and therefore linear methods will not work for the data as shown. Fortunately, it is relatively simple to convert (or code) the data in Table 4-1 as numeric data. You'll learn the technique for coding categorical attributes as numeric attributes in the section titled "Incorporating Non-Numeric Attributes into Linear Methods." Given that the attributes are all real numbers (either in the initial problem formulation or by coding categorical attributes as real numbers), the data for a linear regression problem can be represented by two objects: Y and X, where Y is a column vector of outcomes, and where X is a matrix of real-valued attributes.

$$Y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

Equation 4-1: Vector of outcomes

In the example given in Table 4-1, Y is the column labeled Outcomes.

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{pmatrix}$$

Equation 4-2: Matrix of attributes

In the example given in Table 4-1, X is the set of columns that remains after excluding the Outcomes column.

The ith element from Y (y_i) is from the same instance as the ith row of X. The ith row of X will be denoted by x_i with a single subscript and given by $x_i = (x_{i1}, x_{i2}, \quad, x_{im})$. The ordinary least squares regression problem is to minimize the error between the y_i and a linear function x_i, the ith row of attributes from X (that is, to find a vector of real numbers β).

$$\beta = \begin{pmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{pmatrix}$$

Equation 4-3: β - Vector of model coefficients

and a scalar β_0 so so that each element y_i from Y is approximated by

$$Prediction\ of\ y_i = x_i * \beta + \beta_0$$
$$= x_{i1} * \beta_1 + x_{i2} * \beta_2 + \cdots + x_{im} * \beta_m + \beta_0$$

Equation 4-4: Linear relation between X and prediction of Y

You might be able to find the values for the β's by using your knowledge of the subject matter. In Table 4-1, for example, you might estimate that people will spend 10% more in 2013 than in 2012, that their purchases will increase by $10 per year of age, and that even newborns will purchase $50 of books. That gives you an equation to predict book spending that looks like Equation 4-5.

$$Predicted\ \$Spent\ 2013 = \$50 + 1.1 * (\$\text{Spent}\ 2012) + \$10 * \text{Age}$$

Equation 4-5: Predicting book spending

Equation 4-5 does not use the Gender variable because it's a categorical variable. (That gets covered in "Incorporating Non-Numeric Attributes into Linear Methods" and is ignored for now.) The predictions generated by Equation 4-5 do not exactly match the Outcomes (actual number) in Table 4-1.

Training Linear Models: Minimizing Errors and More

Finding the values for the β's by hand is not usually the best way, although it's always a good sanity check if you can manage it. In many problems, the size of

the problem or the interrelationships between the variables makes guessing the β's impossible. So, the approach taken is to find the multipliers on the attributes (the β's) by solving a minimization problem. The minimization problem is to find the values for the β's that makes the average squared error the smallest (but not zero).

Making the two sides of Equation 4-4 exactly equal usually means the model is overfit. The right side of Equation 4-4 is the predictive model you're going to train. Basically, it says that to make a prediction, you take each attribute, multiply by its corresponding beta, sum these products, and add a constant. *Training the model* means finding the numbers that make up the vector β and the constant, β_0. *Error* is defined as the difference between the actual value of y_i and the prediction o of y_i given by Equation 4-4. The average squared error is used to reduce the individual errors to a single number to be minimized. The square of the error is chosen because it's positive regardless of whether the error is positive or negative and because the square function facilitates some of the math. The formulation of the ordinary least squares regression problem is then to find β_0^*, β^* (the superscript * indicates that these are the best values for β's) that satisfy

$$\beta_0^*, \beta^* = argmin_{\beta_0, \beta}\left(\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \left(x_i * \beta + \beta_0\right)\right)^2\right)$$

Equation 4-6: Minimization problem for OLS

The notation *argmin* means "the arguments that minimize the following expression." The sum is over rows, where a row includes the attribute values and the corresponding labels. The expression inside the $()^2$ is the error between y_i and the linear function that's being used to approximate it. For the predicted $ spent on books in 2013, the expression inside the sum would be the values in the Outcome column minus the prediction calculated from Equation 4-4.

In English, Equation 4-6 says the vector beta star and the constant beta zero star are the values that minimize the expected prediction squared error—that is, the average squared error between y_i and the row of attributes predicted y_i over all data rows (i = 1, . . . , n). The minimization in Equation 4-5 yields the ordinary least squares values for this regression model. This machine learning model is a list of real numbers—the ones included in the vector β^* and the number β_0^*.

Adding a Coefficient Penalty to the OLS Formulation

The mathematical statement of the penalized linear regression problem is very similar to Equation 4-5. Ridge regression, which you saw in Chapter 3, gives an example of penalized linear regression. Ridge regression adds a penalty term to the basic ordinary least squares problem stated in Equation 4-5. The penalty term for ridge regression is shown in Equation 4-7.

$$\frac{\lambda\beta^T\beta}{2} = \frac{\lambda\left(\beta_1^2 + \beta_2^2 + \ldots + \beta_n^2\right)}{2}$$

Equation 4-7: Penalty applied to coefficients (betas)

The OLS problem in Equation 4-6 was to choose β's to minimize the sum of squared errors. The penalized regression problem adds the coefficient penalty in Equation 4-7 to the right-hand side of Equation 4-6. The minimization is then forced to balance the conflicting goals of minimizing the squared prediction error and the squared values of the coefficients. It is easy to minimize the sum of the squared coefficients by themselves. Just make the coefficient all zero. But that results in large prediction error. Similarly, the OLS solution minimizes the prediction errors by themselves but may result in a large coefficient penalty, depending on how large λ is.

Why does this make sense? To help develop some intuition for why this makes sense, think about the subset selection process that you saw in Chapter 3. Using subset selection eliminated overfitting by discarding some of the attributes, or equivalently by setting their coefficients to zero. Penalized regression does the same thing, but instead of reducing the coefficients of a few attributes all the way to zero like subset selection, penalized regression takes a little coefficient away from all of the attributes. Some limiting cases will also help visualize the approach.

The parameter λ can range anywhere between 0 and plus infinity. If λ=0, the penalty term goes away, and the problem reverts to being an ordinary least squares problem. If $\lambda \to \infty$, the penalty on the β's becomes so severe that it forces them all to zero. (Notice, however, that β_0 is not included in the penalty so the prediction becomes a constant independent of the x's.)

As you saw in the examples in Chapter 3, the ridge penalty can have a similar effect to leaving out some of the attributes. The process is to generate a whole family of solutions to the penalized version of the minimization problem shown in Equation 4-6. That meant solving the penalized minimization problem for a variety of different values of λ. Each of these solutions is then tested on out-of-sample data, and the solution that minimizes the out-of-sample error is used for making real-world predictions. Chapter 3 illustrated this sequence of steps using ridge regression.

Other Useful Coefficient Penalties—Manhattan and ElasticNet

The ridge penalty is not the only useful penalty that can be used for penalized regression. Any metric of vector length will work. You can gauge the length of a vector in a number of ways. Using different measures of length changes important properties of the solution. Ridge regression employed the metric of

Euclidean geometry (that is, the sum of the *squared* β's). Another useful algorithm called Lasso regression employs the metric of taxicab geometry called the *Manhattan length* or *L1 norm* (that is, the sum of the *absolute* β's). Lasso regression has some useful properties.

The difference between ridge regression and Lasso regression is the measure of length that each one uses for penalizing β, the vector of linear coefficients. Ridge uses squared Euclidean distance—the sum of the squares of the components of β. Lasso uses the sum of the absolute values of the components of β—called taxicab or Manhattan distance. The ridge penalty is the squared length of a straight line between zero and the vector space point β (distance as the crow flies). The Lasso penalty is like the distance that a taxicab would have to drive in a city where the streets constrain it to move north-south or east-west only. The lasso penalty is given by the following:

$$\lambda\|\beta_1\| = \lambda\left(|\beta_1| + |\beta_2| + \ldots + |\beta_n|\right)$$

Equation 4-8: Equation for Manhattan distance penalty

The double vertical bars are called norm bars. They are used to denote magnitude for things like vectors and operators. The subscript 1 on the right side of the norm bars denotes l_1 norm, which means the sum of absolute values. You'll also see this written with a capital L_1. Norm bars with a subscript 2 mean square root of the sum of squared values—Euclidean distance. These different coefficient penalty functions cause some important and useful changes in the solutions. One of the main differences is that the Lasso coefficient vector β^* is sparse, meaning that many of the coefficients are zero for large to moderate values of λ. By contrast, the ridge regression β^* is completely populated.

Why Lasso Penalty Leads to Sparse Coefficient Vectors

Figures 4-1 and 4-2 illustrate how this sparsity property stems directly from the form of the coefficient penalty function. These figures are for a problem that has two attributes: x1 and x2.

Both Figure 4-1 and 4-2 have two sets of curves. One set of curves is concentric ellipses that represent the ordinary least squares errors in Equation 4-6. The ellipses represent curves of constant sum squared error. You can think of them as being a topographic map of an elliptical depression in the ground. The error gets smaller for the more central ellipsis, just like the altitude of a depression in the ground gets smaller toward the bottom of the depression. The minimum point for the depression is marked with an x. The point x marks the ordinary least squares solution—where the solution lies if there is no coefficient penalty.

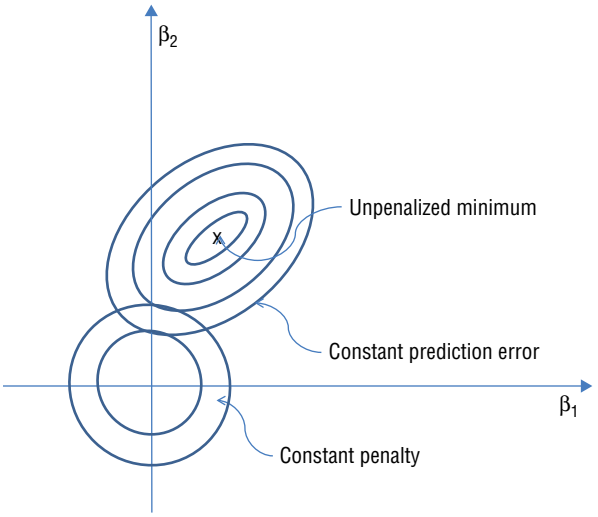


Figure 4-1: Optimum solutions with sum squared coefficient penalty.

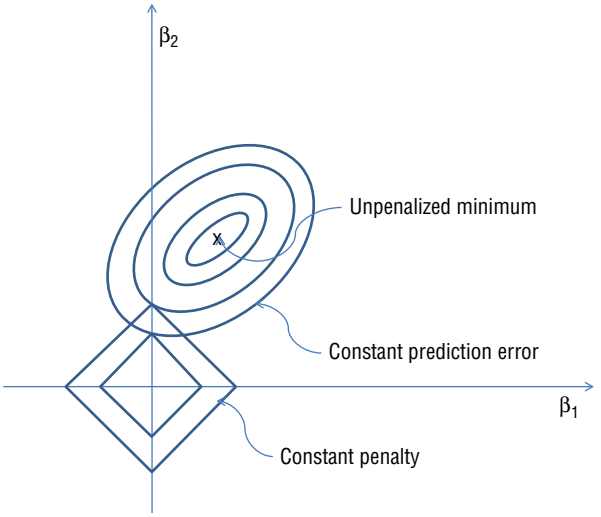


Figure 4-2: Optimum solutions with sum absolute value coefficient penalty.

The other sets of curves in Figures 4-1 and 4-2 represent the coefficient penalties from Equations 4-7 and 4-9—the ridge and Lasso penalties, respectively. In Figure 4-1, the curves representing the coefficient penalty are circles centered at the origin. The set of points where the sum of the squares of β_1 and β_2 is constant defines a circle. The shape of the curves of constant penalty is determined by the nature of the distance measure being used—circles (called hypersphere or l_2 ball in higher dimensions) for sum square penalty function and diamonds (or l_2 ball) for sum of absolute values. Smaller circles (or diamonds) correspond to

smaller value for the distance function. The shape is determined by the nature of the penalty function, but the value associated with each curve is determined by the non-negative parameter λ. Suppose that the two curves in Figures 4-1 correspond to sum of squares of β_1 and β_2 equal to 1.0 and 2.0 for the inner and outer circles. Then if $\lambda = 1$, the penalty associated with the two circles is 1 and 2. If $\lambda = 10$, the associated penalties are 10 and 20. The same is true of the diamonds in Figure 4-2. Increasing λ increases the penalty associated with the concentric diamonds in Figure 4-2.

The elliptical rings corresponding to the sum squares of the prediction error also get larger as the rings get farther from the unconstrained minimum, marked by an x in the figure. Minimizing the sum of these two functions, as indicated in Equation 4-6, requires a compromise somewhere in between the minimum for the prediction error and the coefficient penalty. Larger values of λ will pull the compromise closer to the minimum for the penalty (all zero coefficients). Smaller values of λ will pull the minimum closer to the unconstrained minimum prediction error (the x in Figures 4-1 and 4-2).

Here's where the distinction between sum of squared coefficient penalties and sum of absolute value penalties becomes important. The overall minimum for Equations 4-6 or 4-8 will always be at a point where the curve of constant penalty is tangent to the curve of squared prediction error. Figures 4-1 and 4-2 display two examples illustrating this tangency. The important point to make here is that in Figure 4-1 as λ changes and shifts the minimum point, the point of tangency for the sum of squares penalties (the circles) is generally a point that is not on either of the coordinate axes. Neither β_1 nor β_2 is zero. In Figure 4-2, by contrast, the point of tangency for the sum of absolute value stays stuck to the β_2-axis over a range of solutions. Along the β_2-axis, $\beta_1 = 0$.

A sparse coefficient vector is the algorithm's way of telling you that you can completely ignore some of the variables. When λ gets small enough, the best values of β_2 and β_1 will move off the β_2 axis, and both will be nonzero. The fact that a smaller penalty is required to make β_1 non-zero, gives an order to β_2 and β_1. In some sense, β_2 is more important than β_1 because it gets a nonzero coefficient for larger values of λ. Remember that these coefficients multiply attributes. If the coefficient corresponding to an attribute is zero, the algorithm is telling you that attribute is less important than the attributes that are getting nonzero coefficients. By scanning λ from large values to small ones, you can arrange all of the attributes in order of their importance. The next section shows this for a concrete problem and will show Python code that will make explicit the importance comparison between attributes as part of calculating solutions to Equation 4-8.

ElasticNet Penalty Includes Both Lasso and Ridge

Before seeing how to compute these coefficients, you need to know one more generalized statement of the penalized regression problem. This is called the

ElasticNet formulation. The ElasticNet formulation of the penalized regression problem is to use an adjustable blend of the ridge penalty and the Lasso penalty. ElasticNet introduces another parameter, α, that parameterizes the fraction of the total penalty that is the ridge penalty and the fraction that is Lasso penalty. The end point $\alpha = 1$ corresponds to all Lasso penalty and no ridge penalty.

With the ElasticNet formulation, both λ and α must be specified to solve for the coefficients for a linear model. Usually, the approach is to pick a value for α and solve for a range of λ's. You'll see the computational reasons for that later. In many cases, there's not a big performance difference between $\alpha = 1$ and $\alpha = 0$ or some intermediate value of α. Sometimes it will make a big difference, and it behooves you to check to a few different values of α to make sure that you're not sacrificing performance needlessly.

Solving the Penalized Linear Regression Problem

In the preceding section, you saw that determining a penalized linear regression model amounts to solving an optimization problem. A number of general-purpose numeric optimization algorithms will solve the optimization problems in Equations 4-6, 4-8, and 4-11, but the importance of the penalized linear regression problem has motivated researchers to develop specialized algorithms that generate solutions very rapidly. This section covers the basics of these algorithms and runs the code so that you can understand the mechanics of each algorithm. The section goes through the mechanics of two algorithms *least angle regression* or LARS and Glmnet. These two are chosen because they can be related to one another and to some of the methods you have already seen, such as ridge regression and forward stepwise regression. In addition, they are both very fast algorithms to train and are available as part of Python packages. Chapter 5, "Building Predictive Models Using Penalized Linear Methods," will use the Python packages incorporating these algorithms to explore example problems.

Understanding Least Angle Regression and Its Relationship to Forward Stepwise Regression

One very fast, very clever algorithm is the least-angle regression (LARS) algorithm developed by Bradley Efron, Trevor Hastie, Iain Johnstone and Robert Tibshirani (`http://en.wikipedia.org/wiki/Least-angle_regression`).[1] The LARS algorithm can be understood as a refinement to the forward stepwise algorithm that you saw in Chapter 3. The forward stepwise algorithm is summarized here:

Forward Stepwise Regression Algorithm

- Initialize all the β's equal to zero.

At each step

- Find residuals (errors) after using variables already chosen.
- Determine which unused variable best explains residuals and add it to the mix.

The LARS algorithm is very similar. The main difference with LARS is that instead of unreservedly incorporating each new attribute, it only partially incorporates them. The summary for the LARS algorithm is summarized here:

Least Angle Regression Algorithm

- Initialize all β's to zero.

At Each Step

- Determine which attribute has the largest correlation with the residuals.
- Increment that variable's coefficient by a small amount if the correlation is positive or decrement by a small amount if negative.

The LARS algorithm solves a slightly different problem from those listed earlier. However, the solutions it generates are usually the same as Lasso, and when there are differences, the differences are relatively minor. The reason for looking closely at the LARS algorithm is that it is very closely related to Lasso and to forward stepwise regression, and the LARS algorithm is easy to outline and relatively compact to code. By looking at the code for LARS, you'll get an understanding of what goes on inside more general ElasticNet solvers. More important, you'll see the issues and workarounds that accompany penalized regression solvers. Code implementing the LARS algorithm is shown in Listing 4-1.

There are three major sections to the code, described briefly here and then discussed in more detail:

1. Read in the data and headers and form it into a list of lists for the attributes and the labels.
2. Normalize the attributes and the labels.
3. Solve for the coefficients (β_0^*, β^*) that comprise the solution.

Listing 4-1: LARS Algorithm for Predicting Wine Taste—larsWine2.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
from sklearn import datasets, linear_model
from math import sqrt
import matplotlib.pyplot as plot
```

continues

continued

```
#read data into iterable
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-
databases/"
"wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

#Normalize columns in x and labels

nrows = len(xList)
ncols = len(xList[0])

#calculate means and variances
xMeans = []
xSD = []
for i in range(ncols):
    col = [xList[j][i] for j in range(nrows)]
    mean = sum(col)/nrows
    xMeans.append(mean)
    colDiff = [(xList[j][i] - mean) for j in range(nrows)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrows)])
    stdDev = sqrt(sumSq/nrows)
    xSD.append(stdDev)

#use calculate mean and standard deviation to normalize xList
xNormalized = []
for i in range(nrows):
    rowNormalized = [(xList[i][j] - xMeans[j])/xSD[j] \
         for j in range(ncols)]
    xNormalized.append(rowNormalized)

#Normalize labels
```

```
meanLabel = sum(labels)/nrows
sdLabel = sqrt(sum([(labels[i] - meanLabel) * (labels[i] -
    meanLabel) for i in range(nrows)])/nrows)

labelNormalized = [(labels[i] - meanLabel)/sdLabel \
   for i in range(nrows)]

#initialize a vector of coefficients beta
beta = [0.0] * ncols

#initialize matrix of betas at each step
betaMat = []
betaMat.append(list(beta))


#number of steps to take
nSteps = 350
stepSize = 0.004

for i in range(nSteps):
    #calculate residuals
    residuals = [0.0] * nrows
    for j in range(nrows):
        labelsHat = sum([xNormalized[j][k] * beta[k]
           for k in range(ncols)])
        residuals[j] = labelNormalized[j] - labelsHat

    #calculate correlation between attribute columns from
    #normalized wine and residual
    corr = [0.0] * ncols

    for j in range(ncols):
        corr[j] = sum([xNormalized[k][j] * residuals[k] \
           for k in range(nrows)]) / nrows

    iStar = 0
    corrStar = corr[0]

    for j in range(1, (ncols)):
        if abs(corrStar) < abs(corr[j]):
            iStar = j; corrStar = corr[j]

    beta[iStar] += stepSize * corrStar / abs(corrStar)
    betaMat.append(list(beta))


for i in range(ncols):
    #plot range of beta values for each attribute
    coefCurve = [betaMat[k][i] for k in range(nSteps)]
    xaxis = range(nSteps)
```

continues

continued

```
    plot.plot(xaxis, coefCurve)

plot.xlabel("Steps Taken")
plot.ylabel(("Coefficient Values"))
plot.show()
```

The first section reads in the entire file, separates off the headers and splits on `";"` to form the headers into a list of attribute names, splits the remaining rows into lists of floats, and segregates the attributes into a list of lists and the labels into a list. Ordinary Python lists are used for these data structures because the algorithm is going to want to iterate through the rows and columns, and Pandas data frames seem slow for this purpose.

The second section uses the same normalization that you saw in Chapter 2, "Understand the Problem by Understanding the Data." In Chapter 2, normalization of the attributes was used to bring attributes into commensurate scales so that they'd plot conveniently and fully occupy the same scale. Normalization is usually done as the first step in penalized linear regression for much the same reason.

Each step in the LARS algorithm increments one of the β's by a fixed amount. If the attributes have different scales, this fixed increment means different things to different attributes. Also, changing the scale on one of the attributes (say from miles to feet) makes the answers come out differently. For these reasons, penalized linear regression packages generally normalize using the common normalization that you saw in Chapter 2. They normalize to zero mean (by subtracting the mean) and unit standard deviation (by dividing the result by standard deviation). Packages will often give you the option of not normalizing, but I've never heard a good reason for not normalizing.

The third and final section solves for β_0^*, β^*. Because the algorithm is running on the normalized variables, there's no need for the intercept β_0^*. That would normally account for any difference between the labels and the weighted attributes. Because all the attributes have been normalized to zero mean, there's no offset between them and no purpose for β_0^*. Notice that two beta-related lists are initialized. One is called *beta* and has the same number of elements as the number of attributes—one weight for each attribute. The other is a matrix-like list of lists that will house a list of betas for each step in the LARS algorithm. This gets into a key concept with penalized linear regression and modern machine learning algorithms in general.

How LARS Generates Hundreds of Models of Varying Complexity

Modern machine learning algorithms in general, and penalized linear regression in particular, generate families of solutions, not just single solutions. Look back at Equations 4-6, 4-8, and 4-11. On the left side of those equations are the ,β-′*s*, and on the right hand side are all numeric values that are fixed by the data

available for the problem with one exception. In Equations 4-6 and 4-8, there is a parameter λ that has to be determined some other way. As was pointed out in the discussion of those equations, when λ=*0*, the problems reduce to ordinary least squares regression, and when $\lambda \to \infty$, $\beta^* \to 0$. So, the β's depend on the parameter λ in the problems stated in Equations 4-6, 4-8, and 4-11.

The LARS algorithm doesn't explicitly deal with λ values, but it has the same effect. The LARS algorithm starts with β's equal to zero and then adds a small increment to whichever of the β's will reduce the error the most. The small increment that's added increases the sum of absolute values of the β's by the amount of the increment. If the increment is small and if it's spent on the best of the attributes, the process has the effect of solving the minimization problem in Equation 4-8. You can trace the evolution of this process in Listing 4-1.

The basic iteration is just a few lines of code at the beginning of the for-loop iterating for `nSteps`. The starting point for the iteration is a value for the β's. On the first pass, those are all set to zero. On subsequent passes, they come from the result of the last pass. There are two steps in the iteration. First, the β's are used to calculate residuals. The term *residuals* means the difference between observed outcome and predicted outcome. In this case the predictive method consists of multiplying each attribute times a corresponding element from β and then summing the products. The second step is to find the correlation between each of the attributes and the residuals to determine which attribute will contribute the most to reducing the residual (error). The correlation between two variables is the product of their variations from their means normalized by their individual standard deviations.

Variables that are scaled versions of one another will have correlations of plus one or minus one depending on whether the scaling between them is positive or negative. If two variables vary independently of one another, their correlation is zero. The Wikipedia page on correlation, `http://en.wikipedia.org/wiki/Correlation_and_dependence`, gives good illustrations of variables having other degrees of correlation with one another. The list named corr contains the result of the calculation for each attribute. You may notice that strictly speaking the code omits calculation of the standard deviation of the mean, residuals, and normalized attributes. That works here because the attributes have been normalized to all have standard deviation one and because the resulting values are going to be used to find the biggest correlation and multiplying all the values by a constant won't change that order.

Once the correlations are calculated, it's a simple matter to determine which attribute has the largest correlation with the residuals (largest in absolute value). The corresponding element from the list of β's is incremented by a small amount. The increment is positive if the correlation is positive and negative otherwise. The new value of the β's is then used to rerun the iteration.

The net result from the LARS algorithm are the coefficient curves shown in Figure 4-3. The way to view these is to imagine a point along the "steps taken"

axis in the graph. At that point, a vertical line will pass through all the coefficient curves. The values at which the vertical line intersects the coefficient curves are the coefficients at that step in the evolution of the LARS algorithm. If 350 steps are used to generate the curves, there are 350 sets of coefficients. Each one optimizes Equation 4-8 for some value of λ. That raises the question of which one should you use. That question will be addressed shortly.

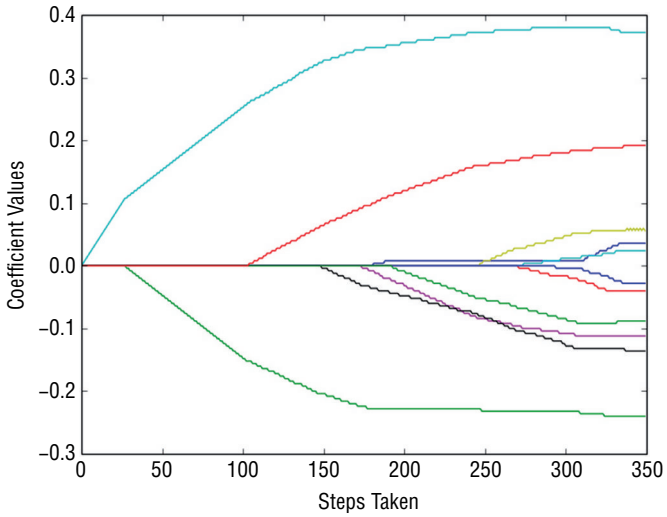


Figure 4-3: Coefficient curves for LARS regression on wine data.

Notice that for the first 25 steps or so, only one of the coefficients is nonzero. This is the sparsity property that comes with Lasso regression. The coefficient that is the first to move off zero is alcohol; for a while, that's the only variable being used by LARS regression. Then a second variable comes into play. This process continues until all the variables are being used in the solution. The order in which coefficients move off zero can be used as an indication of the rank order of importance of the variables. If you had to discard a variable, you'd want to discard one that came in last rather than the one that came in first.

THE IMPORTANCE OF IMPORTANCE

This property of indicating the importance rank of the variables is an important feature of penalized regression methods. It makes them a handy tool to use early in your development process because they'll help you make decisions about what variables to keep and which ones to discard—a process called feature engineering. You'll see later that tree ensembles also yield measures of variable importance. Not all machine learning methods give this sort of information. You could always generate the ordering by trying all combinations of one variable, then two variables, and so on. But even with the mere 10 attributes in the wine data, it's prohibitive to make the 10 factorial training passes required to try all possible subsets.

Choosing the Best Model from the Hundreds LARS Generates

Now you've got 350 possible solutions to the problem of predicting wine taste score from the chemical properties of the wine. How do you choose the best one? To choose which of the curves you'll use, you need to determine how each of the 350 choices performs. As discussed in Chapter 3, *performance* means performance on out of sample data. Chapter 3 outlined several methods for holding out data from the training process to use it to determine performance. Listing 4-2 shows the code for performing 10-fold cross-validation to determine the best set of coefficients to deploy.

Ten-fold cross-validation is the process of dividing the input data into 10 more or less equal groups, removing one of the groups from the data, training on the remainder, and then testing on the removed group. By cycling through all 10 of the groups and removing them one at a time for testing, you can develop a good estimate of the error and of the estimate's variability.

Listing 4-2: 10-Fold Cross-Validation to Determine Best Set of Coefficients—larsWineCV.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
from sklearn import datasets, linear_model
from math import sqrt
import matplotlib.pyplot as plot


#read data into iterable
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
"databases/wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)
```

continues

continued

```
#Normalize columns in x and labels

nrows = len(xList)
ncols = len(xList[0])

#calculate means and variances
xMeans = []
xSD = []
for i in range(ncols):
    col = [xList[j][i] for j in range(nrows)]
    mean = sum(col)/nrows
    xMeans.append(mean)
    colDiff = [(xList[j][i] - mean) for j in range(nrows)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrows)])
    stdDev = sqrt(sumSq/nrows)
    xSD.append(stdDev)

#use calculated mean and standard deviation to normalize xList
xNormalized = []
for i in range(nrows):
    rowNormalized = [(xList[i][j] - xMeans[j])/xSD[j] \
       for j in range(ncols)]
    xNormalized.append(rowNormalized)

#Normalize labels
meanLabel = sum(labels)/nrows
sdLabel = sqrt(sum([(labels[i] - meanLabel) * (labels[i] - meanLabel)
   for i in range(nrows)])/nrows)

labelNormalized = [(labels[i] - meanLabel)/sdLabel \
   for i in range(nrows)]

#Build cross-validation loop to determine best coefficient values.

#number of cross-validation folds
nxval = 10

#number of steps and step size
nSteps = 350
stepSize = 0.004

#initialize list for storing errors.
errors = []
for i in range(nSteps):
    b = []
    errors.append(b)


for ixval in range(nxval):
    #Define test and training index sets
```

```
idxTest = [a for a in range(nrows) if a%nxval == ixval*nxval]
idxTrain = [a for a in range(nrows) if a%nxval != ixval*nxval]

#Define test and training attribute and label sets
xTrain = [xNormalized[r] for r in idxTrain]
xTest = [xNormalized[r] for r in idxTest]
labelTrain = [labelNormalized[r] for r in idxTrain]
labelTest = [labelNormalized[r] for r in idxTest]

#Train LARS regression on Training Data
nrowsTrain = len(idxTrain)
nrowsTest = len(idxTest)

#initialize a vector of coefficients beta
beta = [0.0] * ncols

#initialize matrix of betas at each step
betaMat = []
betaMat.append(list(beta))

for iStep in range(nSteps):
    #calculate residuals
    residuals = [0.0] * nrows
    for j in range(nrowsTrain):
        labelsHat = sum([xTrain[j][k] * beta[k]
           for k in range(ncols)])
        residuals[j] = labelTrain[j] - labelsHat

    #calculate correlation between attribute columns
    #from normalized wine and residual
    corr = [0.0] * ncols

    for j in range(ncols):
        corr[j] = sum([xTrain[k][j] * residuals[k] \
           for k in range(nrowsTrain)]) / nrowsTrain

    iStar = 0
    corrStar = corr[0]

    for j in range(1, (ncols)):
        if abs(corrStar) < abs(corr[j]):
            iStar = j; corrStar = corr[j]

    beta[iStar] += stepSize * corrStar / abs(corrStar)
    betaMat.append(list(beta))

    #Use beta just calculated to predict and accumulate out of
    #sample error - not being used in the calc of beta
    for j in range(nrowsTest):
        labelsHat = sum([xTest[j][k] * beta[k] for k in range
```

continues

continued

```
            (ncols)])
            err = labelTest[j] - labelsHat
            errors[iStep].append(err)

cvCurve = []
for errVect in errors:
    mse = sum([x*x for x in errVect])/len(errVect)
    cvCurve.append(mse)

minMse = min(cvCurve)
minPt = [i for i in range(len(cvCurve)) if cvCurve[i] == minMse ][0]
print("Minimum Mean Square Error", minMse)
print("Index of Minimum Mean Square Error", minPt)

xaxis = range(len(cvCurve))
plot.plot(xaxis, cvCurve)

plot.xlabel("Steps Taken")
plot.ylabel(("Mean Square Error"))
plot.show()

Printed Output:
('Minimum Mean Square Error', 0.5873018933136459)
('Index of Minimum Mean Square Error', 311)
```

Mechanizing Cross-Validation for Model Selection in Python Code

The code in Listing 4-2 begins similarly to the code in Listing 4-1. The differences become clear at the cross-validation loop that is looping `nxval` times. In this case `nxval = 10`, but it could be set to other values as well. The tradeoffs with how many folds to use are that smaller numbers of folds mean that you're training on less of the data. If you take 5 folds, then you're leaving out 20% each training pass. If you take 10 folds, you're only leaving out 10%. As you saw in Chapter 3, training on less data causes deterioration in the accuracy your algorithm will achieve. However, taking more folds means making more passes through the training process. That can be cumbersome in terms of the clock or calendar time required for training.

Just ahead of the cross-validation loop, an error list gets initialized. This error list will consist of a list of errors for each step in the evolution of the LARS algorithm. It will accumulate the errors for each step over all 10 of the cross-validation folds. Just inside the cross-validation loop, you'll see definition of training and test sets. I typically use a modulus function to define these sets unless there's some reason not to. For example, sometimes you may need to do what's called *stratified sampling*. Suppose that you're trying to build a classifier on data that are unbalanced, so there are very few of one of the classes. You want for the training sets to be representative of the full data set. You may need to segregate the data by classes so that the classes are represented in both in-sample and out-of-sample data.

You may prefer to use a random function to define training and test sets. You do need to be aware of any patterning in the data set that would interact with the sampling process adversely (that is, if observations are not exchangeable). For example, if data were taken daily during the work week, then using the modulus function with five-fold cross-validation might result in one set having all the Mondays and another having all the Tuesdays, and so on.

Accumulating Errors on Each Cross-Validation Fold and Evaluating Results

Once the training and test sets are defined along with a few constants, the iteration of the LARS algorithm begins. This is very similar to the process defined in Listing 4-1, with a couple of important differences. First, the basic iteration of the algorithm is carried out on the training set instead of the full data set and second, at each step in the iteration and for each cross-validation fold the current values of the β's are used along with the test attributes and test labels to ascertain the error on the test set for that step. You'll see that calculation at the bottom of the cross-validation loop. Each time β is updated, it is applied to the test data, and the error is accumulated in the appropriate list in "error." It's a simple matter to then square and average each of the lists in error. This produces a curve of the mean square error (MSE) at each iteration, averaged over all 10 of the cross-validation folds.

You might worry whether the test data is being used properly. It's always important to be vigilant about letting the test data leak into the training process. There are numerous ways to trick oneself into violating this necessity. In this case, you'll notice that the test data is not used in the calculation of the increments of β. Only the training data is being used there.

Practical Considerations with Model Selection and Training Sequence

The curve of MSE versus number of steps in the LARS iteration is shown in Figure 4-4. This curve exhibits a fairly common pattern. It decreases more or less monotonically over its whole range. Strictly speaking, it does have a minimum point at around 311, as indicated in the associated printed output from the program. But the graph shows that the minimum is fairly weak, not very sharp. In some cases, this curve will have a sharp minimum at some point and will increase markedly to the right and left of the minimum. You use the result of cross-validation to determine which of the 350 solutions generated by LARS should be used for making predictions. In this case, the minimum is at step 311. The 311th set of β's would be the coefficients to deploy. When there's any ambiguity about the best solution to deploy, it's usually best to deploy the more conservative solution. More conservative for penalized regression means the one with smaller coefficient values. By convention, out-of-sample performance is usually portrayed with the less-complex models on the left and the more-complex models on the right. Less-complex models have better generalization error; that is, they perform more predictably on new data. The more conservative model would be the one more to the left side of the out of sample performance graph.

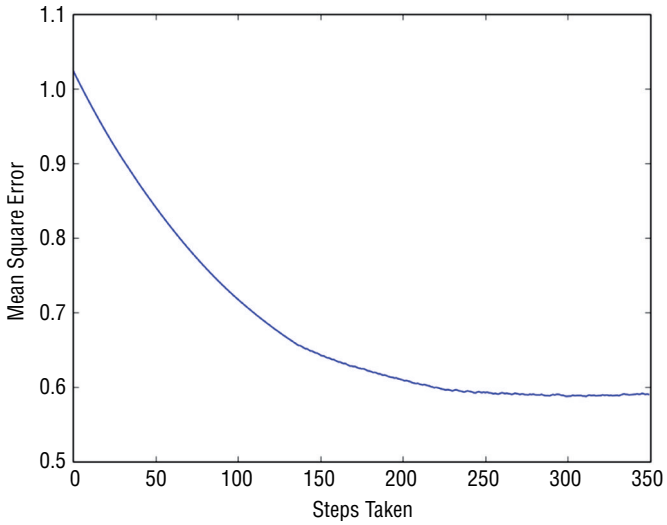


Figure 4-4: Cross-validated mean square error for LARS on wine data.

This description of the LARS algorithm and of the cross-validation process has gone through training the algorithm on the whole data set first, then running cross-validation second. In practice, you'll probably first run cross-validation and then train the algorithm on the whole data set. The purpose of cross-validation is to determine what level of MSE (or other) performance you'll be able to achieve and to learn how complicated a model your data set will sustain. If you recall, Chapter 3 discussed the issues of data set size and model complexity. Cross-validation (or other process for setting aside data to get a sound estimate of performance) is how you determine the best model complexity for the model you will deploy. You determine the complexity but not the specific model (that is, not the specific set of β's). As you can see in Listing 4-2, with 10-fold cross-validation, you've actually trained 10 models, and there's no way to decide among the 10. Best practice is to train on the full data set and to use the cross-validation results to determine which of the models determine which of the models to deploy. In the example shown in Code Listing 4-2, cross-validation gives a minimum MSE of 0.59 at the 311th step in the training process. The coefficient curves in Figure 4-5 were trained on the full data set. The digression into cross-validation was motivated by not knowing which of the 350 sets of coefficients represented in Figure 4-5 should be deployed. Cross-validation has yielded a sound estimate of the MSE and tells us to deploy the 311th model from training on the full data set.

Using Glmnet: Very Fast and Very General

The glmnet algorithm was developed by Professor Jerome Friedman and his colleagues at Stanford.[2] The glmnet algorithm solves the ElasticNet problem given by Equation 4-11. Recall that the ElasticNet problem incorporates a

generalization of the penalty function that includes both the Lasso penalty (sum of absolute values) and the ridge penalty (sum of squares). ElasticNet has a parameter λ that determines how heavily the coefficient penalty is penalized compared to the fit error. It also has a parameter that determines how close the penalty is to ridge (α=*0*) or Lasso (α=*1*). The glmnet algorithm yields the full coefficient curves, similar to the LARS algorithm. Whereas the LARS algorithm accumulates quanta of coefficient into the β's to drive the curves forward, the glmnet algorithm makes steady reductions in the λ's to drive the coefficient curves forward. Equation 4-9 shows the key equation from Friedman's paper—the key iterative equation for the coefficients that solve Equation 11—the ElasticNet equation.

$$\beta_j^{\sim} \leftarrow \frac{S\left(\frac{1}{m}\sum_{i=1}^{m} x_{ij} r_i + \beta_j^{\sim}, \lambda\alpha\right)}{1+\lambda(1-\alpha)}$$

Equation 4-9: Coordinate-wise update for glmnet

Equation 4-9 is a combination of Equations 5 and 8 in Friedman's paper (for those of you who would like to follow the math). It looks complicated, but a little inspection will reveal some similarities and relationships to the LARS method that you saw in the last section.

Comparison of the Mechanics of Glmnet and LARS Algorithms

Equation 4-9 gives the basic update equation for the β's. The update equation for LARS was "find the attribute with the largest magnitude correlation with the residual and increment (or decrement) its coefficient by a small fixed amount." The updated Equation 4-9 is a little more involved. It has an arrow instead of an equals sign. The arrow means something like "gets mapped to." Notice that $\beta_j^{\sim}$ appears on both sides of the arrow. On the right side of the arrow is the old value of $\beta_j^{\sim}$, and on the left side (the direction the arrow points) is the new value of $\beta_j^{\sim}$. After several passes through, the iteration inferred in 4-12, $\beta_j^{\sim}$ stops changing. (More precisely, the change becomes insignificant.) Once $\beta_j^{\sim}$ stops changing, the algorithm has arrived at a solution for the given values of λ and α. It's time to move to the next point in the coefficient curve.

The first thing to notice is the expression $x_{ij} r_i$ inside the sum. The sum of $x_{ij} r_i$ over i (that is over rows of data) yields the correlation between the jth attribute and the residual. Recall that with LARS regression at each step through the algorithm each attribute was correlated against the residuals. In the LARS algorithm, those correlations were tested to see which attribute had the biggest correlation with the residual, and the coefficient corresponding to the attribute with the highest correlation was incremented. With the glmnet algorithm, the correlation is used somewhat differently.

With glmnet, the correlation between the residuals is used to calculate how much each coefficient ought to be changed in magnitude. But the result passes through the function S() before resulting in a change in $\tilde{\beta}_j$. The function S() is the Lasso coefficient shrinkage function. It is plotted in Figure 4-5. As you can see in Figure 4-5, if the first input is smaller than the second, the output is zero. If the first input is larger than the second, the output is the first input reduced in magnitude by the second. This is called a soft limiter.

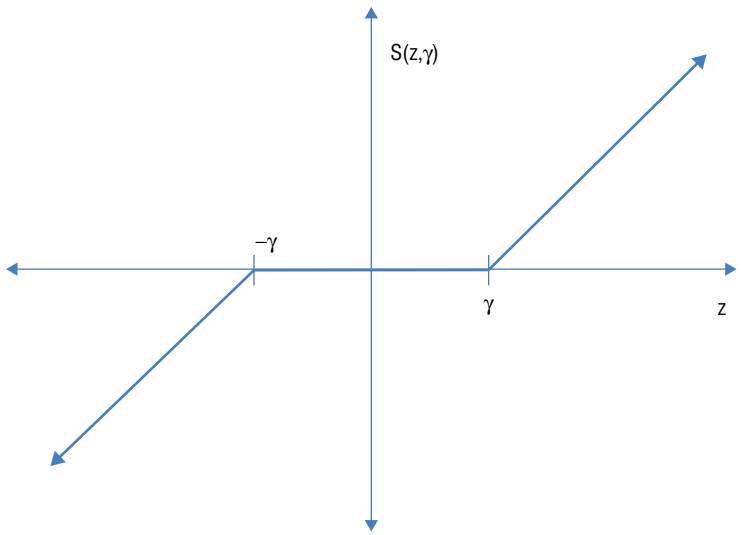


Figure 4-5: Plot of S() function

Listing 4-3 shows code for the glmnet algorithm. You can see in the code how Equation 4-12, for updating the β's, is used to generate ElasticNet coefficient curves. The code in Listing 4-3 is annotated with equation number from Friedman's paper. The paper is very accessible, and you can refer to it to get more mathematical details if you're interested.

Initializing and Iterating the Glmnet Algorithm

The iteration starts with a large value of λ. It begins with a value for λ that is large enough to make all the β's zero. You can see how to calculate the starting value for λ by reference to Equation 4-9. The function S() in Equation 4-12 gives zero for output if its first input (the correlation of $x_{ij}r_i$) is less than the second—λα. The iteration starts with all the β's equal to zero, so the residual is equal to the raw labels. The code for determining the starting lambda calculates the correlations for each of the attributes and the labels, finds the largest in magnitude, and then solves for the value of λ that makes the largest correlation just equal λα. That is the largest value of λ that results in all zero β's.

Then the iteration begins by reducing λ. This is accomplished by multiplying λ by a number slightly less than one. Friedman suggests that the multiplier be

selected so that $\lambda^{100} = 0.001$. That gives a value of roughly 0.93. If the algorithm runs for a long time without converging, then the multiplier on λ needs to be made closer to 1. In Friedman's code, the mechanism for accomplishing this is to increase the number of steps from 100 to, say, 200 so that it takes 200 steps to reduce the starting λ to 0.001 of its starting value. In the Listing 4-3, you've got control of the multiplier directly. The coefficient curves are shown in Figure 4-8.

Listing 4-3: Glmnet Algorithm—glmnetWine.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
from sklearn import datasets, linear_model
from math import sqrt
import matplotlib.pyplot as plot
def S(z, gamma):
    if gamma >= abs(z):
        return 0.0
    return (z/abs(z))*(abs(z) - gamma)

#read data into iterable
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
"databases/wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

#Normalize columns in x and labels

nrows = len(xList)
ncols = len(xList[0])

#calculate means and variances
xMeans = []
```

continues

continued

```
xSD = []
for i in range(ncols):
    col = [xList[j][i] for j in range(nrows)]
    mean = sum(col)/nrows
    xMeans.append(mean)
    colDiff = [(xList[j][i] - mean) for j in range(nrows)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrows)])
    stdDev = sqrt(sumSq/nrows)
    xSD.append(stdDev)

#use calculate mean and standard deviation to normalize xList
xNormalized = []
for i in range(nrows):
    rowNormalized = [(xList[i][j] - xMeans[j])/xSD[j]
                     for j in range(ncols)]
    xNormalized.append(rowNormalized)

#Normalize labels
meanLabel = sum(labels)/nrows
sdLabel = sqrt(sum([(labels[i] - meanLabel) * (labels[i] -
                meanLabel) for i in range(nrows)])/nrows)

labelNormalized = [(labels[i] - meanLabel)/sdLabel for i in
range(nrows)]

#select value for alpha parameter

alpha = 1.0

#make a pass through the data to determine value of lambda that
# just suppresses all coefficients.
#start with betas all equal to zero.


xy = [0.0]*ncols
for i in range(nrows):
    for j in range(ncols):
        xy[j] += xNormalized[i][j] * labelNormalized[i]

maxXY = 0.0
for i in range(ncols):
    val = abs(xy[i])/nrows
    if val > maxXY:
        maxXY = val

#calculate starting value for lambda
lam = maxXY/alpha

#this value of lambda corresponds to beta = list of 0's
#initialize a vector of coefficients beta
beta = [0.0] * ncols

#initialize matrix of betas at each step
```

```
betaMat = []
betaMat.append(list(beta))

#begin iteration
nSteps = 100
lamMult = 0.93 #100 steps gives reduction by factor of 1000 in
               # lambda (recommended by authors)
nzList = []

for iStep in range(nSteps):
    #make lambda smaller so that some coefficient becomes non-zero
    lam = lam * lamMult

    deltaBeta = 100.0
    eps = 0.01
    iterStep = 0
    betaInner = list(beta)
    while deltaBeta > eps:
        iterStep += 1
        if iterStep > 100: break

        #cycle through attributes and update one-at-a-time
        #record starting value for comparison
        betaStart = list(betaInner)
        for iCol in range(ncols):

            xyj = 0.0
            for i in range(nrows):
                #calculate residual with current value of beta
                labelHat = sum([xNormalized[i][k]*betaInner[k]
                                for k in range(ncols)])
                residual = labelNormalized[i] - labelHat

                xyj += xNormalized[i][iCol] * residual

            uncBeta = xyj/nrows + betaInner[iCol]
            betaInner[iCol] = S(uncBeta, lam * alpha) / (1 +
                                          lam * (1 - alpha))

        sumDiff = sum([abs(betaInner[n] - betaStart[n])
                       for n in range(ncols)])
        sumBeta = sum([abs(betaInner[n]) for n in range(ncols)])
        deltaBeta = sumDiff/sumBeta
    print(iStep, iterStep)
    beta = betaInner

    #add newly determined beta to list
    betaMat.append(beta)

    #keep track of the order in which the betas become non-zero
    nzBeta = [index for index in range(ncols) if beta[index] != 0.0]
    for q in nzBeta:
```

continues

continued

```
        if (q in nzList) == False:
            nzList.append(q)

#print out the ordered list of betas
nameList = [names[nzList[i]] for i in range(len(nzList))]
print(nameList)

nPts = len(betaMat)
for i in range(ncols):
    #plot range of beta values for each attribute
    coefCurve = [betaMat[k][i] for k in range(nPts)]
    xaxis = range(nPts)
    plot.plot(xaxis, coefCurve)

plot.xlabel("Steps Taken")
plot.ylabel(("Coefficient Values"))
plot.show()

#Printed Output:
#['"alcohol"', '"volatile acidity"', '"sulphates"',
#'"total sulfur dioxide"', '"chlorides"', '"fixed acidity"', '"pH"',
#'"free sulfur dioxide"', '"residual sugar"', '"citric acid"',
#'"density"']
```

Figure 4-8 shows the coefficient curves generated by Listing 4-3. The curves look similar in character to those generated by LARS and shown in Figure 4-6—similar but not identical. LARS and Lasso often give the same curves, but sometimes give somewhat different results. The only way to tell which one is superior is to try them both against out-of-sample data and see which one gives the best performance.

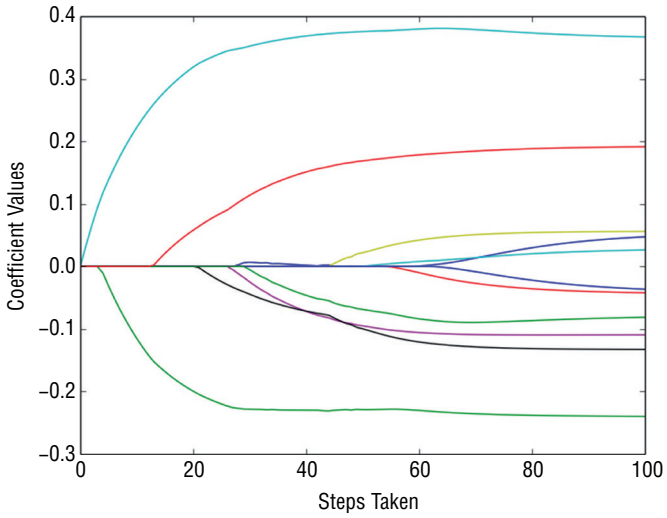


Figure 4-6: Coefficient curves for glmnet models for predicting wine taste

The development process for a Lasso model is the same as for LARS. Use one of the methods described in Chapter 3 for testing on out-of-sample data (n-fold cross-validation, for example). Use the results on out-of-sample data to determine the optimum model complexity. Then train on the full data set to build coefficient curves and pick the step in the coefficient curves that out-of-sample testing shows to be the optimum.

This section has gone through two solution approaches for solving the minimization problems that define penalized linear regression models. You've seen how these two methods work algorithmically, how they relate to one another and what the code looks like to implement them. This should give you a firm foundation for using the packages available in Python that implement these algorithms. It also puts you in a good position to understand various extensions to the models that will be covered in the next section and that will be used in the examples that you'll see in Chapter 5.

Extensions to Linear Regression with Numeric Input

So far, the development has focused on regression problems—problems where the outcomes being predicted take real number values. How can the machinery discussed be applied to classification problems—problems where the outcomes take two (or more) discrete values like "click" or "not click"? There are several ways to extend what you've seen so far to cover classification problems.

Solving Classification Problems with Penalized Regression

For binary classification problems, you'll often get good results by coding the binary values as real numbers. This simple procedure codes one of the two binary values as a 1 and the other as a 0 (or +1 and –1). With that simple arrangement, the list of labels becomes a list of real numbers, and the algorithms already discussed can be employed. This is often a good alternative even though there are more sophisticated approaches. This simple coding approach usually trains faster than more sophisticated approaches and that can be important.

Listing 4-4 gives an example of using the method of substituting numeric 0 or 1 labels for class membership in the rocks versus mines data set. You'll recall from Chapter 2 that the rocks versus mines data set presents a classification problem. The data set comes from an experiment to determine if sonar can be used to detect unexploded mines left in the water. Various other objects besides mines will reflect the sonar's sound waves. The prediction problem is to determine whether the reflected waves come from an unexploded mine or from rocks on the sea floor.

The sonar in the experiment uses what's called a chirped waveform. A chirped waveform is one that rises (or falls) in frequency over the duration of

the transmitted sonar pulse. The 60 attributes in the rocks versus mines data set are the returned pulse sampled at 60 different times, which correspond to 60 different frequencies in the chirped pulse.

Listing 4-4 demonstrates how to convert the classification labels R and M into 0.0 and 1.0 to convert the problem into an ordinary regression problem. The code then uses the LARS algorithm to build a classifier. Listing 4-4 goes through a single pass on the full data set. As discussed in the last section, you'll want to use cross-validation or some other holdout procedure to choose the optimal model complexity. Chapter 5 goes through those design steps and performance comparisons on this data set. The point here is for you to see how to apply the regression tools you've already seen to a classification problem.

Listing 4-4: Converting a Classification Problem to an Ordinary Regression Problem by Assigning Numeric Values to Binary Labels

```
__author__ = 'mike_bowles'
import urllib2
import sys
from math import sqrt
import matplotlib.pyplot as plot

#read data from uci data repository
target_url = "https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")
data = urllib2.urlopen(target_url)


#arrange data into list for labels and list of lists for attributes
xList = []


for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)

#separate labels from attributes, convert from attributes from
#string to numeric and convert "M" to 1 and "R" to 0

xNum = []
labels = []

for row in xList:
    lastCol = row.pop()
    if lastCol == "M":
        labels.append(1.0)
    else:
        labels.append(0.0)
    attrRow = [float(elt) for elt in row]
    xNum.append(attrRow)
```

```
#number of rows and columns in x matrix
nrow = len(xNum)
ncol = len(xNum[1])



#calculate means and variances
xMeans = []
xSD = []
for i in range(ncol):
    col = [xNum[j][i] for j in range(nrow)]
    mean = sum(col)/nrow
    xMeans.append(mean)
    colDiff = [(xNum[j][i] - mean) for j in range(nrow)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrow)])
    stdDev = sqrt(sumSq/nrow)
    xSD.append(stdDev)

#use calculate mean and standard deviation to normalize xNum
xNormalized = []
for i in range(nrow):
    rowNormalized = [(xNum[i][j] - xMeans[j])/xSD[j] \
       for j in range(ncol)]
    xNormalized.append(rowNormalized)

#Normalize labels
meanLabel = sum(labels)/nrow
sdLabel = sqrt(sum([(labels[i] - meanLabel) * (labels[i] -
   meanLabel) for i in range(nrow)])/nrow)

labelNormalized = [(labels[i] - meanLabel)/sdLabel for i in range(nrow)]

#initialize a vector of coefficients beta
beta = [0.0] * ncol

#initialize matrix of betas at each step
betaMat = []
betaMat.append(list(beta))


#number of steps to take
nSteps = 350
stepSize = 0.004
nzList = []

for i in range(nSteps):
    #calculate residuals
    residuals = [0.0] * nrow
    for j in range(nrow):
        labelsHat = sum([xNormalized[j][k] * beta[k]
           for k in range(ncol)])
        residuals[j] = labelNormalized[j] - labelsHat
```

continues

continued

```
    #calculate correlation between attribute columns from
    #normalized X and residual
    corr = [0.0] * ncol

    for j in range(ncol):
        corr[j] = sum([xNormalized[k][j] * residuals[k]
           for k in range(nrow)]) / nrow

    iStar = 0
    corrStar = corr[0]

    for j in range(1, (ncol)):
        if abs(corrStar) < abs(corr[j]):
            iStar = j; corrStar = corr[j]

    beta[iStar] += stepSize * corrStar / abs(corrStar)
    betaMat.append(list(beta))


    nzBeta = [index for index in range(ncol) if beta[index] != 0.0]
    for q in nzBeta:
        if (q in nzList) == False:
            nzList.append(q)

#make up names for columns of xNum
names = ['V' + str(i) for i in range(ncol)]
nameList = [names[nzList[i]] for i in range(len(nzList))]

print(nameList)
for i in range(ncol):
    #plot range of beta values for each attribute
    coefCurve = [betaMat[k][i] for k in range(nSteps)]
    xaxis = range(nSteps)
    plot.plot(xaxis, coefCurve)

plot.xlabel("Steps Taken")
plot.ylabel(("Coefficient Values"))
plot.show()

#Printed Output:
#['V10', 'V48', 'V44', 'V11', 'V35', 'V51', 'V20', 'V3', 'V21', 'V15',
# 'V43', 'V0', 'V22', 'V45', 'V53', 'V27', 'V30', 'V50', 'V58', 'V46',
# 'V56', 'V28', 'V39']
```

Figure 4-7 shows the coefficient curves developed by the LARS algorithm. They are similar in character to the curves you saw for the wine taste prediction problem. However, there are more curves because the rocks versus mines data set has more attributes. (The rock versus mines data has 60 attributes and 208 rows of data.) From the discussion in Chapter 3, you might expect that the

optimum solution won't use all the attributes. You'll see how that tradeoff turns out in Chapter 5, which concentrates on solutions to this and other problems and comparisons between different approaches.

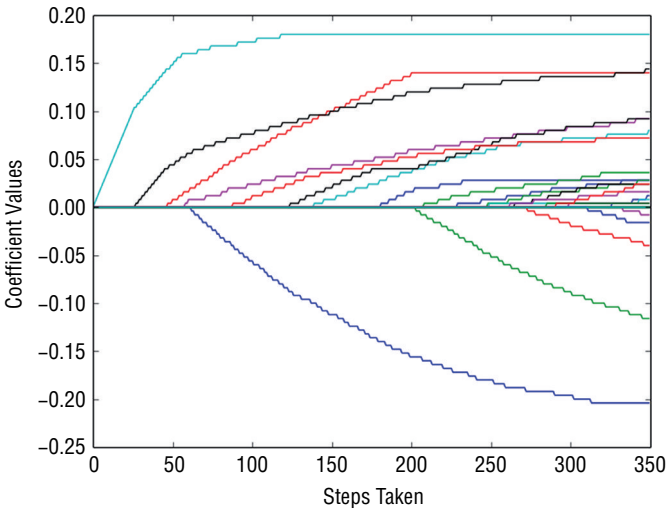


Figure 4-7: Coefficient curves for rocks versus mines classification problem solved by converting to labels

Another approach is to formulate the problem in terms of the likelihoods of the two outcomes in the problem. That leads to what's called *logistic regression*. The glmnet algorithm can be cast in that framework, and Friedman's original paper goes through the development of the logistic regression version of glmnet and of its extension to multiclass problems—problems with more than two discrete outcomes. You'll see the use of the binary and multiclass versions of the algorithm in Chapter 5.

Working with Classification Problems Having More Than Two Outcomes

Some problems require deciding among several alternatives. For example, say you show a visitor to your website several links. The visitor may click on any one of the several links, click the back button, or exit the site entirely. There are several alternatives that aren't ordered like the integer wine taste scores are. A taste score of 4 naturally fits between 3 and 5, and if changing an attribute (like alcohol) makes the score go from 3 to 4, changing it some more seems likely to move the score further in the same direction. Alternative actions a site visitor will take have no such order. This is called a multiclass classification problem.

You can always handle a multiclass problem with an algorithm for binary classification. The technique is called *one versus all* or *one versus the rest*, and the names give you some idea of how the approach works. Basically you pose your multiclass problem as several binary problems. For the example, you could predict whether the visitor would leave the site or choose another option. Another binary classification problem is to predict whether the user would click the back button or take any of the rest of the options available. You'll wind up with as many binary classification problems as you have alternative outcomes. The binary classifiers all give numeric values, like the LARS classifier in Listing 4-4. The outcome that has the largest one-versus-all value is the winner. Chapter 5 implements this method for the glass data set, where there are six different possible outcomes.

Understanding Basis Expansion: Using Linear Methods on Nonlinear Problems

By their nature, linear methods assume classification and regression predictions can be expressed as a linear combination of the attributes that are available to the designer. What if you have reason to suspect that a linear model isn't enough? You can get a linear model to work with strong nonlinearities by using what's called basis expansion. The basic idea behind basis expansion is that the nonlinearities in your problem can be approximated as polynomials of the attributes (or sum of other nonlinear functions of the attributes); then you can add attributes that are powers of the original attributes and let a linear method determine the best set of coefficients for the polynomial.

To get a concrete idea of how this would work, look at the code in Listing 4-5. Listing 4-5 starts with the wine taste data set. If you recall, the linear models that were produced earlier in this chapter both found that alcohol was the most important attribute in determining wine taste. It occurs to you that the relationship might not be a straight line, but might roll off for really high alcohol content and for really low alcohol content.

Listing 4-5 shows you how to test this notion.

Listing 4-5: Basis Expansion for Wine Taste Prediction

```
__author__ = 'mike-bowles'

import urllib2
import matplotlib.pyplot as plot
from math import sqrt, cos, log

#read data into iterable
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
"databases/wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)
```

```
xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)


#extend the alcohol variable (the last column in that attribute matrix
xExtended = []
alchCol = len(xList[1])


for row in xList:
    newRow = list(row)
    alch = row[alchCol - 1]
    newRow.append((alch - 7) * (alch - 7)/10)
    newRow.append(5 * log(alch - 7))
    newRow.append(cos(alch))
    xExtended.append(newRow)

nrow = len(xList)
v1 = [xExtended[j][alchCol - 1] for j in range(nrow)]

for i in range(4):
    v2 = [xExtended[j][alchCol - 1 + i] for j in range(nrow)]
    plot.scatter(v1,v2)

plot.xlabel("Alcohol")
plot.ylabel(("Extension Functions of Alcohol"))
plot.show()
```

The code reads in the data as before. Right after reading in the data (and before it is normalized), the code runs through the rows of data that it's read, adds a few new elements to the row, and then appends the new expanded row to a new set of attributes. The new elements that are appended are all functions of the alcohol attribute in the original data. For example, the first new attribute is ((alch - 7) * (alch - 7)/10), where alch is the alcohol level in the row.

The constants 7 and 10 were introduced so that the resulting new attributes would all plot nicely on one plot. Basically, the new attribute is alcohol squared.

The next step in the process is to take the expanded set of attributes and build a linear model using the tools already developed in this chapter (or another of the methods available for building linear models). Whatever algorithm is used for building a linear model, the model will consist of multipliers (or coefficients) for each of the attributes, including the new ones. If the functions used in the expansion are all powers of the original variable, the linear model yields coefficients in a polynomial function of the original variable. By choosing different functions for the expansion, other function series can be constructed.

Figure 4-8 illustrates the functional dependence of the new attributes (and the original attribute) on the original attribute. You can see the squared, logarithmic, and sinusoidal behavior of the selection of functions in the expansion.

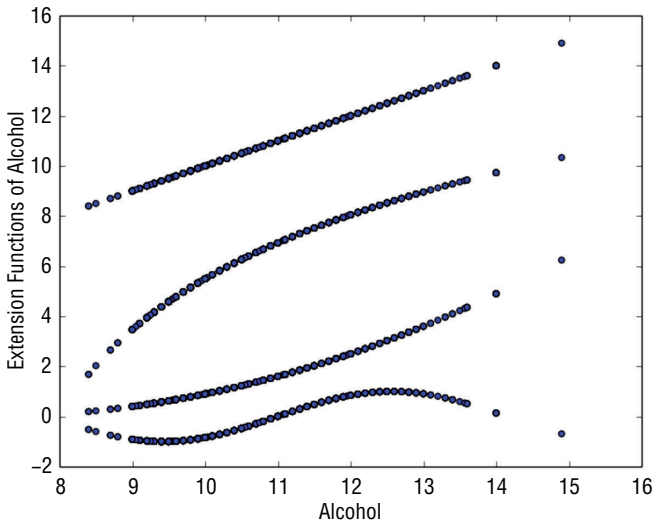


Figure 4-8: Functions generated to expand wine attribute session

Incorporating Non-Numeric Attributes into Linear Methods

Penalized linear regression (and other linear methods) require numeric attributes. What if your problem has some non-numeric attributes (also called categorical or factor attributes)? A familiar example would be a gender attribute where the possibilities are male and female. The standard method for converting categorical variables to numeric is to code them into several new columns of attribute data. If an attribute has N possible values, it gets coded into N - 1 new columns of data as follows. Identify N - 1 columns of data

with N - 1 of the N attributes. In each row enter a 1 in the ith column if the row takes the ith possible value of the categorical variable. Put zeros in the other columns. If the row takes the Nth value of the categorical variable, all the entries will be zero.

Listing 4-6 shows how this technique can be applied to the abalone data set. The task with the abalone data set is to predict the age of abalone from various physical measurements.

Listing 4-6: Coding Categorical Variable for Penalized Linear Regression - Abalone Data—larsAbalone.py

```
__author__ = 'mike_bowles'

import urllib2
from pylab import *
import matplotlib.pyplot as plot

target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
"databases/abalone/abalone.data")
#read abalone data
data = urllib2.urlopen(target_url)

xList = []
labels = []

for line in data:
    #split on semi-colon
    row = line.strip().split(",")

    #put labels in separate array and remove label from row
    labels.append(float(row.pop()))

    #form list of list of attributes (all strings)
    xList.append(row)

names = ['Sex', 'Length', 'Diameter', 'Height', 'Whole weight', \
    'Shucked weight', 'Viscera weight', 'Shell weight', 'Rings']

#code three-valued sex attribute as numeric
xCoded = []
for row in xList:
    #first code the three-valued sex variable
    codedSex = [0.0, 0.0]
    if row[0] == 'M': codedSex[0] = 1.0
    if row[0] == 'F': codedSex[1] = 1.0

    numRow = [float(row[i]) for i in range(1,len(row))]
    rowCoded = list(codedSex) + numRow
    xCoded.append(rowCoded)
```

continues

continued

```
namesCoded = ['Sex1', 'Sex2', 'Length', 'Diameter', 'Height', \
   'Whole weight', 'Shucked weight', 'Viscera weight', \
   'Shell weight', 'Rings']

nrows = len(xCoded)
ncols = len(xCoded[1])

xMeans = []
xSD = []
for i in range(ncols):
    col = [xCoded[j][i] for j in range(nrows)]
    mean = sum(col)/nrows
    xMeans.append(mean)
    colDiff = [(xCoded[j][i] - mean) for j in range(nrows)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrows)])
    stdDev = sqrt(sumSq/nrows)
    xSD.append(stdDev)

#use calculate mean and standard deviation to normalize xCoded
xNormalized = []
for i in range(nrows):
    rowNormalized = [(xCoded[i][j] - xMeans[j])/xSD[j] \
       for j in range(ncols)]
    xNormalized.append(rowNormalized)

#Normalize labels
meanLabel = sum(labels)/nrows
sdLabel = sqrt(sum([(labels[i] - meanLabel) * (labels[i] -
   meanLabel) for i in range(nrows)])/nrows)

labelNormalized = [(labels[i] - meanLabel)/sdLabel \
   for i in range(nrows)]

#initialize a vector of coefficients beta
beta = [0.0] * ncols

#initialize matrix of betas at each step
betaMat = []
betaMat.append(list(beta))


#number of steps to take
nSteps = 350
stepSize = 0.004
nzList = []

for i in range(nSteps):
    #calculate residuals
    residuals = [0.0] * nrows
    for j in range(nrows):
```

```
        labelsHat = sum([xNormalized[j][k] * beta[k]
           for k in range(ncols)])
        residuals[j] = labelNormalized[j] - labelsHat

    #calculate correlation between attribute columns from
    #normalized wine and residual
    corr = [0.0] * ncols

    for j in range(ncols):
        corr[j] = sum([xNormalized[k][j] * residuals[k]
           for k in range(nrows)]) / nrows

    iStar = 0
    corrStar = corr[0]

    for j in range(1, (ncols)):
        if abs(corrStar) < abs(corr[j]):
            iStar = j; corrStar = corr[j]

    beta[iStar] += stepSize * corrStar / abs(corrStar)
    betaMat.append(list(beta))


    nzBeta = [index for index in range(ncols) if beta[index] != 0.0]
    for q in nzBeta:
        if (q in nzList) == False:
            nzList.append(q)

nameList = [namesCoded[nzList[i]] for i in range(len(nzList))]

print(nameList)
for i in range(ncols):
    #plot range of beta values for each attribute
    coefCurve = [betaMat[k][i] for k in range(nSteps)]
    xaxis = range(nSteps)
    plot.plot(xaxis, coefCurve)

plot.xlabel("Steps Taken")
plot.ylabel(("Coefficient Values"))
plot.show()

Printed Output - [filename- larsAbaloneOutput.txt]
['Shell weight', 'Height', 'Sex2', 'Shucked weight', 'Diameter', 'Sex1']
```

The first attribute is the gender of the abalone, which takes three values. When abalone are infants, their sex is indeterminate so the entries in the first column are M, F, and I.

The variable names associated with the columns are shown in a Python list that gets named *names*. With the abalone data set, these names don't come from

the first row of data, but from a separate file on the UC Irvine website. The first variable in the list is Sex—the sex of the animal. The last variable in the list is Rings. These are shell rings that are counted by slicing the shell and counting up the rings through a microscope. The number of rings is essentially the age of the abalone. The objective of the problem is to train a regression system to predict the Rings using easier, less time-consuming and less-expensive measurements.

Coding the Sex attribute is accomplished before the attribute matrix is normalized. The process is to build two columns to represent the three possible values. The logic of the construction is that the first column has a 1 if the corresponding row is from a male (M) and zero otherwise. The second column is 1 for female (F). Both columns are zero if the example is an infant (I). The new columns that replace Sex are given the names Sex1 and Sex2.

Once this coding is accomplished, then the attribute matrix contains all numeric values, and the example proceeds as in earlier examples. It normalizes the variables to zero mean and unit standard deviation, and then it applies the LARS algorithm introduced earlier to develop coefficient curves. The printed output shows the order in which variables enter into the solution of the penalized linear regression solution. You'll observe that both the two columns coding for Sex appear in the solution.

Figure 4-9 shows the coefficient curves that result from LARS applied to this problem. Chapter 5 delves more into performance, with different approaches to this problem.

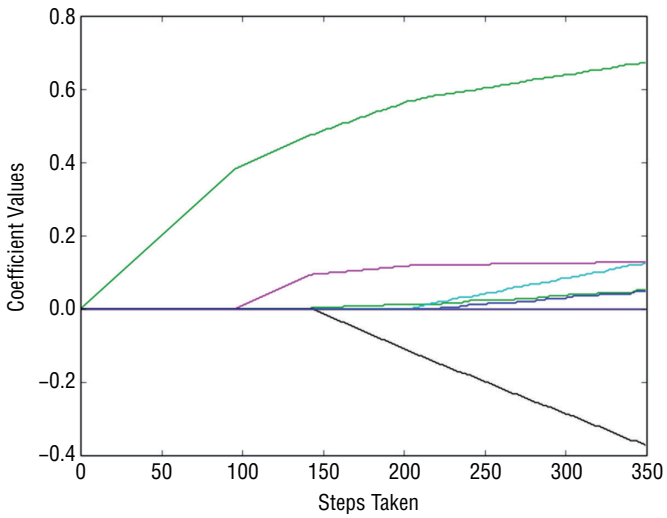


Figure 4-9: Coefficient curves for LARS trained on abalone data with coded categorical variable

This section discussed several extensions to penalized regression that broaden its utility to cover a wide class of problems. The section described a simple and

frequently effective method of converting a classification problem to an ordinary regression problem. It also discussed how to convert a binary classifier into a multiclass classifier. The section went on to discuss how to model nonlinear behaviors using linear regression by adding new attributes that are nonlinear functions of the old attributes. Finally, the section showed how to turn categorical variables into real-valued variables so you can train linear algorithms on categorical variables. This method of converting categorical variables doesn't just work for linear regression. It is also useful for other linear methods such as support vector machines.

Summary

The goal of this chapter was to lay the groundwork for you to confidently understand and use the Python packages implementing the algorithms described here. The chapter described the nature of the input data set as a column vector of outcomes to be predicted and a table of attributes upon which to base the predictions. Chapter 3, the previous chapter, demonstrated that predictive models need to have their complexity tuned to get the best performance for a given problem complexity and data set size. Chapter 3 also showed some methods for introducing a tuning parameter into linear regression. This chapter built on that background and introduced several minimization problems where a tunable coefficient penalty was added to the error penalty from least squares regression. As was demonstrated, this tunable penalty on linear coefficient sizes results in suppression of the coefficients to a greater or lesser degree and thereby adds a complexity adjustment. You saw how to tune the complexity of the resulting models by using the error on out-of-sample data to achieve optimum performance.

The chapter described principles of operation for two modern algorithms for solving the penalized regression minimization problem and python code implementing the main features of the algorithms in order for you to have a concrete instantiation of the core of the algorithms to make the principals of operation clear. The plain regression problem (numeric features and numeric targets) served as the exemplar for in-depth coverage of algorithms. The chapter showed several extensions to broaden the use cases to include binary classification problems, multiclass classification problems, problems with nonlinear relationship between attributes and outcomes, and problems with non-numeric features.

The next chapter, Chapter 5, will use Python packages implementing these algorithms to run through a series of examples that were chosen to exercise a variety of different problem characteristics in order to cement these ideas. Based on what you've learned in this chapter, the various parameters and methods in the Python packages will make sense for you.

References

1. Bradley Efron, Trevor Hastie, Iain Johnstone, and Robert Tibshirani (2004). "Least Angle Regression." *Annals of Statistics*, *32*(2), 407–499.
2. Jerome H. Friedman, Trevor Hastie and Rob Tibshirani (2010). "Regularization Paths for Generalized Linear Models via Coordinate Descent." *Journal of Statistical Software,* vol. 33, issue 1, Feb 2010.

CHAPTER

5

Building Predictive Models Using Penalized Linear Methods

Chapter 2 looked at a number of different data sets with an eye toward understanding the data sets, the relations between the various attributes and labels, and the nature of the problems being posed. This chapter picks those data sets up once again and runs through some case studies demonstrating the process of building predictive models by using the penalized linear methods that you saw in Chapter 4, "Penalized Linear Regression." Generally, the model-building will be segmented into two or more parts.

You'll recall from Chapter 4 that model building with penalized linear regression has two steps. One is to train on the whole data set to trace out coefficient curves. The other is to run cross-validation to determine the best achievable out-of-sample performance and to identify the model that achieves it. The step of determining the achievable performance encompasses the hard design work, and in many of the examples in this chapter, that's the only step that will be presented. The purpose of training on the whole data set is to get the best estimates of the model coefficients. But it does not change your estimate of the errors, which are the gauge of performance.

This chapter runs through a variety of different types of problems: regression problems, classification problems, problems with categorical attributes, and problems with nonlinear dependence of the labels on the attributes. It looks at basis expansion to see whether it improves the prediction performance. In each case, the objective is to work through the steps you'd take to arrive at a deployable linear model and to consider some alternative paths so that you can ensure that you're getting all the performance you can.

Python Packages for Penalized Linear Regression

The examples in Chapter 4 used Python versions of the training algorithms involved: LARS, and coordinate descent with the ElasticNet penalty. The purpose for using the Python code in Chapter 4 was to expose the workings of the algorithms to further your understanding of them. Fortunately, you don't have to code those algorithms each time you want to use them.

Scikit-learn has packages implementing Lasso, LARS, and ElasticNet regression. There are several advantages to using those packages. One advantage is that using them results in fewer lines of code that you have to write and debug. Another big advantage is that they are much faster than the code in Chapter 4. The scikit-learn packages take advantage of practices like not computing correlations for attributes that aren't being used in order to cut way down on the number of calculations. You'll see when you run these packages that they execute very quickly.

The packages used in this chapter are found in sklearn.linear_model. The link `http://scikit-learn.org/stable/modules/classes.html#module-sklearn.linear_model` shows a list including the models you'll see used here. Notice that several of the models come in two flavors. For example, there's a package titled linear_model.ElasticNet and one titled linear_model.ElasticNetCV. These two models correspond to the two tasks discussed at the beginning of this chapter. The Python package linear_model.ElasticNet is used to calculate coefficient curves on the whole data set, and linear_model.ElasticNetCV does the cross-validation run to produce out-of-sample estimates of performance. It's handy to have these two forms.

The same basic input objects fuel both versions (two numpy arrays—one of attributes and one of labels). In some cases, you won't be able to use the cross-validation version because you'll need very specific control of the contents of training and test sets for each fold:

- If your problem has a categorical attribute that takes one of its values very infrequently, you may need to control sampling so that the attribute is represented evenly across the folds.
- You may also need to have access to the separate fold data to compile error statistics for your problem, if you want a different error measure from the *mean squared error* (MSE) that the CV packages deliver. You might prefer *mean absolute error* (MAE) because it better matches the penalty that you'll pay for errors in your real problem.
- Another example of needing fold-by-fold access for error statistics is when you use linear regression to solve classification problems. As discussed in Chapter 3, "Predictive Model Building: Balancing Performance, Complexity, and Big Data," standard error measures for classification problems are

things such as misclassification error or *area under the ROC curve* (AUC). You'll see that case specifically in the "rocks versus mines" and "glass classification" case studies in this chapter.

There are a couple of things for you to keep in mind as you look at these packages and begin thinking about using them. One is that some of them (but not all of them) automatically normalize the attributes before fitting a model. The second thing to be aware of is that the scikit-learn packages name variables differently from Chapter 4 and Friedman's papers. Chapter 4 used the variable λ to represent the multiplier on the coefficient penalty and used the variable α to represent the proportion of Lasso penalty versus ridge penalty in the ElasticNet penalty. The scikit-learn packages use α instead of λ and l1_ratio instead of α. The text that follows switches to the notation used in the scikit-learn packages.

SOME SCIKIT-LEARN CHANGES

The scikit-learn documentation states an intention to bring all the penalized regression packages into conformance with one another by including normalization in all of them. That is in process at the time of writing this book.

Multivariable Regression: Predicting Wine Taste

As discussed in Chapter 2, "Understand the Problem by Understanding the Data," the wine taste data set comes from the UC Irvine data repository (`http://archive.ics.uci.edu/ml/datasets/Wine+Quality`).[1] The data set contains chemical analyses for 1,599 wines along with average taste scores given to each wine by a panel of wine tasters. The predictive problem is to predict the taste given the data on chemical composition. The chemical composition data consist of numeric measurements of 11 different chemical properties—alcohol content, pH and citric acid, and so on. Have a look at the exploration of these data in Chapter 2 or look at the UC Irvine page for the data set for more information.

Predicting the wine taste is a regression problem because the objective of the problem is to predict the quality score, which is an integer between 0 and 10. The data set only includes examples between 3 and 8. Because only integer scores are given, it is also possible to treat this problem as a multiclass classification problem. The multiclass problem would have six possible classifications (the integers from 3 to 8). It would ignore the order relation that exists among the various scores. (For example, 5 is a worse score than 6 and a better score than 4.) Regression is a more natural way to pose the problem because it preserves the order relationship.

Another way to think about how to pose the problem is to consider the different error measures that come with a regression problem versus a multiclass classification problem. The regression error function is the average

squared error. When the true taste is 3, predicting a 5 contributes more to the cumulative error than predicting a 4. The error measure for the multiclass problem is the number of examples that get misclassified. With this error measure, if the true taste is 3, predicting a 5 or 4 contributes the same amount to the cumulative error. Regression seems more natural, but I don't know of a way to prove that it will give superior performance. The only way to know whether this is the best approach is to try both. In the section titled "Multiclass Classification: Classifying Crime Scene Glass Samples," you'll see how to handle multiclass classification problems. You can then come back and try the multiclass approach and see whether it does better or worse. What error measure will you use?

Building and Testing a Model to Predict Wine Taste

The first step in the process of building a model is to generate some out-of-sample performance numbers to see whether they're going to meet your performance requirements. Listing 5-1 shows the code to perform 10-fold cross-validation and plot the results. The first section of the code reads the data from the UCI website into a list of lists and then runs through normalization of the list of lists of attributes and the list of labels. Then the lists get converted to numpy arrays X (matrix of attributes) and Y (vector of labels). There are two versions of these definitions. In one version, the normalized lists are used. In the other, the un-normalized versions are used. You can comment out the second of the two definitions in either case and rerun the code to see what effect normalization the attributes or the labels has on the answers. A single line of code defines the number of cross-validation folds (10) and trains the model. Then the program plots the error versus α curves for each of the 10 folds and also plots the average of the 10. The three plots are shown in Figures 5-1, 5-2, and 5-3. In order, the three cases are as follows:

1. Normalized X and un-normalized Y
2. Normalized X and Y
3. Un-normalized X and Y

Listing 5-1: Using Cross-Validation to Estimate Out-of-Sample Error with Lasso Modeling Wine Taste—wineLassoCV.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
from sklearn import datasets, linear_model
from sklearn.linear_model import LassoCV
from math import sqrt
import matplotlib.pyplot as plot
```

```
#read data into iterable
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
"databases/wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

#Normalize columns in x and labels
#Note: be careful about normalization.  Some penalized
#regression packages include it and some don't.
nrows = len(xList)
ncols = len(xList[0])

#calculate means and variances
xMeans = []
xSD = []
for i in range(ncols):
    col = [xList[j][i] for j in range(nrows)]
    mean = sum(col)/nrows
    xMeans.append(mean)
    colDiff = [(xList[j][i] - mean) for j in range(nrows)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrows)])
    stdDev = sqrt(sumSq/nrows)
    xSD.append(stdDev)

#use calculate mean and standard deviation to normalize xList
xNormalized = []
for i in range(nrows):
    rowNormalized = [(xList[i][j] - xMeans[j])/xSD[j] \
        for j in range(ncols)]
    xNormalized.append(rowNormalized)

#Normalize labels
meanLabel = sum(labels)/nrows
```

continues

continued

```
sdLabel = sqrt(sum([(labels[i] - meanLabel) * (labels[i] -
        meanLabel) for i in range(nrows)])/nrows)

labelNormalized = [(labels[i] - meanLabel)/sdLabel \
        for i in range(nrows)]

#Convert list of list to np array for input to sklearn packages

#Unnormalized labels
Y = numpy.array(labels)

#normalized lables
Y = numpy.array(labelNormalized)

#Unnormalized X's
X = numpy.array(xList)

#Normalized Xss
X = numpy.array(xNormalized)

#Call LassoCV from sklearn.linear_model
wineModel = LassoCV(cv=10).fit(X, Y)

# Display results


plot.figure()
plot.plot(wineModel.alphas_, wineModel.mse_path_, ':')
plot.plot(wineModel.alphas_, wineModel.mse_path_.mean(axis=-1),
         label='Average MSE Across Folds', linewidth=2)
plot.axvline(wineModel.alpha_, linestyle='--',
            label='CV Estimate of Best alpha')
plot.semilogx()
plot.legend()
ax = plot.gca()
ax.invert_xaxis()
plot.xlabel('alpha')
plot.ylabel('Mean Square Error')
plot.axis('tight')
plot.show()

#print out the value of alpha that minimizes the Cv-error
print("alpha Value that Minimizes CV Error  ",wineModel.alpha_)
print("Minimum MSE  ", min(wineModel.mse_path_.mean(axis=-1)))

Printed Output: Normalized X, Un-normalized Y
('alpha Value that Minimizes CV Error  ', 0.010948337166040082)
('Minimum MSE  ', 0.433801987153697)

Printed Output: Normalized X and Y
('alpha Value that Minimizes CV Error  ', 0.013561387700964642)
('Minimum MSE  ', 0.66558492060028562)
```

```
Printed Output: Un-normalized X and Y
('alpha Value that Minimizes CV Error  ', 0.0052692947038249062)
('Minimum MSE  ', 0.43936035436777832)
```

The printed output at the bottom of Listing 5-1 shows a significant increase in the MSE that comes with normalizing Y. In contrast, Figures 5-1 and 5-2 are remarkably similar in shape. The only difference between them is the scale on the Y-axis. Refer to Listing 2-13 to see that the standard deviation of the unscaled wine quality scores is roughly 0.81. That means that the normalization to a standard deviation of 1.0 requires multiplying by roughly 1.2. That results in an increase of 1.2 squared in the MSE. The only issue with normalizing the labels is that the MSE loses its connection to the original data. It's usually handier to be able to extract a square root of the MSE and then relate it directly to the units of the original labels. In this case, the MSE (with un-normalized Y) is 0.433. The square root is roughly 0.65. That means that the +/- 1-sigma errors lie in a band that's 1.3 units of taste-score wide. So, normalizing Y doesn't make a material difference in the results. What about normalizing X? Does normalizing X improve or worsen performance?

The last set of numbers in Listing 5-1 shows a very slight increase in the MSE if X is left un-normalized. However, the plot of CV error versus alpha in Figure 5-3 shows a radical difference from the plots in Figures 5-1 and 5-2. The plot has a scalloped character that's caused by the mishmash of scales that comes from leaving the Xs unscaled. What happens is that the algorithm picks a large variable that requires a correspondingly small coefficient. That can happen if the variable has high correlation with Y or if the variable has low correlation with Y and a large scale. The algorithm uses a somewhat inferior variable for a few iterations until α (formerly known as λ) gets small enough to let in a better variable, at which time the error drops precipitously. The moral of the story is to normalize the Xs or be wary about not normalizing them.

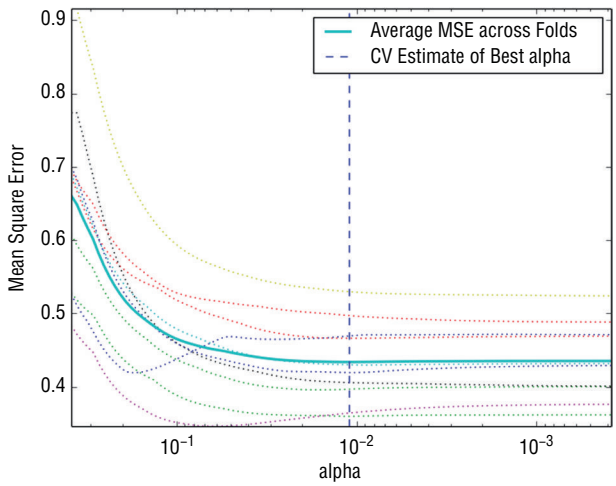


Figure 5-1: Out-of-sample error with un-normalized Y – Lasso model on wine taste data

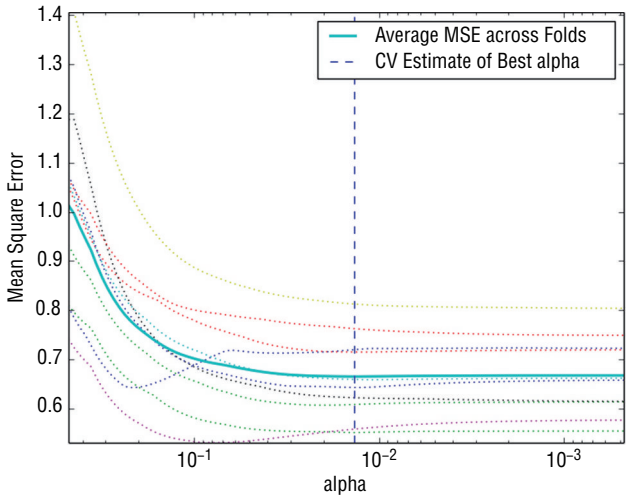


Figure 5-2: Out-of-sample error with normalized Y – Lasso model on wine taste data

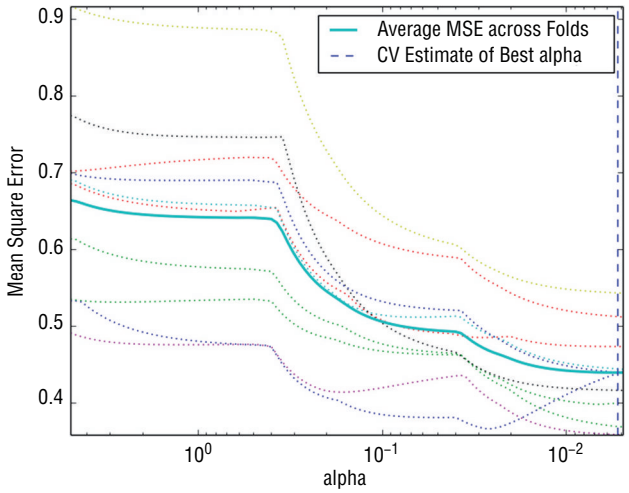


Figure 5-3: Out-of-sample error with un-normalized X and Y – Lasso model on wine taste data

Training on the Whole Data Set before Deployment

Listing 5-2 shows the code for training on the whole data set. As mentioned, the reason for training on the whole data set is to obtain the best set of coefficients for deployment. Cross-validation yields an estimate of the deployed model's performance and gives you the α value that yields the best performance. After reading the wine data from the UC Irvine data repository and normalizing it,

the program converts the data to numpy arrays and then invokes the `lasso_path` method to generate α values (that is, penalties) and the corresponding coefficients. Those coefficient trajectories are plotted in Figure 5-4.

Listing 5-2: Lasso Training on Full Data Set—wineLassoCoefCurves.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
from sklearn import datasets, linear_model
from sklearn.linear_model import LassoCV
from math import sqrt
import matplotlib.pyplot as plot

#read data into iterable
target_url = "http://archive.ics.uci.edu/ml/machine-learning-databases/
wine-quality/winequality-red.csv"
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

#Normalize columns in x and labels
#Note: be careful about normalization.  Some penalized regression
#packages include it and some don't.

nrows = len(xList)
ncols = len(xList[0])

#calculate means and variances
xMeans = []
xSD = []
for i in range(ncols):
    col = [xList[j][i] for j in range(nrows)]
```

continues

continued

```
    mean = sum(col)/nrows
    xMeans.append(mean)
    colDiff = [(xList[j][i] - mean) for j in range(nrows)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrows)])
    stdDev = sqrt(sumSq/nrows)
    xSD.append(stdDev)

#use calculate mean and standard deviation to normalize xList
xNormalized = []
for i in range(nrows):
    rowNormalized = [(xList[i][j] - xMeans[j])/xSD[j] for j in
    range(ncols)]
    xNormalized.append(rowNormalized)

#Normalize labels
meanLabel = sum(labels)/nrows
sdLabel = sqrt(sum([(labels[i] - meanLabel) * (labels[i] - meanLabel)
for i in range(nrows)])/nrows)

labelNormalized = [(labels[i] - meanLabel)/sdLabel for i in
range(nrows)]

#Convert list of list to np array for input to sklearn packages

#Unnormalized labels
Y = numpy.array(labels)

#normalized lables
Y = numpy.array(labelNormalized)

#Unnormalized X's
X = numpy.array(xList)

#Normalized Xss
X = numpy.array(xNormalized)

alphas, coefs, _  = linear_model.lasso_path(X, Y,  return_models=False)


plot.plot(alphas,coefs.T)

plot.xlabel('alpha')
plot.ylabel('Coefficients')
plot.axis('tight')
plot.semilogx()
ax = plot.gca()
ax.invert_xaxis()
plot.show()

nattr, nalpha = coefs.shape
```

```
#find coefficient ordering
nzList = []
for iAlpha in range(1,nalpha):
    coefList = list(coefs[: ,iAlpha])
    nzCoef = [index for index in range(nattr) if coefList[index] != 0.0]
    for q in nzCoef:
        if not(q in nzList):
            nzList.append(q)

nameList = [names[nzList[i]] for i in range(len(nzList))]
print("Attributes Ordered by How Early They Enter the Model", nameList)

#find coefficients corresponding to best alpha value. alpha value
# corresponding to normalized X and normalized Y is 0.013561387700964642

alphaStar = 0.013561387700964642
indexLTalphaStar = [index for index in range(100) if alphas[index] >
alphaStar]
indexStar = max(indexLTalphaStar)

#here's the set of coefficients to deploy
coefStar = list(coefs[:,indexStar])
print("Best Coefficient Values ", coefStar)

#The coefficients on normalized attributes give another slightly
#different ordering

absCoef = [abs(a) for a in coefStar]

#sort by magnitude
coefSorted = sorted(absCoef, reverse=True)

idxCoefSize = [absCoef.index(a) for a in coefSorted if not(a == 0.0)]

namesList2 = [names[idxCoefSize[i]] for i in range(len(idxCoefSize))]

print("Attributes Ordered by Coef Size at Optimum alpha", namesList2)



Printed Output w. Normalized X:
('Attributes Ordered by How Early They Enter the Model',
['"alcohol"', '"volatile acidity"', '"sulphates"',
'"total sulfur dioxide"', '"chlorides"', '"fixed acidity"', '"pH"',
'"free sulfur dioxide"', '"residual sugar"', '"citric acid"',
 '"density"'])

('Best Coefficient Values ',
[0.0, -0.22773815784738916, -0.0, 0.0, -0.094239023363375404,
0.022151948563542922, -0.099036391332770576, -0.0,
```

continues

continued

```
-0.067873612822590218, 0.16804102141830754, 0.37509573430881538])

('Attributes Ordered by Coef Size at Optimum alpha',
['"alcohol"', '"volatile acidity"', '"sulphates"',
'"total sulfur dioxide"', '"chlorides"', '"pH"',
'"free sulfur dioxide"'])


Printed Output w. Un-normalized X:
('Attributes Ordered by How Early They Enter the Model',
['"total sulfur dioxide"', '"free sulfur dioxide"', '"alcohol"',
'"fixed acidity"', '"volatile acidity"', '"sulphates"'])

('Best Coefficient Values ', [0.044339055570034182, -1.0154179864549988,
0.0, 0.0, -0.0, 0.0064112885435006822, -0.0038622920281433199, -0.0,
-0.0, 0.41982634135945091, 0.37812720947996975])

('Attributes Ordered by Coef Size at Optimum alpha',
['"volatile acidity"', '"sulphates"', '"alcohol"', '"fixed acidity"',
'"free sulfur dioxide"', '"total sulfur dioxide"'])
```

The program has hard-coded the α value that gave the best results in cross-validation. The version in the code is the best alpha trained with normalized attributes and labels. Changing either of these to un-normalized will change the corresponding value of the best α. Changing Y to un-normalized changes it by the 1.2 factor that comes from normalizing the standard deviation to 1.0 (as discussed earlier in the context of the MSE difference between normalized and un-normalized labels). The hard-coded value of α is used to identify the vector of coefficients corresponding to the best cross-validation results.

Listing 5-2 shows printed output for three cases: normalized attributes and un-normalized labels, both normalized, and both un-normalized. The printed output for each case includes a list of the attributes in the order that they enter the model as α is decreased. (The α in Python packages corresponds to the penalty term λ in Chapter 4.) The printed output also shows the coefficients at the hard-coded value of α. The third element of the printed output is the order of the attributes as determined by the magnitude of the corresponding coefficient (at the hard-coded value of α). The magnitude of the coefficients is another way to determine the relative importance of attributes. This ranking only makes sense when the attributes are normalized. Observe that with normalized attributes, the two methods discussed for assigning importance to attributes (order in which they appear in the solutions and relative coefficient magnitudes) give essentially the same ordering on the attributes with some disagreement on less important attributes. With un-normalized attributes, this is far from true.

As mentioned earlier, the order in which variables come into the solution (as α decreases) is strongly modified by normalizing the attributes. If a variable isn't normalized, its scale factor determines its usage instead of its inherent value in predicting

the labels. This is obvious from comparing the variable ordering for normalized attributes (the first case in the printout) to the ordering for un-normalized variables.

Figures 5-4 and 5-5 show the Lasso coefficient curves for the case of normalized attributes and un-normalized attributes, respectively. The coefficient curves for un-normalized attributes are less orderly than they are for normalized attributes. Several of the early coefficients hover near zero relative to the magnitudes of coefficients that come into play later along the coefficient trajectories. This is compatible with the radically different ordering between the order that coefficients enter the model and the magnitude of the coefficient at the best solution.

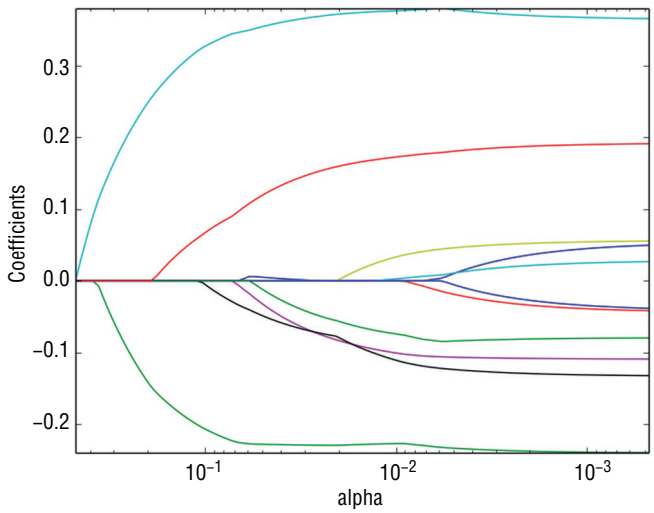


Figure 5-4: Coefficient curves for Lasso trained to predict wine quality

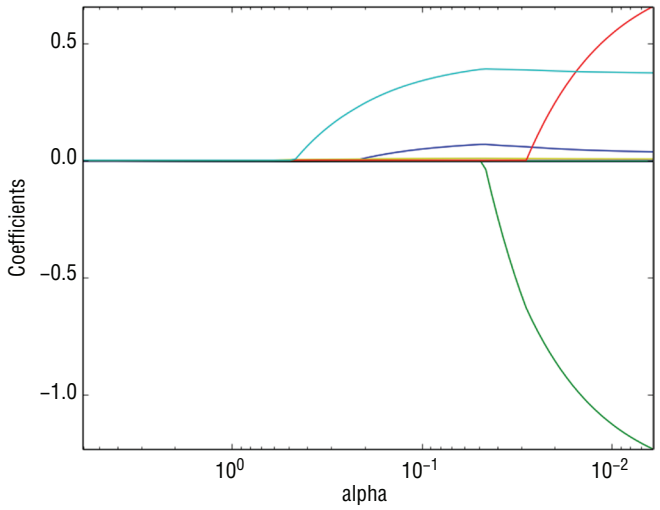


Figure 5-5: Coefficient curves for Lasso trained on un-normalized Xs

Basis Expansion: Improving Performance by Creating New Variables from Old Ones

Chapter 4 discussed adding new attributes in the form of functions of the old attributes. The point of doing that is to see whether it results in improved performance. Listing 5-3 shows how to add two new attributes to the wine data.

Listing 5-3: Using Out-of-Sample Error to Evaluate New Attributes for Predicting Wine Quality—wineExpandedLassoCV.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
from sklearn import datasets, linear_model
from sklearn.linear_model import LassoCV
from math import sqrt
import matplotlib.pyplot as plot

#read data into iterable
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
"databases/wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

#append square of last term (alcohol)

for i in range(len(xList)):
    alcElt = xList[i][-1]
    volAcid = xList[i][1]
    temp = list(xList[i])
    temp.append(alcElt*alcElt)
```

```
    temp.append(alcElt*volAcid)
    xList[i] = list(temp)

#add new name to variable list
names[-1] = "alco^2"
names.append("alco*volAcid")

#Normalize columns in x and labels
#Note: be careful about normalization. Some penalized regression
packages include it and some don't.

nrows = len(xList)
ncols = len(xList[0])

#calculate means and variances
xMeans = []
xSD = []
for i in range(ncols):
    col = [xList[j][i] for j in range(nrows)]
    mean = sum(col)/nrows
    xMeans.append(mean)
    colDiff = [(xList[j][i] - mean) for j in range(nrows)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrows)])
    stdDev = sqrt(sumSq/nrows)
    xSD.append(stdDev)

#use calculate mean and standard deviation to normalize xList
xNormalized = []
for i in range(nrows):
    rowNormalized = [(xList[i][j] - xMeans[j])/xSD[j] \
        for j in range(ncols)]
    xNormalized.append(rowNormalized)

#Normalize labels
meanLabel = sum(labels)/nrows
sdLabel = sqrt(sum([(labels[i] - meanLabel) * (labels[i] - meanLabel) \
        for i in range(nrows)])/nrows)

labelNormalized = [(labels[i] - meanLabel)/sdLabel \
        for i in range(nrows)]

#Convert list of list to np array for input to sklearn packages

#Unnormalized labels
Y = numpy.array(labels)

#normalized labels
#Y = numpy.array(labelNormalized)

#Unnormalized X's
X = numpy.array(xList)
```

continues

continued

```
#Normalized Xss
X = numpy.array(xNormalized)

#Call LassoCV from sklearn.linear_model
wineModel = LassoCV(cv=10).fit(X, Y)

# Display results


plot.figure()
plot.plot(wineModel.alphas_, wineModel.mse_path_, ':')
plot.plot(wineModel.alphas_, wineModel.mse_path_.mean(axis=-1),
         label='Average MSE Across Folds', linewidth=2)
plot.axvline(wineModel.alpha_, linestyle='--',
            label='CV Estimate of Best alpha')
plot.semilogx()
plot.legend()
ax = plot.gca()
ax.invert_xaxis()
plot.xlabel('alpha')
plot.ylabel('Mean Square Error')
plot.axis('tight')
plot.show()

#print out the value of alpha that minimizes the CV-error
print("alpha Value that Minimizes CV Error  ",wineModel.alpha_)
print("Minimum MSE  ", min(wineModel.mse_path_.mean(axis=-1)))

Printed Output: [filename - wineLassoExpandedCVPrintedOutput.txt]
('alpha Value that Minimizes CV Error  ', 0.016640498998569835)
('Minimum MSE  ', 0.43452874043020256)
```

The key step comes right after the attributes are read in and converted to floats. There are a dozen or so lines of code that take each row of attributes, pull out the two variables corresponding to measures of alcohol and volatile acidity, and then append *alcohol squared* and the product *alcohol times volatile acidity*. These are chosen because it makes sense to start with variables that are more important in the solution. A thorough hunt for possible improvements might include several attempts with combinations of the top variables.

The results show that adding these new variables degrades performance slightly. A little hunting might turn up some variables that make a useful difference. You might run out coefficient curves for this example to see whether the new variables replaced any old ones that were important at the optimum solution. That information might lead you to remove the old variables in favor of these new synthetic ones.

Figure 5-6 shows the cross-validation error curves for Lasso trained using the expanded set of attributes. The character of the cross-validation curves doesn't show substantial difference from the curves without basis expansion.

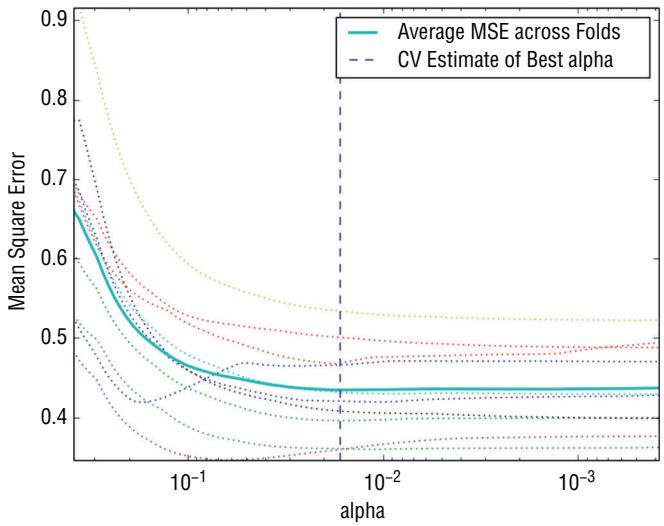


Figure 5-6: Cross-validation error curves for Lasso trained on wine quality data with expanded feature set

This section has demonstrated the use of penalized regression methods on a problem with real number outcomes—a regression problem. The next section shows the use of penalized linear regression methods on a problem where the outcomes are two-valued. The code will look similar to what you have seen in this section, and some of the techniques, like basis expansion, can be used in classification problems. The main difference is how performance is scored for a classification problem.

Binary Classification: Using Penalized Linear Regression to Detect Unexploded Mines

Chapter 4 discussed how you can use penalized linear regression for classification problems and set the process up for the rocks versus mines problem. This section gets into the details of how you would approach and solve a binary classification problem using penalized linear regression. The section incorporates the Python ElasticNet package. You'll recall from Chapter 4 that ElasticNet incorporates a more general penalty function that includes the Lasso and ridge regression penalty functions as special cases. This makes it possible to see how performance of the classifier changes as you make alterations in the penalty function. These are the steps along the path to a solution:

1. Cast the binary classification problem as a regression problem. Construct an outcome vector of real number labels by assigning 0.0 when the class outcome takes one of its two values and assigning 1.0 when it takes the other.

2. Perform cross-validation. The cross-validation becomes a little more complicated because you'll need to calculate an error quantity for each fold. Scikit-learn has some handy utilities to streamline these calculations.

The first step (outlined in Chapter 4) is to cast the binary classification problem as a regression problem by replacing the classification labels with real number labels. The rocks versus mines problem is basically to build a system using sonar to detect unexploded mines on the seabed. You'll recall from the data discovery in Chapter 2 that the data set contains digitized versions of the signals returned from rocks and from metal cylinders shaped like mines. The objective is to build a prediction system that can process the digitized signals to correctly identify whether the object is a rock or a mine. The data set consists of 208 experiments. Of the 208, 111 are mines and 97 are rocks. The data set is 61 columns wide. The first 60 columns contain the digitized sonar return. The last column contains an M or an R, depending on whether the object is in a rock or a mine. The 60 columns of numbers are the attributes for the problem. A regression problem requires numeric labels too. An approach outlined in Chapter 4 is to build the column of numeric labels by assigning the number 1 to one of the two cases and 0 to the other. Listing 5-4 initializes an empty list called *labels* and appends a 1.0 for each M row and appends a 0.0 for each R row.

With numeric attributes and numeric labels, everything is in place to use the regression version of penalized linear regression. The next logical step is to perform cross-validation to get an estimate of out-of-sample performance and identify the best value of α, the penalty parameter. For this problem, doing cross-validation requires building a cross-validation loop to enclose training and testing. Why build a cross-validation loop instead of using the cross-validation package available in Python (like the one used in the wine quality example earlier in this chapter)?

The cross-validation for regression is based on MSE. That's perfectly reasonable for a regression problem, but not for a classification problem. As discussed in Chapter 3, you characterize performance differently for a classification problem than for a regression problem. Chapter 3 discussed several ways to characterize performance. One natural way is to measure the percentage of examples that are misclassified. Another way is to measure the AUC. See Chapter 3 or the Wikipedia page `http://en.wikipedia.org/wiki/Receiver_operating_characteristic` to refresh your memory on the AUC measure. To measure either of these requires that you have access to the predictions and labels in each of the cross-validation folds. You can't judge misclassification error from a summary of the MSE for the fold.

The cross-validation loop breaks the data into training and test sets and then calls the Python `enet_path` method to accomplish training on the training portion of the data. Two inputs to the routine are different from defaults. One is the

`l1_ratio`, which is set equal to `0.8`. This parameter determines what fraction of the penalty is sum of absolute values of coefficients. The value 0.8 means that penalty function is 80 percent sum of absolute values and 20 percent sum of squares. The other nondefault parameter is `fit_intercept`, which is set to `False`. The code is using normalized labels and normalized attributes. Because all of these are zero mean, there's no need to calculate an intercept term. The intercept is required only to adjust any constant offset between the attributes and the labels. Eliminating the need for the intercept term by using normalized labels makes the calculation of predictions a little cleaner. The only downside of normalizing the labels is that it makes the MSE calculation less meaningful relative to a regression problem, but for a classification problem, you're not going to use that metric of performance anyway.

In each fold, after training is completed, the coefficients that are produced are used to generate predictions on the out-of-sample data for the fold. This is accomplished in the code by using the numpy dot function, the attributes for out-of-sample data for the fold, and the coefficients for the fold. This matrix-like multiplication of two numpy arrays leads to another two-dimensional array whose rows correspond to the rows in the out-of-sample test data for the fold and whose columns correspond to the sequence of models generated by `enet_path` (that is, the sequence of coefficient vectors and the corresponding sequence of α's). These matrices of predictions for each fold are concatenated (visualize stacking them atop one another), as are the out-of-sample labels. Then, at the end of the run, these compendia of the fold-by-fold out-of-sample results can be processed easily and efficiently to yield performance data for each model and to select a model complexity (α) for deployment.

Listing 5-4 generates comparisons using two metrics. The first is misclassification error. The second is area under the *receiver operating curve* (ROC). Each column from the matrix of predictions represents predictions generated for the totality of the out-of-sample data for one set of model coefficients. All the data are represented in each column since every row is held out in one (and only one) of the folds. The misclassification comparison considers the prediction data one column at a time and out-of-sample labels (called `yOut` in the code) accumulated fold by fold. Each prediction is compared to a fixed threshold (0.0 in this example) to determine a predicted classification. Then the predicted classification is compared to the corresponding entry in `yOut` to determine whether the predicted classification is correct. The plot in Figure 5-7 shows several points that achieve the same minimum. It's good practice when you have a choice to choose the point farthest to the left on the graph of performance versus α. That's because points to the right have more tendency to be overfit. It's more conservative to choose a solution farther to the left. You'll have a better chance that the errors in deployment will match those you see in cross-validation.

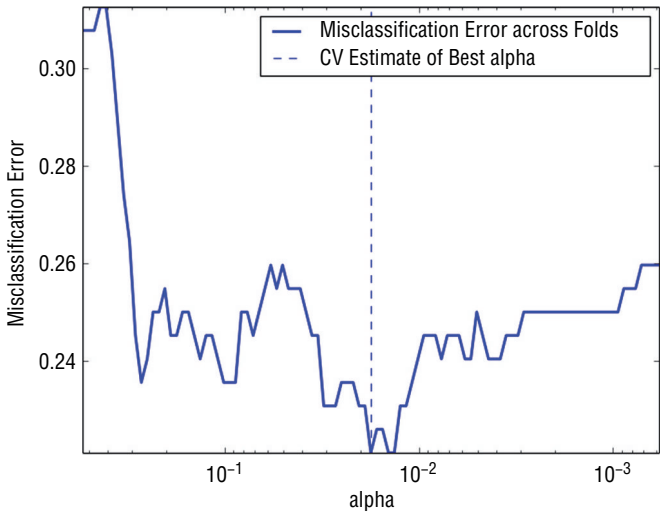


Figure 5-7: Out-of-sample classifier misclassification performance

Another way to measure the performance of your classifier is AUC. AUC has the advantage that in maximizing the AUC you wind up getting the best performance independent of where you intend to operate the system—whether you want more or less equal rates of different types of errors or you'd prefer to bias the errors toward one type. Strictly speaking, maximizing AUC does not guarantee that you'll get optimum performance at a particular error rate. Comparing the model chosen by AUC to the one chosen by minimizing overall error rate and observing the shapes of the curves help you get confidence in your solution and give you some idea about how much more performance is available with more thorough optimization.

The AUC calculations shown in Listing 5-4 use roc_curve and roc_auc_score programs from the sklearn. The process for generating the AUC versus α curve is similar to the process for the misclassification error, except the column of predictions and the true values are passed to the roc_auc_score program to generate the AUC number. Those then get plotted in Figure 5-8. The resulting curve looks roughly like the misclassification error curve upside down—upside down because larger is better for AUC, whereas smaller is better for misclassification error. The printed output at the end of Listing 5-4 shows that the location of the optimum model based on misclassification error isn't exactly the same as the optimum model for AUC, but they're not far apart. Figure 5-9 shows the ROC plot for the classifier that maximizes AUC.

In your problem, some errors might be more expensive than others, causing you to want to bias the results away from the expensive errors in favor of the less expensive errors. For the rocks versus mines problem, there may be much higher expense for incorrectly classifying an unexploded mine as a rock than for classifying a rock as a mine.

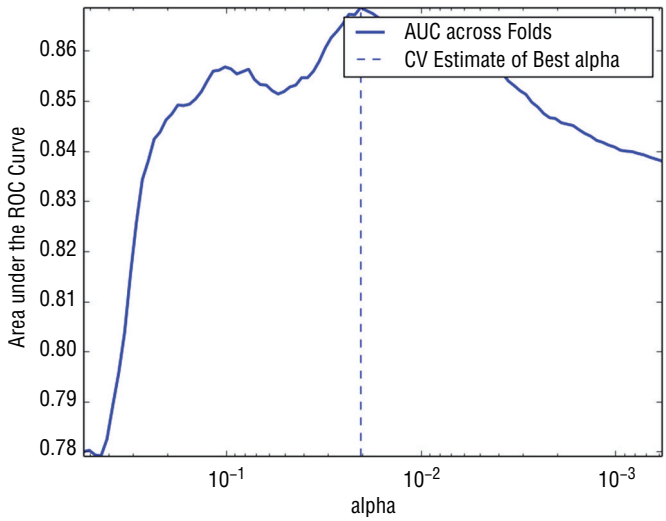


Figure 5-8: Out-of-sample classifier AUC performance

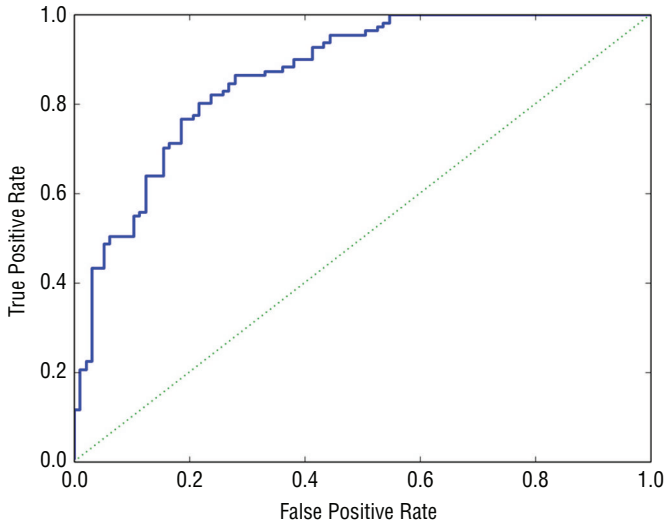


Figure 5-9: Receiver operating characteristic for best performing classifier

One systematic way to deal with this is to use a confusion matrix, discussed in Chapter 3. It's relatively easy to build from the output of the roc_curve program. The points on the ROC curve correspond to different values of threshold. The point (1,1) corresponds to the extreme where the threshold is set so low that all the points are classified as mines. That makes both the true positive rate and false positive rate equal one; the classifier gets all the positive points right, but it also gets all the negative points wrong. Setting the threshold higher than all the points gives the opposite corner of the plot. Getting the details on how points are shifting between the various boxes in the confusion matrix requires

picking some threshold values and printing out the results. Listing 5-4 shows three values of threshold chosen from the range of threshold values at inner quartiles of the threshold values (that is, excluding the end points). Setting the threshold high results in low false positives and high false negatives. Setting the threshold low has the opposite behavior. Setting the threshold in the middle more nearly balances the two types of errors.

You could get a best value of threshold by associating costs with each type of error and finding the value of the threshold that minimizes the total cost. The three confusion matrices in the printed output can serve as an example for how this would work. If false positive and false negative both cost $1, the middle table (corresponding to a threshold value of -0.0455) gives a total cost of $46, whereas the higher threshold gives $68 and the lower threshold gives $54. However, if the cost for false positive is $10 and the cost for false negative is $1, the higher threshold gives $113, the middle gives $226, and the lower gives $504. You might want to test more threshold values at finer granularity. For this approach to work properly, you'll need to get the costs in a reasonable ballpark, and you'll need to make sure that the percentages of positive cases and negative cases match those that you'll see in real examples. The rocks versus mines examples were set up in a laboratory environment and probably don't represent the actual numbers of rocks versus mines in a harbor. That's easy enough to fix by oversampling one class or the other—that is, replicating some of the examples in one class or the other to get the proportions to match those you expect to see in deployment.

The data in the rocks versus mines training set are fairly well balanced. That is, there are roughly the same number of positive and negative examples. In some data sets, there may be many more examples of one class or the other. For example, clicks on Internet ads are a small fraction of 1 percent of the number of times the ads are seen. You may get better training results by over-representing the less numerous examples so that the proportions are closer to equal. You can accomplish this by replicating some of the less numerous cases or removing some of the more numerous ones.

Listing 5-4: Using ElasticNet Regression to Build a Binary (Two-Class) Classifier—rocksVMinesENetRegCV.py

```
__author__ = 'mike_bowles'
import urllib2
from math import sqrt, fabs, exp
import matplotlib.pyplot as plot
from sklearn.linear_model import enet_path
from sklearn.metrics import roc_auc_score, roc_curve
import numpy

#read data from uci data repository
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")
data = urllib2.urlopen(target_url)
```

```
#arrange data into list for labels and list of lists for attributes
xList = []


for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)

#separate labels from attributes, convert from attributes from string
#to numeric and convert "M" to 1 and "R" to 0

xNum = []
labels = []

for row in xList:
    lastCol = row.pop()
    if lastCol == "M":
        labels.append(1.0)
    else:
        labels.append(0.0)
    attrRow = [float(elt) for elt in row]
    xNum.append(attrRow)

#number of rows and columns in x matrix
nrow = len(xNum)
ncol = len(xNum[1])

alpha = 1.0

#calculate means and variances
xMeans = []
xSD = []
for i in range(ncol):
    col = [xNum[j][i] for j in range(nrow)]
    mean = sum(col)/nrow
    xMeans.append(mean)
    colDiff = [(xNum[j][i] - mean) for j in range(nrow)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrow)])
    stdDev = sqrt(sumSq/nrow)
    xSD.append(stdDev)

#use calculate mean and standard deviation to normalize xNum
xNormalized = []
for i in range(nrow):
    rowNormalized = [(xNum[i][j] - xMeans[j])/xSD[j] \
        for j in range(ncol)]
    xNormalized.append(rowNormalized)

#normalize labels to center
#Normalize labels
```

continues

continued

```
meanLabel = sum(labels)/nrow
sdLabel = sqrt(sum([(labels[i] - meanLabel) * (labels[i] - meanLabel) \
        for i in range(nrow)])/nrow)

labelNormalized = [(labels[i] - meanLabel)/sdLabel for i in range(nrow)]


#number of cross-validation folds
nxval = 10


for ixval in range(nxval):
    #Define test and training index sets
    idxTest = [a for a in range(nrow) if a%nxval == ixval%nxval]
    idxTrain = [a for a in range(nrow) if a%nxval != ixval%nxval]

    #Define test and training attribute and label sets
    xTrain = numpy.array([xNormalized[r] for r in idxTrain])
    xTest = numpy.array([xNormalized[r] for r in idxTest])
    labelTrain = numpy.array([labelNormalized[r] for r in idxTrain])
    labelTest = numpy.array([labelNormalized[r] for r in idxTest])
    alphas, coefs, _ = enet_path(xTrain, labelTrain,l1_ratio=0.8,
        fit_intercept=False, return_models=False)

    #apply coefs to test data to produce predictions and accumulate
    if ixval == 0:
        pred = numpy.dot(xTest, coefs)
        yOut = labelTest
    else:
        #accumulate predictions
        yTemp = numpy.array(yOut)
        yOut = numpy.concatenate((yTemp, labelTest), axis=0)

        #accumulate predictions
        predTemp = numpy.array(pred)
        pred = numpy.concatenate((predTemp, numpy.dot(xTest, coefs)),
            axis = 0)


#calculate misclassification error
misClassRate = []
_,nPred = pred.shape
for iPred in range(1, nPred):
    predList = list(pred[:, iPred])
    errCnt = 0.0
    for irow in range(nrow):
        if (predList[irow] < 0.0) and (yOut[irow] >= 0.0):
            errCnt += 1.0
        elif (predList[irow] >= 0.0) and (yOut[irow] < 0.0):
            errCnt += 1.0
    misClassRate.append(errCnt/nrow)
```

```
#find minimum point for plot and for print
minError = min(misClassRate)
idxMin = misClassRate.index(minError)
plotAlphas = list(alphas[1:len(alphas)])

plot.figure()
plot.plot(plotAlphas, misClassRate,
    label='Misclassification Error Across Folds', linewidth=2)
plot.axvline(plotAlphas[idxMin], linestyle='--',
            label='CV Estimate of Best alpha')
plot.legend()
plot.semilogx()
ax = plot.gca()
ax.invert_xaxis()
plot.xlabel('alpha')
plot.ylabel('Misclassification Error')
plot.axis('tight')
plot.show()



#calculate AUC.
idxPos = [i for i in range(nrow) if yOut[i] > 0.0]
yOutBin = [0] * nrow
for i in idxPos: yOutBin[i] = 1

auc = []
for iPred in range(1, nPred):
    predList = list(pred[:, iPred])
    aucCalc = roc_auc_score(yOutBin, predList)
    auc.append(aucCalc)

maxAUC = max(auc)
idxMax = auc.index(maxAUC)

plot.figure()
plot.plot(plotAlphas, auc, label='AUC Across Folds', linewidth=2)
plot.axvline(plotAlphas[idxMax], linestyle='--',
            label='CV Estimate of Best alpha')
plot.legend()
plot.semilogx()
ax = plot.gca()
ax.invert_xaxis()
plot.xlabel('alpha')
plot.ylabel('Area Under the ROC Curve')
plot.axis('tight')
plot.show()


#plot best version of ROC curve
fpr, tpr, thresh = roc_curve(yOutBin, list(pred[:, idxMax]))
```

continues

continued

```
ctClass = [i*0.01 for i in range(101)]

plot.plot(fpr, tpr, linewidth=2)
plot.plot(ctClass, ctClass, linestyle=':')
plot.xlabel('False Positive Rate')
plot.ylabel('True Positive Rate')
plot.show()

print('Best Value of Misclassification Error = ', misClassRate[idxMin])
print('Best alpha for Misclassification Error = ', plotAlphas[idxMin])
print('')
print('Best Value for AUC = ', auc[idxMax])
print('Best alpha for AUC   =  ', plotAlphas[idxMax])

print('')
print('Confusion Matrices for Different Threshold Values')

#pick some points along the curve to print. There are 208 points.
#The extremes aren't useful

#Sample at 52, 104 and 156. Use the calculated values of tpr and fpr
#along with definitions and threshold values.

#Some nomenclature (e.g. see wikipedia "receiver operating curve")


#P = Positive cases
P = len(idxPos)
#N = Negative cases
N = nrow - P
#TP = True positives = tpr * P
TP = tpr[52] * P
#FN = False negatives = P - TP
FN = P - TP
#FP = False positives = fpr * N
FP = fpr[52] * N
#TN = True negatives = N - FP
TN = N - FP

print('Threshold Value =   ', thresh[52])
print('TP = ', TP, 'FP = ', FP)
print('FN = ', FN, 'TN = ', TN)

TP = tpr[104] * P; FN = P - TP; FP = fpr[104] * N; TN = N - FP

print('Threshold Value =   ', thresh[104])
print('TP = ', TP, 'FP = ', FP)
print('FN = ', FN, 'TN = ', TN)

TP = tpr[156] * P; FN = P - TP; FP = fpr[156] * N; TN = N - FP

print('Threshold Value =   ', thresh[156])
```

```
print('TP = ', TP, 'FP = ', FP)
print('FN = ', FN, 'TN = ', TN)


Printed Output: [filename - rocksVMinesENetRegCVPrintedOutput.txt]
('Best Value of Misclassification Error = ', 0.22115384615384615)
('Best alpha for Misclassification Error = ', 0.017686244720179375)

('Best Value for AUC = ', 0.86867279650784812)
('Best alpha for AUC   =  ', 0.020334883589342503)

Confusion Matrices for Different Threshold Values
('Threshold Value =   ', 0.37952298245219962)
('TP = ', 48.0, 'FP = ', 5.0)
('FN = ', 63.0, 'TN = ', 92.0)
('Threshold Value =   ', -0.045503481125357965)
('TP = ', 85.0, 'FP = ', 20.0)
('FN = ', 26.0, 'TN = ', 77.0)
('Threshold Value =   ', -0.4272522354395466)
('TP = ', 107.0, 'FP = ', 49.999999999999993)
('FN = ', 4.0, 'TN = ', 47.000000000000007)
```

Cross-validation gives you a solid estimate of the performance that you are going to see when you deploy this system. If the performance indicated by cross-validation is not good enough, you will have to work to improve it. For example, you might try the basis expansion that was used in the section "Multivariable Regression: Predicting Wine Taste." You might also have a look at the cases giving the worst errors and see if you can discern a pattern, whether they're data-entry errors or if another variable can be added that would account for their being mistaken so badly. If the error satisfies the needs of your problem, you'll want to train a model on the whole data set for deployment. The next section runs through that process.

Build a Rocks versus Mines Classifier for Deployment

As with the wine quality case study, the next step is to retrain the model on the full data set and pull out the coefficients corresponding to the best alpha—the one determined to minimize out-of-sample error, which is estimated in this case study by cross-validation. Listing 5-5 shows the code for accomplishing this.

Listing 5-5: Coefficient Trajectories for ElasticNet Trained on Rocks versus Mines Data—rocksVMinesCoefCurves.py

```
__author__ = 'mike_bowles'
import urllib2
from math import sqrt, fabs, exp
import matplotlib.pyplot as plot
```

continues

continued

```
from sklearn.linear_model import enet_pathsh
from sklearn.metrics import roc_auc_score, roc_curve
import numpy

#read data from uci data repository
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")
data = urllib2.urlopen(target_url)


#arrange data into list for labels and list of lists for attributes
xList = []


for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)

#separate labels from attributes, convert attributes from
#string to numeric and convert "M" to 1 and "R" to 0

xNum = []
labels = []

for row in xList:
    lastCol = row.pop()
    if lastCol == "M":
        labels.append(1.0)
    else:
        labels.append(0.0)
    attrRow = [float(elt) for elt in row]
    xNum.append(attrRow)

#number of rows and columns in x matrix
nrow = len(xNum)
ncol = len(xNum[1])

alpha = 1.0

#calculate means and variances
xMeans = []
xSD = []
for i in range(ncol):
    col = [xNum[j][i] for j in range(nrow)]
    mean = sum(col)/nrow
    xMeans.append(mean)
    colDiff = [(xNum[j][i] - mean) for j in range(nrow)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrow)])
    stdDev = sqrt(sumSq/nrow)
    xSD.append(stdDev)
```

```
#use calculate mean and standard deviation to normalize xNum
xNormalized = []
for i in range(nrow):
    rowNormalized = [(xNum[i][j] - xMeans[j])/xSD[j] \
        for j in range(ncol)]
    xNormalized.append(rowNormalized)

#normalize labels to center

meanLabel = sum(labels)/nrow
sdLabel = sqrt(sum([(labels[i] - meanLabel) * (labels[i] - meanLabel) \
        for i in range(nrow)])/nrow)

labelNormalized = [(labels[i] - meanLabel)/sdLabel for i in range(nrow)]

#Convert normalized labels to numpy array
Y = numpy.array(labelNormalized)

#Convert normalized attributes to numpy array
X = numpy.array(xNormalized)

alphas, coefs, _ = enet_path(X, Y,l1_ratio=0.8, fit_intercept=False,
    return_models=False)

plot.plot(alphas,coefs.T)

plot.xlabel('alpha')
plot.ylabel('Coefficients')
plot.axis('tight')
plot.semilogx()
ax = plot.gca()
ax.invert_xaxis()
plot.show()

nattr, nalpha = coefs.shape

#find coefficient ordering
nzList = []
for iAlpha in range(1,nalpha):
    coefList = list(coefs[: ,iAlpha])
    nzCoef = [index for index in range(nattr) if coefList[index] != 0.0]
    for q in nzCoef:
        if not(q in nzList):
            nzList.append(q)

#make up names for columns of X
names = ['V' + str(i) for i in range(ncol)]
nameList = [names[nzList[i]] for i in range(len(nzList))]
print("Attributes Ordered by How Early They Enter the Model")
```

continues

continued

```
print(nameList)
print('')
#find coefficients corresponding to best alpha value. alpha value
corresponding to normalized X and normalized Y is 0.020334883589342503

alphaStar = 0.020334883589342503
indexLTalphaStar = [index for index in range(100) if \
    alphas[index] > alphaStar]
indexStar = max(indexLTalphaStar)

#here's the set of coefficients to deploy
coefStar = list(coefs[:,indexStar])
print("Best Coefficient Values ")
print(coefStar)
print('')
#The coefficients on normalized attributes give another slightly
#different ordering

absCoef = [abs(a) for a in coefStar]

#sort by magnitude
coefSorted = sorted(absCoef, reverse=True)

idxCoefSize = [absCoef.index(a) for a in coefSorted if not(a == 0.0)]

namesList2 = [names[idxCoefSize[i]] for i in range(len(idxCoefSize))]

print("Attributes Ordered by Coef Size at Optimum alpha")
print(namesList2)

Printed Output: [filename - rocksVMinesCoefCurvesPrintedOutput.txt]
Attributes Ordered by How Early They Enter the Model
['V10', 'V48', 'V11', 'V44', 'V35', 'V51', 'V20', 'V3', 'V21', 'V45',
'V43', 'V15', 'V0', 'V22', 'V27', 'V50', 'V53', 'V30', 'V58', 'V56',
'V28', 'V39', 'V46', 'V19', 'V54', 'V29', 'V57', 'V6', 'V8', 'V7',
'V49', 'V2', 'V23', 'V37', 'V55', 'V4', 'V13', 'V36', 'V38', 'V26',
'V31', 'V1', 'V34', 'V33', 'V24', 'V16', 'V17', 'V5', 'V52', 'V41',
'V40', 'V59', 'V12', 'V9', 'V18', 'V14', 'V47', 'V42']

Best Coefficient Values
[0.082258256813766639, 0.0020619887220043702, -0.11828642590855878,
0.16633956932499627, 0.0042854388193718004, -0.0, -0.04366252474594004,
 -0.07751510487942842, 0.10000054356323497, 0.0, 0.090617207036282038,
0.21210870399915693, -0.0, -0.010655386149821946, -0.0,
-0.13328659558143779, -0.0, 0.0, 0.0, 0.052814854501417867,
0.038531154796719078, 0.0035515348181877982, 0.090854714680378215,
0.030316113904025031, -0.0, 0.0, 0.0086195542357481014, 0.0, 0.0,
0.17497679257272536, -0.2215687804617206, 0.012614243827937584,
0.0, -0.0, 0.0, -0.17160601809439849, -0.080450013824209077,
0.078096790041518344, 0.022035287616766441, -0.072184409273692227,
0.0, -0.0, 0.0, 0.057018816876250704, 0.096478265685721556,
0.039917367637236176, 0.049158231541622875, 0.0, 0.22671917920123755,
```

```
-0.096272735479951091, 0.0, 0.078886784332226484, 0.0,
0.062312821755756878, -0.082785510713295471, 0.014466967172068596,
-0.074326527525632721, 0.068096475974257331,
0.070488864435477847, 0.0]

Attributes Ordered by Coef Size at Optimum alpha
['V48', 'V30', 'V11', 'V29', 'V35', 'V3', 'V15', 'V2', 'V8', 'V44',
'V49', 'V22', 'V10', 'V54', 'V0', 'V36', 'V51', 'V37', 'V7', 'V56',
'V39', 'V58', 'V57', 'V53', 'V43', 'V19', 'V46', 'V6', 'V45', 'V20',
'V23', 'V38', 'V55', 'V31', 'V13', 'V26', 'V4', 'V21', 'V1']
```

The code in Listing 5-5 is structured similarly to the code in Listing 5-4, except that there's no cross-validation loop. The value for alpha at which coefficients are sought is hard-coded and comes directly from the results generated by Listing 5-4. There were two values of alpha generated: one that minimized the misclassification error and one that maximized the AUC. The alpha that maximized AUC was slightly larger and slightly more conservative. It was slightly to the left of the value that minimized misclassification error and therefore slightly more conservative. The coefficients printed by the program are listed at the bottom of the code. Out of the 60 coefficients, 20-some-odd are 0. In this run (as in the cross-validation program), the `l1_ratio` variable was set to `0.8`, which typically results in more coefficients than Lasso regression, which would correspond to `l1_ratio` at `1.0`.

A couple of measures of variable importance are printed at the bottom of the listing. One is the order in which variables come into the solution as alpha is decremented downward. The other ordering is according to the magnitude of the coefficients at the optimum solution. As discussed in conjunction with the wine quality data, these orderings only make sense when the attributes are normalized. Some degree of agreement exists between these two different variable orderings, but they don't agree completely. For example, the variables V48, V11, V35, V44, and V3 appear relatively high in both lists, but V10 appears at the top of the first list and is much further down in the ordering based on coefficient size. Apparently, V10 is important when the coefficient penalty is so large that the algorithm only permits a single attribute, but when the coefficient penalty has shrunk to the point that a multitude of attributes are included, the attribute V10 levels off and drops in importance somewhat as other attributes are added to the mix.

Typically, objects give the strongest reflections for waves whose wavelength is the same order of characteristic dimensions of the object. Mines (metal cylinders) have length and diameter—relatively few and relatively long characteristic dimensions to reflect compared to rocks, which are more fractal in character and reflect a broader range of wavelengths. Because all the attribute values in the data set are positive (power levels), you might expect that the wavelengths corresponding to low frequencies would get positive coefficients and the wavelengths corresponding to high frequencies would get negative coefficients.

You can see how this differencing could easily lead to overfitting the data and building a model that did extremely well on this data set but didn't generalize. The cross-validation process ensures that the model isn't overfit as long as the training data is statistically similar to what the model will see in deployment. The errors seen in cross-validation will match those in deployment to the extent that the rocks and mines encountered in deployment match the nature and proportions of those in the training data.

Figure 5-10 plots the coefficient curves for the ElasticNet regression models trained on the full rocks versus mines data set. The curves emphasize the complexity and changing nature of the relative importance of the available attributes.

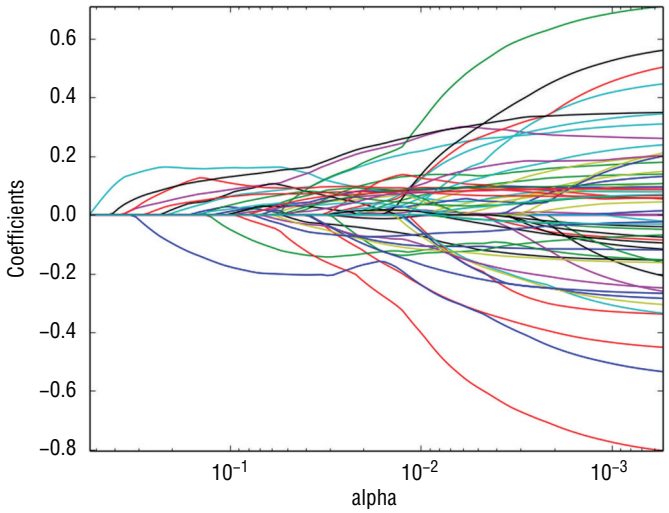


Figure 5-10: Coefficient curves for ElasticNet trained on rocks versus mines data

As mentioned in Chapter 4, an alternative to using penalized regression for classification is to use penalized logistic regression. Listing 5-5 shows code for an implementation of penalized logistic regression to build a classifier for the rocks versus mines data. The listing and the associated results highlight the similarities and differences between the two approaches. The algorithmic differences can be seen in the structure of the iteration. The logistic regression approach involves using linear functions of the attributes to calculate probabilities and likelihoods of each of the training examples being a rock or a mine. (See `http://en.wikipedia.org/wiki/Logistic_regression` for more background on logistic regression and for careful derivations of the associated equations.) The algorithm for nonpenalized logistic regression is called *iteratively reweighted least squares* (IRLS). The name comes from the nature of the algorithm (see `http://en.wikipedia.org/wiki/Iteratively_reweighted_least_squares`). It derives weights based on probability estimates for each example in the training

set. Given the weights, the problem becomes a weighted least squares regression problem. The process has to be iterated until the probabilities (and corresponding weights) stop changing. Basically, the IRLS for logistic regression adds another layer of iteration to the algorithm for (not logistic) penalized regression you saw in Chapter 4.

After reading in the variables and normalizing them, the program initializes weights and probabilities that are central to logistic regression and to the penalized version of it. These probabilities and weights have to be estimated along with the coefficients (β's) each time the penalty parameter decrements. You'll see the letters IRLS added to some of the variables in the code to denote that they are associated with the IRLS layer of the iteration. The iteration to estimate the probabilities is inside the loop for decrementing the λ's and wraps around the loop for iterating the coordinate descent on the β's.

The details of the update are slightly more complicated than the algorithm for plain (not logistic) penalized regression. One complication is the weights that come with IRLS. The weights and probabilities get calculated one input example at a time. Those are denoted by `p` and `w` in the code. The effects of the weights on sums of products like *attributes times residuals* and *squares of attributes* also need to be collected. Those are denoted by variables like `sumWxx`, which is a list containing the sum of the weights times each of the attributes squared. The other complication is that the residuals are now a function of the labels, the probabilities, and (more familiarly) the attributes and their coefficients (β's).

The code runs and produces variable ordering and coefficient curves to compare with those generated using nonlogistic penalized regression. The logistic transformation makes direct comparison of the coefficients problematic because the logistic function causes a nonlinear scale change. Both plain and logistic regression (penalized and nonpenalized) generate vectors of coefficients and then multiply the (same) attributes by them and compare to a threshold. The threshold value is somewhat secondary since it can be determined subsequent to training, as was demonstrated. So the overall scale of the β's doesn't matter as much as the magnitudes of the components relative to one another. One way to judge the relative magnitudes is to look at the order in which the two methods bring in new variables. As you can see by comparing the printed output in Listing 5-5 to the printed output in Listing 5-4, the two methods agree completely on the ordering for the first eight attributes. Of the next eight variables, seven of the eight are common to both lists, although they are ordered somewhat differently. Roughly the same is true of the next eight. There's fairly good general agreement in the ordering between the two methods.

Another question is which one delivers better performance. Assessing that requires running cross-validation with penalized logistic regression. You have the tools and code to carry that out. The code in Listing 5-6 is not at all optimized for speed, but it won't take too long on the rocks versus mines problem.

Listing 5-6: Penalized Logistic Regression Trained on Rocks versus Mines Data—rocksVMinesGlmnet.py

```
__author__ = 'mike_bowles'
import urllib2
import sys
from math import sqrt, fabs, exp
import matplotlib.pyplot as plot

def S(z,gamma):
    if gamma >= fabs(z):
        return 0.0
    if z > 0.0:
        return z - gamma
    else:
        return z + gamma

def Pr(b0,b,x):
    n = len(x)
    sum = b0
    for i in range(n):
        sum += b[i]*x[i]
        if sum < -100: sum = -100
    return 1.0/(1.0 + exp(-sum))


#read data from uci data repository
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")
data = urllib2.urlopen(target_url)


#arrange data into list for labels and list of lists for attributes
xList = []


for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)

#separate labels from attributes, convert from attributes from string
#to numeric and convert "M" to 1 and "R" to 0

xNum = []
labels = []

for row in xList:
    lastCol = row.pop()
    if lastCol == "M":
        labels.append(1.0)
```

```
    else:
        labels.append(0.0)
    attrRow = [float(elt) for elt in row]
    xNum.append(attrRow)

#number of rows and columns in x matrix
nrow = len(xNum)
ncol = len(xNum[1])

alpha = 0.8
#calculate means and variances
xMeans = []
xSD = []
for i in range(ncol):
    col = [xNum[j][i] for j in range(nrow)]
    mean = sum(col)/nrow
    xMeans.append(mean)
    colDiff = [(xNum[j][i] - mean) for j in range(nrow)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrow)])
    stdDev = sqrt(sumSq/nrow)
    xSD.append(stdDev)

#use calculate mean and standard deviation to normalize xNum
xNormalized = []
for i in range(nrow):
    rowNormalized = [(xNum[i][j] - xMeans[j])/xSD[j] \
        for j in range(ncol)]
    xNormalized.append(rowNormalized)

#Do Not Normalize labels but do calculate averages
meanLabel = sum(labels)/nrow
sdLabel = sqrt(sum([(labels[i] - meanLabel) * (labels[i] - meanLabel) \
    for i in range(nrow)])/nrow)

#initialize probabilities and weights
sumWxr = [0.0] * ncol
sumWxx = [0.0] * ncol
sumWr = 0.0
sumW = 0.0

#calculate starting points for betas
for iRow in range(nrow):
    p = meanLabel
    w = p * (1.0 - p)
    #residual for logistic
    r = (labels[iRow] - p) / w
    x = xNormalized[iRow]
    sumWxr = [sumWxr[i] + w * x[i] * r for i in range(ncol)]
    sumWxx = [sumWxx[i] + w * x[i] * x[i] for i in range(ncol)]
    sumWr = sumWr + w * r
    sumW = sumW + w
```

continues

continued

```
avgWxr = [sumWxr[i]/nrow for i in range(ncol)]
avgWxx = [sumWxx[i]/nrow for i in range(ncol)]

maxWxr = 0.0
for i in range(ncol):
    val = abs(avgWxr[i])
    if val > maxWxr:
        maxWxr = val

#calculate starting value for lambda
lam = maxWxr/alpha

#this value of lambda corresponds to beta = list of 0's
#initialize a vector of coefficients beta
beta = [0.0] * ncol
beta0 = sumWr/sumW

#initialize matrix of betas at each step
betaMat = []
betaMat.append(list(beta))

beta0List = []
beta0List.append(beta0)

#begin iteration
nSteps = 100
lamMult = 0.93 #100 steps gives reduction by factor of 1000 in lambda
               #(recommended by authors)
nzList = []
for iStep in range(nSteps):
    #decrease lambda
    lam = lam * lamMult


    #Use incremental change in betas to control inner iteration


    #set middle loop values for betas = to outer values
    #values are used for calculating weights and probabilities
    #inner values are used for calculating penalized regression updates

    #take pass through data to calculate averages over data required
    #for iteration
    #initilize accumulators

    betaIRLS = list(beta)
    beta0IRLS = beta0
    distIRLS = 100.0
    #Middle loop to calculate new betas with fixed IRLS weights and
    #probabilities
    iterIRLS = 0
```

```
while distIRLS > 0.01:
    iterIRLS += 1
    iterInner = 0.0

    betaInner = list(betaIRLS)
    beta0Inner = beta0IRLS
    distInner = 100.0
    while distInner > 0.01:
        iterInner += 1
        if iterInner > 100: break

        #cycle through attributes and update one-at-a-time
        #record starting value for comparison
        betaStart = list(betaInner)
        for iCol in range(ncol):

            sumWxr = 0.0
            sumWxx = 0.0
            sumWr = 0.0
            sumW = 0.0

            for iRow in range(nrow):
                x = list(xNormalized[iRow])
                y = labels[iRow]
                p = Pr(beta0IRLS, betaIRLS, x)
                if abs(p) < 1e-5:
                    p = 0.0
                    w = 1e-5
                elif abs(1.0 - p) < 1e-5:
                    p = 1.0
                    w = 1e-5
                else:
                    w = p * (1.0 - p)

                z = (y - p) / w + beta0IRLS + sum([x[i] *
                    betaIRLS[i] for i in range(ncol)])
                r = z - beta0Inner - sum([x[i] * betaInner[i]
                    for i in range(ncol)])
                sumWxr += w * x[iCol] * r
                sumWxx += w * x[iCol] * x[iCol]
                sumWr += w * r
                sumW += w

            avgWxr = sumWxr / nrow
            avgWxx = sumWxx / nrow

            beta0Inner = beta0Inner + sumWr / sumW
            uncBeta = avgWxr + avgWxx * betaInner[iCol]
            betaInner[iCol] = S(uncBeta, lam * alpha) / (avgWxx +
                lam * (1.0 - alpha))
```

continues

continued

```
            sumDiff = sum([abs(betaInner[n] - betaStart[n]) \
                for n in range(ncol)])
            sumBeta = sum([abs(betaInner[n]) for n in range(ncol)])
            distInner = sumDiff/sumBeta
        #print number of steps for inner and middle loop convergence
        #to monitor behavior
        #print(iStep, iterIRLS, iterInner)

        #if exit inner while loop, then set betaMiddle = betaMiddle
        #and run through middle loop again.

        #Check change in betaMiddle to see if IRLS is converged
        a = sum([abs(betaIRLS[i] - betaInner[i]) for i in range(ncol)])
        b = sum([abs(betaIRLS[i]) for i in range(ncol)])
        distIRLS = a / (b + 0.0001)
        dBeta = [betaInner[i] - betaIRLS[i] for i in range(ncol)]
        gradStep = 1.0
        temp = [betaIRLS[i] + gradStep * dBeta[i] for i in range(ncol)]
        betaIRLS = list(temp)

    beta = list(betaIRLS)
    beta0 = beta0IRLS
    betaMat.append(list(beta))
    beta0List.append(beta0)

    nzBeta = [index for index in range(ncol) if beta[index] != 0.0]
    for q in nzBeta:
        if not(q in nzList):
            nzList.append(q)

#make up names for columns of xNum
names = ['V' + str(i) for i in range(ncol)]
nameList = [names[nzList[i]] for i in range(len(nzList))]

print("Attributes Ordered by How Early They Enter the Model")
print(nameList)
for i in range(ncol):
    #plot range of beta values for each attribute
    coefCurve = [betaMat[k][i] for k in range(nSteps)]
    xaxis = range(nSteps)
    plot.plot(xaxis, coefCurve)

plot.xlabel("Steps Taken")
plot.ylabel("Coefficient Values")
plot.show()

Printed Output: [filename - rocksVMinesGlmnetPrintedOutput.txt]

Attributes Ordered by How Early They Enter the Model
['V10', 'V48', 'V11', 'V44', 'V35', 'V51', 'V20', 'V3', 'V50', 'V21',
'V43', 'V47', 'V15', 'V27', 'V0', 'V22', 'V36', 'V30', 'V53', 'V56',
```

```
'V58', 'V6', 'V19', 'V28', 'V39', 'V49', 'V7', 'V23', 'V54', 'V8',
'V14', 'V2', 'V29', 'V38', 'V57', 'V45', 'V13', 'V32', 'V31', 'V42',
 'V16', 'V37', 'V59', 'V52', 'V25', 'V18', 'V1', 'V33', 'V4', 'V55',
'V17', 'V46', 'V26', 'V12', 'V40', 'V34', 'V5', 'V24', 'V41', 'V9']
```

Figure 5-11 shows the coefficient curves for rocks versus mines using penalized logistic regression. As noted, the scale of the coefficients is different from plain penalized regression because of the logistic function difference between the two methods. Ordinary regression attempts to fit a straight line to targets that are 0.0 and 1.0. Logistic regression attempts to predict probabilities of class membership by fitting a straight line to the "log odds ratio." Suppose p is the predicted probability that an example corresponds to the mines class. Then the odds ratio is the ratio $\frac{p}{1-p}$. The log odds ratio is the natural log of the odds ratio. Whereas p ranges from 0 to 1, the log odds ratio of p ranges from minus infinity to plus infinity. The cases where the log odd is very large and positive corresponds to cases where the prediction is very certain that the case belongs to the mines class. Ones that are large negative numbers correspond to the rocks class.

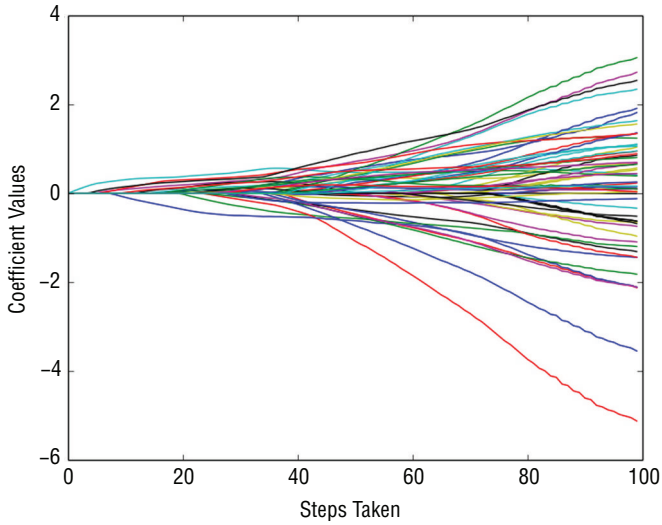


Figure 5-11: Coefficient curves for ElasticNet penalized logistic regression trained on rocks versus mines data

Because the two methods are predicting vastly different quantities, the scale on the predictions is much different and the coefficients are correspondingly different. But as the printed output from the two programs indicates, the order in which the variables appear in the solution is very similar, and the coefficient curves show that the signs are the same for the first several attributes that enter the solution.

Multiclass Classification: Classifying Crime Scene Glass Samples

The rocks versus mines problem that you saw in the last section is called a binary classification problem because the labels and predictions take one of two possible values. (Did the sonar return being processed come from reflections off a rock or a mine?) If labels and predictions can take more than two values, the problem is called a multiclass classification problem. This section uses penalized linear regression for the problem of classifying glass samples. As described more fully in Chapter 2, the glass data set consists of 9 physical chemistry measurements (refractive index and measurements of chemical composition) on 214 samples of 6 different types of glass. The problem is to use the physical chemistry measurements to determine which of the six types a given sample represents. The application for this is forensic analysis of crime and accident scenes. The data set comes from the UCI data repository, and the web page for the data set references a paper that uses support vector machines to solve this same problem. After looking at the code for solving this problem, this section will compare performance with the support vector machine approach.

Listing 5-7 shows code for solving this problem.

Listing 5-7: Multiclass Classification with Penalized Linear Regression - Classifying Crime Scene Glass Samples—glassENetRegCV.py

```
import urllib2
from math import sqrt, fabs, exp
import matplotlib.pyplot as plot
from sklearn.linear_model import enet_path
from sklearn.metrics import roc_auc_score, roc_curve
import numpy

target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/glass/glass.data")
data = urllib2.urlopen(target_url)

#arrange data into list for labels and list of lists for attributes
xList = []
for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)

names = ['RI', 'Na', 'Mg', 'Al', 'Si', 'K', 'Ca', 'Ba', 'Fe', 'Type']

#Separate attributes and labels
xNum = []
```

```
labels = []

for row in xList:
    labels.append(row.pop())
    l = len(row)
    #eliminate ID
    attrRow = [float(row[i]) for i in range(1, l)]
    xNum.append(attrRow)

#number of rows and columns in x matrix
nrow = len(xNum)
ncol = len(xNum[1])

#create one versus all label vectors
#get distinct glass types and assign index to each
yOneVAll = []
labelSet = set(labels)
labelList = list(labelSet)
labelList.sort()
nlabels = len(labelList)
for i in range(nrow):
    yRow = [0.0]*nlabels
    index = labelList.index(labels[i])
    yRow[index] = 1.0
    yOneVAll.append(yRow)

#calculate means and variances
xMeans = []
xSD = []
for i in range(ncol):
    col = [xNum[j][i] for j in range(nrow)]
    mean = sum(col)/nrow
    xMeans.append(mean)
    colDiff = [(xNum[j][i] - mean) for j in range(nrow)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrow)])
    stdDev = sqrt(sumSq/nrow)
    xSD.append(stdDev)

#use calculate mean and standard deviation to normalize xNum
xNormalized = []
for i in range(nrow):
    rowNormalized = [(xNum[i][j] - xMeans[j])/xSD[j] \
        for j in range(ncol)]
    xNormalized.append(rowNormalized)

#normalize y's to center
yMeans = []
ySD = []
for i in range(nlabels):
    col = [yOneVAll[j][i] for j in range(nrow)]
```

continues

continued

```
    mean = sum(col)/nrow
    yMeans.append(mean)
    colDiff = [(yOneVAll[j][i] - mean) for j in range(nrow)]
    sumSq = sum([colDiff[i] * colDiff[i] for i in range(nrow)])
    stdDev = sqrt(sumSq/nrow)
    ySD.append(stdDev)

yNormalized = []
for i in range(nrow):
    rowNormalized = [(yOneVAll[i][j] - yMeans[j])/ySD[j] \
        for j in range(nlabels)]
    yNormalized.append(rowNormalized)


#number of cross-validation folds
nxval = 10
nAlphas=200
misClass = [0.0] * nAlphas

for ixval in range(nxval):
    #Define test and training index sets
    idxTest = [a for a in range(nrow) if a%nxval == ixval%nxval]
    idxTrain = [a for a in range(nrow) if a%nxval != ixval%nxval]

    #Define test and training attribute and label sets
    xTrain = numpy.array([xNormalized[r] for r in idxTrain])
    xTest = numpy.array([xNormalized[r] for r in idxTest])
    yTrain = [yNormalized[r] for r in idxTrain]
    yTest = [yNormalized[r] for r in idxTest]
    labelsTest = [labels[r] for r in idxTest]

    #build model for each column in yTrain
    models = []
    lenTrain = len(yTrain)
    lenTest = nrow - lenTrain
    for iModel in range(nlabels):
        yTemp = numpy.array([yTrain[j][iModel]
            for j in range(lenTrain)])
        models.append(enet_path(xTrain, yTemp,l1_ratio=1.0,
            fit_intercept=False, eps=0.5e-3, n_alphas=nAlphas ,
            return_models=False))

    for iStep in range(1,nAlphas):
        #Assemble the predictions for all the models, find largest
        #prediction and calc error
        allPredictions = []
        for iModel in range(nlabels):
            _, coefs, _ = models[iModel]
            predTemp = list(numpy.dot(xTest, coefs[:,iStep]))
            #un-normalize the prediction for comparison
```

```
            predUnNorm = [(predTemp[j]*ySD[iModel] + yMeans[iModel]) \
                for j in range(len(predTemp))]
            allPredictions.append(predUnNorm)

        predictions = []
        for i in range(lenTest):
            listOfPredictions = [allPredictions[j][i] \
                for j in range(nlabels) ]
            idxMax = listOfPredictions.index(max(listOfPredictions))
            if labelList[idxMax] != labelsTest[i]:
                misClass[iStep] += 1.0

misClassPlot = [misClass[i]/nrow for i in range(1, nAlphas)]

plot.plot(misClassPlot)

plot.xlabel("Penalty Parameter Steps")
plot.ylabel(("Misclassification Error Rate"))
plot.show()
```

The first part of the code deals with reading the data from the UCI website and separating the labels from the attributes. The attributes get normalized in the usual way. The one-versus-all approach leads to some distinctive changes in the treatment of the labels. Instead of having a single set of labels, the one-versus-all approach leads to as many vectors of labels as there are distinct labels. In the glass problem, there are six different labels. So, where the regression and binary classification problems had a single vector of labels, the glass problem has six vectors of labels. The intuition behind this is as follows: If you've got a problem of dividing a set of points into two groups, one plane will do it. If your problem is to divide a set of points into six groups, you'll need more than a single plane.

One versus all trains as many different binary classifiers as there are distinct labels. The difference between these different classifiers is that they are trained to different labels. The code in Listing 5-7 shows how these labels are constructed from the original multiclass labels given by the problem. The approach is very similar to the approach you saw in Chapter 4 for converting categorical variables to numeric variables. The code listing extracts the distinct labels using a Python set, orders them from smallest to largest (not really necessary but helpful for keeping things straight), and then forms a column of labels where the first column has a 1 if the original labels take the first distinct label and has a 0 otherwise, the second column has a 1 if the original label takes the second distinct label, and so on. You can see why this is called one versus all. The labels in the first column will lead to a binary classifier predicting whether the sample takes the value of the first distinct label. Each of the six classifiers has a similar binary decision to make.

The code goes on to build a cross-validation loop along familiar lines. One minor difference is that the raw labels are also sliced into a test set to facilitate measuring misclassification error later on. There's a signal difference in the model training because on each cross-validation fold six models are trained and the trained models are stored in a list for later use. There are a couple of changes to the call to `enet_path` that are useful to discuss. One is that the `eps` parameter is spelled out and is exactly half the default value of `1e-3`. This is one of the parameters that you have at your disposal to control the range of penalty parameter values that are covered in the training. Recall from the discussion (and code example) in Chapter 4 that the coordinate descent algorithm progresses by decrementing the penalty parameter. The `eps` parameter tells the algorithm where to stop decrementing. The input `eps` is the ratio of the stopping value of the penalty parameter divided by the starting value. The parameter `n_alphas` controls the number of steps. Be aware that taking steps that are too large my result in the algorithm not converging. It will give you a warning message if it fails to converge. You can then either make `eps` a little larger so the penalty parameter doesn't get decremented by quite so much each step or you can take more steps by making `n_alphas` larger and thereby making each individual step smaller.

Another factor to consider is whether you're seeing enough of the curve. The plot in Figure 5-12 shows a minimum that's fairly close to the right edge of the graph. It would be useful to see a little more of the curve to be sure that the minimum isn't further to the right. Decreasing `eps` will show portions of the curve to the right of where the curve currently ends.

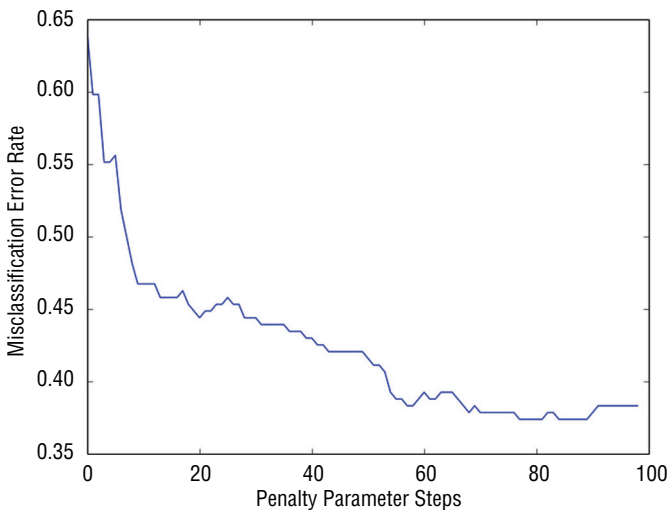


Figure 5-12: Misclassification error rates using penalized linear regression for glass classification

After the six models are trained, they are used to make six predictions. The code checks to see which of the six predictions has the largest numerical value and chooses the corresponding value for the prediction. Then that is compared to the actual value and the error is accumulated.

Figure 5-12 shows a plot of the misclassification error rate versus the number of decrementing steps that the penalty parameter has undergone. The plot shows a marked improvement at the minimum from the simplest model at the left-hand edge of the plot. The minimum value for misclassification error is roughly 35 percent. This is a better value than reported for a linear kernel support vector machine. That paper does achieve misclassification errors of 35 percent for some choices of nonlinear kernels and gets errors as low as 30 percent for some nonlinear kernels. Using nonlinear kernels in a support vector machine is roughly equivalent to basis expansion that you saw used in the wine quality example earlier in this chapter. Basis expansion didn't prove effective in the wine quality problem, but the fact that nonlinear kernels gave marked performance improvement for support vector machines makes that a promising method for improving performance in the glass classification problem.

Summary

This chapter demonstrated the use of penalized regression along with a number of general tools for predictive modeling. The chapter showed several different types of problems that you'll frequently encounter in real problems. These include regression, binary classification, and multiclass classification. The chapter used Python packages incarnating various different flavors of penalized regression for these tasks. In addition, the chapter illustrated the use of several tools that you may need in order to solve the modeling problems that you encounter. These include techniques for coding factor variables as numeric, for using a binary classifier to solve multiclass classification problems, and for extending linear methods to predict nonlinear relationships between attributes and outcomes.

The chapter also demonstrated a variety of ways to quantify performance for your predictive models. Regression problems are easiest to quantify because their errors can naturally be expressed in real number terms. Classification problems can be more involved. You saw classification performance quantified as misclassification error rates, area under the receiver operating curve, and economic costs. You should pick the method that comes closest to measuring performance in terms of your actual objectives (business objectives, science objectives, and so forth).

References

1. P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, Elsevier, 47(4):547–553.
2. T. Hastie, R. Tibshirani, and J. Friedman. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction.* 2nd ed. Springer-Verlag, New York.
3. J. Friedman, T. Hastie, and R. Tibshirani. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1).
4. K. Bache and M. Lichman. (2013). UCI Machine Learning Repository. Irvine, CA: University of California, School of Information and Computer Science. `http://archive.ics.uci.edu/ml`.

CHAPTER

6

Ensemble Methods

Ensemble methods stem from the observation that multiple models give better performance than a single model if the models are somewhat independent of one another. A classifier that will give you the correct result 55% of the time is only mediocre, but if you've got 100 classifiers that are correct 55% of the time, the probability that the majority of them are right jumps to 82%. (Google "cumulative binomial probability" and try some numbers yourself.)

One way to get a variety of models that are somewhat independent from one another is to use different machine learning algorithms. For example, you can build models with support vector machine, linear regression, k nearest neighbors, binary decision tree, and so on. But it's difficult to get a very long list of models that way. And, besides, it's tedious because the different models all have different parameters that need to be tweaked separately and may have different requirements on the input data. So, the models need to be coded separately. That's not going to work for generating hundreds or thousands of models (which you'll be doing soon).

Therefore, the key with ensemble methods is to develop an algorithm approach to generate numerous somewhat independent models that will then be combined into an ensemble. In this chapter you learn how the most popular methods accomplish this. The chapter teaches you the mechanics of the most popular ensemble methods. It outlines the basic structure of the algorithms and demonstrates the algorithms in Python code to give you a firm understanding of their workings.

Ensemble methods employ a hierarchy of two algorithms. The low-level algorithm is called a *base learner*. The base learner is a single machine learning algorithm

that gets used for all of the models that will get aggregated into an ensemble. This chapter will primarily use binary decision trees as base learners. The upper-level algorithm manipulates the inputs to the base learners so that the models they generate are somewhat independent. How can the same algorithm generate different models? There are several upper-level algorithms that are widely used. They go by the names *bagging*, *boosting*, and *random forests*. (Strictly speaking, random forests is a combination of an upper-level algorithm and particular modification to binary decision trees. You will see more detail on that in the section "Random Forests").

A number of different algorithms could conceivably be used as base learners—binary decision trees, support vector machine, and so on—but as a practical matter binary decision trees are the most widely used. They are the base learners in the open source and commercial packages that you'll be able to use in your projects. The ensembles are collections of hundreds or thousands of binary decision trees, and many of the properties of these ensembles are ones they inherit from binary decision trees. So, this chapter begins with an introduction to binary decision trees.

Binary Decision Trees

Binary decision trees operate by subjecting attributes to a series of binary (yes/no) decisions. Each decision leads to one of two possibilities. Each decision leads to another decision or it leads to prediction. An example of a trained tree will help cement the idea. You'll learn how training works after understanding the result of training.

Listing 6-1 shows the code to use scikitlearn's DecisionTreeRegressor package to build a binary decision tree for the wine quality data. Figure 6-1 depicts the trained tree produced by Listing 6-1.

Listing 6-1: Building a Decision Tree to Predict Wine Quality—wineTree.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
from sklearn import tree
from sklearn.tree import DecisionTreeRegressor
from sklearn.externals.six import StringIO
from math import sqrt
import matplotlib.pyplot as plot

#read data into iterable
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
"databases/wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
```

```
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

nrows = len(xList)
ncols = len(xList[0])

wineTree = DecisionTreeRegressor(max_depth=3)

wineTree.fit(xList, labels)

with open("wineTree.dot", 'w') as f:
    f = tree.export_graphviz(wineTree, out_file=f)
#Note: The code above exports the trained tree info to a
#Graphviz "dot" file.
#Drawing the graph requires installing GraphViz and the running the
#following on the command line
#dot -Tpng wineTree.dot -o wineTree.png
# In Windows, you can also open the .dot file in the GraphViz
#gui (GVedit.exe)]
```

Figure 6-1 shows the series of decisions produced as an outcome of the training on the wine quality data. The block diagram of the trained tree shows a number of boxes, which are called *nodes* in decision tree parlance. There are two types of nodes: Nodes can either pose a yes/no question of the data, or they can be terminal nodes that assign a prediction to examples that end up in them. Terminal nodes are often referred to as *leaf* nodes. In Figure 6-1, the terminal nodes are the nodes at the bottom of the figure that have no branches or further decision nodes below them.

How a Binary Decision Tree Generates Predictions

When an observation or row is passed to a nonterminal node, the row answers the node's question. If it answers yes, the row of attributes is passed to the leaf node below and to the left of the current node. If the row answers no, the row of

attributes is passed to the leaf node below and to the right of the current node. The process continues recursively until the row arrives at a terminal (that is, leaf) node where a prediction value is assigned to the row. The value assigned by the leaf node is the mean of the outcomes of the all the training observations that wound up in the leaf node.

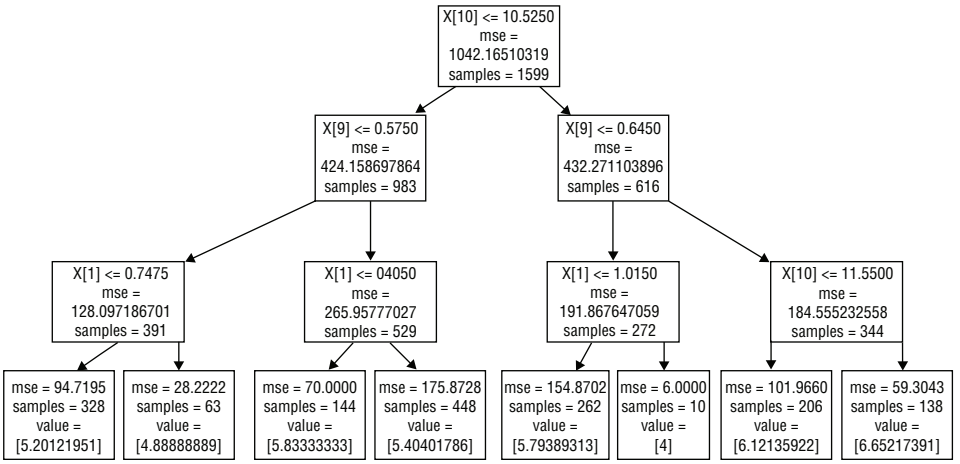


Figure 6-1: Decision tree for determining wine quality

While in this tree the second decision regards the variable X[9] in both branches of the tree, the two decisions can be about different attributes. (For example, see the third layer of decisions.)

Look at the top node, known as the *root* node. That node poses the question X[10] <= 10.525. In binary decision trees, important variables are split on early (or near the top of the tree), so the decision tree deems variable X[10], or alcohol content, important. In this respect, it agrees with the penalized linear regression in Chapter 5, "Building Predictive Models Using Penalized Linear Methods," which also deemed alcohol content most important in determining wine quality.

The tree in Figure 6-1 is said to have a depth of 3. The depth of a tree is defined as the number of decisions that have to be made down the longest path through the tree. The discussion of training in the section "Tree Training Equals Split Point Selection" will show you that there's no reason that all the paths to the terminal nodes have to be the same length (as they are in Figure 6-1).

You now have an idea what a trained tree looks like and you have seen how to how to use a trained tree to make predictions. Now you'll see how a tree gets trained.

How to Train a Binary Decision Tree

The easiest way to see how a tree gets trained is to look at a simple example. Listing 6-2 shows an example of predicting a real number label given a real

number attribute. The data set for this is created in the code (so called synthetic data). The basic idea is that the single attribute `x` has `100` equally spaced values between `-0.5` and `+0.5`. The labels `y` are equal to `x`, with some random noise added.

Listing 6-2: Training a Decision Tree for Simple Regression Problem—simpleTree.py

```
__author__ = 'mike-bowles'

import numpy
import matplotlib.pyplot as plot
from sklearn import tree
from sklearn.tree import DecisionTreeRegressor
from sklearn.externals.six import StringIO

#Build a simple data set with y = x + random
nPoints = 100

#x values for plotting
xPlot = [(float(i)/float(nPoints) - 0.5) for i in range(nPoints + 1)]

#x needs to be list of lists.
x = [[s] for s in xPlot]

#y (labels) has random noise added to x-value
#set seed
numpy.random.seed(1)
y = [s + numpy.random.normal(scale=0.1) for s in xPlot]

plot.plot(xPlot,y)
plot.axis('tight')
plot.xlabel('x')
plot.ylabel('y')
plot.show()

simpleTree = DecisionTreeRegressor(max_depth=1)
simpleTree.fit(x, y)

#draw the tree
with open("simpleTree.dot", 'w') as f:
    f = tree.export_graphviz(simpleTree, out_file=f)

#compare prediction from tree with true values

yHat  = simpleTree.predict(x)

plot.figure()
plot.plot(xPlot, y, label='True y')
plot.plot(xPlot, yHat, label='Tree Prediction ', linestyle='--')
```

continues

continued

```
plot.legend(bbox_to_anchor=(1,0.2))
plot.axis('tight')
plot.xlabel('x')
plot.ylabel('y')
plot.show()

simpleTree2 = DecisionTreeRegressor(max_depth=2)
simpleTree2.fit(x, y)

#draw the tree
with open("simpleTree2.dot", 'w') as f:
    f = tree.export_graphviz(simpleTree2, out_file=f)

#compare prediction from tree with true values

yHat  = simpleTree2.predict(x)

plot.figure()
plot.plot(xPlot, y, label='True y')
plot.plot(xPlot, yHat, label='Tree Prediction ', linestyle='--')
plot.legend(bbox_to_anchor=(1,0.2))
plot.axis('tight')
plot.xlabel('x')
plot.ylabel('y')
plot.show()

#split point calculations - try every possible split point to
#find the best one
sse = []
xMin = []
for i in range(1, len(xPlot)):
    #divide list into points on left and right of split point
    lhList = list(xPlot[0:i])
    rhList = list(xPlot[i:len(xPlot)])

    #calculate averages on each side
    lhAvg = sum(lhList) / len(lhList)
    rhAvg = sum(rhList) / len(rhList)

    #calculate sum square error on left, right and total
    lhSse = sum([(s - lhAvg) * (s - lhAvg) for s in lhList])
    rhSse = sum([(s - rhAvg) * (s - rhAvg) for s in rhList])

    #add sum of left and right to list of errors

    sse.append(lhSse + rhSse)
    xMin.append(max(lhList))

plot.plot(range(1, len(xPlot)), sse)
plot.xlabel('Split Point Index')
```

```
plot.ylabel('Sum Squared Error')
plot.show()

minSse = min(sse)
idxMin = sse.index(minSse)
print(xMin[idxMin])

#what happens if the depth is really high?
simpleTree6 = DecisionTreeRegressor(max_depth=6)
simpleTree6.fit(x, y)

#too many nodes to draw the tree
#with open("simpleTree2.dot", 'w') as f:
#    f = tree.export_graphviz(simpleTree6, out_file=f)

#compare prediction from tree with true values

yHat  = simpleTree6.predict(x)

plot.figure()
plot.plot(xPlot, y, label='True y')
plot.plot(xPlot, yHat, label='Tree Prediction ', linestyle='-')
plot.legend(bbox_to_anchor=(1,0.2))
plot.axis('tight')
plot.xlabel('x')
plot.ylabel('y')
plot.show()
```

Figure 6-2 plots the labels y versus the single attribute x. As you'd expect, y roughly follows x but with some randomness.

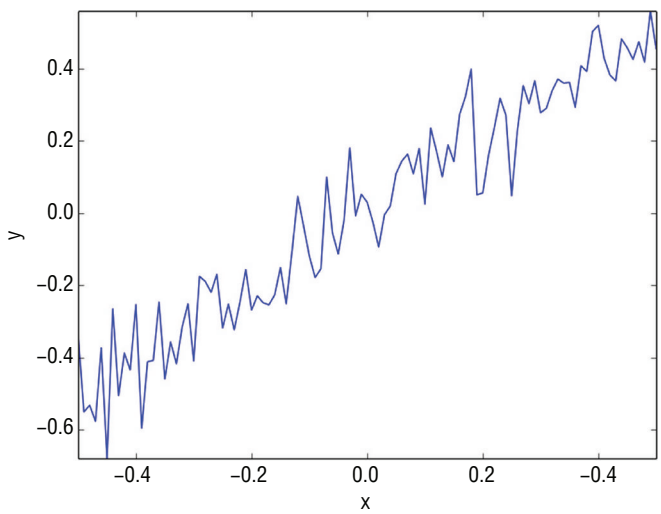


Figure 6-2: Label versus attribute for simple example

Tree Training Equals Split Point Selection

The first step in Listing 6-2 is to run scikitlearn's regression tree package with a depth of `1` specified. The results of that process are shown plotted in Figure 6-3. Figure 6-3 shows the block diagram for a depth 1 tree. Depth 1 trees are also called *stumps*. The single decision at the root node is to compare the attribute value with `-0.075`. This number is called the *split point* because it splits the data into two groups. The two boxes that emanate from the decision indicate that 43 of the 101 input examples go down the left leg of the tree, and the remaining 58 examples go down the right leg. If the attribute is less than the split point, the prediction from the tree is what's indicated as value in the block diagram—roughly –0.302.

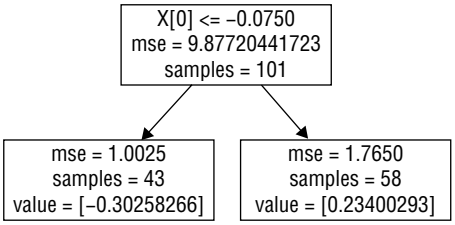


Figure 6-3: Block diagram of depth 1 tree for simple problem

How Split Point Selection Affects Predictions

Another way to view the trained tree is to see how its predictions compare with the true value of the labels. Because the simple synthetic problem has a single attribute only, the plot of the prediction generated by the trained tree alongside the actual values begins to give an idea about how the training of this simple tree was accomplished. The predicted values shown in Figure 6-4 follow a simple recipe. The prediction is a step function of the attribute. The step occurs at the split point.

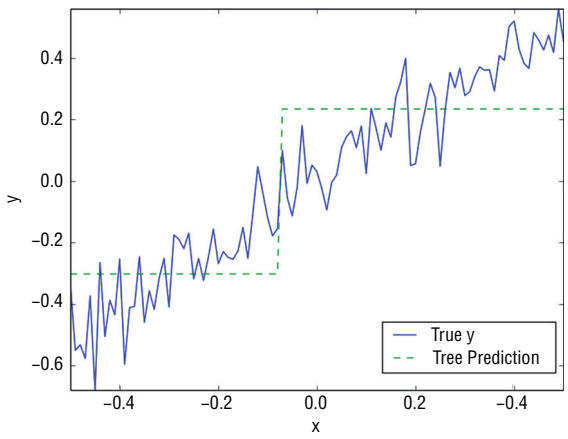


Figure 6-4: Comparison of predictions and actual values versus attribute for simple example

Algorithm for Selecting Split Points

Only three quantities are required to specify this simple tree: the split point value and the values assigned to the prediction if it falls into either of the two possible groups of points. Arriving at those quantities is accomplished during training of the tree. Here's how that works. The tree is trained to minimize the squared error of its predictions. Suppose first that the split point is given. Once the split point is given, the values assigned to the two groups are also determined. The average of each group is the single quantity that minimizes the mean squared error. That only leaves the question of how the split point is determined. Listing 6-2 has a small section of code that goes through the process of determining the split. The process is to try every possible split point. This is accomplished by dividing the data into two groups, approximating each group by its average, and then calculating the resulting sum squared error.

Figure 6-5 shows how the sum squared error varies as a function of the split point location. As you can see, there's a well-defined minimum at roughly the midpoint of the data set. Training a decision tree entails exhaustively searching all possible split points to determine which one minimizes the sum squared error. That takes care of this simple example.

Multivariable Tree Training—Which Attribute to Split?

What if the problem has more than one attribute? Then the algorithm checks all possible split points for all of the attributes to see which split point gives the best sum squared error for each attribute and then which attributes gives the overall minimum.

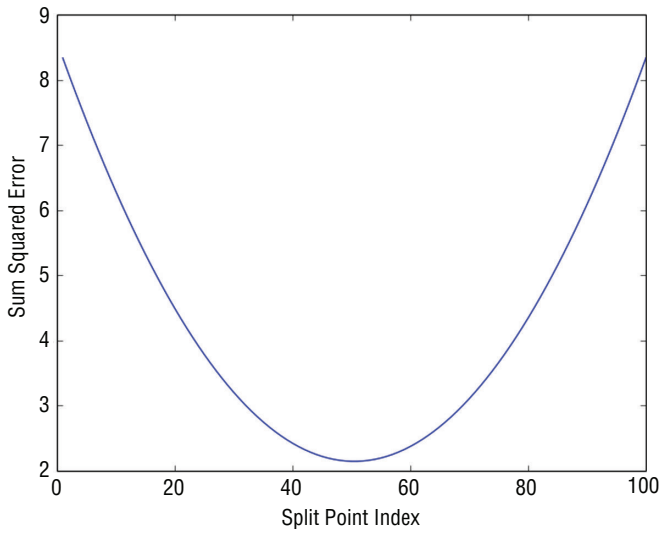


Figure 6-5: Sum squared error resulting from every possible split point location

This split point calculation is where all the computation cycles go in training a decision tree—and, by extension, where they go in training ensembles of trees. If the attribute being split doesn't have any repeat values, there's a split point to check for every data point (minus one).

As the data set gets larger, the number of split point calculations grows in direct proportion to the size of the data set. The split points that are checked can also get ridiculously close together. Algorithms designed to run on very large data sets allow split point checking to be considerably coarser than the raw granularity of the data. An approach to this is spelled out in "PLANET: Massively Parallel Learning of Tree Ensembles with MapReduce,"[1] which outlines the approach taken by engineers at Google to build a decision tree algorithm on large data sets. As mentioned in the paper, they wanted the decision tree algorithm so that they could implement gradient boosting (one of the ensemble algorithms you'll learn about later in this chapter).

Recursive Splitting for More Tree Depth

Listing 6-2 shows what happens to the prediction curve as the tree depth increases from 1 to 2. The resulting prediction curves are shown in Figure 6-6, and the block diagram for the tree is shown in Figure 6-7. Instead of having a single step, the prediction curve now has three steps. The second set of split points is determined in the same manner as the first one. Each node in the tree deals with the subset of points determined by the splits above it. The split point for each node is determined to minimize the sum squared error in the two nodes below. The curve in Figure 6-6 approximates the actual curve with a finer stair-step function. More tree depth results in finer steps and higher fidelity to the training data. Will that continue indefinitely?

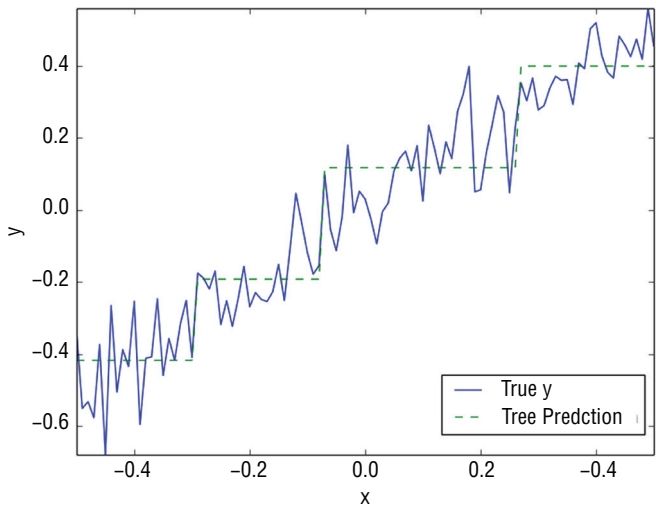


Figure 6-6: Prediction using depth 2 tree

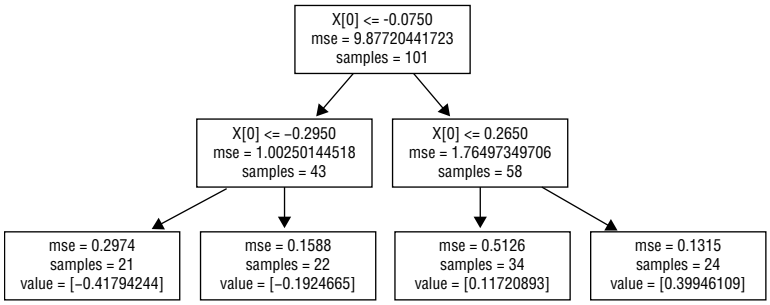


Figure 6-7: Block diagram for depth 2 tree

As splitting continues, the number of examples in the deepest nodes decreases. This can cause the splitting to terminate before the specified depth is reached. If there is only one example in a node, splitting certainly cannot continue. Tree training algorithms usually have a parameter to allow you to control how small a population will be split. Small populations in the nodes can lead to high variance in the resulting predictions.

Overfitting Binary Trees

The previous section showed how to train a binary decision tree of any depth. Is it possible to overfit a binary tree? This section discusses how to measure and regulate overfitting with binary trees. The mechanisms for overfitting binary trees are different from what you saw in Chapter 4, "Penalized Linear Regression," and Chapter 5, but you will see some similarities in the symptoms and how to measure overfitting. You will see that binary trees have parameters (tree depth and minimum leaf node size, for example) that can be used to regulate model complexity, similar to the process you saw in Chapters 4 and 5.

Measuring Overfit with Binary Trees

Figure 6-8 shows what happens when the tree depth is increased to 6. In Figure 6-8, it's hard to see the difference between the true value and the prediction. The prediction follows almost every zig and zag. That begins to suggest that the model is overfitting the data. The way the data were generated indicates that the best possible prediction would be for the prediction to equal the attribute value. The noise that was added to the attribute is unpredictable, and yet the prediction is following the noise-driven deviations of the label from the attribute. Synthetic data afford the luxury of knowing the correct answer.

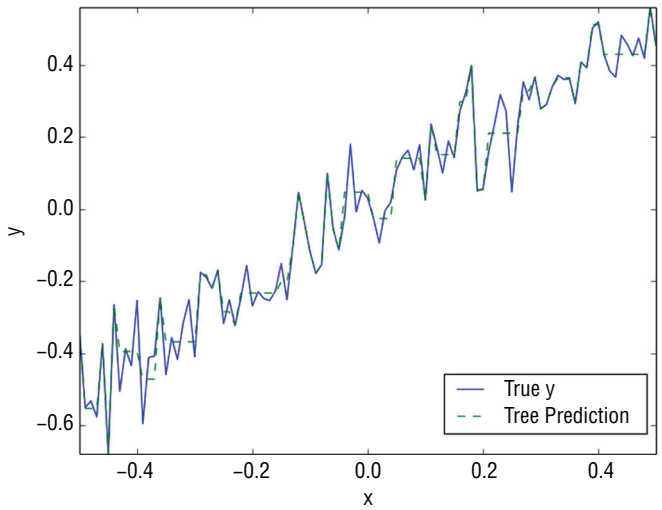


Figure 6-8: Prediction using depth 6 tree

Another way to look at overfitting with a binary tree is to consider the number of terminal nodes in the tree compared to the amount of data available. The tree that generated the prediction shown in Figure 6-8 was depth 6. That means that it has 64 terminal nodes (2^6). There are 100 points in the dataset. That means a lot of the points are the sole occupants of a terminal node, so their predicted value exactly matches their observed value. No wonder the graph of the prediction is matching the wiggles due to noise.

Balancing Binary Tree Complexity for Best Performance

In real problems, cross-validation can be performed to control overfitting. Listing 6-3 shows 10-fold cross-validation run on trees of a variety of depths for this simple problem. The code listing shows two loops. The outer one defines the tree depth for the inner cross-validation loop. The inner loop divides the data up and makes 10 passes to calculate out of sample errors. The mean squared error (MSE) results for each depth are plotted in Figure 6-9.

Listing 6-3: Cross-Validation at a Range of Tree Depths—simpleTreeCV.py

```
__author__ = 'mike-bowles'

import numpy
import matplotlib.pyplot as plot
from sklearn import tree
from sklearn.tree import DecisionTreeRegressor
from sklearn.externals.six import StringIO

#Build a simple data set with y = x + random
```

```
nPoints = 100

#x values for plotting
xPlot = [(float(i)/float(nPoints) - 0.5) for i in range(nPoints + 1)]

#x needs to be list of lists.
x = [[s] for s in xPlot]

#y (labels) has random noise added to x-value
#set seed
numpy.random.seed(1)
y = [s + numpy.random.normal(scale=0.1) for s in xPlot]

nrow = len(x)

#fit trees with several different values for depth and use
#x-validation to see which works best.

depthList = [1, 2, 3, 4, 5, 6, 7]
xvalMSE = []
nxval = 10

for iDepth in depthList:

    #build cross-validation loop to fit tree and evaluate on
    #out of sample data
    for ixval in range(nxval):

        #Define test and training index sets
        idxTest = [a for a in range(nrow) if a%nxval == ixval%nxval]
        idxTrain = [a for a in range(nrow) if a%nxval != ixval%nxval]

        #Define test and training attribute and label sets
        xTrain = [x[r] for r in idxTrain]
        xTest = [x[r] for r in idxTest]
        yTrain = [y[r] for r in idxTrain]
        yTest = [y[r] for r in idxTest]

        #train tree of appropriate depth and accumulate
        #out of sample (oos) errors
        treeModel = DecisionTreeRegressor(max_depth=iDepth)
        treeModel.fit(xTrain, yTrain)

        treePrediction = treeModel.predict(xTest)
        error = [yTest[r] - treePrediction[r] \
            for r in range(len(yTest))]

        #accumulate squared errors
        if ixval == 0:
            oosErrors = sum([e * e for e in error])
        else:
```

continues

continued

```
            #accumulate predictions
            oosErrors += sum([e * e for e in error])

    #average the squared errors and accumulate by tree depth

    mse = oosErrors/nrow
    xvalMSE.append(mse)

plot.plot(depthList, xvalMSE)
plot.axis('tight')
plot.xlabel('Tree Depth')
plot.ylabel('Mean Squared Error')
plot.show()
```

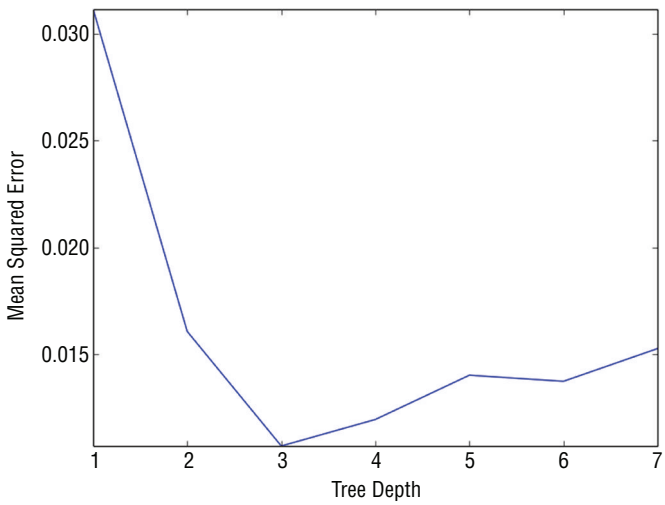


Figure 6-9: Out-of-sample error versus tree depth for simple problem

Tree depth is one way to regulate the complexity of a binary tree model. It has a similar effect to the coefficient penalty in the penalized regression model in Chapter 4 and Chapter 5. More tree depth makes it possible for the model to extract more complicated behaviors from the data at the cost of additional complexity. Figure 6-8 shows that depth 3 gives the best MSE performance for the synthetic problem from Listing 6-2. That depth makes the best trade off between reproducing the underlying relationships and overfitting the problem.

Recall from Chapter 3, "Predictive Model Building: Balancing Performance, Complexity, and Big Data," that the optimum model complexity is a function of the data set size. This synthetic data problem offers an opportunity to demonstrate how that works. Figure 6-10 shows how the optimum model complexity and performance change if the number of data points is increased to 1000.

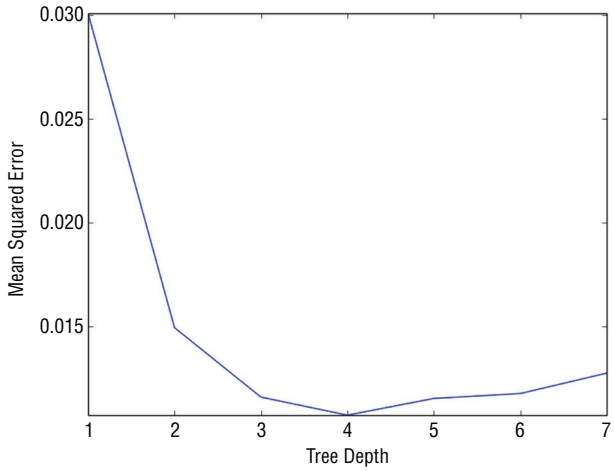


Figure 6-10: Out-of-sample MSE versus tree depth with 1,000 data points

You can run the plot for yourself by changing the variable `nPoints` in Listing 6-3 to `1000`. Two things happen as a result of adding more data. For one thing, the best tree depth increases from 3 to 4. The added data supports a more complicated model. For another thing, the MSE drops slightly. The added depth permits finer steps in the stair-step approximation of the real model. The added fidelity of the model is what excites people about really large data sets.

Modifications for Classification and Categorical Features

For you to have a complete picture of how decision trees are trained, there are a couple of other details to discuss. One is this: How does this work for a classification problem? The earlier criteria used to judge splits, MSE, makes sense only for regression problems. As you've seen elsewhere in the book, classification problems have different figures of merit than regression problems. Several figures of merit can be used with classification problems to judge splits in place of MSE. One that you're already familiar with is misclassification error. The other two commonly used measures are Gini impurity measure and information gain. For more information on these, see `http://en.wikipedia.org/wiki/Decision_tree_learning#Gini_impurity`. These other two measures have somewhat different properties from misclassification error, but aren't conceptually different.

The last detail is how trees can be trained on attributes that are categorical instead of being numeric. The (nonterminal) nodes in the tree pose a yes/no question. For numeric variables, the question is in the form of whether the given attribute is less than a parameter. Splitting a categorical variable into two subsets consists of trying all the possible divisions of the categories into two sets. If the categories are A, B, and C, the possible splits are A in the first group and B and

C in the second, B in the first group and A and C in the second, and so forth. There are some mathematical results that simplify this in some circumstances.

This section furnished some background on binary decision trees. On their own, binary decision trees are a legitimate prediction tool and worthy of study, but the main purpose of outlining them here is as background for ensemble methods, which incorporate hordes of binary decision trees. You will see that some of the issues that come up using an individual tree (multiple parameters to adjust, structural instability, and overfitting for large trees) will recede into the background when the hundreds or thousands of these trees are combined. That was the intent behind the development of ensemble methods which are remarkably robust, easy to train, and accurate. The next sections discuss the three main ensemble methods one at a time.

Bootstrap Aggregation: "Bagging"

Bootstrap aggregation was developed by Leo Breiman.[2] This method starts with picking a base learner. The method will be implemented here using binary decision trees as the base learners. You'll see as we go through the method that other machine learning algorithms could be used as base learners. Binary decision trees are a logical choice because they naturally model problems with complicated decision boundaries, but binary decision trees can exhibit excessive performance variance. Variance can be overcome by combining a multitude of tree-based models.

How Does the Bagging Algorithm Work?

The bootstrap aggregation algorithm uses what is called a *bootstrap* sample. The bootstrap sample is often used for generating sample statistics from a modest data set. A (nonparametric) bootstrap sample is a random selection of several elements from the data set with replacement (that is, a bootstrap sample can contain multiple copies of a row from the original data). Bootstrap aggregation takes a number of bootstrap samples from the training data set and then trains a base learner on each of these samples. The resulting models are averaged in regression problems. For classification problems, the models can either be averaged or probabilities can be developed based on the percentages of different classes. Listing 6-4 shows code for the bagging algorithm applied to the synthetic problem introduced at the beginning of the chapter.

The code holds out 30% of the data for measuring out-of-sample performance instead of using cross-validation. The parameter `numTreesMax` determines the maximum number of trees that will be included in the ensemble. The code builds models from the first tree, the first two trees, the first three trees, and so on, up to `numTreesMax` trees, to see how the accuracy depends on the number of trees included in the ensemble. The code stores the trained models in a list and stores the predictions on the data that were held out for out-of-sample error testing.

The code produces two plots. One plots shows how the MSE changes as more trees are included in the ensemble. The second plot shows how the predictions from the first tree, the average of the first 10 trees and the average of the first 20 trees, compare. The comparison plot is similar to the plot of the prediction curve relative to the actual labels as functions of the single attribute.

Listing 6-4: Bootstrap Aggregation Algorithm—simpleBagging.py

```
__author__ = 'mike-bowles'

import numpy
import matplotlib.pyplot as plot
from sklearn import tree
from sklearn.tree import DecisionTreeRegressor
from math import floor
import random

#Build a simple data set with y = x + random
nPoints = 1000

#x values for plotting
xPlot = [(float(i)/float(nPoints) - 0.5) for i in range(nPoints + 1)]

#x needs to be list of lists.
x = [[s] for s in xPlot]

#y (labels) has random noise added to x-value
#set seed
random.seed(1)
y = [s + random.normal(scale=0.1) for s in xPlot]

#take fixed test set 30% of sample
nSample = int(nPoints * 0.30)
idxTest = random.sample(range(nPoints), nSample)
idxTest.sort()
idxTrain = [idx for idx in range(nPoints) if not(idx in idxTest)]

#Define test and training attribute and label sets
xTrain = [x[r] for r in idxTrain]
xTest = [x[r] for r in idxTest]
yTrain = [y[r] for r in idxTrain]
yTest = [y[r] for r in idxTest]

#train a series of models on random subsets of the training data
#collect the models in a list and check error of composite as list grows

#maximum number of models to generate
numTreesMax = 20
```

continues

continued

```
#tree depth - typically at the high end
treeDepth = 1

#initialize a list to hold models
modelList = []
predList = []

#number of samples to draw for stochastic bagging
nBagSamples = int(len(xTrain) * 0.5)

for iTrees in range(numTreesMax):
    idxBag = random.sample(range(len(xTrain)), nBagSamples)
    xTrainBag = [xTrain[i] for i in idxBag]
    yTrainBag = [yTrain[i] for i in idxBag]

    modelList.append(DecisionTreeRegressor(max_depth=treeDepth))
    modelList[-1].fit(xTrainBag, yTrainBag)

    #make prediction with latest model and add to list of predictions
    latestPrediction = modelList[-1].predict(xTest)
    predList.append(list(latestPrediction))


#build cumulative prediction from first "n" models
mse = []
allPredictions = []
for iModels in range(len(modelList)):

    #average first "iModels" of the predictions
    prediction = []
    for iPred in range(len(xTest)):
        prediction.append(sum([predList[i][iPred] \
            for i in range(iModels + 1)])/(iModels + 1))

    allPredictions.append(prediction)
    errors = [(yTest[i] - prediction[i]) for i in range(len(yTest))]
    mse.append(sum([e * e for e in errors]) / len(yTest))


nModels = [i + 1 for i in range(len(modelList))]

plot.plot(nModels,mse)
plot.axis('tight')
plot.xlabel('Number of Models in Ensemble')
plot.ylabel('Mean Squared Error')
plot.ylim((0.0, max(mse)))
plot.show()

plotList = [0, 9, 19]
for iPlot in plotList:
```

```
    plot.plot(xTest, allPredictions[iPlot])
plot.plot(xTest, yTest, linestyle="--")
plot.axis('tight')
plot.xlabel('x value')
plot.ylabel('Predictions')
plot.show()
```

Figure 6-11 shows how the MSE varies as the number of trees is increased. The error more or less levels out at around 0.025. This isn't really very good. The noise that was added had a standard deviation of 0.1. The very best MSE a predictive algorithm could generate is the square of that standard deviation or 0.01. The single binary tree that was trained earlier in the chapter was getting close to 0.01. Why is this more sophisticated algorithm underperforming?

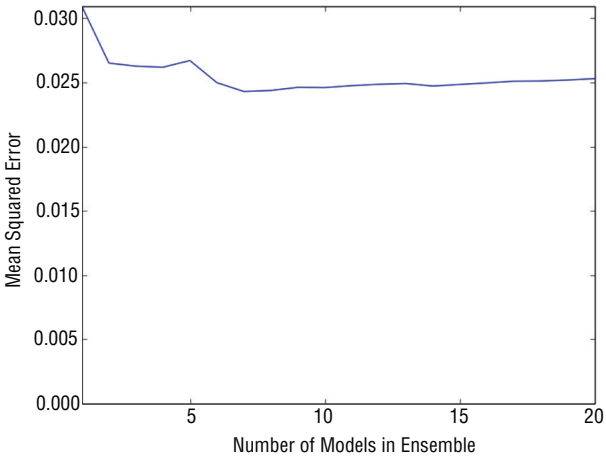


Figure 6-11: MSE versus number of trees in Bagging ensemble

Bagging Performance—Bias versus Variance

A look at Figure 6-12 gives some insight into the problem and raises a point that is important to illustrate because it's relevant to other problems too. Figure 6-12 shows the single tree prediction, the 10-tree prediction, and the 20-tree prediction. The prediction from the single tree is easy to discern because there's a single step. The 10- and 20-tree predictions superpose a number of slightly different trees so they have a series of finer steps that are in the neighborhood of the single step taken by the first tree. The steps of the multiple trees aren't all in exactly the same spot because they are trained on different samples of the data and that leads to some randomness in the split points. But that randomness only jiggles the split points in a relatively small neighborhood near the center of the graph. So, the resulting ensemble doesn't see much variety because all the trees in the ensemble roughly agree about where the single split point should go.

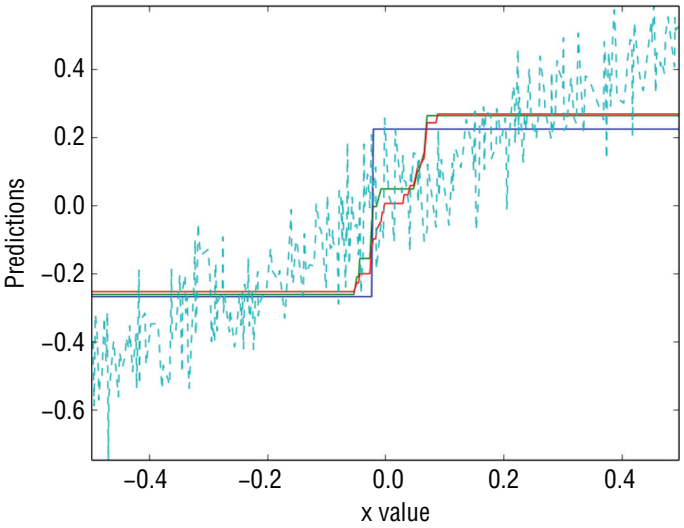


Figure 6-12: Comparison of prediction and actual label as functions of attribute

There are two types of error: bias and variance. Consider trying to fit a wiggly curve with a straight line. Getting more data can reduce the effect of noise in the data being used for fitting, but more data will not make a straight line into a wiggly curve. Errors that do not get smaller as more data points are added are called *bias errors*. Fitting depth-1 trees to the synthetic problem suffers from a bias error. All the split points are chosen near the center of the data, and the model accuracy suffers at the edges of the data.

The bias error with depth 1 trees comes from the basic model being too simple and sharing a common limitation. Bagging reduces variance between models. But with depth 1 trees, it gets a bias error, which can't be averaged. The way to overcome this problem is to use trees with more depth.

Figure 6-13 shows the curve of MSE versus number of trees in the ensemble for depth 5 trees. The MSE with depth 5 trees is somewhat smaller than 0.01 (probably due to randomness in the noise data), clearly much better performance than with depth 1 trees.

Figure 6-14 shows plots for the prediction based on the first tree, the first 10 trees, and the first 20 trees. The single tree prediction stands out from the others because it has a number of sharp spikes where it's making severe errors. In other words, it has a high variance. The other single trees undoubtedly show similar performance. But when they're average, the variance is reduced; the curve representing the prediction from the bagging algorithm is much smoother and closer to the true answer.

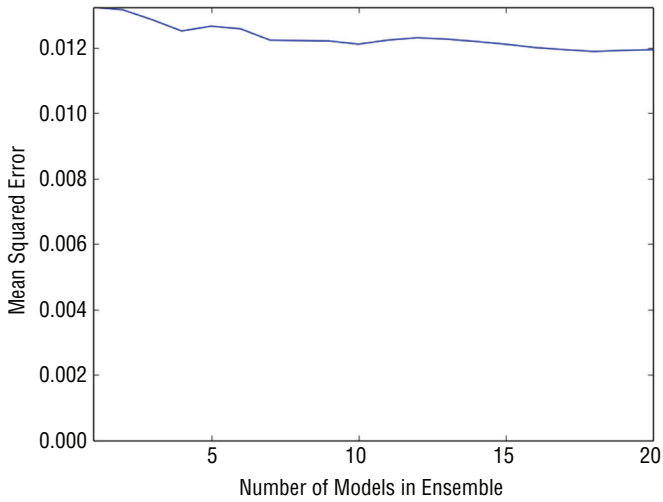


Figure 6-13: MSE versus number of trees with depth 5 trees

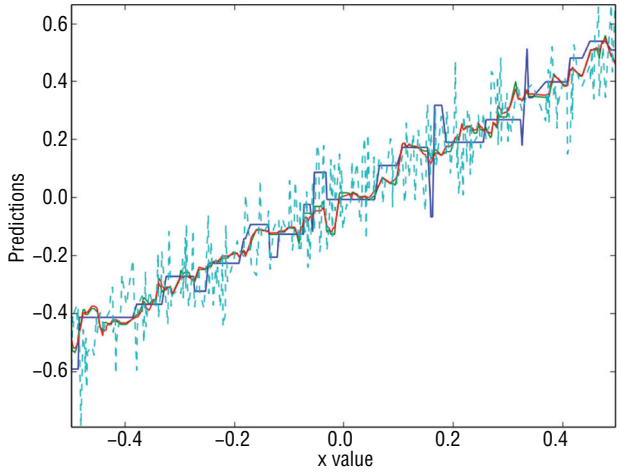


Figure 6-14: Comparison of prediction and actual labels with depth 5 trees

How Bagging Behaves on Multivariable Problem

Listing 6-5 shows the application of the bagging algorithm for the task of predicting wine quality. The wine example demonstrates some of the same principles as you saw with the synthetic data. These are best seen in Figures 6-15 through 6-17, which come from running Listing 6-4 with different parameter settings.

Listing 6-5: Predicting Wine Quality with Bagging—wineBagging.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
from sklearn import tree
from sklearn.tree import DecisionTreeRegressor
import random
from math import sqrt
import matplotlib.pyplot as plot

#read data into iterable
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
"databases/wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

nrows = len(xList)
ncols = len(xList[0])

#take fixed test set 30% of sample
random.seed(1)
nSample = int(nrows * 0.30)
idxTest = random.sample(range(nrows), nSample)
idxTest.sort()
idxTrain = [idx for idx in range(nrows) if not(idx in idxTest)]

#Define test and training attribute and label sets
xTrain = [xList[r] for r in idxTrain]
xTest = [xList[r] for r in idxTest]
yTrain = [labels[r] for r in idxTrain]
```

```
yTest = [labels[r] for r in idxTest]

#train a series of models on random subsets of the training data
#collect the models in a list and check error of composite as list grows

#maximum number of models to generate
numTreesMax = 30

#tree depth - typically at the high end
treeDepth = 1

#initialize a list to hold models
modelList = []
predList = []

#number of samples to draw for stochastic bagging
nBagSamples = int(len(xTrain) * 0.5)

for iTrees in range(numTreesMax):
    idxBag = []
    for i in range(nBagSamples):
        idxBag.append(random.choice(range(len(xTrain))))
    xTrainBag = [xTrain[i] for i in idxBag]
    yTrainBag = [yTrain[i] for i in idxBag]

    modelList.append(DecisionTreeRegressor(max_depth=treeDepth))
    modelList[-1].fit(xTrainBag, yTrainBag)

    #make prediction with latest model and add to list of predictions
    latestPrediction = modelList[-1].predict(xTest)
    predList.append(list(latestPrediction))


#build cumulative prediction from first "n" models
mse = []
allPredictions = []
for iModels in range(len(modelList)):

    #average first "iModels" of the predictions
    prediction = []
    for iPred in range(len(xTest)):
        prediction.append(sum([predList[i][iPred] \
            for i in range(iModels + 1)])/(iModels + 1))

    allPredictions.append(prediction)
    errors = [(yTest[i] - prediction[i]) for i in range(len(yTest))]
    mse.append(sum([e * e for e in errors]) / len(yTest))


nModels = [i + 1 for i in range(len(modelList))]
```

continues

continued

```
plot.plot(nModels,mse)
plot.axis('tight')
plot.xlabel('Number of Tree Models in Ensemble')
plot.ylabel('Mean Squared Error')
plot.ylim((0.0, max(mse)))
plot.show()

print('Minimum MSE')
print(min(mse))

#with treeDepth = 1
#Minimum MSE
#0.516236026081


#with treeDepth = 5
#Minimum MSE
#0.39815421341

#with treeDepth = 12 & numTreesMax = 100
#Minimum MSE
#0.350749027669
```

Figure 6-15 shows how MSE changes as more trees are included in the bagging ensemble. The ensemble of stumps (depth 1 trees) on the wine quality data shows negligible improvement in MSE over the single tree. The lack of improvement on the wine data is much more dramatic than with the synthetic data. This might be true for a couple of reasons. One possibility is that the errors at the edges of the data are more significant with the wine quality data than with the synthetic data. Another possibility is that interaction between variables is more important with the wine data.

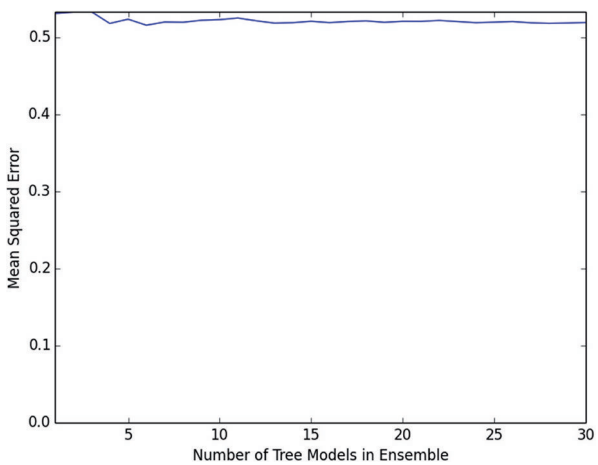


Figure 6-15: Predicting wine quality with Bagging on depth 1 trees

The synthetic data had only one variable, so no interaction between variables was possible. The wine data has multiple attributes, and so it's possible that the attributes in combination are more important than the sum of their individual contributions. If you stumble while walking, it won't likely be important. If you walk along the edge of a cliff, it won't likely be important. But if you stumble while walking along the edge of a cliff, it could be important. The two conditions have to be considered together. A depth 1 tree can only consider solitary attributes and therefore cannot account for strong interactions between variables.

Bagging Needs Tree Depth for Performance

Figure 6-16 shows how the MSE depends on number of trees when the trees all have depth 5. The Bagging ensemble shows clear improvement as more trees are added. The resulting performance is much better than that achieved by Bagging depth 1 trees. The improvement suggests that perhaps even more tree depth would yield further improvement.

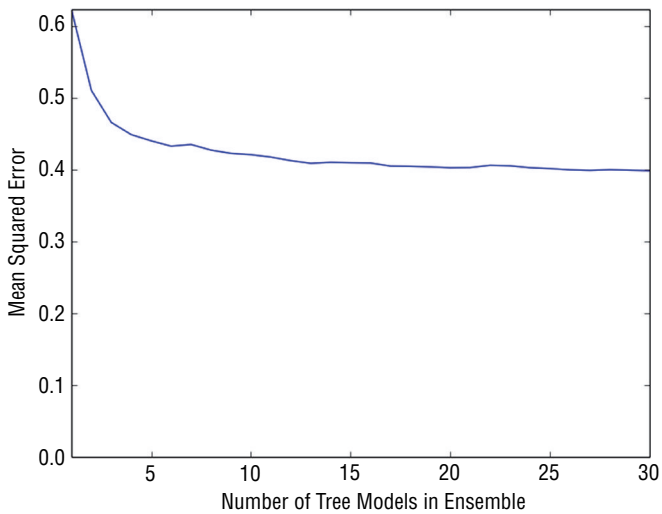


Figure 6-16: Predicting wine quality with Bagging on depth 5 trees

Figure 6-17 shows MSE versus number of trees in the Bagging ensemble when the trees are depth 12. In addition to employing deeper trees, the ensemble runs 100 trees rather than 30 to get a better picture of how much performance improvement is available by training larger numbers of trees for the Bagging ensemble. Figure 6-17 shows the lowest MSE of the three runs.

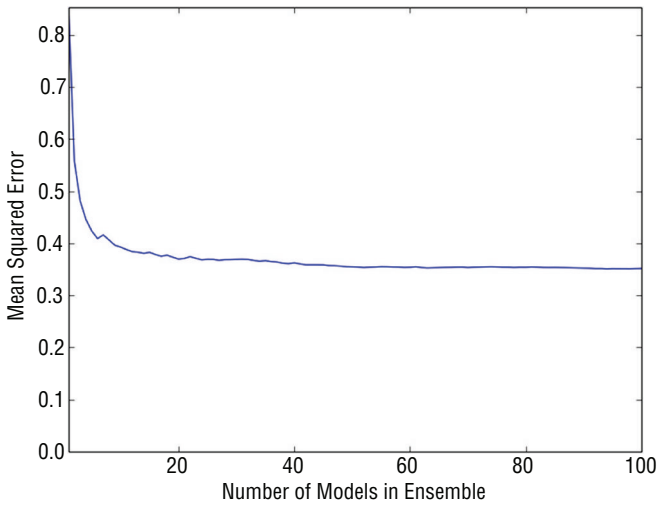


Figure 6-17: Predicting wine quality with Bagging on depth 12 trees

Summary of Bagging

Now you have seen a first example of an ensemble method. Bagging clearly demonstrates the two-level hierarchy common to ensemble methods. Properly speaking, bagging is the higher-level algorithm defining a series of subproblems to be solved by base learners and then averaging their predictions. The individual problems making up a bagging ensemble are derived by taking random bootstrap samples of the original training data. Bagging reduces the variance exhibited by individual binary trees. For bagging to work properly, the trees in a bagging ensemble need to be grown to sufficient depth.

Bagging serves as a good introduction to ensemble methods because it is relatively easy to understand and because it is relatively easy to demonstrate its variance reduction properties. The next two algorithms covered are gradient boosting and random forests. They take different approaches to building ensembles and exhibit some advantages over bagging. Most of the current practitioners I know use either gradient boosting or random forests first and do not regularly use bagging.

Gradient Boosting

Gradient boosting was developed by Stanford professor Jerome Friedman[4,5], who also developed the coordinate descent algorithm used to solve the ElasticNet problem (in Chapters 4 and 5). Gradient boosting develops an ensemble of tree-based models by training each of the trees in the ensemble on different labels and then combining the trees. For a regression problem where the objective is

to minimize MSE, each successive tree is trained on the errors left over by the collection of earlier trees. For the derivation of the algorithm see the References section at the end of this chapter. The easiest way to see how gradient boosting works is to look at some code implementing the algorithm.

Basic Principle of Gradient Boosting Algorithm

Listing 6-6 details the gradient boosting algorithm for the synthetic problem introduced earlier in this chapter. The early part of the code uses the process from earlier to build the synthetic data set.

Listing 6-6: Gradient Boosting for Simple Problem—simpleGBM.py

```
__author__ = 'mike-bowles'

import numpy
import matplotlib.pyplot as plot
from sklearn import tree
from sklearn.tree import DecisionTreeRegressor
from math import floor
import random

#Build a simple data set with y = x + random
nPoints = 1000

#x values for plotting
xPlot = [(float(i)/float(nPoints) - 0.5) for i in range(nPoints + 1)]

#x needs to be list of lists.
x = [[s] for s in xPlot]

#y (labels) has random noise added to x-value
#set seed
numpy.random.seed(1)
y = [s + numpy.random.normal(scale=0.1) for s in xPlot]

#take fixed test set 30% of sample
nSample = int(nPoints * 0.30)
idxTest = random.sample(range(nPoints), nSample)
idxTest.sort()
idxTrain = [idx for idx in range(nPoints) if not(idx in idxTest)]

#Define test and training attribute and label sets
xTrain = [x[r] for r in idxTrain]
xTest = [x[r] for r in idxTest]
yTrain = [y[r] for r in idxTrain]
yTest = [y[r] for r in idxTest]

#train a series of models on random subsets of the training data
#collect the models in a list and check error of composite as list grows
```

continues

continued

```
#maximum number of models to generate
numTreesMax = 30

#tree depth - typically at the high end
treeDepth = 5

#initialize a list to hold models
modelList = []
predList = []
eps = 0.3

#initialize residuals to be the labels y
residuals = list(yTrain)

for iTrees in range(numTreesMax):

    modelList.append(DecisionTreeRegressor(max_depth=treeDepth))
    modelList[-1].fit(xTrain, residuals)

    #make prediction with latest model and add to list of predictions
    latestInSamplePrediction = modelList[-1].predict(xTrain)

    #use new predictions to update residuals
    residuals = [residuals[i] - eps * latestInSamplePrediction[i] \
        for i in range(len(residuals))]

    latestOutSamplePrediction = modelList[-1].predict(xTest)
    predList.append(list(latestOutSamplePrediction))


#build cumulative prediction from first "n" models
mse = []
allPredictions = []
for iModels in range(len(modelList)):

    #add the first "iModels" of the predictions and multiply by eps
    prediction = []
    for iPred in range(len(xTest)):
        prediction.append(sum([predList[i][iPred]
            for i in range(iModels + 1)]) * eps)

    allPredictions.append(prediction)
    errors = [(yTest[i] - prediction[i]) for i in range(len(yTest))]
    mse.append(sum([e * e for e in errors]) / len(yTest))


nModels = [i + 1 for i in range(len(modelList))]

plot.plot(nModels,mse)
plot.axis('tight')
```

```
plot.xlabel('Number of Models in Ensemble')
plot.ylabel('Mean Squared Error')
plot.ylim((0.0, max(mse)))
plot.show()

plotList = [0, 14, 29]
lineType = [':', '-.', '--']
plot.figure()
for i in range(len(plotList)):
    iPlot = plotList[i]
    textLegend = 'Prediction with ' + str(iPlot) + ' Trees'
    plot.plot(xTest, allPredictions[iPlot], label = textLegend,
        linestyle = lineType[i])
plot.plot(xTest, yTest, label='True y Value', alpha=0.25)
plot.legend(bbox_to_anchor=(1,0.3))
plot.axis('tight')
plot.xlabel('x value')
plot.ylabel('Predictions')
plot.show()
```

Parameter Settings for Gradient Boosting

The first thing that looks unfamiliar is the comment about setting the depth parameter for the individual trees being trained in a gradient boosting ensemble. Gradient boosting differs from bagging and random forests in that it can reduce bias in addition to reducing variance. Gradient boosting has the useful property that it will often perform as well as low MSE values with stumps as with deeper trees. With gradient boosting, tree depth is only required to the extent that there's a significant interaction between variables. Performance improvement from increasing tree depth serves as a gauge of variable interaction in your problem.

The next thing that looks a little different is the definition of a variable called `eps`. This variable is a step size control of the sort that you may be familiar with from optimization problems. Gradient boosting takes gradient descent steps and, as with other gradient descent processes, if the steps are too large the optimization can diverge instead of converging. If the step size is too small, the process can take too many iterations. After generating some results, the chapter will talk about how to tune `eps`, the step size.

The next unfamiliar element of the code is the definition of a variable called `residuals`. The term *residuals* is commonly used to denote prediction errors (that is, observed values minus predicted values). The gradient boosting algorithm will make a series of refinements to its predictions of the labels. At each step along the way, the residuals will get recalculated. At the beginning of the process, gradient boosting initializes predictions to null (or zero) values so that the residuals are equal to the observed labels.

How Gradient Boosting Iterates Toward a Predictive Model

The loop on iTrees begins by training a tree using the attributes, but training on the residuals instead of the labels. Only for the first pass are the raw labels used for training targets. Subsequent passes take the predictions generated by training and subtract `eps` of them from the residuals before training. As mentioned, the subtraction of the residuals amount to a gradient descent and the reason for multiplying by the step size control parameter `eps` is to make sure that the iterative process converges. The code uses a fixed holdout set to measure out-of-sample performance and then plots the MSE as a function of the number of trees trained and also plots the function showing predicted values versus the single attribute.

Getting the Best Performance from Gradient Boosting

The first pair of plots (see Figures 6-18 and 6-19) shows the MSE versus number of trees and the plot of the prediction functions with eps = 0.1 and treeDepth = 1. Figure 6-18 shows that the error decreases smoothly and reaches roughly 0.014 after training 30 trees, and the MSE is heading down, indicating that it could be reduced further by training still more trees.

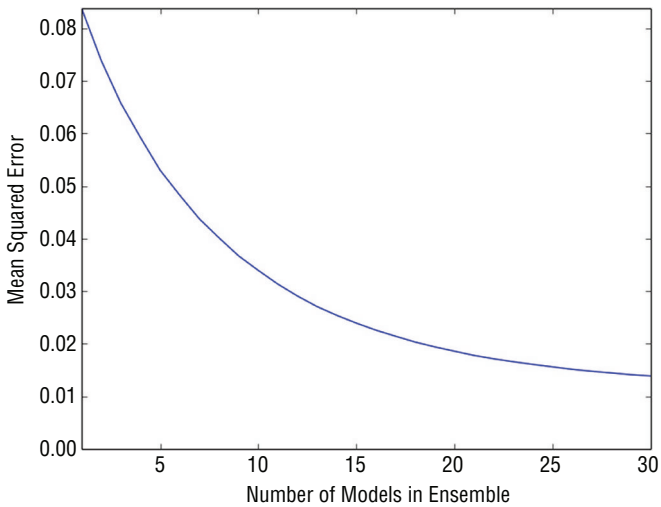


Figure 6-18: MSE versus number of trees for synthetic problem - eps = 0.1, treeDepth = 1

Figure 6-19 shows the prediction versus attribute value for three gradient boosting models—one that only trains one tree, one that trains 15 trees, and one that trains 30 trees. The model incorporating a single tree looks like a diminished version of the tree models that you saw in the introductory section about decision trees. As described, it is indeed a single depth 1 tree trained on the labels and then multiplied by 0.1—the value of `eps`. Things get more interesting with the

model built on 10 trees. That model makes a nice approximation to the correct answer—a straight line at 45 degrees on the graph. The model incorporating 10 trees correctly predicts roughly half of the range right, and predicts the right and left sides as constant. The model incorporating 30 trees gets a little further toward good approximation all the way to the edges of the data. This is distinct from the behavior that bagging showed with using stumps.

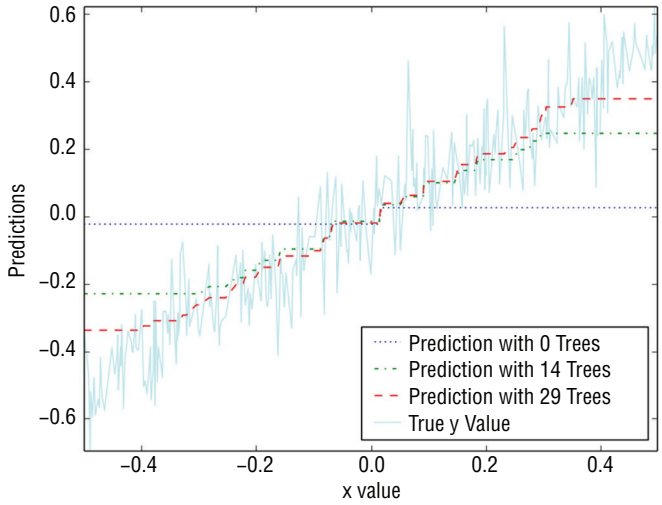


Figure 6-19: Gradient boosting predictions versus attribute value problem - eps = 0.1, treeDepth = 1

Bagging couldn't get beyond the bias error inherent in using shallow trees to build predictions for a number of problems not much different from one another. Gradient boosting starts in the same manner, but as it begins to reduce the errors in the middle of the data, it begins to pay more attention to the areas where it's making mistakes. That moves the split points out into the regions where there are mistakes. That process leads to a nice approximation without needing tree depth to get it.

What happens as the parameters controlling the training are changed? Figures 6-20 and 6-21 show how the picture changes if trees are of depth 5. The MSE plot in Figure 6-20 shows a similar smooth reduction in MSE as the number of trees increases. The MSE value gets very close to perfection (0.01) after training 30 depth 5 trees—lower than with depth 1 trees. What the plot doesn't show is training time. Each level in a tree takes about the same time to train. At each layer, all the possible split points have to be compared for MSE. A depth 5 tree takes five times as long as five depth 1 trees. A fair comparison would be to see what error the depth 1 trees reached after 150 trees compared to depth 5 trees after 30.

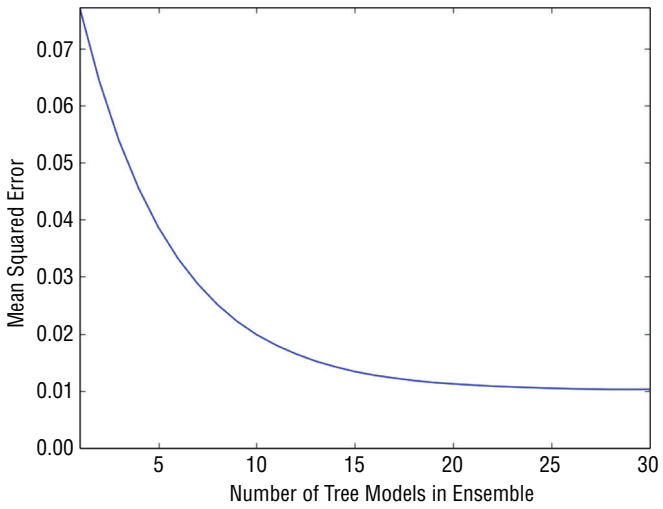


Figure 6-20: MSE versus number of trees for synthetic problem - eps = 0.1, treeDepth = 5

Figure 6-21 clearly reflects the deeper trees being used to build the gradient boosting ensemble. Even the first prediction based on a single tree shows some structure all across the range of the attribute. The models based on 15 trees and 30 trees still exhibit higher levels of error at the edges of the data.

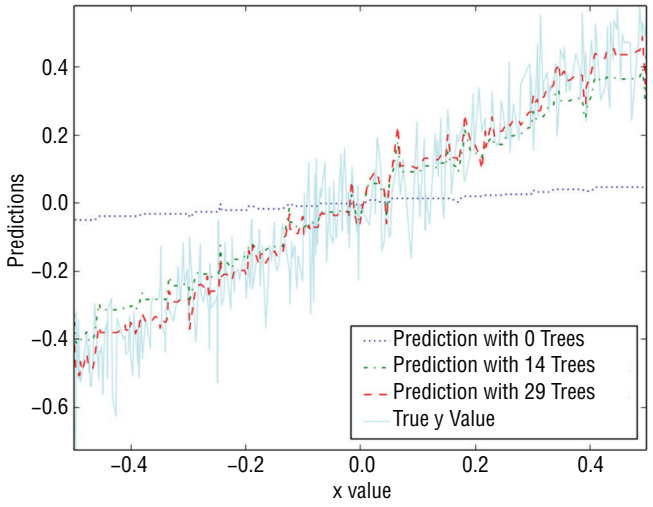


Figure 6-21: Gradient boosting predictions versus attribute value problem - eps = 0.1, treeDepth = 5

Figures 6-22 and 6-23 show what happens as the step size parameter `eps` is increased. Figure 6-22 shows behavior that's characteristic of too large a step size parameter (named `eps` here). The graph of MSE versus the number of trees decreases

sharply but then increases again toward the right side. The minimum is on the left side of the graph, near the one-third point. You want to adjust `eps` so that the minimum is at or near the right edge of the graph. That usually gives better performance.

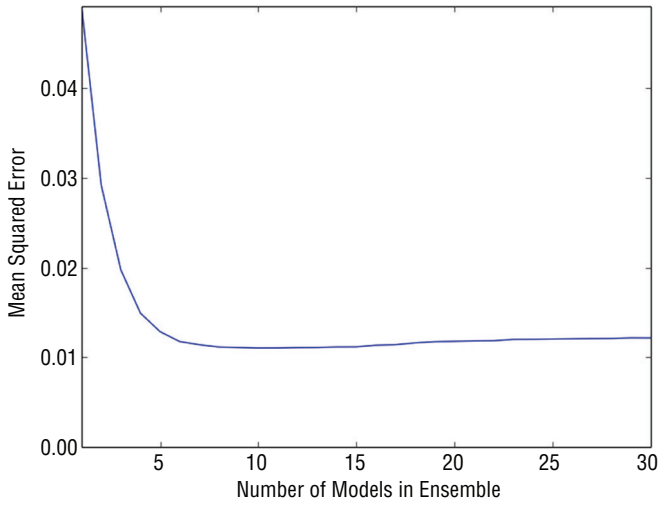


Figure 6-22: MSE versus number of trees for synthetic problem - eps = 0.3, treeDepth = 5

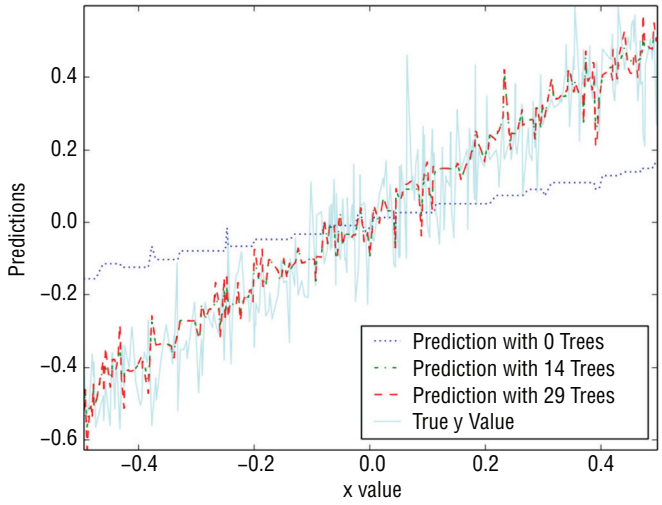


Figure 6-23: Gradient Boosting predictions versus attribute value problem - eps = 0.3, treeDepth = 5

The picture of the predictions as a function of the attribute shows more spiky diversions from the correct 45% line than either the versions using *eps=0.1*. Overall, the version with depth 1 trees is the best behaved. It looks like training more trees might improve the performance at the edges of the depth 1 model and lead to the best answer for gradient boosting.

Gradient Boosting on a Multivariable Problem

Listing 6-7 shows application of gradient boosting to the task of predicting wine quality. With the exception using the wine data set for input, the code in Listing 6-6 is very similar to the code used on the simple synthetic data set.

Listing 6-7: Gradient Boosting for Prediction Wine Quality—wineGBM.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
from sklearn import tree
from sklearn.tree import DecisionTreeRegressor
import random
from math import sqrt
import matplotlib.pyplot as plot

#read data into iterable
target_url = "http://archive.ics.uci.edu/ml/machine-learning-"
"databases/wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

nrows = len(xList)
ncols = len(xList[0])

#take fixed test set 30% of sample
nSample = int(nrows * 0.30)
idxTest = random.sample(range(nrows), nSample)
idxTest.sort()
idxTrain = [idx for idx in range(nrows) if not(idx in idxTest)]
```

```
#Define test and training attribute and label sets
xTrain = [xList[r] for r in idxTrain]
xTest = [xList[r] for r in idxTest]
yTrain = [labels[r] for r in idxTrain]
yTest = [labels[r] for r in idxTest]

#train a series of models on random subsets of the training data
#collect the models in a list and check error of composite as list grows

#maximum number of models to generate
numTreesMax = 30

#tree depth - typically at the high end
treeDepth = 5

#initialize a list to hold models
modelList = []
predList = []
eps = 0.1

#initialize residuals to be the labels y
residuals = list(yTrain)

for iTrees in range(numTreesMax):

    modelList.append(DecisionTreeRegressor(max_depth=treeDepth))
    modelList[-1].fit(xTrain, residuals)

    #make prediction with latest model and add to list of predictions
    latestInSamplePrediction = modelList[-1].predict(xTrain)

    #use new predictions to update residuals
    residuals = [residuals[i] - eps * latestInSamplePrediction[i] \
        for i in range(len(residuals))]

    latestOutSamplePrediction = modelList[-1].predict(xTest)
    predList.append(list(latestOutSamplePrediction))


#build cumulative prediction from first "n" models
mse = []
allPredictions = []
for iModels in range(len(modelList)):

    #add the first "iModels" of the predictions and multiply by eps
    prediction = []
    for iPred in range(len(xTest)):
        prediction.append(sum([predList[i][iPred]
            for i in range(iModels + 1)]) * eps)

    allPredictions.append(prediction)
```

continues

continued

```
    errors = [(yTest[i] - prediction[i]) for i in range(len(yTest))]
    mse.append(sum([e * e for e in errors]) / len(yTest))

nModels = [i + 1 for i in range(len(modelList))]

plot.plot(nModels,mse)
plot.axis('tight')
plot.xlabel('Number of Trees in Ensemble')
plot.ylabel('Mean Squared Error')
plot.ylim((0.0, max(mse)))
plot.show()

print('Minimum MSE')
print(min(mse))

#printed output
#Minimum MSE
#0.405031864814
```

The parameter selections shown in the code are for 30 depth 5 trees and *eps=0.1*. This parameter set yields MSE of roughly 0.4. That's about 10% worse than the performance bagging got on the same problem. Try adjusting the number of trees, `eps`, the step size parameter, and the tree depth to see whether you can get better results.

The curve of MSE versus number of trees looks fairly flat at the right edge (see Figure 6-24). It might still be possible to get some more performance by adding more trees to the ensemble. The other possible approaches to squeezing out a little more performance would be to tweak the tree depth or step size parameter.

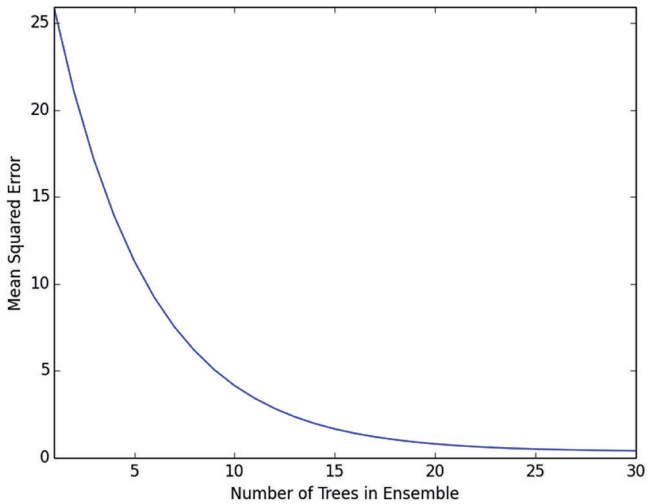


Figure 6-24: MSE versus number of trees for Gradient Boosting model of wine quality

Summary for Gradient Boosting

This section has shown how gradient boosting operates and demonstrated how to control its behavior to get the best performance. The section talked about the effect of changing step size, tree depth, and number of trees. You've seen how gradient boosting avoids the bias errors that bagging experienced with shallow trees. The basic difference in principle between bagging and boosting is that boosting constantly monitors its cumulative error and uses that residual for subsequent training. That difference accounts for gradient boosting only needing tree depth when there's significant interaction among the various attributes in the problem.

Random Forest

The random forests algorithm was developed by the late Berkeley professor Leo Breiman and Adele Cutler.[3] Random forests generates its sequence of models by training them on subsets of the data. The subsets are drawn at random from the full training set. One way in which the subset is selected is to randomly sample rows with replacement in the same manner as Brieman's bootstrap aggregation algorithm. The other random element is that the training sets for the individual trees in the random forests ensemble don't incorporate all the attributes but take a random subset of the attributes also. Listing 6-8 approximates random forests using Python DecisionTreeRegression.

Listing 6-8: Bagging with Random Attribute Selection—wineRF.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
from sklearn import tree
from sklearn.tree import DecisionTreeRegressor
import random
from math import sqrt
import matplotlib.pyplot as plot

#read data into iterable
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
"databases/wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
```

continues

continued

```
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

nrows = len(xList)
ncols = len(xList[0])

#take fixed test set 30% of sample
random.seed(1)  #set seed so results are the same each run
nSample = int(nrows * 0.30)
idxTest = random.sample(range(nrows), nSample)
idxTest.sort()
idxTrain = [idx for idx in range(nrows) if not(idx in idxTest)]

#Define test and training attribute and label sets
xTrain = [xList[r] for r in idxTrain]
xTest = [xList[r] for r in idxTest]
yTrain = [labels[r] for r in idxTrain]
yTest = [labels[r] for r in idxTest]

#train a series of models on random subsets of the training data
#collect the models in a list and check error of composite as list grows

#maximum number of models to generate
numTreesMax = 30

#tree depth - typically at the high end
treeDepth = 12

#pick how many attributes will be used in each model.
# authors recommend 1/3 for regression problem
nAttr = 4

#initialize a list to hold models
modelList = []
indexList = []
predList = []
nTrainRows = len(yTrain)

for iTrees in range(numTreesMax):

    modelList.append(DecisionTreeRegressor(max_depth=treeDepth))
```

```
    #take random sample of attributes
    idxAttr = random.sample(range(ncols), nAttr)
    idxAttr.sort()
    indexList.append(idxAttr)

    #take a random sample of training rows
    idxRows = []
    for i in range(int(0.5 * nTrainRows)):
        idxRows.append(random.choice(range(len(xTrain))))
    idxRows.sort()

    #build training set
    xRfTrain = []
    yRfTrain = []

    for i in range(len(idxRows)):
        temp = [xTrain[idxRows[i]][j] for j in idxAttr]
        xRfTrain.append(temp)
        yRfTrain.append(yTrain[idxRows[i]])

    modelList[-1].fit(xRfTrain, yRfTrain)

    #restrict xTest to attributes selected for training
    xRfTest = []
    for xx in xTest:
        temp = [xx[i] for i in idxAttr]
        xRfTest.append(temp)

    latestOutSamplePrediction = modelList[-1].predict(xRfTest)
    predList.append(list(latestOutSamplePrediction))


#build cumulative prediction from first "n" models
mse = []
allPredictions = []
for iModels in range(len(modelList)):

    #add the first "iModels" of the predictions and multiply by eps
    prediction = []
    for iPred in range(len(xTest)):
        prediction.append(sum([predList[i][iPred]
            for i in range(iModels + 1)]) / (iModels + 1))

    allPredictions.append(prediction)
    errors = [(yTest[i] - prediction[i]) for i in range(len(yTest))]
    mse.append(sum([e * e for e in errors]) / len(yTest))


nModels = [i + 1 for i in range(len(modelList))]

plot.plot(nModels,mse)
```

continues

continued

```
plot.axis('tight')
plot.xlabel('Number of Trees in Ensemble')
plot.ylabel('Mean Squared Error')
plot.ylim((0.0, max(mse)))
plot.show()

print('Minimum MSE')
print(min(mse))

#printed output

#Depth 1
#Minimum MSE
#0.52666715461

#Depth 5
#Minimum MSE
#0.426116327584

#Depth 12
#Minimum MSE
#0.38508387863
```

Random Forests: Bagging Plus Random Attribute Subsets

The example shown in Listing 6-5 trains on the wine quality data set. The simple single-attribute example that was used earlier to illustrate bagging and gradient boosting algorithms won't work with random forests. That example had only one attribute. It does not make sense to take a random draw of a single item. The code in Listing 6-5 looks a lot like the code for bagging. The only difference between the two that shows up before the loop on iTrees is the specification of a variable called `nAttr`. The random draw on the attributes needs to know how many attributes to select. The authors of the original paper recommend one third the number of attributes for a regression problem (and the square root of the number of attributes for a classification problem). Inside the iTrees loop, there's a random sample on rows of the attribute matrix—just like with bagging. There's also a random draw without replacement on the columns of the attribute matrix (or what would be rows and columns if the list of lists were converted to a numpy array). Then a tree gets trained and used to make a prediction on the out-of-sample data.

There is a difference between what's implemented in Listing 6-5 and the random forests algorithm. The algorithm in Listing 6-5 takes a random subset of the attributes and trains a tree with that subset. Breiman's original version of the random forests algorithm takes a different random set of attributes for each node in the tree. To implement Breiman's original version of the algorithm requires access to the innards of the tree growing algorithm. The example, nonetheless, gives a feel for how the algorithm operates. Some people argue that there's not much advantage to make the random draw on attributes at every node.

Random Forests Performance Drivers

Figures 6-25 through 6-27 show how the addition of random attribute selection affect the curves of MSE versus the number of trees included in the ensemble. Figure 6-25 shows the result when the individual trees are depth 1 trees. The picture is very similar to bagging in that the ensemble doesn't improve performance very much. Depth 1 trees mostly cause bias error not variance error. Bias can't be averaged away.

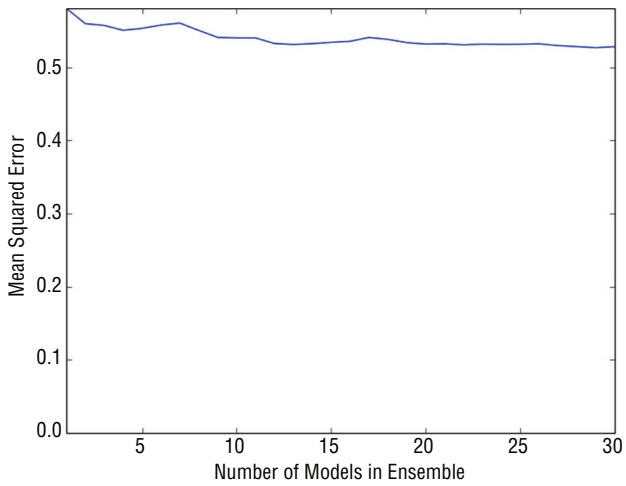


Figure 6-25: MSE versus number of trees for bagging + random attribute selection – Depth 1 trees

Figure 6-26 shows the MSE curve using depth 5 trees. Now the variance reduction with bagging plus random attribute selection begins to show some performance. The improvement with this combination gets similar performance to other methods demonstrated.

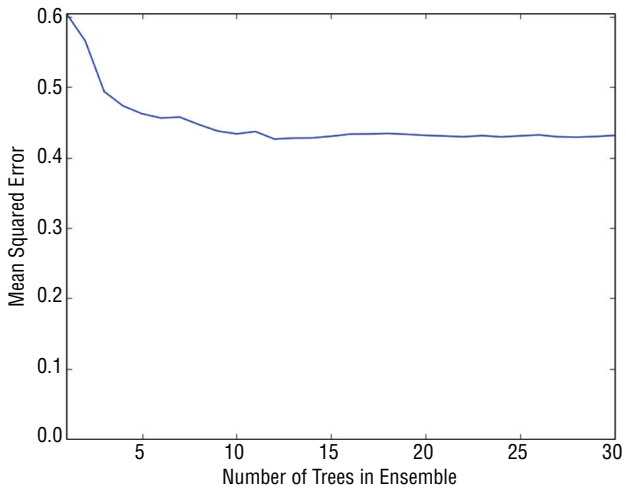


Figure 6-26: MSE versus number of trees for bagging + random attribute selection – Depth 5 trees

Figure 6-27 shows that a little more performance is available by using depth-12 trees.

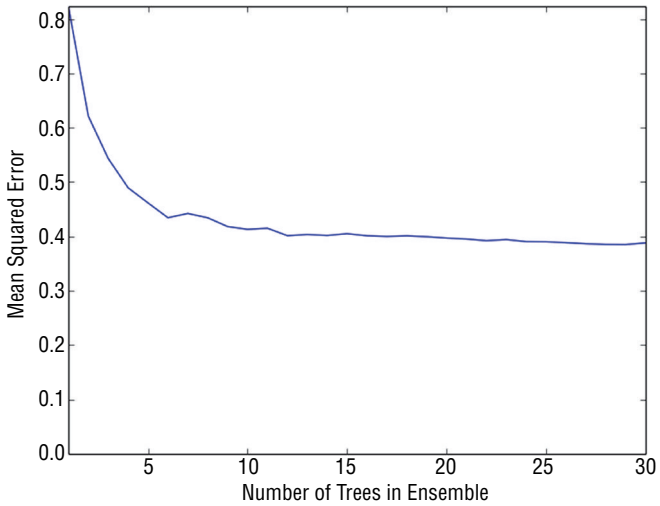


Figure 6-27: MSE versus number of trees for bagging + random attribute selection – Depth 12 trees

Random Forests Summary

Random forests is a combination of bagging and a random attribute selection modification to the binary tree base learners. These differences may not seem substantial, but they give random forests different performance characteristics from bagging and gradient boosting. Some results suggest that random forests has an advantage with wide sparse attribute spaces such as occur in text mining problems. Random forests is a little easier to parallelize than gradient boosting because the individual base learners can be trained independently of one another whereas with gradient boosting each base learner needs the results from the ones before it.

Differences like these mean that you may want to try both random forests in addition to gradient boosting, if you need to wring as much performance as possible from your data.

Summary

This chapter gave you some background on basic ensemble algorithms. It explained that ensemble methods consisted of a hierarchy of two algorithms. Ensemble methods train hundreds or thousands of the low-level algorithms called *base learners*. The higher-level algorithm controls the training of the base learners in order that their models turn out somewhat independent from one

another so that combining them will reduce the variance of the combination. For bagging, the higher-level algorithm is to take bootstrap samples of the training set and train base learners on these samples. For gradient boosting, the higher-level algorithm at each stage takes a sample of the input data and trains a base learner on it. With gradient boosting, the target used to train each base learner is the error from the accumulation of all the earlier base learners. Random forests is a combination of bagging as a higher-level algorithm and base learners that are modified versions of binary decision trees. The base learners with random forests are binary trees where, at each node, the split point decisions are restricted to a random sample of the available attributes instead of considering all the attributes in each split. The packages available for doing gradient boosting in Python permit you to use random forests base learners with gradient boosting. You will see that use in the next chapter, "Building Ensemble Methods with Python."

The chapter coded each of the high-level algorithms and showed a facsimile of the random forests base learners. The purpose for coding these is for you to gain an understanding of the mechanisms at work in each of the algorithms. The idea behind that is that you will better understand the options, input variables, nominal starting values, and so on for the Python packages for these algorithms. The next chapter uses available Python packages to generate solutions to some of the problems you've seen solved by penalized linear regression.

References

1. Panda Biswanath, Joshua S. Herbach, Sugato Basu, and Roberto J. Bayardo. (2009). PLANET: Massively Parallel Learning of Tree Ensembles with MapReduce. Proceedings of the 35th International Conference on Very Large Data Bases. Retrieved from `http://research.google.com/pubs/pub36296.html`.
2. Leo Breiman. (September, 1994). Bagging Predictors. Technical Report No. 421. Department of Statistics, UC Berkeley. Retrieved from `http://statistics.berkeley.edu/sites/default/files/tech-reports/421.pdf`.
3. Leo Breiman. (2001). Random forests. *Machine Learning*, *45*:5–32. Retrieved from `http://oz.berkeley.edu/~breiman/randomforest2001.pdf`.
4. J.H. Friedman. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, *29*(5):1189–1232. Retrieved from `http://statweb.stanford.edu/~jhf/ftp/trebst.pdf`.
5. J.H. Friedman. (2002). Stochastic Gradient Boosting. *Computational Statistics and Data Analysis*, *38*(4):367–378. Retrieved from `http://statweb.stanford.edu/~jhf/ftp/stobst.pdf`.

CHAPTER

7

Building Ensemble Models with Python

This chapter uses several available Python packages to build predictive models using the ensemble algorithms that you saw in Chapter 6, "Ensemble Methods." The problems used to illustrate them were introduced in Chapter 2, "Understand the Problem by Understanding the Data." You saw in Chapter 5, "Building Predictive Models Using Penalized Linear Methods," how to build predictive models for them using penalized linear regression. This chapter uses ensemble methods to solve the same problems. That will enable you to compare the algorithms and the available Python packages in terms of how easy the packages are to use, what kinds of accuracy is achievable with ensemble methods versus penalized linear regression, how the training times compare, and so on. The end of the chapter shows some summary comparisons of the various algorithms you've become familiar with.

Solving Regression Problems with Python Ensemble Packages

The next several sections demonstrate the application of available Python packages for building ensemble models. You will see the things you learned in Chapter 6 in action. The methods explained in Chapter 6 will be used on the series of problems explored in Chapter 2 and then used to demonstrate the application of penalized linear regression in Chapter 5. Using the same problems makes it possible to compare the algorithms covered here along several dimensions, including raw performance, training time, and ease of use. The chapter also covers the available Python packages. The background given in Chapter 6 helps you understand why the Python packages are structured

the way they are and helps you see how to get the most from these methods. This section goes through a variety of different problem types, beginning with regression problems.

Building a Random Forest Model to Predict Wine Taste

The wine quality data set provides an opportunity to predict wine taste scores based on the chemical composition of wine. As you know by now, this problem type is called a regression problem because the predictions take the form of real numbers. The Python scikit-learn ensemble module houses a Random Forest algorithm and a Gradient Boosting algorithm, both of which are for regression problems. First, this section explains the parameters required to instantiate a member of the `RandomForestRegressor` class. Then this section uses the `RandomForestRegressor` class to train a Random Forests model for the wine taste data and to explore the performance of the model.

Constructing a RandomForestRegressor Object[1]

Here is the class constructor for `sklearn.ensemble.RandomForestRegressor`:

```
sklearn.ensemble.RandomForestRegressor(n_estimators=10, criterion='mse',
max_depth=None, min_samples_split=2, min_samples_leaf=1, max_features=
'auto', max_leaf_nodes=None, bootstrap=True, oob_score=False, n_jobs=1,
random_state=None, verbose=0, min_density=None, compute_importances=
None)
```

The following description mirrors sklearn documentation, but covers only the parameter values that you're most likely to want to alter.[1] For those parameters, the list describes how to choose alternatives to the default values. To see descriptions of the parameters not covered here, see the sklearn package documentation. The following list describes the parameters:

- **n_estimators**

  integer, optional (default = 10)

  This is the number of trees in the ensemble. The default is okay to use if you coded things correctly, but you'll generally want more than 10 trees to gain the best performance. You can experiment with the number and get a feel for how many are required. As emphasized throughout this book, the appropriate model complexity (tree depth and number of trees) depends on the complexity of the underlying problem and the amount of data that you have. A good starting point is 100–500.

- **max_depth**

  integer or None, optional (default=None)

  If this parameter is set to `None`, the tree will be grown until all the leaf nodes are either pure or they hold fewer than `min_samples_split` examples. As an alternative to specifying the tree depth, you can use `max_leaf_nodes` to specify the number of leaf nodes in the tree. If you specify `max_leaf_nodes`, `max_depth` is ignored. There might be a performance advantage to leaving `max_depth` set to auto and growing full-depth trees. This is also a training time cost associated with full-depth trees. You may want to experiment with the depth if you need several training runs to complete your modeling process.

- **min_samples_split**

  integer, optional (default=2)

  Nodes will not be split that have fewer than `min_samples_split` examples. Splitting nodes that are small is a source of overfitting.

- **min_samples_leaf**

  integer, optional (default=1)

  A split is not taken if the split leads to nodes that have fewer than `min_samples_leaf`. The default value for this parameter results in the parameter being ignored, which is often okay—particularly when you're making the first few training runs on your data set. You can think about selecting a meaningful value for this parameter in a couple of ways. One is that the value assigned to a leaf is the average of the examples in the leaf and that you'll get a lower variance average if there's more than one sample in the leaf node. Another way to think about this parameter is as an alternative way to control tree depth.

- **max_features**

  integer, float or string, optional (default=None)

  The number of features to consider when looking for the best split depends on the value set for `max_features` and on the number of features in the problem. Call the number of features in the problem `nFeatures`. Then:

  - If the type of `max_features` is `int`, consider `max_features` features at each split. Note: `max_features gt; nFeatures` throws an error.
  - If the type of `max_features` is `float`, `max_features` is the fraction of features to consider: `int(max_features * nFeatures)`.
  - Possible string values include the following:

    ```
    auto  max_features=nFeatures
    sqrt  max_features=sqrt(nFeatures)
    log2   max_features=log2(nFeatures)
    ```

- If `max_features=None`, then `max_features=nFeatures`.

  Brieman and Cutler[2] recommend `sqrt(nFeatures)` for regression problems. The answers aren't generally terribly sensitive to `max_features`, but this parameter can have some effect, so you'll want to test a few alternative values.

- **random_state**

  int, RandomState instance, or None (default=None)

  - If the type is integer, the integer is used as the seed for the random number generator.
  - If the `random_state` is an instance of `RandomState`, that instance is used as the random number generator.
  - If `random_state` is `None`, the random number generator is the instance of `RandomState` used by `numpy.random`.

`RandomForestRegressor` has several attributes, including the trained trees that make up the ensemble. There's a predict method that will use the trained trees to make predictions, so you will not generally access those directly. You will want to access the variable `importances`. Here is a description:

- **feature_importances**

  This is an array whose length is equal to the number of features in the problem (called `nFeatures` earlier). The values in the array are positive floats indicating relative importance of the corresponding attribute. The `importance`s are determined by a procedure Brieman invented in the original paper on Random Forests.[2] The basic idea is that, one at a time, values of each attribute are randomly permuted, and the change in the model's prediction accuracy is determined. The more the prediction accuracy suffers, the more important the attribute.

Here are descriptions of the methods used:

- **fit(XTrain, yTrain, sample_weight=None)**

  `XTrain` is an array of attribute values. It has `nInstances` rows and `nFeature` columns. `yTrain` is an array of targets. `y` also has `nInstances` rows. In the examples you'll see in this chapter, `yTrain` will have a single column, but the method can fit several models having different targets. For that, `y` would have `nTargets` columns—one column for each set of outcomes. `sample_weight` makes it possible to assign different weights to each of the instances in the training data. It can take one of two forms. The default value of `None` results in equal weighting of all input instances. To apply different weights to each instance, `sample_weight` should be an array with `nInstances` rows and one column.

- **predict(XTest)**

  `XTest` is an array of attribute values for which predictions are produced. The array input to `predict()` has the same number of columns as the array used in `fit()` method for training, but can have a different number of rows, including perhaps a single row. The rows in the output from `predict()` have the same form as rows in the target array `y` used in training.

Modeling Wine Taste with RandomForestRegressor

Listing 7-1 shows how to use the sklearn version of the Random Forest algorithm to build an ensemble model to predict wine taste.

The code reads the wine data set from UCI data repository; does some manipulation to get the attributes, labels, and attribute names into lists; and converts the lists to numpy arrays as required for input to `RandomForestRegressor`. A side benefit of having these input objects in the form of numpy arrays is that it enables the use of a sklearn utility `train_test_split` for building training and test versions of the inputs. The code sets `random_state` to a specified integer value instead of letting the random number generator pick an unrepeatable internal value. That's so that you'll get the same graphs and numeric values when you run the code as the results shown here. Setting `random_state` can also prove handy during development because randomness in the results can mask changes you're making. During real model training, you'll probably want to set `random_state` to its default value, `None`. Fixing `random_state` fixes the holdout set and, as a result, repeated parameter adjustments and retraining may start to overtrain on your holdout set.

The next step in the code is to define a list of ensemble sizes to produce performance graphs that show how the performance varies as the number of trees in the ensemble is changed. For producing detailed plots, the number chosen in Listing 7-2 results in roughly 45 separate runs. That many are useful here so that you can see the shape of the curve of error versus number of trees, but now that you've got that mental picture, you won't want to run so many points in the curve. You might run two or three different numbers of trees early in the development process and then settle on a good number and only run for a single value most of the time.

Most of the parameters affecting training are set as part of the constructor that instantiates a `RandomForestRegressor` object. The call to the constructor is pretty simple in this case. The only parameter that is not left at default values is the `max_features` parameter. The default value (`None`) results in all the features being considered at each node of the tree, which means that it's actually implementing Bagging[3] because no random selection of attributes is involved.[4, 5, 6]

After instantiating a `RandomForestRegressor` object, the next step is to invoke the `fit()` method the training sets as arguments. Once that is done, invoking

the `predict()` method with the attributes from the test set generates predictions that can be compared to the test set labels. The code in the listing uses the sklearn.metrics function `mean_squared_error` to calculate the prediction error. The resulting mean squared error numbers are collected in a list and then plotted. Figure 7-1 shows the resulting plot.

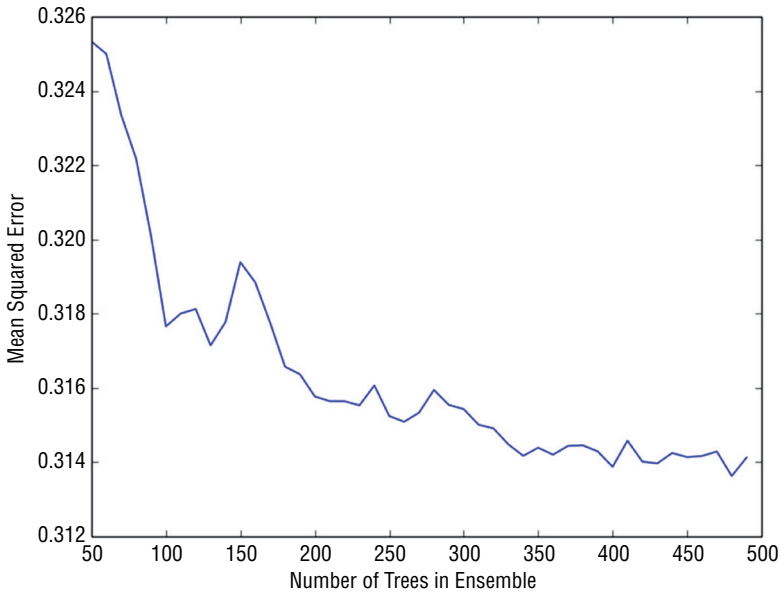


Figure 7-1: Wine taste prediction performance with Random Forest: errors versus ensemble size

The last value of mean squared error is also printed and copied at the bottom of Listing 7-1. Notice that the last value is printed as representative of the mean squared error, not the minimum value. Random Forest generates somewhat independent predictions and then averages them. Adding more trees to the average cannot lead to overfitting, so the minimum point in the curve of Figure 7-1 represents deviation due to statistical fluctuation, not a reproducible minimum.

Listing 7-1: Using RandomForestRegressor to Build a Regression Model—wineRF.py

```
import urllib2
import numpy
from sklearn.cross_validation import train_test_split
from sklearn import ensemble
from sklearn.metrics import mean_squared_error
import pylab as plot


# Read wine quality data from UCI website
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
databases/wine-quality/winequality-red.csv")
```

```
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

nrows = len(xList)
ncols = len(xList[0])

X = numpy.array(xList)
y = numpy.array(labels)
wineNames = numpy.array(names)

#take fixed holdout set 30% of data rows
xTrain, xTest, yTrain, yTest = train_test_split(X, y, test_size=0.30,
  random_state=531)

#train Random Forest at a range of ensemble sizes in order to
#see how the mse changes
mseOos = []
nTreeList = range(50, 500, 10)
for iTrees in nTreeList:
    depth = None
    maxFeat  = 4 #try tweaking
    wineRFModel = ensemble.RandomForestRegressor(n_estimators=iTrees,
        max_depth=depth, max_features=maxFeat,
        oob_score=False, random_state=531)

    wineRFModel.fit(xTrain,yTrain)

    #Accumulate mse on test set
    prediction = wineRFModel.predict(xTest)
    mseOos.append(mean_squared_error(yTest, prediction))


print("MSE" )
```

continues

continued

```
print(mseOos[-1])


#plot training and test errors vs number of trees in ensemble
plot.plot(nTreeList, mseOos)
plot.xlabel('Number of Trees in Ensemble')
plot.ylabel('Mean Squared Error')
#plot.ylim([0.0, 1.1*max(mseOob)])
plot.show()

# Plot feature importance
featureImportance = wineRFModel.feature_importances_

#scale by max importance
featureImportance = featureImportance / featureImportance.max()
sorted_idx = numpy.argsort(featureImportance)
barPos = numpy.arange(sorted_idx.shape[0]) + .5
plot.barh(barPos, featureImportance[sorted_idx], align='center')
plot.yticks(barPos, wineNames[sorted_idx])
plot.xlabel('Variable Importance')
plot.show()

#printed output
#MSE
#0.314125711509
```

Visualizing the Performance of a Random Forests Regression Model

The curve in Figure 7-1 demonstrates the variance reduction properties of the Random Forest algorithm. The level of the error decreases as more trees are added, and the amount of statistical fluctuation in the curve also decreases.

NOTE **To get a feel for the behavior of the algorithm, try changing some of the parameters used in Listing 7-1 and see how the plots change. Try running more trees to see whether you can reduce the error further. Try something like** `nTreeList=range(100,1000,100)`**. Try altering the tree depth parameter to see how sensitive the answers are to tree depth. The wine quality data set has roughly 1,600 instances (rows), so a depth of 10 or 11 could result in almost every point having its own leaf node. A depth of 8 could ideally have 256 leaf nodes, so each one would have an average of about 6 instances. Try some depths in that range to determine whether it affects performance.**

Random Forest generates estimates of how important each variable is to the accuracy of predictions. Listing 7-1 extracts the data member `feature_importance_`, rescales importance values to between `0` and `1`, orders the resulting importance values, and then plots them in a bar chart. Figure 7-2 shows that plot. The most important variable has scaled importance of `1.0` and is the top bar in

the bar chart. It shouldn't be too surprising that alcohol is the most important variable in the Random Forest model. It was also the most important in the penalized linear regression models that you saw in Chapter 5.

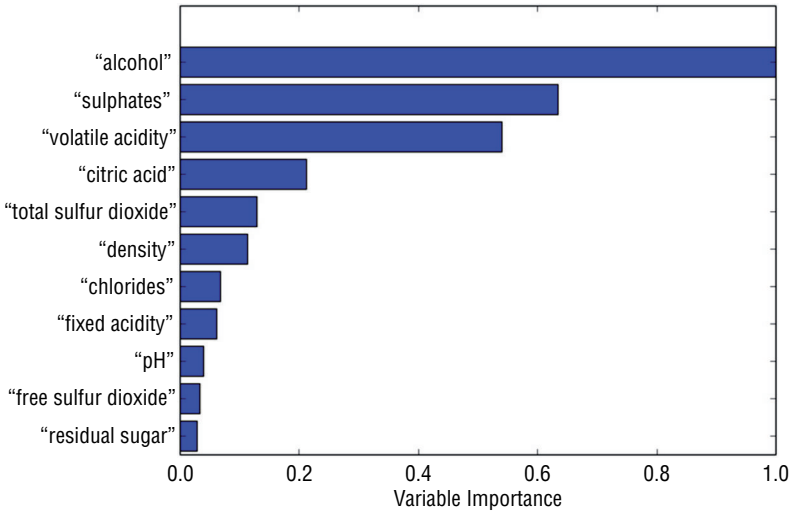


Figure 7-2: Relative importance of variables for Random Forest predicting wine taste

Using Gradient Boosting to Predict Wine Taste

As you saw in Chapter 6, Gradient Boosting[7,8] takes an error-minimization approach to building an ensemble of trees, instead of the variance-reduction approach that Bagging and Random Forest take. Because Gradient Boosting incorporates binary trees as its base learners, it shares some tree-related parameters. However, because Gradient Boosting takes steps directed by the gradient, you'll also see parameters such as step size. In addition, Gradient Boosting's error-minimization approach will lead to different rationale and choices for setting tree depth. There's also a surprise variable that allows you to build models that are a hybrid between Random Forest and Gradient Boosting. You can use Gradient Boosting error-minimization structure while employing the Random Forest random attribute selection for base learners. The sklearn ensemble module is the only place that I've seen that combination available.

Using the Class Constructor for GradientBoostingRegressor[9]

Here is the class constructor for `sklearn.ensemble.GradientBoostingRegressor`:

```
class sklearn.ensemble.GradientBoostingRegressor(loss='ls', learning_
rate=0.1, n_estimators=100, subsample=1.0, min_samples_split=2, min_
samples_leaf=1, max_depth=3, init=None, random_state=None, max_features=
None, alpha=0.9, verbose=0, max_leaf_nodes=None, warm_start=False)
```

The following lists describe the parameters and methods that you'll want to be familiar with and give some comment on the choices and tradeoffs for them where appropriate. This list describes the parameters:

- **loss**

  string, optional (default='ls')

  Gradient Boosting uses trees to approximate the gradient of an overall loss function. The most commonly used overall loss is sum squared error, as is the penalty from ordinary least squares regression. Least sum squared error is a handy choice because the squared error makes the math work out neatly. But other loss functions may better describe your real problem. As an example, I worked on algorithms for automated trading and noticed that using squared error penalty led to algorithms that would avoid large losses but that would accept small losses that in aggregate were more significant. Sum of absolute value of the errors gave much better overall performance; it better matched the real problem. Least mean absolute value is generally less sensitive to outliers. Gradient Boosting is one of the few algorithms that gives you wide flexibility in your choice of penalty functions.

  Possible string values include the following:

  - `ls` Least mean squared error.
  - `lad` Least mean absolute value of error.
  - `huber` Huberized loss is a hybrid between squared error for small values and absolute value of error for large values.[10,11]
  - `quantile` Quantile regression. Predicts quantile (indicated by alpha parameter).

- **learning_rate**

  float, optional (default=0.1)

  As mentioned, Gradient Boosting is based on a gradient descent algorithm. The learning rate is the size of the step taken in the gradient direction. If it is too large, you'll see a rapid decline in the error and then a rapid rise in the error (as a function of the number of trees in the ensemble.) If it is too small, the errors will decrease very slowly, and it will require training more trees than necessary. The best value for `learning_rate` is problem dependent and also depends on the tree depth chosen. The default value of `0.1` is a relatively large value, but a good choice for a starting point. Try it. See whether it leads to instability and overfitting. Adjust if necessary.

- **n_estimators**

  int, optional (default=100)

  This parameter is the number of trees in the ensemble. As you saw in Chapter 6, you can also think of this as the number of steps taken toward

the minimum in a gradient descent sequence. It is also the number of terms in an additive approximation (that is, the sum of the trained models). Because each successive approximation (each successive tree) gets multiplied by the learning rate, a larger learning rate requires fewer trees to be trained to make the same progress toward the minimum. However (as discussed in the section on learning rate), if the learning rate is too high, it can lead to overfitting and may achieve the best performance. It usually takes a few tries to learn what parameter ranges work best on a new problem. The default value of `100` is a good starting point (particularly in conjunction with the default value for the learning rate).

- **subsample**

  float, optional (default=1.0)

  Gradient Boosting becomes stochastic Gradient Boosting when the individual trees are trained on a subsample of the data, similar to Random Forest. Friedman (algorithm inventor) recommends using `subsample=0.5`.[12] That's a good starting point.

- **max_depth**

  integer, optional (default=3)

  As with Random Forests, `max_depth` is the depth of the individual trees in the ensemble. As you saw in the simple example in Chapter 6, Random Forests needs some tree depth to generate a high-fidelity model, whereas Gradient Boosting, by continually focusing on the residual error, was able to get a high-fidelity approximation with trees of depth `1` (called *stumps*). Gradient Boosting's need for deep trees is driven by the degree of interaction between attributes. If they act independently, a depth of 1 will get as good a model as depth 2. Generally, you want to start with a tree depth equal to `1` to get the other parameters set and then try a tree depth of `2` to see whether it gives you an improvement. I've never encountered a problem that needed depth `10`.

- **max_features**

  int, float, string, or None, optional (default = None)

  The number of features to consider when looking for the best split depends on the value set for `max_features` and on the number of features in the problem. Call the number of features in the problem `nFeatures`. Then:

  - If the type of `max_features` is `int`, consider `max_features` at each split.
  - If the type of `max_features` is `float`, then `max_features` is the fraction of features to consider: `int(max_features * nFeatures)`.
  - Possible string values include the following:

    `auto  max_features=nFeatures`

```
sqrt   max_features=sqrt(nFeatures)
log2   max_features=log2(nFeatures)
```

- If `max_features=None`, then `max_features=nFeatures`.

  `max_features` in the Python implementation of Gradient Boosting plays the same role as in Random Forest. It determines how many attributes will be considered for splitting at each node in the trees. That gives the Python implementation of Gradient Boosting a unique capability. It can incorporate Random Forest base learners in the place of trees grown on the full attribute space.

- **warm_start**

  bool, optional(default=False)

  If `warm_start` is set to `True`, subsequent applications of the `fit()` function start from last stopping point in training and continue to accumulate the results of adding further gradient steps.

Here are descriptions of the attributes used:

- **feature_importances**

  An array whose length is equal to the number of features in the problem (called `nFeatures` earlier). The values in the array are positive floats indicating relative importance of the corresponding attribute. A large number corresponds to large influence.

- **train_score**

  An array whose length is equal to the number of trees in the ensemble. This contains the error on the training set at each stage in training the sequence of trees.

Here are descriptions of the methods used:

- **fit(XTrain, yTrain, monitor=None)**

  `XTrain` and `yTrain` have the same form as for Random Forest. `XTrain` is an (`nInstances` x `nAttributes`) numpy array, where `nInstances` is the number of rows in the training data set and `nAttributes` is the number of attributes. `yTrain` is an (`nInstances` x `1`) numpy array of targets. The object "monitor" is a callable that can be used to stop training early.

- **predict(X)**

  `predict(X)` generates a prediction from an array of attributes `X`. `X` needs to have the same number of columns (attributes) as the training set. `X` can have any number of rows.

- **staged_predict(X)**

  This function acts like the `predict()` function except that it's iterable and generates a sequence of predictions corresponding to the sequence of

model produced by the Gradient Boosting algorithm. Each call generates a prediction incorporating one additional tree in the sequence generated by Gradient Boosting.

Getting the parameters set for Gradient Boosting can be a little bewildering for a new user. The following list suggests a sequence of parameter settings and adjustments for Gradient Boosting:

1. Start with default settings, except set `subsample=0.5`. Train a model and look at the curve of out-of-sample (oos) performance versus the number of trees in the ensemble. After the first and subsequent runs, look at the shape of the oos performance curve.
2. If the oos performance is improving rapidly at the right end of the graph either increase `n_estimators` or increase `learning_rate`.
3. If the oos performance is deteriorating rapidly at the right end of the graph, decrease `learning_rate`.
4. Once the oos performance curve improves over its whole length (or only deteriorates very slightly) and levels out at the right side of the graph, try altering `max_depth` and `max_features`.

Using GradientBoostingRegressor to Implement a Regression Model

Listing 7-2 shows what's required to build a Gradient Boosting model for the wine quality data set.

Listing 7-2: Using Gradient Boosting to Build a Regression Model—wineGBM.py

```
import urllib2
import numpy
from sklearn.cross_validation import train_test_split
from sklearn import ensemble
from sklearn.metrics import mean_squared_error
import pylab as plot

# Read wine quality data from UCI website
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-databases"
"/wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
```

continues

continued

```
        firstLine = False
    else:
        #split on semi-colon
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

nrows = len(xList)
ncols = len(xList[0])

X = numpy.array(xList)
y = numpy.array(labels)
wineNames = numpy.array(names)

#take fixed holdout set 30% of data rows
xTrain, xTest, yTrain, yTest = train_test_split(X, y, test_size=0.30,
    random_state=531)

# Train Gradient Boosting model to minimize mean squared error
nEst = 2000
depth = 7
learnRate = 0.01
subSamp = 0.5
wineGBMModel = ensemble.GradientBoostingRegressor(n_estimators=nEst,
                                max_depth=depth,
                                learning_rate=learnRate,
                                subsample = subSamp,
                                loss='ls')

wineGBMModel.fit(xTrain, yTrain)

# compute mse on test set
msError = []
predictions = wineGBMModel.staged_predict(xTest)
for p in predictions:
    msError.append(mean_squared_error(yTest, p))

print("MSE" )
print(min(msError))
print(msError.index(min(msError)))

#plot training and test errors vs number of trees in ensemble
plot.figure()
plot.plot(range(1, nEst + 1), wineGBMModel.train_score_,
    label='Training Set MSE')
```

```
plot.plot(range(1, nEst + 1), msError, label='Test Set MSE')
plot.legend(loc='upper right')
plot.xlabel('Number of Trees in Ensemble')
plot.ylabel('Mean Squared Error')
plot.show()

# Plot feature importance
featureImportance = wineGBMModel.feature_importances_

# normalize by max importance
featureImportance = featureImportance / featureImportance.max()
idxSorted = numpy.argsort(featureImportance)
barPos = numpy.arange(idxSorted.shape[0]) + .5
plot.barh(barPos, featureImportance[idxSorted], align='center')
plot.yticks(barPos, wineNames[idxSorted])
plot.xlabel('Variable Importance')
plot.subplots_adjust(left=0.2, right=0.9, top=0.9, bottom=0.1)
plot.show()

# Printed Output:
# for:
#nEst = 2000
#depth = 7
#learnRate = 0.01
#subSamp = 0.5
#
# MSE
# 0.313361215728
# 840
```

The first section of code follows the same process as for Random Forest: read the data set, separate the attribute matrix from the targets, convert to numpy arrays, and then form train and test subsets. The training sequence is a little simpler for Gradient Boosting. The code for Random Forest used a loop to generate several models for different values of `n_estimator` to see how the oos error behaved as a function of the number of trees in the ensemble. The Python Gradient Boosting implementation has an iterable (`staged_predict` for regression problems and `staged_decision_function` for classification problems) that simplifies that process. Using these functions, you can train a model incorporating `n_estimator` trees and then generate the oos performance curve for models of all sizes (not greater than `n_estimator`).

Assessing the Performance of a Gradient Boosting Model

Figure 7-3 and the printed output shown in Listing 7-2 show that Gradient Boosting gets about the same level of performance as Random Forest. This is usually the case. There are problems where one or the other will achieve

significantly better performance, so you might want to try them both to make sure. The plot in Figure 7-3 shows an oos error that increases very slightly on the right side of the plot. To see that it is increasing requires looking at the numbers. The increase is not enough to warrant reducing `learning_rate` and retraining.

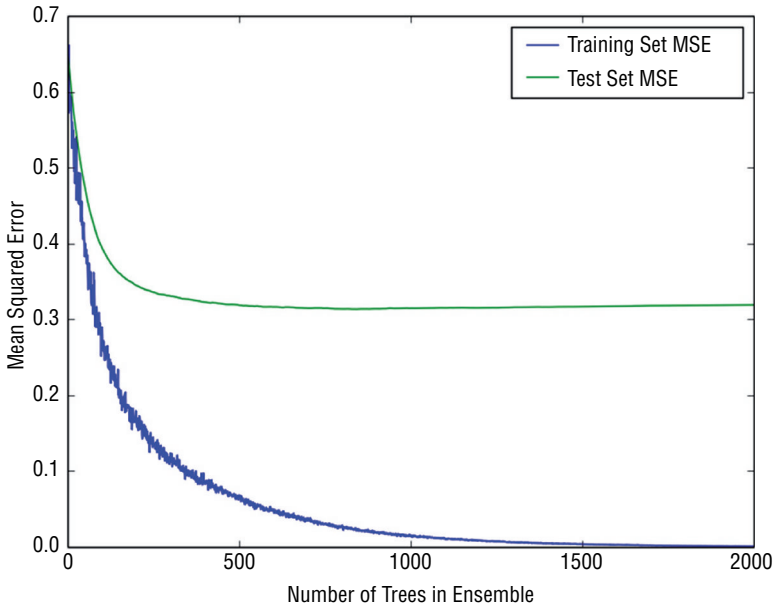


Figure 7-3: Wine taste prediction performance with Gradient Boosting: errors versus ensemble size

Figure 7-4 shows the variable importance determined as part of the Gradient Boosting implementation. Comparing with the variable importance generated by Random Forest reveals that the two are fairly similar but not identical. They agree that the most important variable is alcohol and have several of the same variables in the top four or five.

Coding Bagging to Predict Wine Taste

Listing 7-3 shows the code for generating a bootstrap sample from the wine data, training trees on it and then averaging the resulting models. This is called *Bagging*. It's purely a variance reduction technique, and it's useful to compare the performance that bagging achieves with the performance of Random Forest and Gradient Boosting.

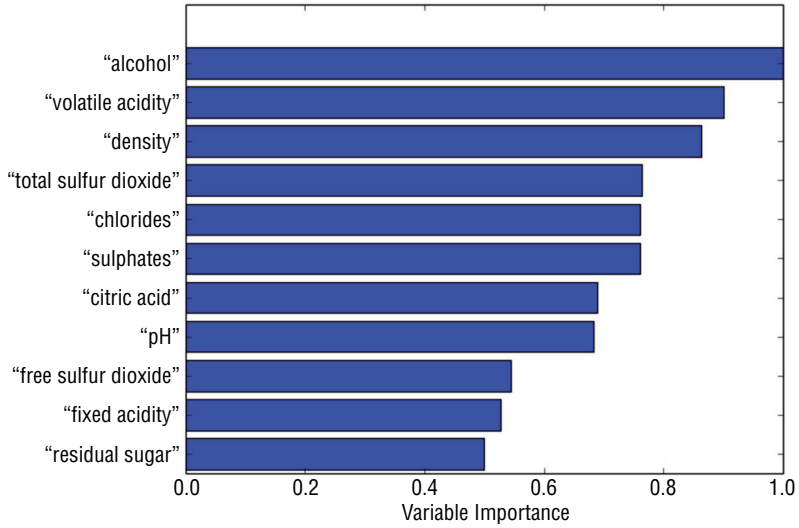


Figure 7-4: Relative importance of variables for Gradient Boosting predicting wine taste

Listing 7-3: Building a Regression Model for Wine Taste Using Bagging (Bootstrap Aggregation)—wineBagging.py

```
__author__ = 'mike-bowles'

import urllib2
import numpy
import matplotlib.pyplot as plot
from sklearn import tree
from sklearn.tree import DecisionTreeRegressor
from math import floor
import random


# Read wine quality data from UCI website
target_url = ("http://archive.ics.uci.edu/ml/machine-learning-databases"
"/wine-quality/winequality-red.csv")
data = urllib2.urlopen(target_url)

xList = []
labels = []
names = []
firstLine = True
for line in data:
    if firstLine:
        names = line.strip().split(";")
        firstLine = False
    else:
        #split on semi-colon
```

continues

continued

```
        row = line.strip().split(";")
        #put labels in separate array
        labels.append(float(row[-1]))
        #remove label from row
        row.pop()
        #convert row to floats
        floatRow = [float(num) for num in row]
        xList.append(floatRow)

nrows = len(xList)
ncols = len(xList[0])


#take fixed test set 30% of sample
nSample = int(nrows * 0.30)
idxTest = random.sample(range(nrows), nSample)
idxTest.sort()
idxTrain = [idx for idx in range(nrows) if not(idx in idxTest)]

#Define test and training attribute and label sets
xTrain = [xList[r] for r in idxTrain]
xTest = [xList[r] for r in idxTest]
yTrain = [labels[r] for r in idxTrain]
yTest = [labels[r] for r in idxTest]

#train a series of models on random subsets of the training data
#collect the models in a list and check error of composite as list grows

#maximum number of models to generate
numTreesMax = 100

#tree depth - typically at the high end
treeDepth = 5

#initialize a list to hold models
modelList = []
predList = []

#number of samples to draw for stochastic bagging
bagFract = 0.5
nBagSamples = int(len(xTrain) * bagFract)

for iTrees in range(numTreesMax):
    idxBag = []
    for i in range(nBagSamples):
        idxBag.append(random.choice(range(len(xTrain))))
    xTrainBag = [xTrain[i] for i in idxBag]
    yTrainBag = [yTrain[i] for i in idxBag]

    modelList.append(DecisionTreeRegressor(max_depth=treeDepth))
```

```
    modelList[-1].fit(xTrainBag, yTrainBag)

    #make prediction with latest model and add to list of predictions
    latestPrediction = modelList[-1].predict(xTest)
    predList.append(list(latestPrediction))


#build cumulative prediction from first "n" models
mse = []
allPredictions = []
for iModels in range(len(modelList)):

    #average first "iModels" of the predictions
    prediction = []
    for iPred in range(len(xTest)):
        prediction.append(sum([predList[i][iPred] for i in
            range(iModels + 1)])/(iModels + 1))

    allPredictions.append(prediction)
    errors = [(yTest[i] - prediction[i]) for i in range(len(yTest))]
    mse.append(sum([e * e for e in errors]) / len(yTest))

nModels = [i + 1 for i in range(len(modelList))]

plot.plot(nModels,mse)
plot.axis('tight')
plot.xlabel('Number of Models in Ensemble')
plot.ylabel('Mean Squared Error')
plot.ylim((0.0, max(mse)))
plot.show()

print('Minimum MSE')
print(min(mse))


#With treeDepth = 5
#     bagFract = 0.5
#Minimum MSE
#0.429310223079

#With treeDepth = 8
#     bagFract = 0.5
#Minimum MSE
#0.395838627928

#With treeDepth = 10
#     bagFract = 1.0
#Minimum MSE
#0.313120547589
```

Listing 7-3 includes three parameters that can be tweaked. The first is `numTreesMax`, which determines the number of trees that will be built; the second is `treeDepth`; the third is `bagFract`. As discussed in Chapter 6, Bagging operates by taking a bootstrap sample from the input data. The sample is taken with replacement so some of the data may be repeated. The variable `bagFract` determines how many samples are taken. The original paper on the algorithm recommended that the bootstrap samples be the same size as the original data set, which would correspond to `bagFract = 1.0`. The program in the listing generates `numTreesMax` models from the trees it builds. The first model is the first tree. The second model is the average of the first two trees. The third model is the average of the first three trees, etc. Then the program plots curves of the error versus the number of trees in the model.

Figures 7-5 and 7-6 plot the results for two different parameters settings. The ensemble for Bagging applied to the wine taste prediction data set. The printed results are shown at the bottom of the code listing. Figure 7-5 shows performance versus number of trees where the trees are depth 10 and are trained on bootstrap sample sets as large as the original data set (`bag fraction = 1.0`). With these parameters, Bagging achieves the same level of performance as Random Forest and Gradient Boosting.

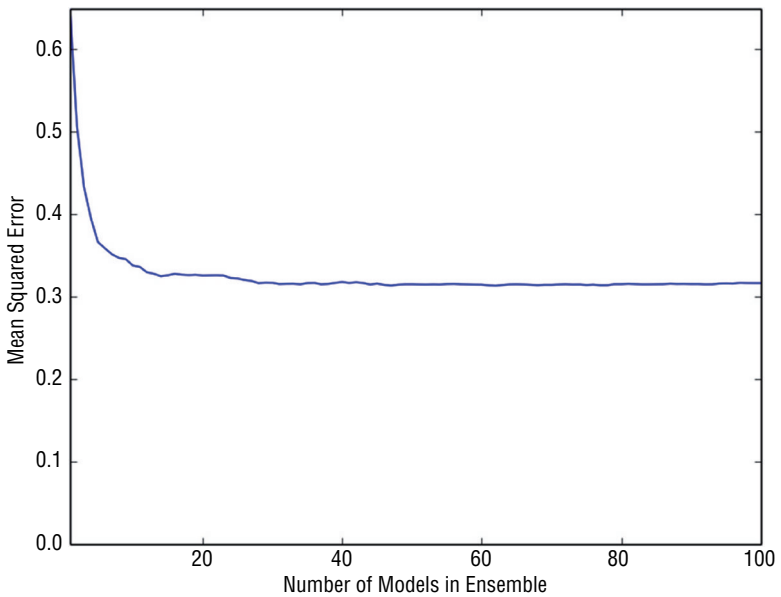


Figure 7-5: Wine taste error for Bagged regression trees (tree depth = 10, bag fraction = 1.0)

Figure 7-6 shows the performance for Bagging using trees of depth 8 and bootstrap data sets half as large as the original data (`bag fraction = 0.5`). As the plot and the printed results indicate, the performance is noticeably worse with this parameter selection.

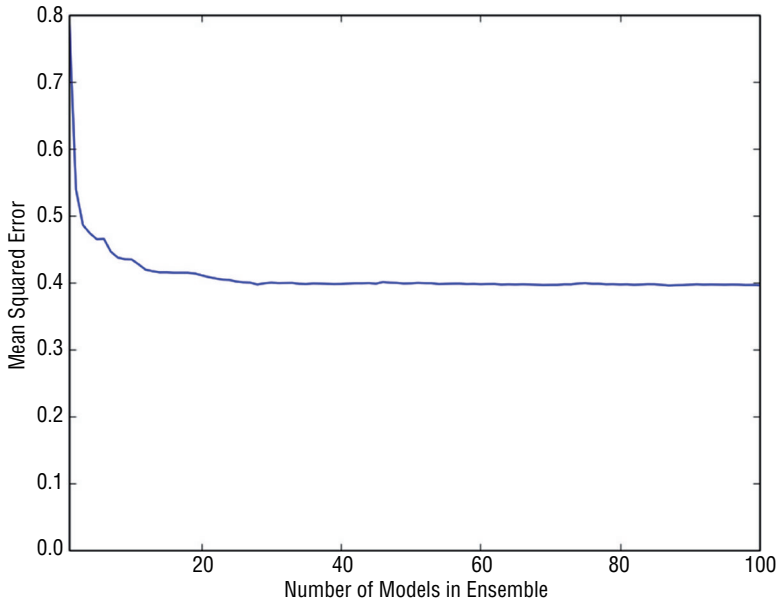


Figure 7-6: Wine taste error for Bagged regression trees (tree depth = 8, bag fraction = 0.5)

Incorporating Non-Numeric Attributes in Python Ensemble Models

Non-numeric attributes are ones that take several discrete non-numeric values. A census record has myriad non-numeric attributes—married, single, divorced, for example; state in which the household is located is another. Non-numeric attributes can improve prediction accuracy, but Python ensemble methods need numeric input. In Chapters 4, "Penalized Linear Regression," and 5, "Building Predictive Models Using Penalized Linear Methods," you saw how to code factor variables so that they could be incorporated in penalized linear regression. The same technique will work here. The problem of estimating the age of abalone will serve as an example to illustrate the technique.

Coding the Sex of Abalone for Input to Random Forest Regression in Python

Suppose that your problem has an attribute that takes n values. The attribute "States in the US" takes 50 values, and "Marital Status" takes 3. To code the n-valued factor variable, you create $n - 1$ new dummy attributes. If the variable takes its ith value, the ith dummy variable is 1 and all other dummies are 0. If the factor variable takes its nth value, all the dummy variables are 0. The abalone data will illustrate.

Listing 7-4 shows the steps training a Random Forest model to predict abalone age from data on the abalone's weight, shell size, and so forth. The objective in this problem is to predict the age of the abalone from various physical measurements (weights of various parts of the abalone, dimensions, and so on). That makes this a regression problem amenable to the algorithms used for building models for predicting taste scores for wines in the previous two sections.

Listing 7-4: Predicting Abalone Age with Random Forest—abaloneRF.py

```
__author__ = 'mike_bowles'

import urllib2
from pylab import *
import matplotlib.pyplot as plot
import numpy
from sklearn.cross_validation import train_test_split
from sklearn import ensemble
from sklearn.metrics import mean_squared_error


target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
"databases/abalone/abalone.data")
#read abalone data
data = urllib2.urlopen(target_url)

xList = []
labels = []
for line in data:
    #split on semi-colon
    row = line.strip().split(",")

    #put labels in separate array and remove label from row
    labels.append(float(row.pop()))

    #form list of list of attributes (all strings)
    xList.append(row)

#code three-valued sex attribute as numeric
xCoded = []
for row in xList:
    #first code the three-valued sex variable
    codedSex = [0.0, 0.0]
    if row[0] == 'M': codedSex[0] = 1.0
    if row[0] == 'F': codedSex[1] = 1.0

    numRow = [float(row[i]) for i in range(1,len(row))]
    rowCoded = list(codedSex) + numRow
    xCoded.append(rowCoded)
```

```
#list of names for
abaloneNames = numpy.array(['Sex1', 'Sex2', 'Length', 'Diameter',
    'Height', 'Whole weight', 'Shucked weight', 'Viscera weight',
    'Shell weight', 'Rings'])

#number of rows and columns in x matrix
nrows = len(xCoded)
ncols = len(xCoded[1])

#form x and y into numpy arrays and make up column names
X = numpy.array(xCoded)
y = numpy.array(labels)

#break into training and test sets.
xTrain, xTest, yTrain, yTest = train_test_split(X, y, test_size=0.30,
    random_state=531)

#train Random Forest at a range of ensemble sizes in
#order to see how the mse changes
mseOos = []
nTreeList = range(50, 500, 10)
for iTrees in nTreeList:
    depth = None
    maxFeat  = 4 #try tweaking
    abaloneRFModel = ensemble.RandomForestRegressor(n_estimators=iTrees,
        max_depth=depth, max_features=maxFeat,
         oob_score=False, random_state=531)

    abaloneRFModel.fit(xTrain,yTrain)

    #Accumulate mse on test set
    prediction = abaloneRFModel.predict(xTest)
    mseOos.append(mean_squared_error(yTest, prediction))


print("MSE" )
print(mseOos[-1])


#plot training and test errors vs number of trees in ensemble
plot.plot(nTreeList, mseOos)
plot.xlabel('Number of Trees in Ensemble')
plot.ylabel('Mean Squared Error')
#plot.ylim([0.0, 1.1*max(mseOob)])
plot.show()

# Plot feature importance
featureImportance = abaloneRFModel.feature_importances_

# normalize by max importance
```

continues

continued

```
featureImportance = featureImportance / featureImportance.max()
sortedIdx = numpy.argsort(featureImportance)
barPos = numpy.arange(sortedIdx.shape[0]) + .5
plot.barh(barPos, featureImportance[sortedIdx], align='center')
plot.yticks(barPos, abaloneNames[sortedIdx])
plot.xlabel('Variable Importance')
plot.subplots_adjust(left=0.2, right=0.9, top=0.9, bottom=0.1)
plot.show()

# Printed Output:
# MSE
# 4.30971555911
```

One of the attributes in the data set is the sex of the abalone. There are three possible values for an abalone's gender: male, female, and infant (although the gender of an abalone is indeterminate in infancy). So, the gender attribute is a three-valued factor variable. In the data set, the gender attribute is one of three character variables: `M`, `F`, or `I`. The section of the program that codes this attribute starts with a list filled with two float zeros. If the attribute value is `M`, the first list element is changed to a `1.0`. If the attribute value is `F`, the second list element is changed to a `1.0`. Otherwise, the list is left with two zeros (that is, if the attribute value is `I`). Then the new two-element list replaces the old character variable and the result is used to build a Random Forest model.

Assessing Performance and the Importance of Coded Variables

Figure 7-7 shows how the mean square prediction error decreases as the number of trees in the Random Forest ensemble is changed. The mean squared error in predicting the age of abalone was 4.31. Compare that to the summary statistics that you saw in Chapter 2. The standard deviation of the age (shell rings) was 3.22, meaning that the mean squared variation in the age was 10.37. Therefore, Random Forest is able to predict about 58% of the squared variation in the age of the abalone in the population that was tested.

Figure 7-8 shows the relative variable importance for the Random Forest model. The gender-related variables that were created to deal with the non-numeric gender variable do not turn out to be very important in this model.

Coding the Sex of Abalone for Gradient Boosting Regression in Python

The process of doing the coding for the gender variable is the same for Gradient Boosting as it was for Random Forest. Listing 7-5 contains the code to train a Gradient Boosting model.

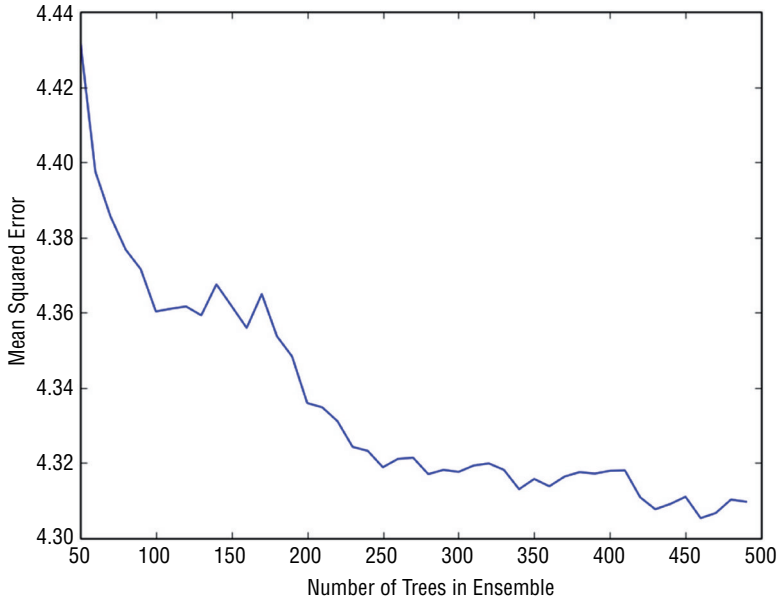


Figure 7-7: Abalone age prediction error with Random Forest

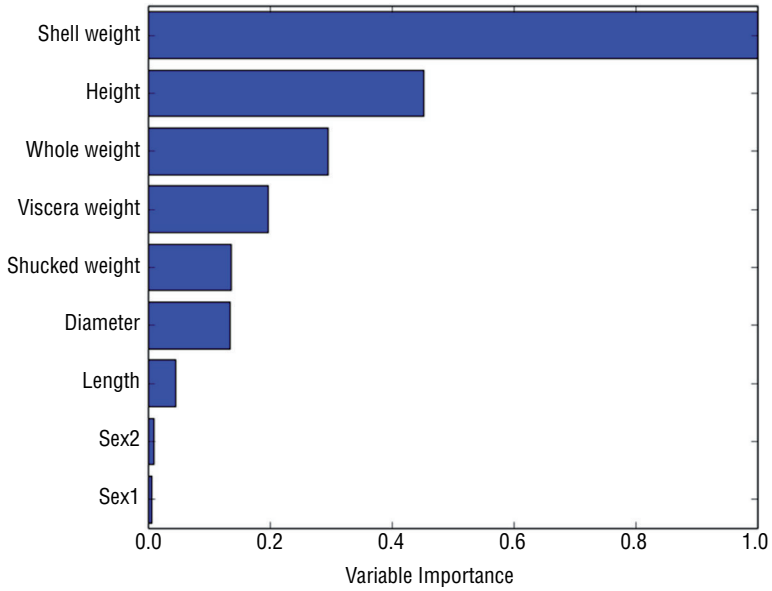


Figure 7-8: Variable importance for abalone age prediction with Random Forest

Listing 7-5: Predicting Abalone Age with Gradient Boosting—abaloneGBM.py

```
__author__ = 'mike_bowles'

import urllib2
```

continues

continued

```
from pylab import *
import matplotlib.pyplot as plot
import numpy
from sklearn.cross_validation import train_test_split
from sklearn import ensemble
from sklearn.metrics import mean_squared_error

target_url = ("http://archive.ics.uci.edu/ml/machine-learning-"
"databases/abalone/abalone.data")
#read abalone data
data = urllib2.urlopen(target_url)

xList = []
labels = []
for line in data:
    #split on semi-colon
    row = line.strip().split(",")

    #put labels in separate array and remove label from row
    labels.append(float(row.pop()))

    #form list of list of attributes (all strings)
    xList.append(row)

#code three-valued sex attribute as numeric
xCoded = []
for row in xList:
    #first code the three-valued sex variable
    codedSex = [0.0, 0.0]
    if row[0] == 'M': codedSex[0] = 1.0
    if row[0] == 'F': codedSex[1] = 1.0

    numRow = [float(row[i]) for i in range(1,len(row))]
    rowCoded = list(codedSex) + numRow
    xCoded.append(rowCoded)

#list of names for
abaloneNames = numpy.array(['Sex1', 'Sex2', 'Length', 'Diameter',
    'Height', 'Whole weight', 'Shucked weight',
    'Viscera weight', 'Shell weight', 'Rings'])

#number of rows and columns in x matrix
nrows = len(xCoded)
ncols = len(xCoded[1])

#form x and y into numpy arrays and make up column names
X = numpy.array(xCoded)
y = numpy.array(labels)

#break into training and test sets.
```

```
xTrain, xTest, yTrain, yTest = train_test_split(X, y, test_size=0.30,
    random_state=531)

#instantiate model
nEst = 2000
depth = 5
learnRate = 0.005
maxFeatures = 3
subsamp = 0.5
abaloneGBMModel = ensemble.GradientBoostingRegressor(n_estimators=nEst,
                        max_depth=depth, learning_rate=learnRate,
                         max_features=maxFeatures,subsample=subsamp,
                         loss='ls')

#train
abaloneGBMModel.fit(xTrain, yTrain)

# compute mse on test set
msError = []
predictions = abaloneGBMModel.staged_decision_function(xTest)
for p in predictions:
    msError.append(mean_squared_error(yTest, p))

print("MSE" )
print(min(msError))
print(msError.index(min(msError)))

#plot training and test errors vs number of trees in ensemble
plot.figure()
plot.plot(range(1, nEst + 1), abaloneGBMModel.train_score_,
    label='Training Set MSE', linestyle=":")
plot.plot(range(1, nEst + 1), msError, label='Test Set MSE')
plot.legend(loc='upper right')
plot.xlabel('Number of Trees in Ensemble')
plot.ylabel('Mean Squared Error')
plot.show()

# Plot feature importance
featureImportance = abaloneGBMModel.feature_importances_

# normalize by max importance
featureImportance = featureImportance / featureImportance.max()
idxSorted = numpy.argsort(featureImportance)
barPos = numpy.arange(idxSorted.shape[0]) + .5
plot.barh(barPos, featureImportance[idxSorted], align='center')
plot.yticks(barPos, abaloneNames[idxSorted])
plot.xlabel('Variable Importance')
plot.subplots_adjust(left=0.2, right=0.9, top=0.9, bottom=0.1)
plot.show()
```

continues

continued

```
# Printed Output:

# for Gradient Boosting
# nEst = 2000
# depth = 5
# learnRate = 0.003
# maxFeatures = None
# subsamp = 0.5
#
# MSE
# 4.22969363284
# 1736

#for Gradient Boosting with RF base learners
# nEst = 2000
# depth = 5
# learnRate = 0.005
# maxFeatures = 3
# subsamp = 0.5
#
# MSE
# 4.27564515749
# 1687
```

Assessing Performance and the Importance of Coded Variables with Gradient Boosting

There are a couple of things to highlight in the training and results. One is to have a look at the variable importances that Gradient Boosting determines to see whether they agree that the coded gender variables are the least important.

The other thing to check is Python's implementation to incorporate Random Forest base learners for gradient boosting. Will that help or hurt performance? The only thing required to make Gradient Boosting use Random Forest base learners is to change the `max_features` variable from `None` to an integer value less than the number of attributes or a float less than `1.0`. When `max_features` is set to `None`, all nine of the features are considered when the Tree Growing algorithm is searching for the best attribute for splitting the data at each of the nodes. When `max_features` is set to an integer less than 9, the features are chosen from a set attributes of length `max_features` chosen at random for each node.

The printed output from the code in Listing 7-5 is shown at the bottom of the listing. The mean squared error numbers indicate that there's not much performance difference between Random Forest and Gradient Boosting for predicting abalone age. There's also not much difference between using simple trees as base learners versus using Random Forest base learners in Gradient Boosting when they're used to predict abalone age.

The use of simple trees versus Random Forest similarly makes little difference in trajectories of prediction error versus ensemble size, as seen by comparing Figures 7-9 through 7-11.

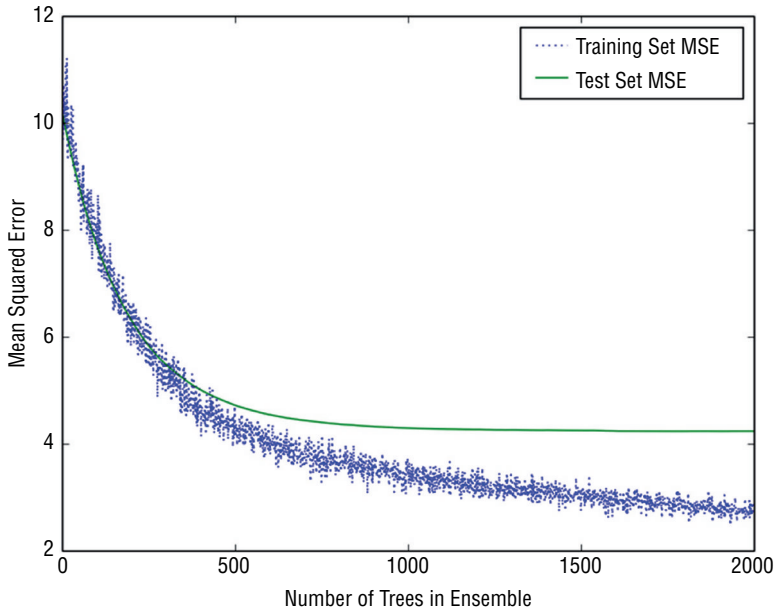


Figure 7-9: Abalone age prediction error with Gradient

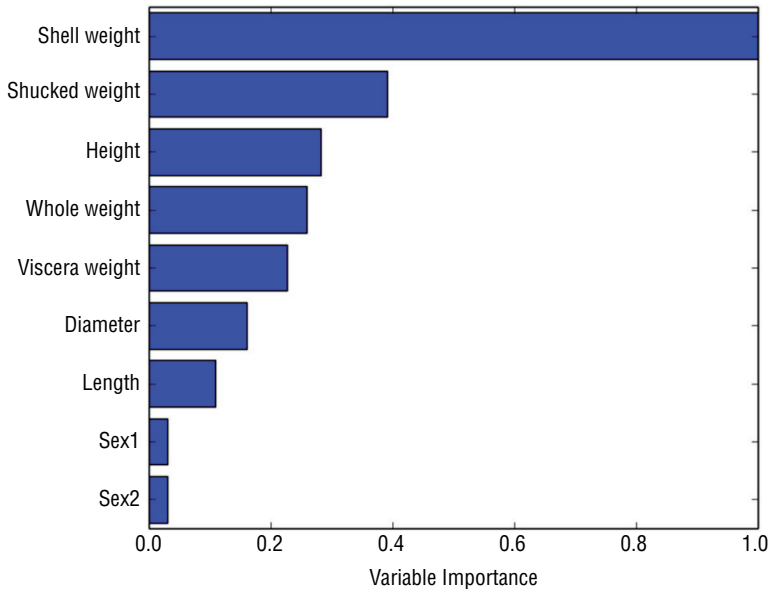


Figure 7-10: Variable importance for abalone age prediction with Gradient Boosting

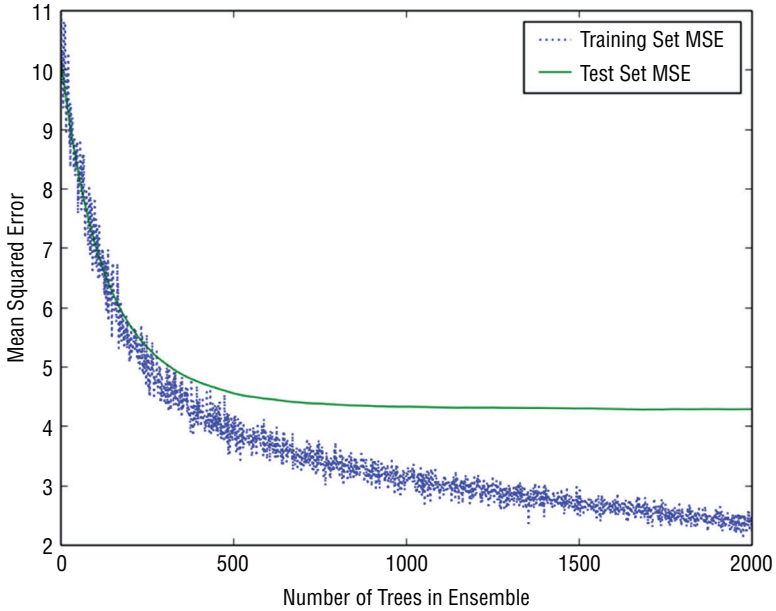


Figure 7-11: Abalone age prediction error with Gradient Boosting using Random Forest base learners

Figures 7-10 and 7-12 show the variable importance for Gradient Boosting based on simple trees and based on Random Forest base learners, respectively. The only difference between the two lists is that viscera weight and height (fourth and fifth most important variables) are swapped in position between the two. Similarly, there is little difference between the order of the importance list generated by Random Forest and either of the two lists generated by Gradient Boosting.

There's some belief that Random Forest has an advantage on wider attribute spaces, particularly for sparse ones such as in text-mining problems. The next section compares the two algorithms on a binary classification problem: classifying rock versus mines using sonar output. That problem has 60 attributes—not as wide as a text-mining problem, but perhaps that will show some performance difference between Gradient Boosting using ordinary binary decision trees versus using Random Forest base learners.

Solving Binary Classification Problems with Python Ensemble Methods

This section covers two basic types of classification problems: binary classification and multiclass classification. Binary classification problems are ones where there are two possible outcomes. Those outcomes might be "clicked on the ad"

or "didn't click on the ad," for example. The example used here to illustrate the use of ensemble methods is the rocks versus mines problem, where the task is to use sonar returns to determine whether the object being scanned by the sonar is a rock or a mine.

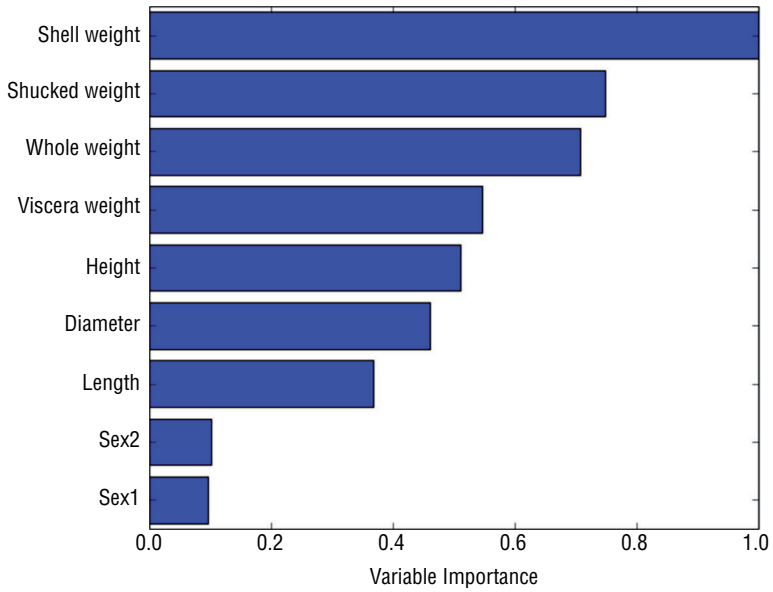


Figure 7-12: Variable importance for abalone age prediction with Gradient Boosting using Random Forest base learners

Multiclass problems are ones where there are more than two possible outcomes. Classifying glass samples according their chemical composition serves to illustrate the use of Python ensemble methods for this class of problems.

Detecting Unexploded Mines with Python Random Forest

The lists that follow show the constructor and its arguments for `RandomForestClassifier`.[13] Most of the arguments for the `RandomForestClassifier` are the same as for `RandomForestRegressor`. The arguments for `RandomForestRegressor` were outlined and discussed in the section on using `RandomForestRegressor` for predicting wine quality. This section highlights only the elements of the `RandomForestClassifier` class that differ from their regression counterparts.

The first difference is the criterion used for judging the quality of splits. Recall from Chapter 6 that the process of training a tree involves trying all possible attributes and all possible split points for each attribute and then picking the

attribute and split point that give the best split. For regression trees, the quality of the split was judged on the basis of sum squared error. Sum squared error does not work for classification problems. Something more like misclassification error is required.

Here is the class constructor for `sklearn.ensemble.RandomForestClassifier`:

```
sklearn.ensemble.RandomForestClassifier(n_estimators=10, criterion=
'gini', max_depth=None, min_samples_split=2, min_samples_leaf=1,
max_features='auto', max_leaf_nodes=None, bootstrap=True, oob_score=
False, n_jobs=1, random_state=None, verbose=0, min_density=None,
compute_importances=None)
```

The following list describes the parameter:

- **criterion**

  string, optional (default='gini')

  Possible values include the following:

  `gini` Use gini impurity measure

  `entropy` Use entropy-based information gain

  For more information on these two measures of node impurity, see the Wikipedia page on binary decision trees at `http://en.wikipedia.org/wiki/Decision_tree_learning`. As a practical matter, the choice does not make a lot of difference for ensemble performance.

Classification trees naturally produce probabilities of class membership based on the percentages of different classes from the training data that wind up in each of the leaf nodes. Depending on the application you have, for the answers you might prefer to work directly with those probabilities or you may want to have the value of the most numerous class returned as the prediction for those examples that wind up in the leaf node. If you're going to adjust thresholds used in conjunction with the prediction, you'll want to have the probabilities. For generating area under the curve (AUC), you'll get better fidelity on the receiver operating curve (ROC) with probabilities. If you want to calculate misclassification errors, you'll want the probabilities converted to a prediction of a specific class.

The following list describes the methods:

- **fit**(X, y, sample_weight=None)

  The description of the arguments for the classification version of Random Forest differs only in the nature of the labels `y`. For a classification problem, the labels are integers taking values from `0` to the number of different classes minus `1`. For binary classification the labels are `0` or `1`. For a multiclass problem with `nClass` different classes they are integers from `0` to `nClass - 1`.

- **predict**(X)

  For an attribute matrix (two-dimensional numpy array) `X`, this function produces a specific class prediction. It yields a single column array with the same number of rows as `X`. Each entry is a predicted class, whether the problem is a binary classification problem or a multiclass problem.

- **predict_proba**(X)

  This version of the prediction function produces a two-dimensional array. The number of rows matches the number of rows in `X`. The number of columns is equal to the number of classes being predicted (two columns for a binary classification problem, for example). The entry in each row is the probability of the associated class.

- **predict_log_proba**(X)

  This version of the prediction function produces a two-dimensional array similar to the `predict_proba`. Instead of showing probabilities, this version shows log of probability.

Constructing a Random Forests Model to Detect Unexploded Mines

Listing 7-8 shows how to build a Random Forest model for detecting unexploded mines using sonar. The overall structure of the data setup and training should be familiar from the other Random Forest examples earlier in this chapter and in Chapter 6. Differences stem from properties of classification problems. First you'll notice that the labels are changed from `M` and `R` to `0` and `1`. That's an input requirement for `RandomForestClassifier`. The next differences show up after training when evaluating performance on the test set. For a binary classification problem, there is choice of using area under the ROC curve (AUC) or misclassification error. I usually prefer AUC when it is available because it gives an overall measure of performance.

To calculate AUC, the `predict_proba` version of the `predict()` function is used. You cannot get a useful ROC curve with predictions that are already reduced to a specific class. (More correctly, the ROC curve you calculate only has three points on it: the two end points and one point in the middle.) The sklearn metric utilities make calculating the AUC simple, with just a couple of lines of code. Those get accumulated into a list to plot AUC performance as a function of the number of trees in the ensemble. The code in Listing 7-7 then plots the AUC versus number of trees, the feature importance for the 30 most important features, and the ROC curve for the largest ensemble of the ones that are generated. The last section of the code picks three different threshold levels and prints out the confusion matrix for each of these threshold levels. The threshold levels are chosen at the three quartile boundaries, and the results

show how false positives and false negatives change as the threshold moves to favor one versus the others.

Listing 7-7: Classifying Sonar Returns as Rocks or Mines with Random Forest—rocksVMinesRF.py

```
__author__ = 'mike_bowles'

import urllib2
from math import sqrt, fabs, exp
import matplotlib.pyplot as plot
from sklearn.cross_validation import train_test_split
from sklearn import ensemble
from sklearn.metrics import roc_auc_score, roc_curve
import numpy

#read data from uci data repository
target_url = ("https://archive.ics.uci.edu/ml/machine-learning"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")
data = urllib2.urlopen(target_url)

#arrange data into list for labels and list of lists for attributes
xList = []

for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)

#separate labels from attributes, convert from attributes from
#string to numeric and convert "M" to 1 and "R" to 0

xNum = []
labels = []

for row in xList:
    lastCol = row.pop()
    if lastCol == "M":
        labels.append(1)
    else:
        labels.append(0)
    attrRow = [float(elt) for elt in row]
    xNum.append(attrRow)

#number of rows and columns in x matrix
nrows = len(xNum)
ncols = len(xNum[1])

#form x and y into numpy arrays and make up column names
X = numpy.array(xNum)
y = numpy.array(labels)
```

```
rocksVMinesNames = numpy.array(['V' + str(i) for i in range(ncols)])

#break into training and test sets.
xTrain, xTest, yTrain, yTest = train_test_split(X, y, test_size=0.30,
    random_state=531)

auc = []
nTreeList = range(50, 2000, 50)
for iTrees in nTreeList:
    depth = None
    maxFeat  = 8 #try tweaking
    rocksVMinesRFModel = ensemble.RandomForestClassifier(n_estimators=
                            iTrees, max_depth=depth, max_features=
                            maxFeat, oob_score=False, random_state=531)
                            rocksVMinesRFModel.fit(xTrain,yTrain)

    #Accumulate auc on test set
    prediction = rocksVMinesRFModel.predict_proba(xTest)
    aucCalc = roc_auc_score(yTest, prediction[:,1:2])
    auc.append(aucCalc)

print("AUC" )
print(auc[-1])


#plot training and test errors vs number of trees in ensemble
plot.plot(nTreeList, auc)
plot.xlabel('Number of Trees in Ensemble')
plot.ylabel('Area Under ROC Curve - AUC')
#plot.ylim([0.0, 1.1*max(mseOob)])
plot.show()

# Plot feature importance
featureImportance = rocksVMinesRFModel.feature_importances_

# normalize by max importance
featureImportance = featureImportance / featureImportance.max()

#plot importance of top 30
idxSorted = numpy.argsort(featureImportance)[30:60]
idxTemp = numpy.argsort(featureImportance)[::-1]
print(idxTemp)
barPos = numpy.arange(idxSorted.shape[0]) + .5
plot.barh(barPos, featureImportance[idxSorted], align='center')
plot.yticks(barPos, rocksVMinesNames[idxSorted])
plot.xlabel('Variable Importance')
plot.show()

#plot best version of ROC curve
fpr, tpr, thresh = roc_curve(yTest, list(prediction[:,1:2]))
```

continues

continued

```
ctClass = [i*0.01 for i in range(101)]

plot.plot(fpr, tpr, linewidth=2)
plot.plot(ctClass, ctClass, linestyle=':')
plot.xlabel('False Positive Rate')
plot.ylabel('True Positive Rate')
plot.show()

#pick some threshold values and calc confusion matrix for
#best predictions

#notice that GBM predictions don't fall in range of (0, 1)
#pick threshold values at 25th, 50th and 75th percentiles
idx25 = int(len(thresh) * 0.25)
idx50 = int(len(thresh) * 0.50)
idx75 = int(len(thresh) * 0.75)

#calculate total points, total positives and total negatives
totalPts = len(yTest)
P = sum(yTest)
N = totalPts - P

print('')
print('Confusion Matrices for Different Threshold Values')

#25th
TP = tpr[idx25] * P; FN = P - TP; FP = fpr[idx25] * N; TN = N - FP
print('')
print('Threshold Value =   ', thresh[idx25])
print('TP = ', TP/totalPts, 'FP = ', FP/totalPts)
print('FN = ', FN/totalPts, 'TN = ', TN/totalPts)

#50th
TP = tpr[idx50] * P; FN = P - TP; FP = fpr[idx50] * N; TN = N - FP
print('')
print('Threshold Value =   ', thresh[idx50])
print('TP = ', TP/totalPts, 'FP = ', FP/totalPts)
print('FN = ', FN/totalPts, 'TN = ', TN/totalPts)

#75th
TP = tpr[idx75] * P; FN = P - TP; FP = fpr[idx75] * N; TN = N - FP
print('')
print('Threshold Value =   ', thresh[idx75])
print('TP = ', TP/totalPts, 'FP = ', FP/totalPts)
print('FN = ', FN/totalPts, 'TN = ', TN/totalPts)


# Printed Output:
#
# AUC
```

```
# 0.950304259635
#
# Confusion Matrices for Different Threshold Values
#
# ('Threshold Value =   ', 0.76051282051282054)
# ('TP = ', 0.25396825396825395, 'FP = ', 0.0)
# ('FN = ', 0.2857142857142857, 'TN = ', 0.46031746031746029)
#
# ('Threshold Value =   ', 0.62461538461538457)
# ('TP = ', 0.46031746031746029, 'FP = ', 0.047619047619047616)
# ('FN = ', 0.079365079365079361, 'TN = ', 0.41269841269841268)
#
# ('Threshold Value =   ', 0.46564102564102566)
# ('TP = ', 0.53968253968253965, 'FP = ', 0.22222222222222221)
# ('FN = ', 0.0, 'TN = ', 0.23809523809523808)
```

Determining the Performance of a Random Forests Classifier

Figure 7-13 shows a plot of AUC versus number of trees. The plot appears upside down from the plots you've seen involving mean squared error or misclassification error. For mean squared error and misclassification error, smaller is better. For AUC, `1.0` is perfect, and `0.5` is perfectly bad. So, with AUC, larger is better, and instead of looking for a valley in the plot, you're looking for a peak. Figure 7-13 shows a peak toward the left side of the plot. However, because Random Forest only reduces variance and does not overfit, the peak can be attributed to random fluctuation. As was the case with some of the regression problem earlier in the chapter, the best choice of model is the one including all the trees whose performance is the rightmost point on the curve.

Figure 7-14 plots the variable importance for the most important 30 variables in the Random Forest mine detector. The different attributes in the mine detection problem correspond to different frequencies of sonar signal and therefore different wavelengths. If you were given the problem of designing the machine learning for this problem, your next step might be to determine the wavelengths corresponding to these variables and compare those wavelengths to the characteristic dimensions of the rocks and mines in the test and training set. That could help you get some faith and understanding of the model.

The model is getting remarkably high AUC, and the ROC curve is correspondingly good. It doesn't quite square the corner in the upper left, but it comes pretty close.

Detecting Unexploded Mines with Python Gradient Boosting

Listing 7-7 shows the form of the constructor for Gradient Boosting in sci-kit learn. Most of the arguments and methods for `GradientBoostingClassifier`[14] are the same as for `GradientBoostingRegressor`, so the following descriptions

are limited to the elements that are different with the classifier than with the regression version.

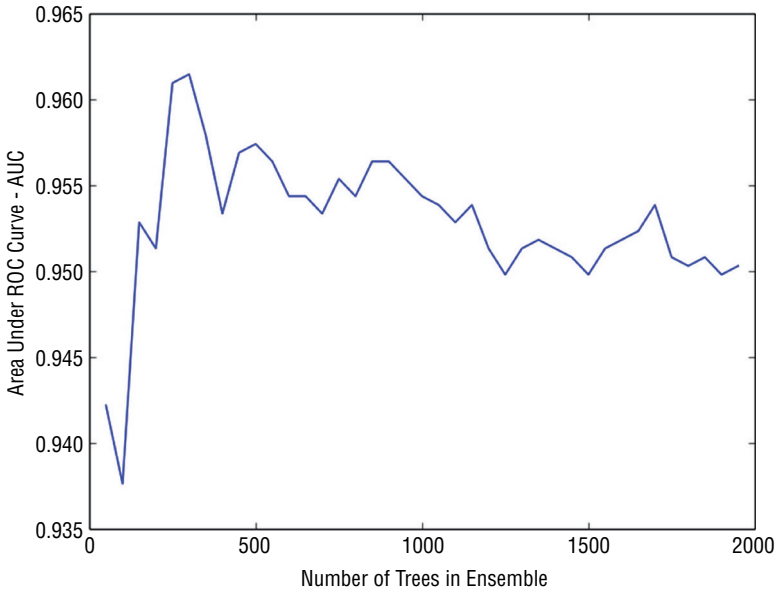


Figure 7-13: AUC versus ensemble size for Random Forest models for detecting mines using sonar

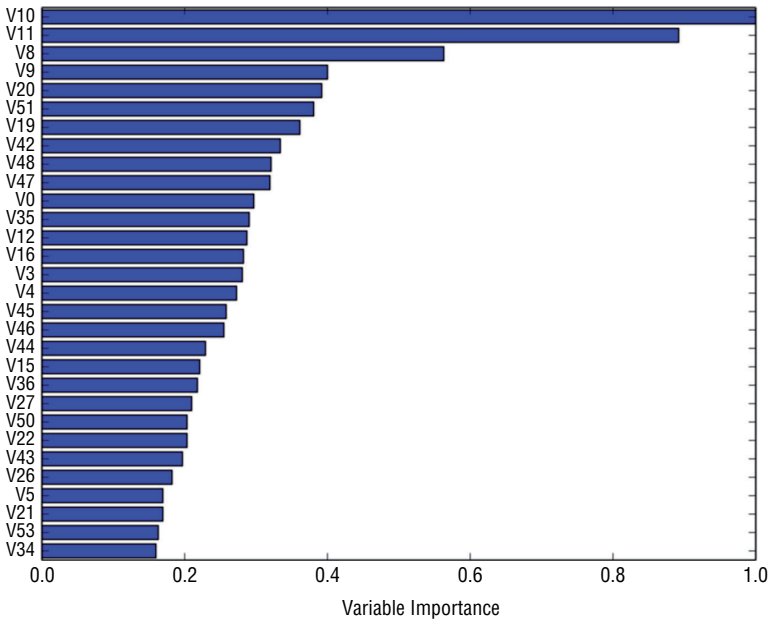


Figure 7-14: Variable importance for Random Forest mine detection model

Here is the class constructor for `sklearn.ensemble.GradientBoostingClassifier`:

```
sklearn.ensemble.GradientBoostingClassifier(loss='deviance', learning_
rate=0.1, n_estimators=100, subsample=1.0, min_samples_split=2,
min_samples_leaf=1, max_depth=3, init=None, random_state=None,
max_features=None, verbose=0, max_leaf_nodes=None, warm_start=False)
```

The following list describes the parameters:

- **loss**

  `deviance` is the default and the only option for classification.

The following list describes the methods:

- **fit**(X, y, monitor=None)

  The description of the arguments for the classification version of Random Forest differs only in the nature of the labels `y`. For a classification problem the labels are integers taking values from `0` to the number of different classes minus `1`. For binary classification the labels are `0` or `1`. For a multiclass problem with `nClass` different classes they are integers from `0` to `nClass - 1`.

- **decision_function**(X)

  Under the hood of a Gradient Boosting classifier is a sum of regression trees. These generate a real number estimate related to the probability of class membership. These real number estimates have to be passed through an inverse logistic function to turn them into probabilities. The real number values before being converted are available through the decision function and can be used just as easily as probabilities for ROC curve calculations.

- **predict**(X)

  This function predicts class membership.

- **predict_proba**(X)

  This function predicts class probabilities. It has a column of probabilities for each class. For a binary problem, there are two columns. For multiclass problems, there are `nClass` columns.

The staged versions of these functions are iterable and will generate as many values as there are trees in the ensemble (which is the same as the number of steps in the training).

- **staged_decision_function**(X)

  This is the staged version of the `decision` function.

- **staged_predict**(X)

  This is the staged version of the `predict` function.

- **staged_predict_proba**(X)

  This is the staged version of the `predict_proba` function.

The code in Listing 7-8 applies the sklearn `GradientBoostingClassifier` to the task of detecting mines.

Listing 7-8: Classifying Sonar Returns as Rocks or Mines with Gradient Boosting—rocksVMinesGBM.py

```
__author__ = 'mike_bowles'

import urllib2
from math import sqrt, fabs, exp
import matplotlib.pyplot as plot
from sklearn.cross_validation import train_test_split
from sklearn import ensemble
from sklearn.metrics import roc_auc_score, roc_curve
import numpy

#read data from uci data repository
target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/undocumented/connectionist-bench/sonar/sonar.all-data")
data = urllib2.urlopen(target_url)


#arrange data into list for labels and list of lists for attributes
xList = []


for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)

#separate labels from attributes, convert from attributes from
#string to numeric and convert "M" to 1 and "R" to 0

xNum = []
labels = []

for row in xList:
    lastCol = row.pop()
    if lastCol == "M":
        labels.append(1)
    else:
        labels.append(0)
    attrRow = [float(elt) for elt in row]
    xNum.append(attrRow)

#number of rows and columns in x matrix
nrows = len(xNum)
ncols = len(xNum[1])

#form x and y into numpy arrays and make up column names
```

```
X = numpy.array(xNum)
y = numpy.array(labels)
rockVMinesNames = numpy.array(['V' + str(i) for i in range(ncols)])

#break into training and test sets.
xTrain, xTest, yTrain, yTest = train_test_split(X, y, test_size=0.30,
    random_state=531)

#instantiate model
nEst = 2000
depth = 3
learnRate = 0.007
maxFeatures = 20
rockVMinesGBMModel = ensemble.GradientBoostingClassifier(
                        n_estimators=nEst, max_depth=depth,
                        learning_rate=learnRate,
                        max_features=maxFeatures)
#train
rockVMinesGBMModel.fit(xTrain, yTrain)

# compute auc on test set as function of ensemble size
auc = []
aucBest = 0.0
predictions = rockVMinesGBMModel.staged_decision_function(xTest)
for p in predictions:
    aucCalc = roc_auc_score(yTest, p)
    auc.append(aucCalc)

    #capture best predictions
    if aucCalc > aucBest:
        aucBest = aucCalc
        pBest = p

idxBest = auc.index(max(auc))

#print best values
print("Best AUC" )
print(auc[idxBest])
print("Number of Trees for Best AUC")
print(idxBest)

#plot training deviance and test auc's vs number of trees in ensemble
plot.figure()
plot.plot(range(1, nEst + 1), rockVMinesGBMModel.train_score_,
    label='Training Set Deviance', linestyle=":")
plot.plot(range(1, nEst + 1), auc, label='Test Set AUC')
plot.legend(loc='upper right')
plot.xlabel('Number of Trees in Ensemble')
plot.ylabel('Deviance / AUC')
plot.show()
```

continues

continued

```
# Plot feature importance
featureImportance = rockVMinesGBMModel.feature_importances_

# normalize by max importance
featureImportance = featureImportance / featureImportance.max()

#plot importance of top 30
idxSorted = numpy.argsort(featureImportance)[30:60]

barPos = numpy.arange(idxSorted.shape[0]) + .5
plot.barh(barPos, featureImportance[idxSorted], align='center')
plot.yticks(barPos, rockVMinesNames[idxSorted])
plot.xlabel('Variable Importance')
plot.show()

#pick threshold values and calc confusion matrix for best predictions
#notice that GBM predictions don't fall in range of (0, 1)

#plot best version of ROC curve
fpr, tpr, thresh = roc_curve(yTest, list(pBest))
ctClass = [i*0.01 for i in range(101)]

plot.plot(fpr, tpr, linewidth=2)
plot.plot(ctClass, ctClass, linestyle=':')
plot.xlabel('False Positive Rate')
plot.ylabel('True Positive Rate')
plot.show()

#pick threshold values and calc confusion matrix for best predictions
#notice that GBM predictions don't fall in range of (0, 1)
#pick threshold values at 25th, 50th and 75th percentiles
idx25 = int(len(thresh) * 0.25)
idx50 = int(len(thresh) * 0.50)
idx75 = int(len(thresh) * 0.75)

#calculate total points, total positives and total negatives
totalPts = len(yTest)
P = sum(yTest)
N = totalPts - P

print('')
print('Confusion Matrices for Different Threshold Values')

#25th
TP = tpr[idx25] * P; FN = P - TP; FP = fpr[idx25] * N; TN = N - FP
print('')
print('Threshold Value =   ', thresh[idx25])
print('TP = ', TP/totalPts, 'FP = ', FP/totalPts)
print('FN = ', FN/totalPts, 'TN = ', TN/totalPts)

#50th
```

```
TP = tpr[idx50] * P; FN = P - TP; FP = fpr[idx50] * N; TN = N - FP
print('')
print('Threshold Value =   ', thresh[idx50])
print('TP = ', TP/totalPts, 'FP = ', FP/totalPts)
print('FN = ', FN/totalPts, 'TN = ', TN/totalPts)

#75th
TP = tpr[idx75] * P; FN = P - TP; FP = fpr[idx75] * N; TN = N - FP
print('')
print('Threshold Value =   ', thresh[idx75])
print('TP = ', TP/totalPts, 'FP = ', FP/totalPts)
print('FN = ', FN/totalPts, 'TN = ', TN/totalPts)


# Printed Output:
#
# Best AUC
# 0.936105476673
# Number of Trees for Best AUC
# 1989
#
# Confusion Matrices for Different Threshold Values
#
# ('Threshold Value =   ', 6.2941249291909935)
# ('TP = ', 0.23809523809523808, 'FP = ', 0.015873015873015872)
# ('FN = ', 0.30158730158730157, 'TN = ', 0.44444444444444442)
#
# ('Threshold Value =   ', 2.2710265370949441)
# ('TP = ', 0.44444444444444442, 'FP = ', 0.063492063492063489)
# ('FN = ', 0.095238095238095233, 'TN = ', 0.3968253968253968)
#
# ('Threshold Value =   ', -3.0947902666953317)
# ('TP = ', 0.53968253968253965, 'FP = ', 0.22222222222222221)
# ('FN = ', 0.0, 'TN = ', 0.23809523809523808)
#
#
# Printed Output with max_features = 20 (Random Forest base learners):
#
# Best AUC
# 0.956389452333
# Number of Trees for Best AUC
# 1426
#
# Confusion Matrices for Different Threshold Values
#
# ('Threshold Value =   ', 5.8332200248698536)
# ('TP = ', 0.23809523809523808, 'FP = ', 0.015873015873015872)
# ('FN = ', 0.30158730158730157, 'TN = ', 0.44444444444444442)
#
# ('Threshold Value =   ', 2.0281780133610567)
# ('TP = ', 0.47619047619047616, 'FP = ', 0.031746031746031744)
```

continues

continued

```
# ('FN = ', 0.063492063492063489, 'TN = ', 0.42857142857142855)
#
# ('Threshold Value =   ', -1.2965629080181333)
# ('TP = ', 0.53968253968253965, 'FP = ', 0.22222222222222221)
# ('FN = ', 0.0, 'TN = ', 0.23809523809523808)
```

The code follows the same general progression as was followed for Random Forest. One difference is that Gradient Boosting can overfit, and so the program keeps track of the best value of AUC as it accumulates AUCs into a list to be plotted. The best version is then used for generating a ROC curve and the tables of false positives, false negatives, and so on. Another difference is that Gradient Boosting is run twice—once incorporating ordinary trees and once using Random Forest base learners. Both ways have very good classification performance. The version using Random Forest base learners achieved better performance, unlike the models for predicting abalone age, where the performance was not markedly changed.

Determining the Performance of a Gradient Boosting Classifier

Figure 7-16 plots two curves. One is the deviance on the training set. Deviance is related to how far the probability estimates are from correct but differs slightly from misclassification error. Deviance is plotted because that quantity is what gradient boosting is training to improve. It's included in the plot to show the progress of training. The AUC (on oos data) is also plotted to show how the oos performance is changing as the number of trees increases (or equivalently more gradient steps are taken; each step results in training an additional tree).

Figure 7-17 plots the variable importance for the most important 30 variables in the Gradient Boosting mine detector. The variable importances in Figure 7-17 have a somewhat different order than the ones for Random Forest (shown in Figure 7-14). There is some commonality; for example, variables V10, V11, V20 and V51 are near the top of both lists although not in quite the same order.

Figure 7-19 shows model training progress for Gradient Boosting that is using Random Forest base learners. Gradient Boosting does get better results using Random Forest base learners, but the difference isn't large enough to be obvious in the graph.

Using Random Forest base learners doesn't change the variable importance very much, as you can see by comparing Figure 7-20 with Figure 7-17.

Figure 7-21 shows the ROC curve for the mine detector model built with Gradient Boosting using Random Forest base learners.

In this section you have seen how ensemble methods can be used to solve binary classification problems. In most respects, using the application of ensemble methods to binary classification problems is the same as for regression problems. As an illustration of the similarity, notice how many of the parameters required to instantiate a randomForestRegressor object are the same as the ones for a

randomForestClassification object. Based on what you saw in Chapter 6, you can understand the basis for this similarity.

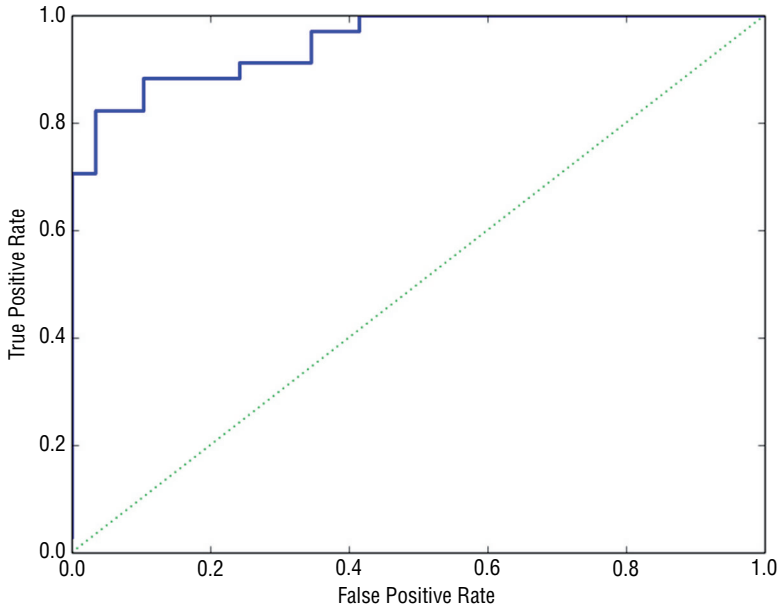


Figure 7-15: ROC curve for Random Forest mine detection model

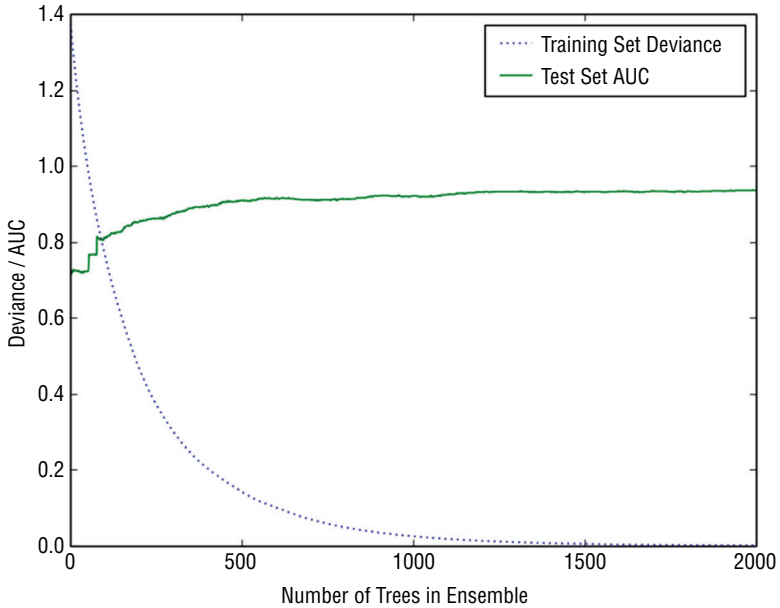


Figure 7-16: AUC versus ensemble size for Gradient Boosting models for detecting mines using sonar

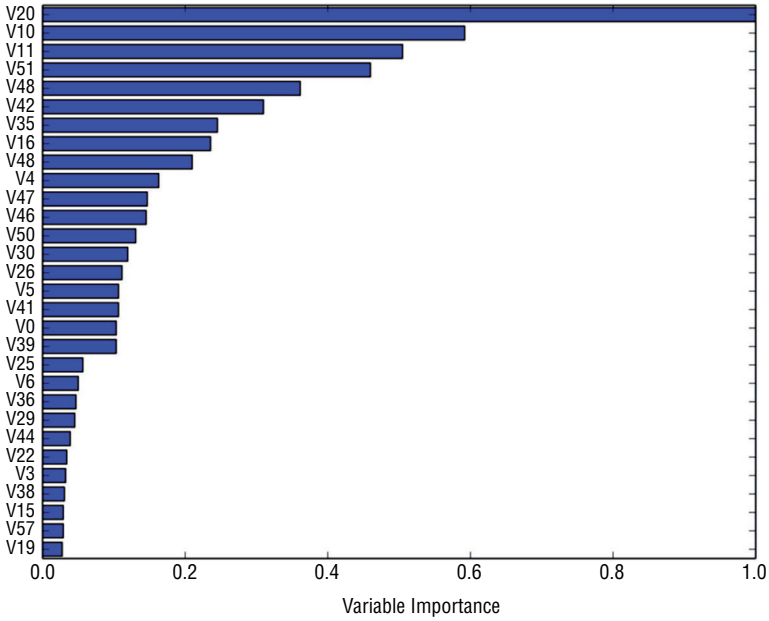


Figure 7-17: Variable importance for Gradient Boosting mine detection model

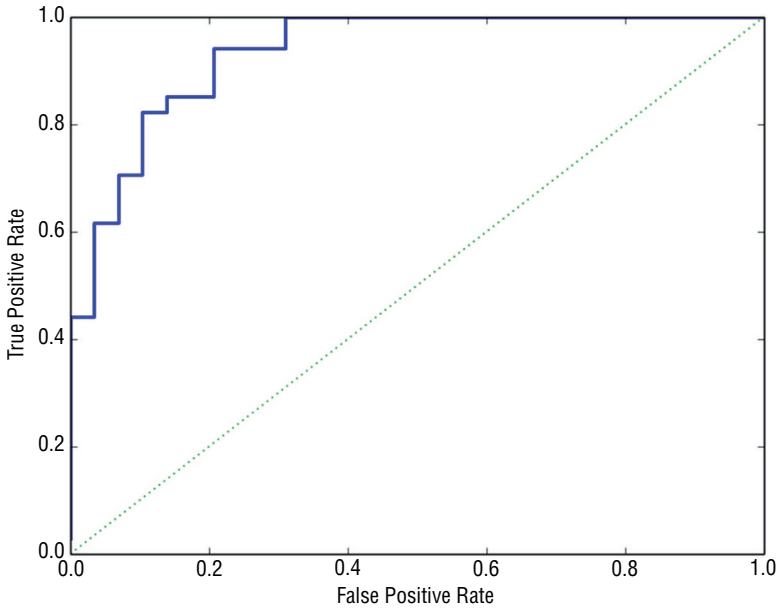


Figure 7-18: Mine detection ROC curve for Gradient Boosting

You also understand that many of the differences between building ensemble models for classification and regression stem from differences in measuring errors and otherwise characterizing errors between the two classes of problems.

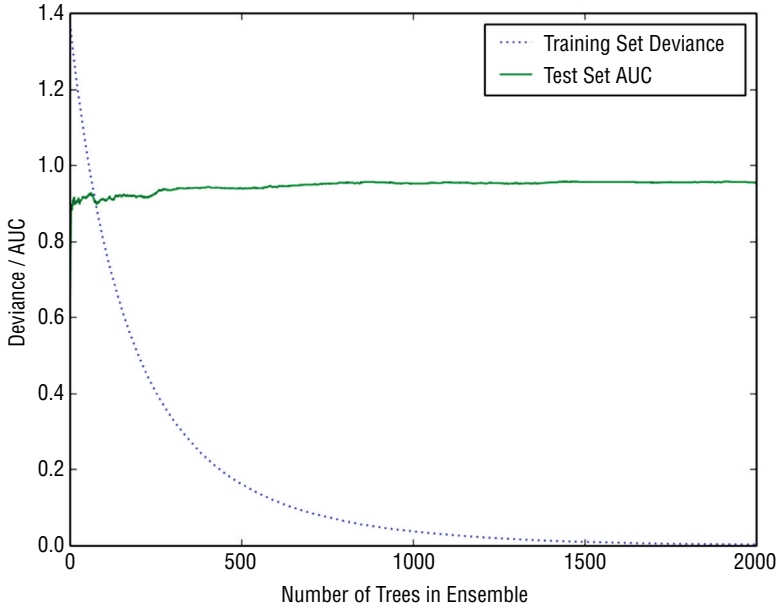


Figure 7-19: Mine detection AUC versus ensemble size for Gradient Boosting with Random Forest base learners

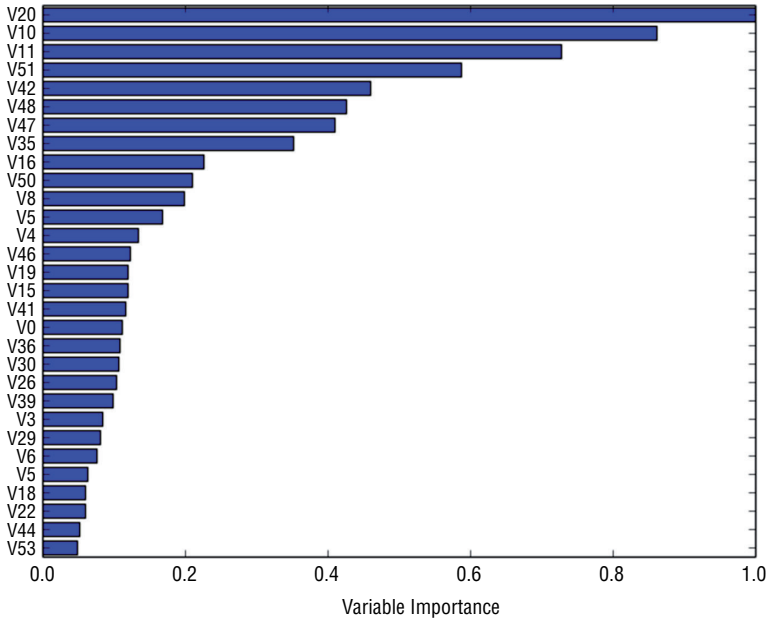


Figure 7-20: Variable importance for Gradient Boosting with Random Forest base learners

The next section shows how these methods can be used for multiclass problems.

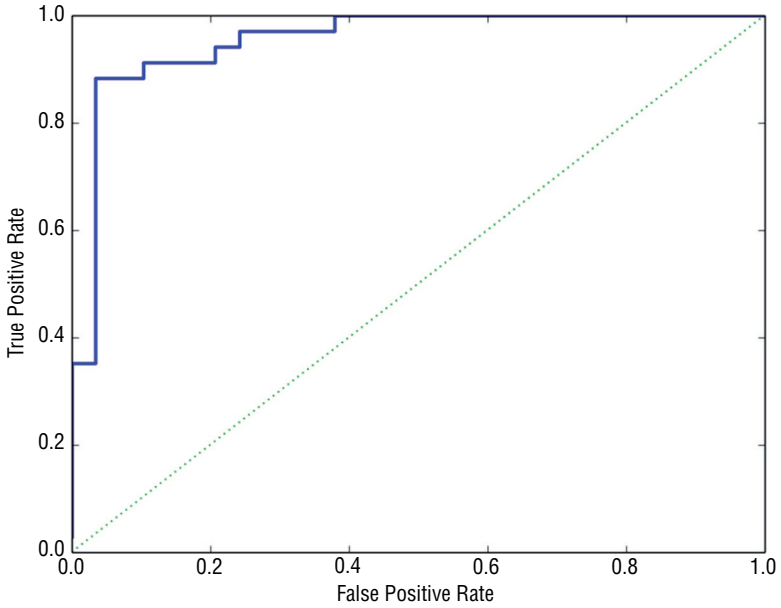


Figure 7-21: Mine detection ROC curve for Gradient Boosting using Random Forest base learners

Solving Multiclass Classification Problems with Python Ensemble Methods

The Random Forest and Gradient Boosting packages in Python will build both binary and multiclass classification models. The two types of models have a few natural differences between them. One is that the labels (`y`) take more values. The discussion of the Random Forest and Gradient Boosting packages described the manner in which the labels are specified. For a classification problem having `nClass` different classes, the labels take integer values from `0` to `nClass - 1`. Another manifestation of the number of classes is the output of the various predict methods. The predict methods that are predicting classes generate the same integer values that the labels take. The methods predicting probabilities yield probabilities for `nClass` possible classes.

The other area where there is a noticeable difference is in specifying performance. Misclassification error still makes sense, and you'll see that the example code uses that to measure oos performance. AUC is more complicated to use when there are more than two classes, and trading off different error types becomes more challenging.

Classifying Glass with Random Forests

Listing 7-9 follows a similar outline to the code used for detecting mines.

Listing 7-9: Classifying Glass Types Using Random Forests—glassRF.py

```
__author__ = 'mike_bowles'

import urllib2
from math import sqrt, fabs, exp
import matplotlib.pyplot as plot
from sklearn.linear_model import enet_path
from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve
from sklearn.cross_validation import train_test_split
from sklearn import ensemble
import numpy

target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
"databases/glass/glass.data")
data = urllib2.urlopen(target_url)

#arrange data into list for labels and list of lists for attributes
xList = []
for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)

glassNames = numpy.array(['RI', 'Na', 'Mg', 'Al', 'Si', 'K', 'Ca',
    'Ba', 'Fe', 'Type'])

#Separate attributes and labels
xNum = []
labels = []

for row in xList:
    labels.append(row.pop())
    l = len(row)
    #eliminate ID
    attrRow = [float(row[i]) for i in range(1, l)]
    xNum.append(attrRow)

#number of rows and columns in x matrix
nrows = len(xNum)
ncols = len(xNum[1])

#Labels are integers from 1 to 7 with no examples of 4.
#gb requires consecutive integers starting at 0
newLabels = []
labelSet = set(labels)
labelList = list(labelSet)
labelList.sort()
nlabels = len(labelList)
for l in labels:
    index = labelList.index(l)
```

continues

continued

```
    newLabels.append(index)

#Class populations:
#old label     new label     num of examples
#1              0               70
#2              1               76
#3              2               17
#5              3               13
#6              4               9
#7              5               29
#
#Drawing 30% test sample may not preserve population proportions

#stratified sampling by labels.
xTemp = [xNum[i] for i in range(nrows) if newLabels[i] == 0]
yTemp = [newLabels[i] for i in range(nrows) if newLabels[i] == 0]
xTrain, xTest, yTrain, yTest = train_test_split(xTemp, yTemp,
    test_size=0.30, random_state=531)
for iLabel in range(1, len(labelList)):
    #segregate x and y according to labels
    xTemp = [xNum[i] for i in range(nrows) if newLabels[i] == iLabel]
    yTemp = [newLabels[i] for i in range(nrows) if \
        newLabels[i] == iLabel]

    #form train and test sets on segregated subset of examples
    xTrainTemp, xTestTemp, yTrainTemp, yTestTemp = train_test_split(
        xTemp, yTemp, test_size=0.30, random_state=531)

    #accumulate
    xTrain = numpy.append(xTrain, xTrainTemp, axis=0)
    xTest = numpy.append(xTest, xTestTemp, axis=0)
    yTrain = numpy.append(yTrain, yTrainTemp, axis=0)
    yTest = numpy.append(yTest, yTestTemp, axis=0)

missCLassError = []
nTreeList = range(50, 2000, 50)
for iTrees in nTreeList:
    depth = None
    maxFeat  = 4 #try tweaking
    glassRFModel = ensemble.RandomForestClassifier(n_estimators=iTrees,
                      max_depth=depth, max_features=maxFeat,
                      oob_score=False, random_state=531)

    glassRFModel.fit(xTrain,yTrain)

    #Accumulate auc on test set
    prediction = glassRFModel.predict(xTest)
    correct = accuracy_score(yTest, prediction)

    missCLassError.append(1.0 - correct)
```

```
print("Missclassification Error" )
print(missCLassError[-1])

#generate confusion matrix
pList = prediction.tolist()
confusionMat = confusion_matrix(yTest, pList)
print('')
print("Confusion Matrix")
print(confusionMat)

#plot training and test errors vs number of trees in ensemble
plot.plot(nTreeList, missCLassError)
plot.xlabel('Number of Trees in Ensemble')
plot.ylabel('Missclassification Error Rate')
#plot.ylim([0.0, 1.1*max(mseOob)])
plot.show()

# Plot feature importance
featureImportance = glassRFModel.feature_importances_

# normalize by max importance
featureImportance = featureImportance / featureImportance.max()

#plot variable importance
idxSorted = numpy.argsort(featureImportance)
barPos = numpy.arange(idxSorted.shape[0]) + .5
plot.barh(barPos, featureImportance[idxSorted], align='center')
plot.yticks(barPos, glassNames[idxSorted])
plot.xlabel('Variable Importance')
plot.show()


# Printed Output:
# Missclassification Error
# 0.227272727273
#
# Confusion Matrix
# [[17  1  2  0  0  1]
#  [ 2 18  1  2  0  0]
#  [ 3  0  3  0  0  0]
#  [ 0  0  0  4  0  0]
#  [ 0  1  0  0  2  0]
#  [ 0  2  0  0  0  7]]
```

Dealing with Class Imbalances

As stated, this listing follows a similar outline to the code used for detecting mines. There are a couple of key differences. In the code, you'll see a list of the different glass types using the numbering system from the original data set and

the corresponding integer used as labels to meet the specification required for Random Forest. The table also shows how many examples there are of each type of glass. Some of the types have relatively many examples (in the 70s). Some types of glass are not as well represented. One in particular only has nine examples.

Imbalanced classes can sometimes cause problems because random sampling of the underrepresented classes may result in wildly different proportions in the sample than in the original data. To avoid that, the code goes through a process called *stratified sampling*. What that means in this case is that the data are segregated according to labels (stratified), and then each of those groups is sampled to obtain training and test sets within each class. Then the class-specific training sets are combined into a training set that has proportions of different classes that exactly match the original data.

The code generates Random Forest models and plots the training progress and the variable importance. It also prints out a confusion matrix that shows for each true class how many of the class were predicted to be each class. If the classifier is perfect, there should be no off-diagonal entries in the matrix.

Figure 7-22 shows how the performance of Random Forests improves as more trees are included in the ensemble. The curve generally drops as more trees are added. The rate of improvement decreases as more trees are added. It has slowed considerably at the point where the graph stops.

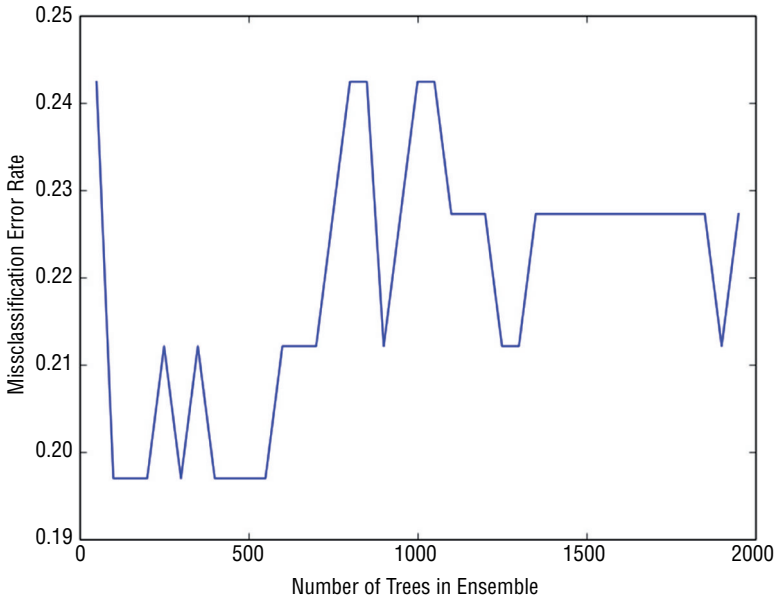


Figure 7-22: The overall performance of Random Forests

Figure 7-23 is a bar chart showing the relative importance of the variables used by Random Forests. The chart shows that a number of the variables are roughly

equal in performance. This is unusual behavior. In many cases the variable importances drop off quickly after the first few variables. In this problem there are several equally important variables.

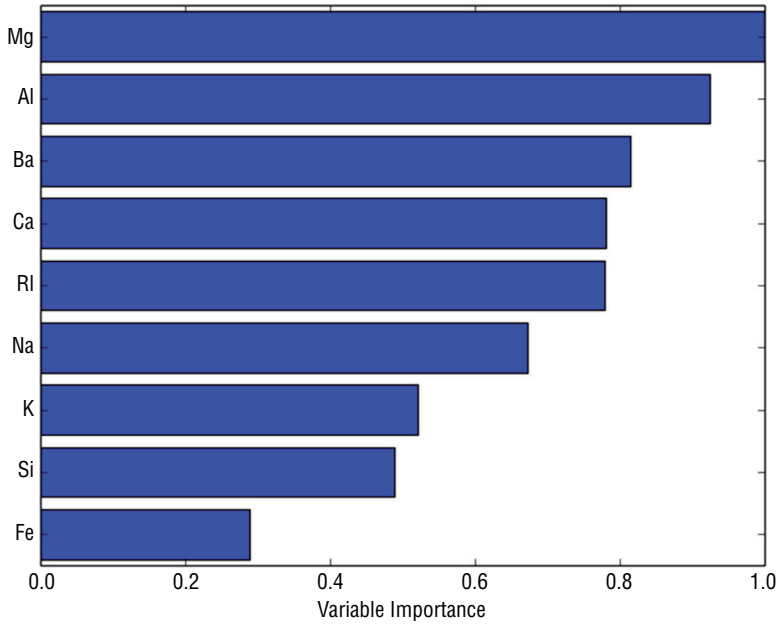


Figure 7-23: The relative importance of the variables used by Random Forest

Classifying Glass Using Gradient Boosting

Listing 7-10 runs through the same steps as the Random Forest glass classifier in the preceding section, with a couple of minor differences.

Listing 7-10: Classifying Glass with Gradient Boosting—glassGbm.py

```
__author__ = 'mike_bowles'

import urllib2
from math import sqrt, fabs, exp
import matplotlib.pyplot as plot
from sklearn.linear_model import enet_path
from sklearn.metrics import roc_auc_score, roc_curve, confusion_matrix
from sklearn.cross_validation import train_test_split
from sklearn import ensemble
import numpy


target_url = ("https://archive.ics.uci.edu/ml/machine-learning-"
```

continues

continued

```
"databases/glass/glass.data")
data = urllib2.urlopen(target_url)

#arrange data into list for labels and list of lists for attributes
xList = []
for line in data:
    #split on comma
    row = line.strip().split(",")
    xList.append(row)

glassNames = numpy.array(['RI', 'Na', 'Mg', 'Al', 'Si', 'K', 'Ca',
    'Ba', 'Fe', 'Type'])

#Separate attributes and labels
xNum = []
labels = []

for row in xList:
    labels.append(row.pop())
    l = len(row)
    #eliminate ID
    attrRow = [float(row[i]) for i in range(1, l)]
    xNum.append(attrRow)

#number of rows and columns in x matrix
nrows = len(xNum)
ncols = len(xNum[1])

#Labels are integers from 1 to 7 with no examples of 4.
#gb requires consecutive integers starting at 0
newLabels = []
labelSet = set(labels)
labelList = list(labelSet)
labelList.sort()
nlabels = len(labelList)
for l in labels:
    index = labelList.index(l)
    newLabels.append(index)

#Class populations:
#old label     new label     num of examples
#1              0              70
#2              1              76
#3              2              17
#5              3              13
#6              4              9
#7              5              29
#
#Drawing 30% test sample may not preserve population proportions

#stratified sampling by labels.
```

```
xTemp = [xNum[i] for i in range(nrows) if newLabels[i] == 0]
yTemp = [newLabels[i] for i in range(nrows) if newLabels[i] == 0]
xTrain, xTest, yTrain, yTest = train_test_split(xTemp, yTemp,
    test_size=0.30, random_state=531)
for iLabel in range(1, len(labelList)):
    #segregate x and y according to labels
    xTemp = [xNum[i] for i in range(nrows) if newLabels[i] == iLabel]
    yTemp = [newLabels[i] for i in range(nrows) if \
        newLabels[i] == iLabel]

    #form train and test sets on segregated subset of examples
    xTrainTemp, xTestTemp, yTrainTemp, yTestTemp = train_test_split(
        xTemp, yTemp, test_size=0.30, random_state=531)

    #accumulate
    xTrain = numpy.append(xTrain, xTrainTemp, axis=0)
    xTest = numpy.append(xTest, xTestTemp, axis=0)
    yTrain = numpy.append(yTrain, yTrainTemp, axis=0)
    yTest = numpy.append(yTest, yTestTemp, axis=0)

#instantiate model
nEst = 500
depth = 3
learnRate = 0.003
maxFeatures = 3
subSamp = 0.5
glassGBMModel = ensemble.GradientBoostingClassifier(n_estimators=nEst,
                    max_depth=depth,learning_rate=learnRate,
                    max_features=maxFeatures,subsample=subSamp)

#train
glassGBMModel.fit(xTrain, yTrain)

# compute auc on test set as function of ensemble size
missClassError = []
missClassBest = 1.0
predictions = glassGBMModel.staged_decision_function(xTest)
for p in predictions:
    missClass = 0
    for i in range(len(p)):
        listP = p[i].tolist()
        if listP.index(max(listP)) != yTest[i]:
            missClass += 1
    missClass = float(missClass)/len(p)

    missClassError.append(missClass)

    #capture best predictions
    if missClass < missClassBest:
        missClassBest = missClass
        pBest = p
```

continues

continued

```
idxBest = missClassError.index(min(missClassError))

#print best values
print("Best Missclassification Error" )
print(missClassBest)
print("Number of Trees for Best Missclassification Error")
print(idxBest)

#plot training deviance and test auc's vs number of trees in ensemble
missClassError = [100*mce for mce in missClassError]
plot.figure()
plot.plot(range(1, nEst + 1), glassGBMModel.train_score_,
    label='Training Set Deviance', linestyle=":")
plot.plot(range(1, nEst + 1), missClassError, label='Test Set Error')
plot.legend(loc='upper right')
plot.xlabel('Number of Trees in Ensemble')
plot.ylabel('Deviance / Classification Error')
plot.show()

# Plot feature importance
featureImportance = glassGBMModel.feature_importances_

# normalize by max importance
featureImportance = featureImportance / featureImportance.max()

#plot variable importance
idxSorted = numpy.argsort(featureImportance)
barPos = numpy.arange(idxSorted.shape[0]) + .5
plot.barh(barPos, featureImportance[idxSorted], align='center')
plot.yticks(barPos, glassNames[idxSorted])
plot.xlabel('Variable Importance')
plot.show()

#generate confusion matrix for best prediction.
pBestList = pBest.tolist()
bestPrediction = [r.index(max(r)) for r in pBestList]
confusionMat = confusion_matrix(yTest, bestPrediction)
print('')
print("Confusion Matrix")
print(confusionMat)


# Printed Output:
#
# nEst = 500
# depth = 3
# learnRate = 0.003
# maxFeatures = None
# subSamp = 0.5
#
```

```
#
# Best Missclassification Error
# 0.242424242424
# Number of Trees for Best Missclassification Error
# 113
#
# Confusion Matrix
# [[19  1  0  0  0  1]
#  [ 3 19  0  1  0  0]
#  [ 4  1  0  0  1  0]
#  [ 0  3  0  1  0  0]
#  [ 0  0  0  0  3  0]
#  [ 0  1  0  1  0  7]]
#



# For Gradient Boosting using Random Forest base learners
# nEst = 500
# depth = 3
# learnRate = 0.003
# maxFeatures = 3
# subSamp = 0.5
#
#
#
# Best Missclassification Error
# 0.227272727273
# Number of Trees for Best Missclassification Error
# 267
#
# Confusion Matrix
# [[20  1  0  0  0  0]
#  [ 3 20  0  0  0  0]
#  [ 3  3  0  0  0  0]
#  [ 0  4  0  0  0  0]
#  [ 0  0  0  0  3  0]
#  [ 0  2  0  0  0  7]]
```

As before, the Gradient Boosting version uses the "staged" methods available in the `GradientBoostingClassifier` class to generate predictions at each step in the Gradient Boosting training process.

Assessing the Advantage of Using Random Forest Base Learners with Gradient Boosting

At the end of the code, you'll see results reported for both Gradient Boosting with `max_features=None` and for `max_features=20`. The first parameter setting trains ordinary trees as suggested in the original Gradient Boosting papers.

The second parameter setting incorporates trees like the ones used in Random Forest, where not all the features are considered for splitting at each node. Instead of all the features being considered, `max_features` are selected at random for consideration as the splitting variable. This gives a sort of hybrid between the usual Gradient Boosting implementation and Random Forest.

Figure 7-24 plots the deviance on the training set and the misclassification error on the test set. The deviance indicates the progress of the training process. Misclassification on the test set is used to determine whether the model is overfitting. The algorithm does not overfit, but it also does not improve past 200 or so trees and could be terminated sooner.

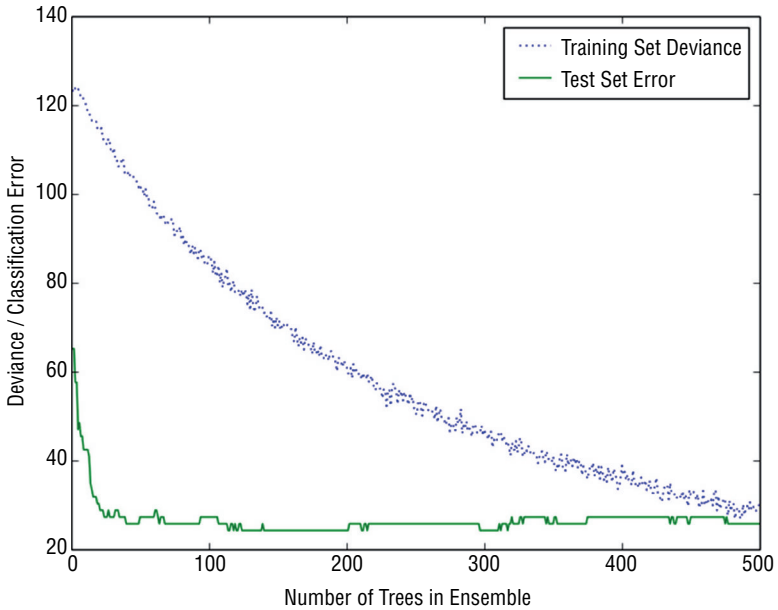


Figure 7-24: Glass classifier built using Gradient Boosting: training performance

Figure 7-25 plots the variable importance for Gradient Boosting. The variables show unusually equal importance. It's more usual to have a few variables be very important and for the importances to drop off more rapidly.

Figure 7-26 plots the deviance and oos misclassification error `max_features=20`, which results in Random Forest base learners being used in the ensemble, as discussed earlier. This leads to an improvement of about 10% in the misclassification error rate. That's not really perceptible from the graph in Figure 7-26, and the slight improvement in the end number does not change the basic character of the plot.

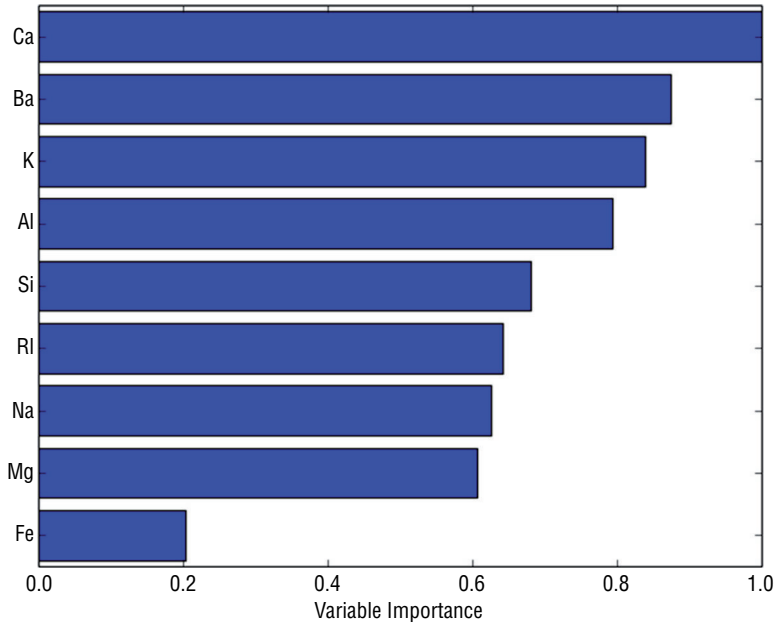


Figure 7-25: Glass classifier built using Gradient Boosting: variable importance

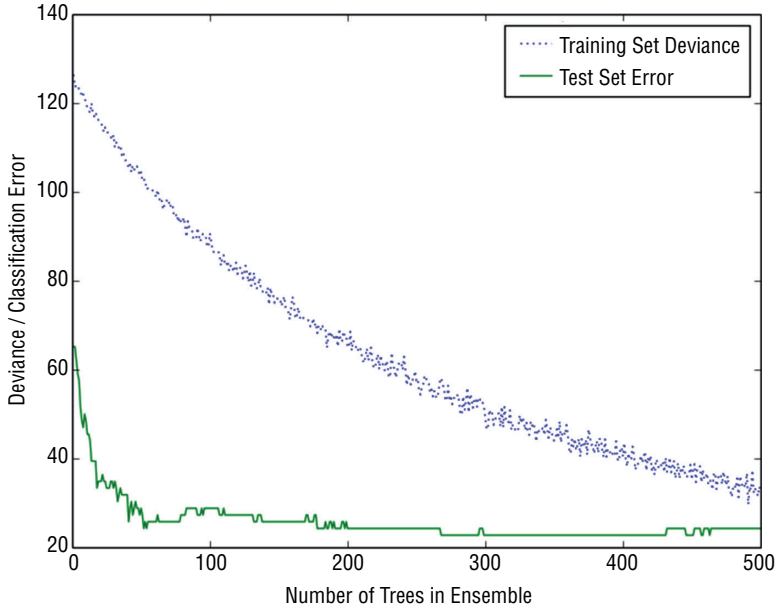


Figure 7-26: Glass classifier built using Gradient Boosting with Random Forest base learners: training performance.

Figure 7-27 shows the plot of variable importance for Gradient Boosting with Random Forest base learners. The order between this figure and Figure 7-25 is

somewhat altered. Some of the same variables appear in the top five, but some other in the top five for one are in the bottom for the other. These plots both show a surprisingly uniform level of importance, and that may be the cause of the instability in the importance order between the two.

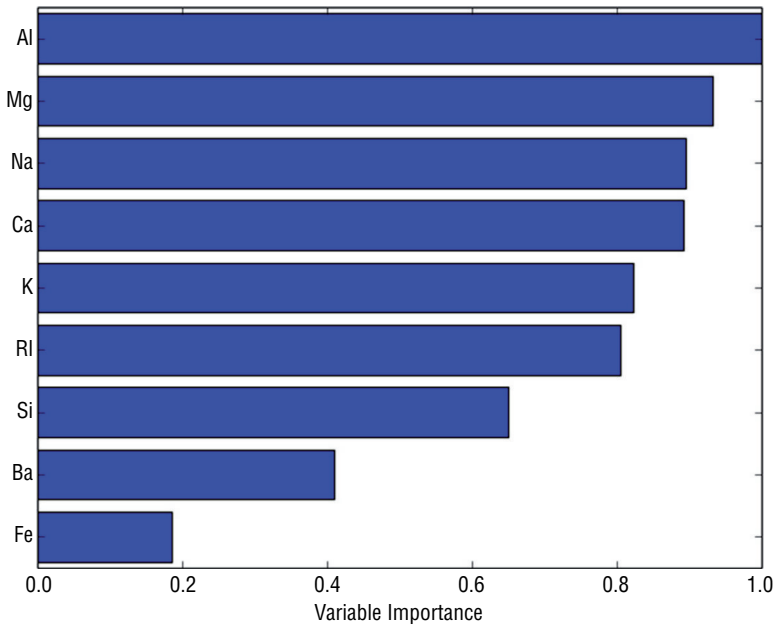


Figure 7-27: Glass classifier built using Gradient Boosting with Random Forest base learners: variable importance

Comparing Algorithms

Table 7-1 gives timing and performance comparisons for the algorithms presented here. The times shown are the training times for one complete pass through training. Some of the code for training Random Forest trained a series of different-sized models. In that case, only the last (and longest) training pass is counted. The others were done to illustrate the behavior as a function of the number of trees in the training set. Similarly, for penalized linear regression, many of the runs incorporated 10-fold cross-validation, whereas other examples used a single holdout set. The single holdout set requires one training pass, whereas 10-fold cross-validation requires 10 training passes. For examples that incorporated 10-fold cross-validation, the time for 1 of the 10 training passes is shown.

Except for the glass data set (a multiclass classification problem), the training times for penalized linear regression are an order of magnitude faster

than Gradient Boosting and Random Forest. Generally, the performance with Random Forest and Gradient Boosting is superior to penalized linear regression. Penalized linear regression is somewhat close on some of the data sets. Getting close on the wine data required employing basis expansion. Basis expansion was not used on other data sets and might lead to some further improvement.

Table 7-1: Performance and Training Time Comparisons

DATA SET	ALGORITHM	TRAIN TIME	PERFORMANCE	PERF METRIC
glass	RF 2000 trees	2.354401	0.227272727273	class error
glass	gbm 500 trees	3.879308	0.227272727273	class error
glass	lasso	12.296948	0.373831775701	class error
rvmines	rf 2000 trees	2.760755	0.950304259635	auc
rvmines	gbm 2000 trees	4.201122	0.956389452333	auc
rvmines	enet	0.519870*	0.868672796508	auc
abalone	rf 500 trees	8.060850	4.30971555911	mse
abalone	gbm 2000 trees	22.726849	4.22969363284	mse
wine	rf 500 trees	2.665874	0.314125711509	mse
wine	gbm 2000 trees	13.081342	0.313361215728	mse
wine	lasso-expanded	0.646788*	0.434528740430	mse

*The times marked with an asterisk are time per cross-validation fold. These techniques were trained several times in repetition in accordance with the n-fold cross-validation technique whereas other methods were trained using a single holdout test set. Using the time per cross-validation fold puts the comparisons on the same footing.

Random Forest and Gradient Boosting have very close performance to one another, although sometimes one or the other of them requires more trees than the other to achieve it. The training times for Random Forest and Gradient Boosting are roughly equivalent. In some of the cases where they differ, one of them is getting trained much longer than required. In the abalone data set, for example, the oos error has flattened by 1,000 steps (trees), but training continues until 2,000. Changing that would cut the training time for Gradient Boosting in half and bring the training times for that data set more into agreement. The same is true for the wine data set.

Summary

This chapter demonstrated ensemble methods available as Python packages. The examples show these methods at work building models on a variety of different types of problems. The chapter also covered regression, binary classification,

and multiclass classification problems, and discussed variations on these themes such as the workings of coding categorical variables for input to Python ensemble methods and stratified sampling. These examples cover many of the problem types that you're likely to encounter in practice.

The examples also demonstrate some of the important features of ensemble algorithms—the reasons why they are a first choice among data scientists. Ensemble methods are relatively easy to use. They do not have many parameters to tune. They give variable importance data to help in the early stages of model development, and they very often give the best performance achievable.

The chapter demonstrated the use of available Python packages. The background given in Chapter 6 helps you to understand the parameters and adjustments that you see in the Python packages. Seeing them exercised in the example code can help you get started using these packages.

The comparisons given at the end of the chapter demonstrate how these algorithms compare. The ensemble methods frequently give the best performance. The penalized regression methods are blindingly much faster than ensemble methods and in some cases yield similar performance.

References

1. sklearn documentation for RandomForestRegressor, `http://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html`
2. Leo Breiman. (2001). "Random Forests." *Machine Learning*, *45*(1): 5–32. doi:10.1023/A:1010933404324
3. J. H. Friedman. "Greedy Function Approximation: A Gradient Boosting Machine," `https://statweb.stanford.edu/~jhf/ftp/trebst.pdf`
4. sklearn documentation for RandomForestRegressor, `http://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html`
5. L. Breiman, "Bagging predictors," `http://statistics.berkeley.edu/sites/default/files/tech-reports/421.pdf`
6. Tin Ho. (1998). "The Random Subspace Method for Constructing Decision Forests." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *20*(8): 832–844. doi:10.1109/34.709601
7. J. H. Friedman. "Greedy Function Approximation: A Gradient Boosting Machine," `https://statweb.stanford.edu/~jhf/ftp/trebst.pdf`

8. J. H. Friedman. "Stochastic Gradient Boosting," https://statweb.stanford.edu/~jhf/ftp/stobst.pdf
9. sklearn documentation for GradientBoostingRegressor, http://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html
10. J. H. Friedman. "Greedy Function Approximation: A Gradient Boosting Machine," https://statweb.stanford.edu/~jhf/ftp/trebst.pdf
11. J. H. Friedman. "Stochastic Gradient Boosting," https://statweb.stanford.edu/~jhf/ftp/stobst.pdf
12. J. H. Friedman. "Stochastic Gradient Boosting," https://statweb.stanford.edu/~jhf/ftp/stobst.pdf
13. sklearn documentation for RandomForestClassifier, http://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
14. sklearn documentation for GradientBoostingClassifier, http://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html

Index

Index

A

algorithms
  bagged decision trees, 4
  base learners, 211–212
  boosted decision trees, 4
  bootstrap aggregation, 226–236
  choosing, 11–13
  comparison, 6
  ensemble methods, 1
  linear, compared to nonlinear, 87–88
  logistic regression, 4
  multiclass classification problems, 314–315
  nonlinear, compared to linear, 87–88
  penalized linear regression methods, 1
  Random Forests, 4
ANNs (artificial neural nets), 4
argmin, 110–111
attributes. *See also* features; independent variables; inputs; predictors
  categorical variables, 26, 77
    statistical characterization, 37
  cross plots, 42–43
  factor variables, 26, 77
  features, 25
  function approximation and, 76
  increase, 5
  labels, relationship visualization, 42–49
  numeric variables, 26, 77
  predictions and, 3
  real-valued, 62–68, 77
  squares of, 197
  targets, correlation, 44–47
  times residuals, 197
AUC (area under the curve), 88

B

bagging, 11, 212, 226–236, 270–275
  bias *versus* variance, 229–231
  decision trees, 235–236
  multivariable problems and, 231–235
  random forests and, 247–250
base learners, 9, 211–212
basis expansion, 19
  linear methods/nonlinear problems, 156–158
best subset selection, 103
bias *versus* variance, 229–231

binary classification problems, 78
  ensemble methods, 284–302
  penalized linear regression methods and, 181–191
binary decision trees, 9–10, 212–213
  bagging, 11
  categorical features, 225–226
  classification features, 225–226
  overfitting, 221–225
  predictions and, 213–214
  training, 214–217
    tree training, 218–221
boosting, 212
bootstrap aggregating. *See* bagging
box and whisker plots, 54–55
  normalization and, 55

C

categorical variables, 19, 26
  binary decision trees, 225–226
  classification problems, 27
  statistical characterization, 37
chapter content and dependencies, 18–20
chirped signals, 28
chirped waveform, 151
class imbalances, 305–307
classification problems
  algorithms and, 2–3
  binary, penalized linear regression and, 181–191
  binary decision trees, 225–226
  categorical variables, 27
  chirped signals, 28
  class imbalances, 305–307
  converting to regression, 152–154
  multiclass, 68–73, 204–209
    ensemble methods, 302–314
  multiple outcomes, 155–156
  penalized linear regression methods, 151–155
coefficient estimation
  Lasso penalty and, 129–131
  penalized linear regression and, 122
coefficient penalized regression, 111
complex models, compared to simple models, 82–86
complexity
  balancing, 102–103
  simple problems *versus* complex problems, 80–82
complexity parameter, 110
confusion matrix, 91
contingency tables, 91
correlations
  heat map and, 49–50
    regression problems, 60–62
  Pearson's, 47–49
  targets and attributes, 44–47
cross plots, 42–43
cross-validation
  out-of-sample error, 168–172
  regression, 182–183

D

data frames, 37–38
data sets
  examples, 24
  instances, 24
  items to check, 27–28
  labels, 25
  observations, 24
  points, 7–8
  problems, 24–28
  shape, 29–32
  size, 29–32
  statistical summaries, 32–35
  unique ID, 25
  user ID, 25
deciles, 34
decision trees, binary, 9–10
  bagging, 11
degree of freedom, 86–87
dependencies, chapters in book, 18–20
dependent variables, 26

E

ElasticNet package, 128–129, 131–132, 181–191

ensemble methods, 1, 20, 211–212
bagged decision trees, 4
base learners, 9–11
binary decision trees, 9–10
bagging, 11
boosted decision trees, 4
multiclass classification problems, 302–314
penalized linear regression methods and, 124
penalized linear regression methods comparison, 11–13
Random Forests, 4
speed, 11
ensemble models
binary classification problems, 284–302
non-numeric attributes
coded variables, 278, 282–284
gradient boosting regression, 278–282
random forest regression, 275–278
ensemble packages, 255–256
random forest model, 256–270
errors, out-of-sample, 80

F

factor variables, 26
predictions, 50–62
false negatives, 92
false positives, 92
feature engineering, 7, 17–18, 76
feature extraction, 17–18
feature selection, 7
features, 25
function approximation and, 76
forward stepwise regression, 102
LARS and, 132–144
overfitting and, 103–108
function approximation, 1, 76, 124–125
performance, 78–79
training data, 76–78

G

Glmnet, 132, 144–145
initialization, 146–151
iterating, 146–151
LARS comparison, 145–146
gradient boosting, 236–239, 256–262, 291–298
classifier performance, 298–302, 307–311
multivariable problems and, 244–246
parameter settings, 239
performance, 240–243
predictive models and, 240
random forest model base learners, 311–314
GradientBoostingRegressor, 263–267
model performance, 269–270
regression model implementation, 267–269

H

heat map, correlations, 49–50
regression problems, 60–62

I

importance, 138
independent variables, 26
inputs, 11, 26

K

KNNs (k nearest neighbors), 4

L

labels, 16. *See also* dependent variables; outcomes; responses; targets
attributes, relationship visualization, 42–49
categorical, classification problems, 27
data sets, 25
function approximation and, 76
numeric, regression problems, 27
LARS (least-angle regression), 132

forward stepwise regression and, 132–144
Glmnet comparison, 145–146
model selection, 139–142
cross-validation in Python Code, 142–143
errors on cross-validation fold, 143
practical considerations, 143–144
Lasso penalty, 129–131
lasso training, data sets, 173–176
linear algorithms *versus* nonlinear, 87–88
linear methods
nonlinear problems and, 156–158
non-numeric attributes, 158–163
linear models, penalized linear regression and, 124
linear regression, 1
model training, 126–132
numeric input and, classification problems, 151–155
penalized linear regression methods, 1, 124–132
logistic regression, 1, 4, 155

M

MACD (moving average convergence divergence), 17
machine learning, problem formulation, 15–17
MAE (mean absolute error), 78–79, 88
mean, Pandas, 39
misclassification errors, 96
mixture model, 81
models
inputs, 11
LARS and, 136–138
MSE (mean squared error), 78–79, 88
multiclass classification problems, 68–73, 78, 204–209
algorithm comparison, 314–315
class imbalances, 305–307
ensemble methods, 302–314
multivariable regression, 167–168
bagging and, 231–235
gradient boosting and, 244–246
model building, 168–172
testing model, 168–172

N

n-fold cross-validation, 100
nonlinear algorithms, *versus* linear, 87–88
nonlinear problems, linear methods and, 156–158
non-numeric attributes, linear methods and, 158–163
normalization, box plots and, 55
notation, predictors, 77
numeric values, assigning to binary labels, 152–154
numeric variables, 26, 77
regression problems, 27

O

OLS (ordinary least squares), 7, 101, 121
coefficient penalties, 127–128
L1 norm, 129
Manhatten length, 129
outcomes, 26
function approximation and, 76
outliers, quantile-quantile plot, 35–37
out-of-sample errors, 80
cross-validation and, 168–172
overfitting
binary decision trees, 221–225
forward stepwise regression and, 103–108
ridge regression and, 110–119

P

packages
ElasticNet, 181–191
penalized linear regression methods, 166–167
Pandas, 37–39

parallel coordinates plots, 40–42, 64–66
regression problems, 56–60
Pearson's correlation, 47–49
penalized linear regression methods, 1, 20, 121
binary classification, 181–191
classification problems, 151–155
coefficient estimation, 122
coefficient penalized regression, 111
ensemble methods and, 124
ensemble methods comparison, 11–13
evaluation speed, 123
function approximation and, 124
Glmnet, 144–145
initialization, 146–151
iterating, 146–151
LARS comparison, 145–146
linear models and, 124
linear regression regulation, 124–132
multiclass classification, 204–209
OLS (ordinary least squares) and, 7
packages, 166–167
reliable performance, 123
sparse solution, 123
speed, 11
variable importance information, 122–123
percentiles, 34
plots
box and whisker, 54–55
cross plots, 42–43
parallel coordinates, 40–42
quantile-quantile, 35–37
scatter plots, 42
points, data sets, 7–8
pred( ) function, 79
predictions
attributes and, 3
binary decision trees, 212–213
factor variables and, 50–62
real-valued, 62–68
wine taste, 168–172
predictive models
building, 13–18
feature engineering, 7, 17–18
feature extraction, 17–18
feature selection, 7
function approximation, 76
performance, 78–79
training data, 76–78
gradient boosting and, 240
labels, 16
mathematical description, 19
performance factors, 86–87
performance measures, 88–99
targets, 14
trained, 25
performance evaluation, 18
predictors, 25
function approximation and, 76
notation, 77
problem formulation, 15–17

Q

quantiles, Pandas, 39
quantil-quantile plot, 35–37
quartiles, 34
quintiles, 34

R

random forest model, 256–270
base learners, gradient boosting and, 311–314
classification, 302–305
classifier performance, 291
random forests, 212
bagging and, 247–250
performance and, 251–252
RandomForestRegressor object, 256–262
real-valued attributes, 77
regression
penalized linear regression, 121
ridge regression, 121
step-wise, 121
regression problems
correlation heat map, 60–62

numeric variables, 27
parallel coordinates, 56–60
regressors, function approximation and, 76
relationships
attributes/labels, visualization, 42–49
variable, 56–60
reliable performance, 123
residuals, 137
attributes times residuals, 197
responses, 26
ridge regression, 102, 121
overfitting and, 110–119
RMSE (root MSE), 88
ROC (receiver operating curves), 88, 183
RSI (relative strength index), 17

S

scatter plots, 42
scikit-learn packages, 166
simple models, compared to complex models, 82–86
sklearn.linear_model, 166
sparse solution, 123
squares of attributes, 197
statistics, data sets, 32–35
stepwise regression, 121
stratified sampling, 37, 306
summaries
data sets, 32–35
Pandas, 38–39
supervised learning, 1
SVMs (support vector machines), 4

T

targets, 14, 26
attributes, correlation, 44–47
binary classification problem, 78
function approximation and, 76
multiclass classification problem, 78
trained models, 25
linear, 126–132
performance evaluation, 18
training
binary decision trees, 214–217
tree training, 218–221
training data, 76–78
deployment and, 172–181
tree training, 218–221

U

user ID, 25

V

validation, cross-validation, out-of-sample errors, 168–172
variable importance information, 122–123
variables
categorical, 19, 26
classification problems, 27
statistical characterization, 37
creating from old, 178–181
factor, 26
numeric, 26
regression problems, 27
relationships, 56–60
variance
versus bias, 229–231
Pandas, 39
visualization
attributes/labels relationship, 42–49
parallel coordinates plots, 40–42
variable relationships, 56–60

WILEY END USER LICENSE AGREEMENT

Go to www.wiley.com/go/eula to access Wiley’s ebook EULA.